

UNIVERSITÀ DEGLI STUDI DI SALERNO

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE ED ELETTRICA
E MATEMATICA APPLICATA

Corso:
DESIGN AND ANALYSIS OF ALGORITHMS



APPUNTI

Carmine Terracciano

Mat. IE22700109

ANNO ACCADEMICO 2025/2026

Sommario

1	Introduzione	7
1.1	Introduzione all'Approccio Orientato agli Oggetti	7
1.1.1	Concetti Fondamentali	7
1.1.2	Design Patterns	7
1.2	Abstract Data Types (ADT) vs Strutture Dati	8
1.2.1	Come si crea un ADT?	8
1.2.2	Come si utilizza un ADT?	8
1.3	ADT Stack (Pila)	9
1.3.1	Operazioni Fondamentali	9
1.3.2	Implementazione basata su Array	9
1.3.3	Pattern Adapter	10
1.4	Alberi (Trees)	10
1.4.1	Terminologia	10
1.4.2	ADT Tree	10
1.4.3	Calcolo dell'Altezza e Profondità	11
1.5	Alberi Binari (Binary Trees)	11
1.5.1	ADT BinaryTree	11
1.5.2	Implementazione Collegata (Linked Structure)	11
1.5.3	Il Pattern Position	12
1.5.4	Visite dell'Albero (Traversals)	13
1.6	ADT Map	14
1.6.1	Implementazione mediante lista doppiamente linkata non ordinata	14
1.6.2	Sorted Map	14
2	Binary Search Tree (BST)	17
2.1	BST Insert	17
2.2	BST Delete	18
2.2.1	Cancellazione di un nodo con al massimo un figlio	18
2.2.2	Cancellazione di un nodo con due figli	18
2.3	Analisi delle prestazioni	19
2.4	Richiami: Tipi di alberi binari	20
3	AVL Tree (AVLT)	21
3.1	Alberi bilanciati e Balance Factor	21

3.2	Definizione di Albero AVL	22
3.3	Altezza di un AVL Tree	22
3.4	Mantenimento della Proprietà AVL: Ristrutturazione	23
3.4.1	Caso 1 e 2: Rotazioni Singole (Casi Esterni)	24
3.4.2	Caso 3 e 4: Rotazioni Doppie (Casi Interni)	25
3.5	AVL Insert	26
3.6	AVL Delete	27
3.7	Analisi delle prestazioni	27
4	Multi-Way Search Tree (MWST)	29
4.1	(a, b)-Tree	30
4.2	Insert	32
4.3	Delete	33
4.4	B-Tree	35
5	Red-Black Trees e (2, 4) Trees	37
5.1	(2, 4) Trees	37
5.2	Red-Black Trees	38
5.3	Dal (2, 4)-Tree al Red-Black Tree	39
5.4	Dal Red-Black Tree al (2, 4)-Tree	39
5.5	Altezza di un Red-Black Tree	40
5.6	Insert	40
5.6.1	Come risolvere un double red	40
5.6.2	Complessità dell'inserimento	41
5.7	Delete	42
5.7.1	Caso 1: Delete di un nodo rosso	42
5.7.2	Caso 2: Delete di un nodo nero con un solo figlio (rosso)	42
5.7.3	Caso 3: Delete di un nodo nero senza figli	43
5.7.4	Complessità della cancellazione	48
6	Hash Tables	49
6.1	Hash Functions	50
6.1.1	Esempi di Hash Code	52
6.1.2	Esempi di Funzione di Compressione	52
6.2	Gestione delle Collisioni	53
6.2.1	Separate Chaining	53
6.2.2	Open Addressing	54
6.3	Load Factor	57
6.4	Analisi delle prestazioni	57
7	Priority Queues	59
7.1	Chiavi (Priorità)	59
7.2	Possibili implementazioni di Priority Queue	60
7.3	Heap	61
7.4	Implementazione di una Priority Queue attraverso Heap	61

7.5	Implementazione Array-based di un Heap	66
7.6	Analisi delle prestazioni	67
7.7	Costruzione di un Heap da una lista di elementi: Heapify	67
7.8	Heap-Sort	70
7.9	Adaptable Priority Queue	73
8	Pattern Matching	75
8.1	Brute Force	76
8.2	L'algoritmo di Boyer-Moore	78
8.3	L'algoritmo di Knuth-Morris-Pratt	83
8.3.1	Failure-Function	83
9	Tries	87
9.1	Standard Tries	87
9.2	Compressed Tries	90
9.3	Suffix Tries	92
9.4	Pattern Matching con Suffix Tries	93
9.5	The Substring Problem	94
9.6	Longest Common Substring: two strings	95
9.7	Longest Common Substring: many strings	96
10	Greedy Algorithms	97
10.1	Modello generale	98
10.2	Coin Change: The Cashier Algorithm	99
10.3	Scheduling	100
10.3.1	Task Scheduling	100
10.3.2	Interval Scheduling	102
10.4	Fractional Knapsack	104
10.5	Text Compression	106
10.5.1	Codifica di Huffman	106
11	Dynamic Programming	109
11.1	Modello generale	109
11.2	Longest Common Subsequence (LCS)	111
11.3	Edit Distance: Levenshtein Distance	115
11.4	Sequence Alignment	119
11.5	Coin Change per sistemi non canonici	122
11.6	0-1 Knapsack	125
11.7	Matrix Chain-Product	128
12	Local Search	131
12.1	Problema: Vertex Cover	133
12.1.1	Gradient Descent per Vertex Cover	133
12.2	Neighbor Relation: Algoritmo Metropolis	134
12.2.1	La Funzione di Gibbs-Boltzmann	135
12.2.2	Funzionamento dell'Algoritmo	135

12.3	Neighbor Relation: Simulated Annealing	136
12.4	Problema: Max Cut	137
12.5	Problema: Hopfield Neural Networks	140
12.5.1	Configurazioni Stabili	140
12.5.2	Algoritmo State-Flipping	141
12.5.3	Convergenza e Complessità	142
13	Graphs	143
13.1	Rappresentazioni dell'ADT Grafo	148
13.2	Rappresentazione mediante Edge List	149
13.3	Rappresentazione mediante Adjacency List	150
13.4	Rappresentazione mediante Adjacency Map	152
13.5	Rappresentazione mediante Adjacency Matrix	154
13.6	Implementazione Python (Adjacency Map)	156
14	Graphs Traversal	161
14.1	Depth-First Search (DFS)	162
14.1.1	Analisi e implementazione dell'algoritmo DFS	165
14.1.2	DFS su grafi orientati	166
14.1.3	Estensioni della DFS	167
14.2	Breadth-First Search (BFS)	170
14.2.1	Analisi e implementazione dell'algoritmo BFS	173
14.2.2	Estensioni della BFS	174
15	Transitive Closure	175
15.1	Algoritmo di Floyd-Warshall per la Chiusura Transitiva	176
15.1.1	Analisi e implementazione dell'algoritmo	178
16	Directed Acyclic Graph - DAG	181
16.1	Topological Sorting	181
16.1.1	Implementazione Python	182
16.1.2	Analisi della complessità	183
17	Shortest Paths	185
17.1	Weighted Graphs	185
17.2	Algoritmo di Dijkstra	187
17.2.1	Edge Relaxation - Greedy Choice	187
17.2.2	Pseudocodice ed Esempio	188
17.2.3	Dimostrazione della correttezza dell'algoritmo	190
17.2.4	Analisi della complessità	192
17.2.5	Implementazione Python dell'algoritmo di Dijkstra	194
17.3	Algoritmo di Bellman-Ford	196
17.4	Shortest Paths in un DAG	199
17.5	All-Pairs Shortest Paths	201
17.6	Riepilogo Finale: Algoritmi di Shortest Path	202
18	Minimum Spanning Trees (MST)	203

18.1	L'algoritmo di Prim-Jarník	206
18.2	L'algoritmo di Kruskal	210
18.3	Reverse-Delete Algorithm	214
19	Network Flow	215
19.1	Flow Network - Rete di flusso	216
19.2	Maximum Flow Problem	218
19.2.1	Residual Graph	219
19.2.2	Augmenting Path	220
19.2.3	MaxFlow Algorithm	222
19.2.4	s-t Cuts	224
19.3	Il problema del Bipartite Matching	229

Capitolo 1

Introduzione

1.1 Introduzione all'Approccio Orientato agli Oggetti

Lo sviluppo di software efficiente si basa sulla progettazione e analisi di algoritmi e strutture dati. L'approccio moderno privilegia la programmazione orientata agli oggetti (OOP) per promuovere la robustezza, la riusabilità e l'adattabilità del software.

1.1.1 Concetti Fondamentali

In questo contesto, i dati non sono entità isolate ma sono incapsulati insieme ai metodi necessari per accedervi e modificarli.

- **Incapsulamento:** I dati sono protetti e accessibili solo tramite un'interfaccia pubblica.
- **Information Hiding:** L'utente interagisce con l'interfaccia senza conoscere i dettagli implementativi interni.

1.1.2 Design Patterns

Un *Design Pattern* è una soluzione standard a un problema tipico di progettazione software. Fornisce un modello generale che può essere specializzato per adattarsi a problemi specifici. Un pattern è composto da:

1. **Nome:** Identifica il pattern.
2. **Contesto:** Descrive quando può essere applicato.
3. **Template:** Descrive come viene applicato.
4. **Risultato:** Analizza ciò che il pattern produce.

Esempi di pattern includono *Iterator*, *Adapter*, *Factory* (Software Engineering) e *Divide-and-Conquer*, *Greedy* (Algorithm Design).

1.2 Abstract Data Types (ADT) vs Strutture Dati

È fondamentale distinguere tra il modello astratto e la sua realizzazione concreta.

- **Abstract Data Type (ADT):** È un modello matematico che descrive un tipo di dato attraverso i suoi comportamenti (semantica) e i parametri delle operazioni. Descrive *cosa* fanno le operazioni, ma non *come* (punto di vista dell'utente). È indipendente dall'implementazione.
- **Struttura Dati:** È la realizzazione concreta di un ADT. Descrive la rappresentazione in memoria dei dati e l'implementazione degli algoritmi per le operazioni (punto di vista del progettista).

In un linguaggio object-oriented, ADT e struttura dati sono realizzati tramite delle classi. Una classe definisce sia l'interfaccia pubblica (ADT) sia l'implementazione effettiva (struttura dati).

1.2.1 Come si crea un ADT?

Per progettare un ADT bisogna seguire i seguenti passi:

1. Identificare le operazioni.
 - Definire le operazioni che l'ADT deve offrire e quali parametri richiedono.
 - Descrivere il significato (semantica) di ciascuna operazione.
2. Definire la classe tramite l'interfaccia.
 - Scrivere la classe che rappresenta l'ADT, specificando solo i metodi pubblici (l'interfaccia) necessari all'uso dell'astrazione.
3. Implementazione di un testing client.
 - Creare un programma di test che utilizzi l'ADT per verificare correttezza e efficienza delle operazioni.

Per ogni operazione dell'ADT bisogna specificare:

- **Precondizioni:** condizione che deve essere vera prima dell'esecuzione dell'operazione. Se non è soddisfatta, viene generato un errore e lanciata un'eccezione.
- **Post-condizione:** condizione che deve essere vera dopo l'esecuzione dell'operazione, se la precondizione era validata.

1.2.2 Come si utilizza un ADT?

Una volta progettato e implementato un ADT, il suo utilizzo avviene esclusivamente tramite la sua interfaccia pubblica, senza conoscere i dettagli interni dell'implementazione.

1. Si crea un'istanza dell'ADT, si dichiara un oggetto della classe che implementa l'ADT.
2. Usare le operazioni definite dall'interfaccia, chiamare i metodi pubblici per accedere o modificare i dati.
3. Non accedere mai ai dati interni direttamente: i campi interni sono incapsulati e protetti.
4. Gestire eccezioni e precondizioni: controllare che le precondizioni delle operazioni siano rispettate; se falliscono, l'ADT può lanciare un'eccezione.

1.3 ADT Stack (Pila)

Uno **Stack** è una collezione di oggetti inseriti e rimossi secondo il principio **LIFO** (Last-In, First-Out).

1.3.1 Operazioni Fondamentali

- `push(e)`: Inserisce l'elemento e in cima alla pila.
- `pop()`: Rimuove e restituisce l'elemento in cima. Se vuota, lancia un'eccezione.
- `top()`: Restituisce (senza rimuovere) l'elemento in cima.
- `is_empty()`: Restituisce `True` se la pila è vuota.
- `len()`: Restituisce il numero di elementi.

1.3.2 Implementazione basata su Array

Una semplice implementazione in Python utilizza una lista dinamica (array dinamico). I dati vengono inseriti da sinistra a destra e una variabile mantiene l'indice dell'ultimo elemento aggiunto (*top*).

Gestione della Memoria e Analisi Ammortizzata

Quando l'array è pieno, è necessario ridimensionarlo ("stretching").

- Non si può semplicemente estendere la memoria contigua esistente.
- Bisogna allocare un nuovo array più grande e copiare gli elementi.

Se si incrementasse la dimensione di una sola unità per ogni inserimento, il costo sarebbe quadratico nel tempo. La strategia efficiente è il **Raddoppiamento** della dimensione.

Analisi Ammortizzata: Considera il costo totale di una sequenza di operazioni. Sebbene il raddoppiamento costi $O(n)$, esso avviene raramente. Ammortizzando questo costo sulle operazioni economiche ($O(1)$) di inserimento semplice:

- **Costo reale:** 1 (array non pieno) o $O(n)$ (resize).
- **Costo ammortizzato:** $O(1)$ per ogni operazione di push.

1.3.3 Pattern Adapter

L'implementazione dello Stack può utilizzare il pattern *Adapter* (o *Wrapper*). Si utilizza una classe esistente (es. `list` di Python) e si adatta la sua interfaccia a quella richiesta dall'ADT Stack.

Listing 1.1: Esempio di Classe ArrayStack

```
1 class ArrayStack:
2     def __init__(self):
3         self._data = [] # Lista privata
4
5     def push(self, e):
6         self._data.append(e) # Adatta append a push
7
8     def pop(self):
9         if self.is_empty():
10            raise Empty('Stack is empty')
11        return self._data.pop() # Adatta pop a pop
```

1.4 Alberi (Trees)

Un albero è un modello astratto per rappresentare strutture gerarchiche non lineari. È composto da nodi con relazioni padre-figlio.

1.4.1 Terminologia

- **Radice (Root):** Nodo senza padre.
- **Nodo Interno:** Nodo con almeno un figlio.
- **Foglia (Leaf) / Nodo Esterno:** Nodo senza figli.
- **Profondità (Depth) di un nodo:** Numero di antenati.
- **Altezza (Height) dell'albero:** Massima profondità tra i nodi (o altezza della radice).

Un albero è ordinato se esiste un ordine lineare significativo tra i figli di ciascun nodo. Ogni figlio può essere identificato come primo, secondo, terzo... La visualizzazione tipica è quella in cui i fratelli sono allineati da sinistra a destra.

1.4.2 ADT Tree

Un albero T è un insieme di nodi e un insieme di relazioni padre-figlio che soddisfano le seguenti proprietà:

- Se T è non vuoto, esiste un nodo speciale chiamato **radice** che non ha padre.

- Ogni nodo v di T , diverso dalla radice, ha un unico nodo padre w ; tutti i nodi che hanno w come padre sono chiamati **figli** di w , e sono tra di loro **fratelli**.

L'ADT Tree definisce metodi di accesso e navigazione generici:

- `root()`: Restituisce la posizione della radice.
- `parent(p)`: Restituisce il genitore della posizione p .
- `children(p)`: Restituisce un iteratore sui figli di p .
- `is_leaf(p)`, `is_root(p)`: Query booleane.

1.4.3 Calcolo dell'Altezza e Profondità

Questi algoritmi sono spesso implementati ricorsivamente.

$$\text{depth}(p) = \begin{cases} 0 & \text{se } p \text{ è la radice} \\ 1 + \text{depth}(\text{parent}(p)) & \text{altrimenti} \end{cases} \quad (1.1)$$

$$\text{height}(p) = \begin{cases} 0 & \text{se } p \text{ è una foglia} \\ 1 + \max(\{\text{height}(c) \mid c \in \text{children}(p)\}) & \text{altrimenti} \end{cases} \quad (1.2)$$

1.5 Alberi Binari (Binary Trees)

Un albero binario è un albero ordinato in cui ogni nodo ha **al massimo due figli**, distinti in *figlio sinistro* e *figlio destro*.

1.5.1 ADT BinaryTree

Estende l'ADT Tree con metodi specifici:

- `left(p)`: Restituisce il figlio sinistro di p .
- `right(p)`: Restituisce il figlio destro di p .
- `sibling(p)`: Restituisce il fratello di p .

1.5.2 Implementazione Collegata (Linked Structure)

A differenza degli array, gli alberi sono spesso implementati come strutture collegate per ottimizzare inserimenti e cancellazioni ($O(1)$ per aggiornamenti locali). Si utilizza una classe annidata `_Node` che contiene:

- `_element`: Il dato memorizzato.
- `_parent`: Riferimento al nodo padre.

- `_left`: Riferimento al figlio sinistro.
- `_right`: Riferimento al figlio destro.

Inoltre, si utilizza il concetto di **ADT Position** per incapsulare il nodo. L'utente manipola oggetti di tipo *Position*, non direttamente i nodi, proteggendo la struttura interna.

1.5.3 Il Pattern Position

Per implementare collezioni di elementi, come gli alberi, in modo robusto e astratto, si utilizza il pattern (o ADT) `**Position**`.

Concetto e Scopo

Una *Position* rappresenta un'astrazione di un "nodo" all'interno della struttura dati. A differenza di un semplice nodo, che potrebbe esporre direttamente i puntatori al padre e ai figli, una *Position* funge da "segnalibro" o "puntatore sicuro" all'elemento:

- È un oggetto che incapsula un elemento.
- Protegge la struttura interna: l'utente manipola le posizioni, ma solo la classe albero può accedere ai collegamenti (link) tra i nodi reali sottostanti.
- Una *Position* *p* rimane valida e non è influenzata dai cambiamenti negli altri nodi, a meno che non venga esplicitamente cancellata.

L'unica operazione pubblica principale dell'ADT *Position* è il metodo per accedere al dato contenuto:

```
p.element() # Restituisce l'elemento memorizzato in P
```

Implementazione in Python

Nel contesto dell'implementazione `'LinkedBinaryTree'`, la classe *Position* è definita come un wrapper pubblico intorno alla classe privata `_Node`. Ogni istanza di *Position* memorizza:

1. Un riferimento al nodo reale (`_node`).
2. Un riferimento al contenitore a cui appartiene (`_container`), utile per verificare che la posizione sia utilizzata con l'albero corretto.

Ecco un esempio dell'implementazione della classe *Position*:

Listing 1.2: Classe Position (Wrapper)

```

1 class Position(BinaryTree.Position):
2     """Classe pubblica che avvolge un nodo."""
3     def __init__(self, container, node):
4         """Costruttore interno, non invocato dall'utente."""
5         self._container = container
6         self._node = node
7
8     def element(self):
9         """Restituisce l'elemento contenuto nella Position."""
10        return self._node._element
11
12    def __eq__(self, other):
13        """Restituisce True se other si riferisce alla stessa Position."""
14        return type(other) is type(self) and other._node is self._node

```

Meccanismo di Validazione: Per garantire la robustezza, l'albero implementa un metodo interno `_validate(p)` che controlla se una data posizione è valida prima di usarla. Una posizione è considerata valida se:

- È un'istanza della classe corretta.
- Appartiene all'istanza dell'albero corrente (`p._container is self`).
- Il nodo sottostante non è stato eliminato (spesso i nodi eliminati vengono marcati facendo puntare il loro padre a se stessi).

1.5.4 Visite dell'Albero (Traversals)

Esistono diverse strategie per visitare sistematicamente i nodi:

1. **Preorder (Anticipata):** Visita la radice, poi ricorsivamente i sottoalberi.
2. **Postorder (Posticipata):** Visita ricorsivamente i sottoalberi, poi la radice.
3. **Inorder (Simmetrica):** Specifica per alberi binari. Visita sottoalbero sinistro, poi radice, poi sottoalbero destro.
4. **Breadth-First (Visita in Ampiezza):** Visita i nodi livello per livello. Richiede l'uso di una **Coda** (Queue) per memorizzare i nodi da visitare.

Listing 1.3: Algoritmo Breadth-First

```

1 def breadthfirst(self):
2     if not self.is_empty():
3         fringe = ArrayQueue()
4         fringe.enqueue(self.root())
5         while not fringe.is_empty():
6             p = fringe.dequeue()
7             yield p
8             for c in self.children(p):
9                 fringe.enqueue(c)

```

1.6 ADT Map

Una mappa è una struttura dati usata per implementare l'astrazione del dizionario: permette di cercare, inserire e aggiornare informazioni a partire da una chiave. Si parla di **contenitore associativo** perché la mappa associa una variabile di tipo ordinabile, detta *key*, alle posizioni che individuano un dato, detto *value*. In altre parole, a ogni chiave è associato un valore memorizzato nella struttura. Una mappa può essere vista come una collezione di elementi a coppia, chiamati **item**, ciascuno della forma $\langle key, value \rangle$.

Fondamentale è il **vincolo di unicità** sulle chiavi: ogni chiave deve identificare univocamente un item nella mappa. I valori, invece, non è obbligatorio che siano univoci: lo stesso *value* può essere associato a più chiavi diverse, ma non possono esistere due item con la stessa *key*.

Python contiene già una implementazione di Dizionario (questo per capire quanto sia importante come struttura dati). L'implementazione che fornisce Python è una di quelle che studieremo, ma non sarà l'unica possibile. Le varie implementazioni saranno:

- Mediante un array non ordinato.
- Mediante un array ordinato.
- Usando alberi di vario tipo.
- Usando tabella hash.

1.6.1 Implementazione mediante lista doppiamente linkata non ordinata

Implementare una Map mediante una lista doppiamente linkata non ordinata non è il massimo per quanto riguarda le performances.

- Inserimento: Tempo $O(1)$
- Ricerca e rimozione: Tempo $O(n)$

Nel caso pessimo bisogna scorrere l'intera lista per cercare una chiave (nel caso in cui l'elemento è l'ultimo dell'array) o renderti conto che non è presente (nel caso in cui l'elemento appunto non è contenuto).

1.6.2 Sorted Map

Una **Sorted Map** è una mappa che mantiene gli item ordinati in base alle chiavi. Questo ovviamente implica che le chiavi debbano essere di un tipo di valore ordinabile.

Gli item sono ordinati in base all'ordinamento delle chiavi. La sorted map supporta un'operazione chiamata "Nearest Neighbour". Questa operazione implica che, rispetto alla mappa non ordinata, possiamo fare una ricerca e sapere con tempo $O(1)$ il successore ed il predecessore, questo può essere utile quando cerchiamo una chiave che non è contenuta nella SortedMap per conoscere la chiave più vicina al valore oggetto della nostra ricerca.

- Successore: Data una chiave K mi dici qual è la chiave più piccola tra tutte quelle che sono maggiori k .
- Predecessore: Data una chiave K mi dici qual è la chiave più grande tra tutte quelle che sono minori k .

Non solo, sfruttando sempre la proprietà dell'ordinamento possiamo dire il massimo ed il minimo dell'intera mappa.

Una possibile implementazione di Sorted Map è quella mediante un array ordinato. In questo caso, per inserire un nuovo item, dobbiamo trovare la posizione corretta per mantenere l'ordinamento. Questo richiede una ricerca binaria, che ha un costo di $O(\log n)$, seguita da uno spostamento degli elementi per fare spazio al nuovo item, che ha un costo di $O(n)$ nel caso pessimo. Quindi, l'inserimento in una Sorted Map implementata con un array ordinato ha un costo complessivo di $O(n)$.

Questa implementazione può essere efficiente se la struttura dati viene utilizzata per effettuare ricerche frequenti con pochissime operazioni di inserimento/cancellazione.

Capitolo 2

Binary Search Tree (BST)

Definizione: La struttura dati **albero binario di ricerca (BST)** è un albero binario i cui nodi contengono elementi del tipo chiave-valore, e soddisfa le seguenti proprietà:

- La chiave di ogni nodo è maggiore di tutte le chiavi nel suo sottoalbero sinistro.
- La chiave di ogni nodo è minore di tutte le chiavi nel suo sottoalbero destro.
- Tutti i nodi esterni non contengono elementi e sono considerati come nodi nulli (None) o nodi foglia.

Questa struttura (che non ammette chiavi duplicate) consente di eseguire le operazioni fondamentali di ricerca, inserimento e cancellazione in tempo proporzionale all'altezza dell'albero ($O(h)$).

2.1 BST Insert

L'inserimento di una coppia chiave-valore (k, v) in un BST avviene seguendo questi passaggi:

- Per prima cosa si esegue una ricerca per la chiave k nell'albero, per verificare se k esiste già nell'albero o in alternativa per trovare la posizione corretta per l'inserimento.
 - Partendo dalla radice, si confronta k con la chiave del nodo corrente:
 - Se k è minore, si procede nel sottoalbero sinistro.
 - Se k è maggiore, si procede nel sottoalbero destro.
 - Questo processo continua fino a raggiungere un nodo p che sarà il padre del nuovo nodo o fino a trovare k .
- Se k è già presente, si aggiorna il valore associato a k con v .
- Se k non è presente, si crea un nuovo nodo con la coppia (k, v) e lo si inserisce come figlio di p .
 - Se $k < p.key()$, il nuovo nodo viene inserito come figlio sinistro di p .
 - Se $k > p.key()$, viene inserito come figlio destro di p .

2.2 BST Delete

Per prima cosa si esegue una ricerca per la chiave k per verificare che k esista nell'albero.

- Se k non è presente, l'operazione termina senza modifiche all'albero.
- Se k è presente, si procede con la cancellazione facendo una distinzione tra due casi:
 - Il nodo da cancellare ha al massimo un figlio.
 - Il nodo da cancellare ha due figli.

2.2.1 Cancellazione di un nodo con al massimo un figlio

In questo caso, possiamo semplicemente rimuovere il nodo da cancellare p e collegare il suo unico figlio r (se esiste) al padre.

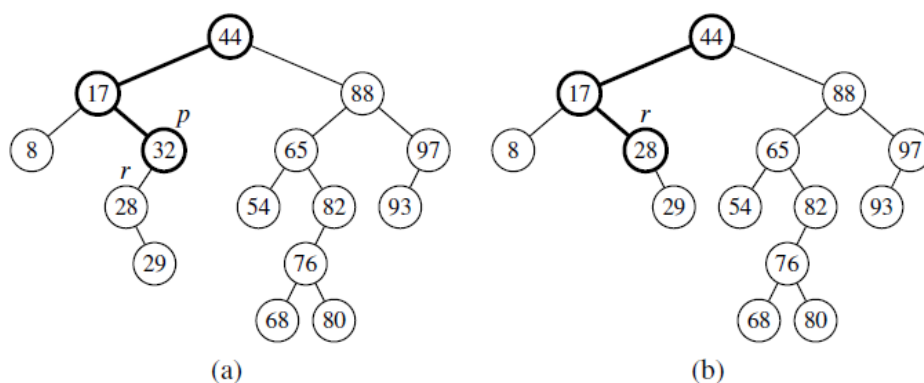


Figure 11.5: Deletion from the binary search tree of Figure 11.4b, where the item to delete (with key 32) is stored at a position p with one child r : (a) before the deletion; (b) after the deletion.

2.2.2 Cancellazione di un nodo con due figli

In questo caso, dobbiamo trovare un nodo sostituto per mantenere le proprietà del BST. Il nodo sostituto può essere (arbitrariamente) il *successore* (il nodo con la chiave più piccola [il nodo più a sinistra] nel sottoalbero destro) o il *predecessore* (il nodo con la chiave più grande [il nodo più a destra] nel sottoalbero sinistro).

Ipotizziamo di utilizzare il predecessore: $r = \text{before}(p)$. Una volta trovato il nodo sostituto r , copiamo la sua chiave e valore nel nodo da cancellare p e poi cancelliamo il nodo r , che avrà al massimo un figlio (quello sinistro) in quanto è il nodo più a destra nel sottoalbero sinistro. Dunque, dopo aver scambiato i valori, possiamo procedere con la cancellazione di r come nel primo caso, collegando il suo figlio (se esiste) al padre di r .

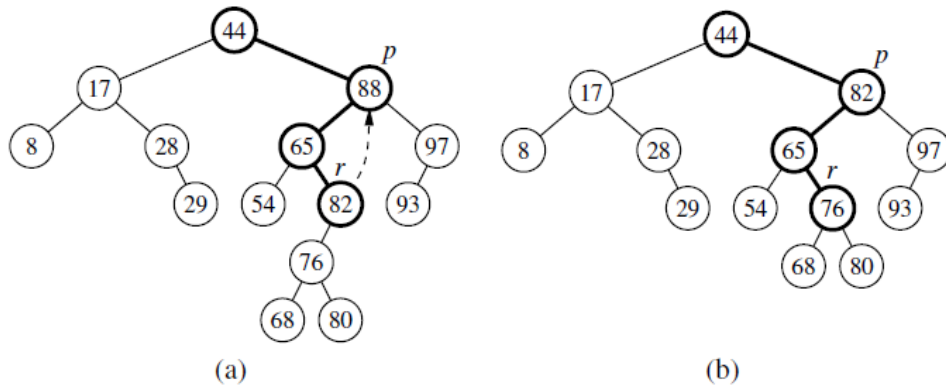


Figure 11.6: Deletion from the binary search tree of Figure 11.5b, where the item to delete (with key 88) is stored at a position p with two children, and replaced by its predecessor r : (a) before the deletion; (b) after the deletion.

2.3 Analisi delle prestazioni

Operation	Running Time
$k \text{ in } T$	$O(h)$
$T[k], T[k] = v$	$O(h)$
$T.\text{delete}(p), \text{del } T[k]$	$O(h)$
$T.\text{find_position}(k)$	$O(h)$
$T.\text{first}(), T.\text{last}(), T.\text{find_min}(), T.\text{find_max}()$	$O(h)$
$T.\text{before}(p), T.\text{after}(p)$	$O(h)$
$T.\text{find_lt}(k), T.\text{find_le}(k), T.\text{find_gt}(k), T.\text{find_ge}(k)$	$O(h)$
$T.\text{find_range}(\text{start}, \text{stop})$	$O(s + h)$
$\text{iter}(T), \text{reversed}(T)$	$O(n)$

Table 11.1: Worst-case running times of the operations for a TreeMap T . We denote the current height of the tree with h , and the number of items reported by `find_range` as s . The space usage is $O(n)$, where n is the number of items stored in the map.

Un BST è un'implementazione efficiente di un Map solo quando la sua altezza h è piccola. Partiamo dalla relazione che lega il numero di nodi n all'altezza h nel caso migliore (un albero completo). Un albero di altezza h può contenere al massimo $2^{h+1} - 1$ nodi.

Quindi, per contenere n nodi, l'altezza h deve soddisfare:

$$\begin{aligned}
 n &\leq 2^{h+1} - 1 && \text{(Nodi massimi per altezza } h) \\
 n + 1 &\leq 2^{h+1} && \text{(Aggiungo 1 a entrambi i lati)} \\
 \log_2(n + 1) &\leq \log_2(2^{h+1}) && \text{(Applico } \log_2 \text{ a entrambi i lati)} \\
 \log_2(n + 1) &\leq h + 1 && \text{(Semplifico } \log_2(2^x) = x) \\
 \log_2(n + 1) - 1 &\leq h && \text{(Sottraggo 1)}
 \end{aligned}$$

Dato che h deve essere il più piccolo intero che soddisfa questa disequazione, applichiamo la funzione **ceiling** (soffitto, $\lceil \dots \rceil$) al lato sinistro:

$$h = \lceil \log_2(n + 1) - 1 \rceil$$

Che è matematicamente equivalente a scrivere:

$$h = \lceil \log_2(n + 1) \rceil - 1$$

Per definizione, la funzione **floor** (pavimento, $\lfloor \dots \rfloor$) restituisce l'unico intero h che soddisfa $h \leq x < h + 1$. Di conseguenza, possiamo scrivere direttamente:

$$h = \lfloor \log_2(n) \rfloor$$

2.4 Richiami: Tipi di alberi binari

Esistono diverse definizioni precise per classificare gli alberi binari in base alla loro struttura e al loro bilanciamento. Queste distinzioni sono fondamentali, perché la forma dell'albero determina l'efficienza delle operazioni su di esso.¹

La differenza tra un albero "completo" e un albero "perfetto" è particolarmente importante.

¹Ecco le definizioni chiave (spesso confuse tra loro) dei vari tipi di alberi:

- **Albero Binario Pieno (Full Binary Tree):** Un albero in cui ogni nodo ha **zero o due figli**. Non sono ammessi nodi con un solo figlio.
- **Albero Binario Perfetto (Perfect Binary Tree):** Un albero *pieno* in cui tutte le **foglie si trovano allo stesso livello** (stessa profondità). È la forma "perfetta" che massimizza il numero di nodi per una data altezza h .
- **Albero Binario Completo (Complete Binary Tree):** Un albero in cui tutti i livelli, **tranne eventualmente l'ultimo**, sono completamente pieni. Se l'ultimo livello non è pieno, i suoi nodi sono "impacchettati" il più possibile a sinistra. Questa è la struttura usata dagli *Heap*.
- **Albero Bilanciato (Balanced Binary Tree):** Termine generico per un albero la cui altezza è garantita essere $O(\log n)$. Esistono diverse definizioni specifiche:
 - **Bilanciato in altezza (es. Albero AVL):** Un albero binario di ricerca in cui, per *ogni nodo*, la differenza tra l'altezza del sottoalbero sinistro e quella del sottoalbero destro è al massimo 1.
 - **Perfettamente bilanciato (o bilanciato in peso):** Un albero in cui, per *ogni nodo*, il numero di nodi nel sottoalbero sinistro differisce al massimo di 1 dal numero di nodi nel sottoalbero destro. Questa definizione è molto più restrittiva e raramente usata.
- **Albero Degenerato (o Sbilanciato):** Il caso peggiore. Un albero in cui ogni nodo genitore ha un solo nodo figlio, trasformandosi essenzialmente in una lista concatenata. L'altezza è $O(n)$.

Capitolo 3

AVL Tree (AVLT)

Inserire n elementi casuali in un BST produce, in media, un albero di altezza $O(\log n)$. Tuttavia, nel caso peggiore (ad esempio, inserendo gli elementi in ordine crescente), l'altezza dell'albero può diventare $O(n)$, degradando le prestazioni delle operazioni di ricerca, inserimento e cancellazione a tempo lineare.

Per evitare questo problema, vogliamo modificare le operazioni di inserimento e cancellazione in modo da mantenere l'altezza dell'albero sempre logaritmica rispetto al numero di nodi n , ossia $h = O(\log n)$, garantendo così prestazioni logaritmiche anche nel caso peggiore. Gli **alberi AVL** sono una delle strutture dati che implementano questa idea di bilanciamento automatico.

3.1 Alberi bilanciati e Balance Factor

Per misurare quantitativamente il bilanciamento di un albero, introduciamo una metrica chiamata **Balance Factor**.

Il balance factor di un nodo v , che indichiamo con $\beta(v)$, è definito come la differenza tra l'altezza del suo sottoalbero sinistro e l'altezza del suo sottoalbero destro.

La formula per il calcolo è:

$$\beta(v) = \text{height}(\text{left}(v)) - \text{height}(\text{right}(v))$$

Dove $\text{height}(\dots)$ è la funzione che calcola l'altezza di un sottoalbero. Per convenzione, **l'altezza di un sottoalbero nullo (inesistente) è -1** .

Il valore del balance factor ci dice lo stato del nodo:

- $\beta(v) = 0$: Il nodo è bilanciato (i due sottoalberi hanno la stessa altezza).
- $\beta(v) = +1$: Il sottoalbero sinistro è più alto di 1 (l'albero è "pendente a sinistra").
- $\beta(v) = -1$: Il sottoalbero destro è più alto di 1 (l'albero è "pendente a destra").

3.2 Definizione di Albero AVL

Un albero **AVL** è un albero binario di ricerca (BST) che soddisfa una specifica proprietà di bilanciamento basata sul *balance factor*.

Definizione: Un albero binario di ricerca è un **albero AVL** se, **per ogni nodo** v appartenente all'albero, il valore assoluto del suo balance factor $\beta(v)$ è al più 1.

$$|\beta(v)| = |\text{height}(\text{left}(v)) - \text{height}(\text{right}(v))| \leq 1$$

Questo vincolo, se mantenuto dopo ogni operazione, è sufficiente a garantire che l'altezza totale dell'albero h rimanga sempre logaritmica ($h = O(\log n)$). Se un'operazione di inserimento o cancellazione viola questa proprietà (creando un nodo con $\beta(v) = +2$ o -2), l'albero esegue delle specifiche operazioni chiamate **rotazioni** per ripristinare il bilanciamento.

3.3 Altezza di un AVL Tree

L'altezza di un AVL Tree con n nodi è $O(\log n)$.

Dimostrazione Definiamo $n(h)$ come il numero minimo di nodi che un albero AVL di altezza h può avere. Notiamo subito che $n(1) = 1$ e $n(2) = 2$ (considerando i nodi esterni None come nodi foglia, e quindi considerando che un albero con solo la radice abbia altezza 1).

Per $h \geq 3$, il numero minimo di nodi in un albero AVL di altezza h si ottiene quando uno dei sottoalberi ha altezza $h - 1$ e l'altro ha altezza $h - 2$ (la differenza massima consentita di 1 tra le altezze dei sottoalberi). Quindi, possiamo definire la seguente relazione ricorsiva:

$$n(h) = 1 + n(h - 1) + n(h - 2)$$

Ciò implica che $n(h)$ è una funzione strettamente crescente di h , e quindi sappiamo che $n(h) > n(h - 1) > n(h - 2)$. Sostituendo $n(h - 1)$ con $n(h - 2)$ nella relazione ricorsiva, ed eliminando 1, otteniamo:

$$n(h) > 2 \cdot n(h - 2)$$

Questo ci indica che $n(h)$ almeno raddoppia ogni volta che h aumenta di 2, il che intuitivamente significa che $n(h)$ cresce esponenzialmente.

$$n(h) > 2 \cdot n(h - 2) > 4 \cdot n(h - 4) > 8 \cdot n(h - 6) > \dots > 2^i \cdot n(h - 2i)$$

Questa relazione vale per ogni intero i tale che $h - 2i \geq 1$. Dato che conosciamo i valori di $n(1)$ e $n(2)$, scegliamo i in modo che $h - 2i$ sia uguale a 1 o 2. In altre parole, scegliamo:

$$i = \left\lceil \frac{h}{2} \right\rceil - 1$$

Sostituendo questo valore di i nella disuguaglianza precedente, otteniamo:

$$n(h) > 2^{\lceil \frac{h}{2} \rceil - 1} \cdot n(1) \geq 2^{\frac{h}{2} - 1}$$

Applicando il logaritmo in base 2 a entrambi i membri, otteniamo:

$$\log_2(n(h)) > \frac{h}{2} - 1$$

Riorganizzando questa disuguaglianza, otteniamo:

$$h < 2 \log_2(n(h)) + 2$$

Che dimostra che un AVL Tree con n elementi ha altezza al più $2 \log_2(n)$, ovvero $O(\log n)$.

3.4 Mantenimento della Proprietà AVL: Ristrutturazione

Come abbiamo visto, un albero AVL è un BST che deve obbedire alla proprietà di bilanciamento. Le operazioni standard di inserimento e cancellazione di un BST possono violare questa proprietà, creando un nodo con fattore di bilanciamento $+2$ o -2 .

Quando questo accade, l'albero deve essere "riparato". L'operazione di riparazione è chiamata **ristrutturazione** (o ribilanciamento) e viene implementata attraverso una o più operazioni primitive chiamate **rotazioni**.

Dopo un inserimento (o una cancellazione), risaliamo dall'elemento inserito p (o dal padre p dell'elemento cancellato) verso la radice per aggiornare i fattori di bilanciamento. Chiamiamo z il **primo nodo antenato che incontriamo che risulta sbilanciato**, ovvero con $\beta(z) = +2$ o $\beta(z) = -2$.

Una volta identificato z , definiamo:

- y : il **figlio di z con altezza maggiore** (e nota che y deve essere un antenato di p).
- x : il **figlio di y con altezza maggiore**. (non può esserci un pareggio e la posizione x deve anche essere un antenato di p , possibilmente p stesso).

L'operazione di ribilanciamento, chiamata **ristrutturazione trinodale**, coinvolge sempre e solo questi tre nodi (x, y, z) e i loro 4 possibili sottoalberi. L'obiettivo è riordinare x , y e z in modo da ottenere un albero binario di ricerca bilanciato. Si identificano i tre nodi (in ordine crescente di chiave) come a , b , c . Il nodo con la chiave mediana (b) diventerà la nuova radice, a diventerà il suo figlio sinistro e c il suo figlio destro. I 4 sottoalberi vengono poi riagganciati ordinatamente. Questo processo logico unificato si traduce in quattro diversi casi, che richiedono due tipi di operazioni meccaniche: **le rotazioni singole e le rotazioni doppie**.

Un'operazione di rotazione singola o doppia ha sempre una complessità temporale di $O(1)$, poiché coinvolge solo un numero costante di nodi e puntatori.

3.4.1 Caso 1 e 2: Rotazioni Singole (Casi Esterni)

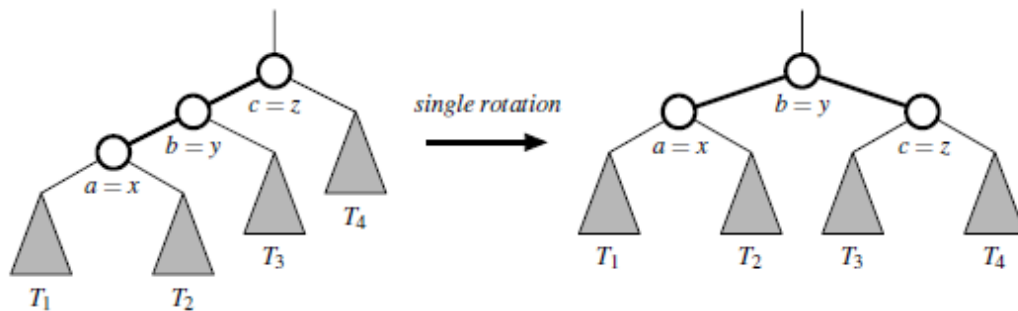
Si ha una rotazione singola quando x , y e z sono allineati sullo stesso lato.

Caso Sinistra-Sinistra (LL)

Questo caso si verifica quando y è il figlio sinistro di z e x è il figlio sinistro di y .

- z : Nodo sbilanciato ($\beta(z) = +2$)
- y : Figlio sinistro di z
- x : Figlio sinistro di y

Soluzione: Rotazione Singola a Destra (su z) Il nodo b diventa la nuova radice del sottoalbero. Il nodo a rimane il figlio sinistro di b , mentre c diventa il figlio destro di b . Inoltre, il sottoalbero destro di b viene agganciato come nuovo sottoalbero sinistro di c .

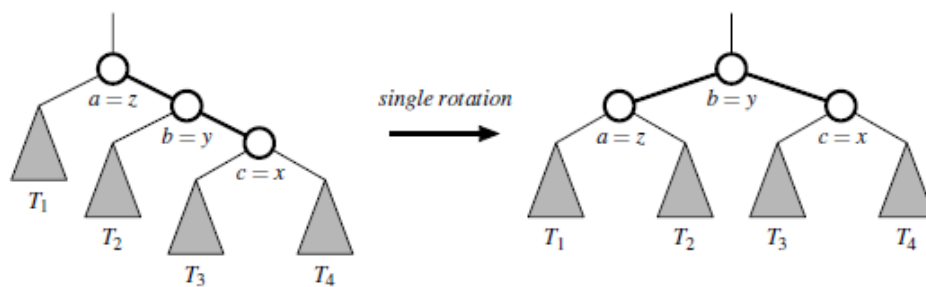


Caso Destra-Destra (RR)

Questo è il caso speculare. y è il figlio destro di z e x è il figlio destro di y .

- z : Nodo sbilanciato ($\beta(z) = -2$)
- y : Figlio destro di z
- x : Figlio destro di y

Soluzione: Rotazione Singola a Sinistra (su z) Il nodo b diventa la nuova radice del sottoalbero. Il nodo a diventa il figlio sinistro di b , mentre c rimane il figlio destro di b . Inoltre, il sottoalbero sinistro di b viene agganciato come nuovo sottoalbero destro di a .



3.4.2 Caso 3 e 4: Rotazioni Doppie (Casi Interni)

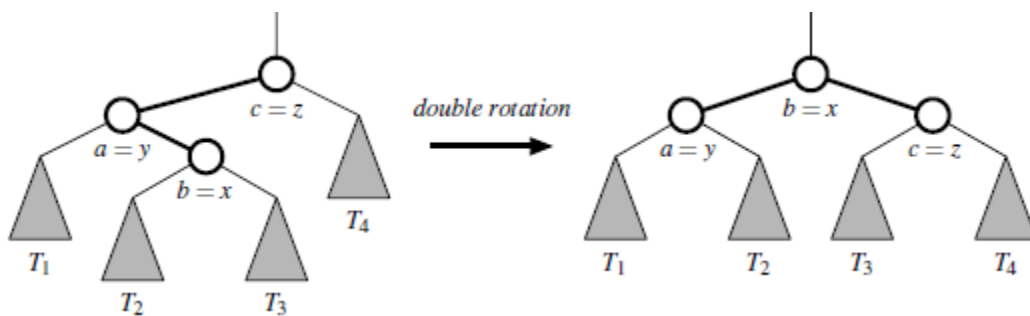
Si ha una rotazione doppia quando x , y e z formano un "gomito" (o "zig-zag").

Caso Sinistra-Destra (LR)

Questo caso si verifica quando y è il figlio sinistro di z , ma x è il figlio destro di y .

- z : Nodo sbilanciato ($\beta(z) = +2$)
- y : Figlio sinistro di z
- x : Figlio **destro** di y

Soluzione: Doppia Rotazione (Sinistra-Destra) È sempre b a diventare la nuova radice del sottoalbero, con a come figlio sinistro e c come figlio destro. Inoltre, il sottoalbero sinistro di b viene agganciato come nuovo sottoalbero destro di a , e il sottoalbero destro di b viene agganciato come nuovo sottoalbero sinistro di c .

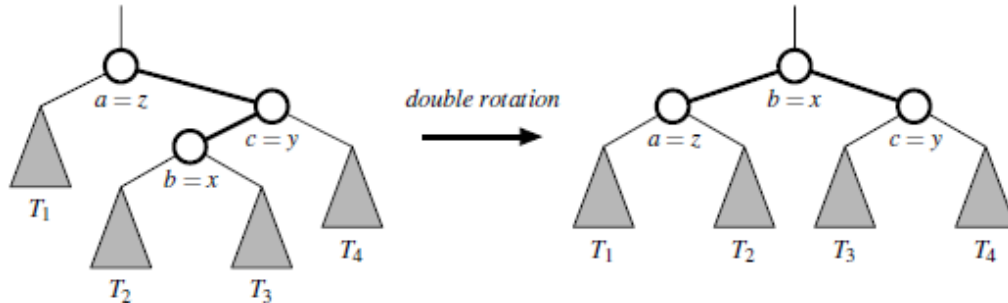


Caso Destra-Sinistra (RL)

Questo è il caso speculare. y è il figlio destro di z , ma x è il figlio *sinistro* di y .

- z : Nodo sbilanciato ($\beta(z) = -2$)
- y : Figlio destro di z
- x : Figlio **sinistro** di y

Soluzione: Doppia Rotazione (Destra-Sinistra) È sempre b a diventare la nuova radice del sottoalbero, con a come figlio sinistro e c come figlio destro. Inoltre, il sottoalbero sinistro di b viene agganciato come nuovo sottoalbero destro di a , e il sottoalbero destro di b viene agganciato come nuovo sottoalbero sinistro di c .



3.5 AVL Insert

L'inserimento in un albero AVL segue le stesse regole di un normale albero binario di ricerca, con l'aggiunta della necessità di mantenere l'equilibrio dell'albero. Dopo aver inserito un nuovo nodo, denotiamo questo nodo come p , e si risale lungo il cammino verso la radice, aggiornando i fattori di bilanciamento e applicando le rotazioni necessarie per ripristinare l'equilibrio al primo nodo z che risulta essere non bilanciato.

In particolare, in un albero AVL, l'applicazione di una singola ristrutturazione è sufficiente per ripristinare l'equilibrio dell'intero albero dopo un inserimento problematico.

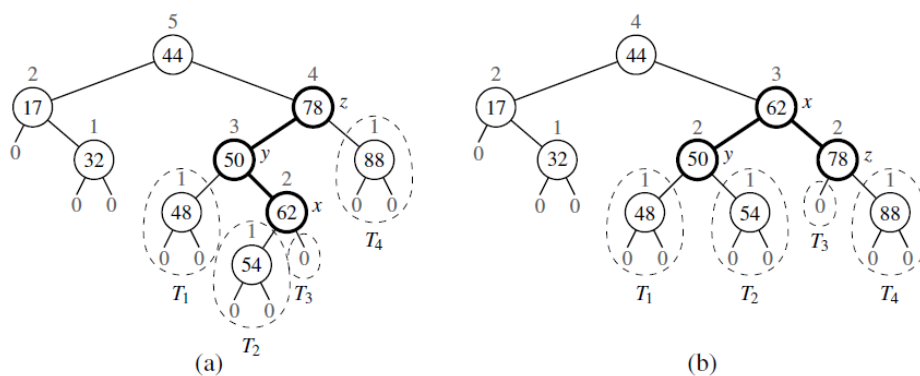


Figure 11.12: An example insertion of an item with key 54 in the AVL tree of Figure 11.11: (a) after adding a new node for key 54, the nodes storing keys 78 and 44 become unbalanced; (b) a trinode restructuring restores the height-balance property. We show the heights of nodes above them, and we identify the nodes x , y , and z and subtrees T_1 , T_2 , T_3 , and T_4 participating in the trinode restructuring.

3.6 AVL Delete

La cancellazione in un albero AVL segue le stesse regole di un normale albero binario di ricerca, con l'aggiunta della necessità di mantenere l'equilibrio dell'albero. Dopo aver cancellato un nodo, denotiamo il padre del nodo cancellato come p , e si risale lungo il cammino verso la radice, aggiornando i fattori di bilanciamento e applicando le rotazioni necessarie per ripristinare l'equilibrio a ogni nodo z che risulta essere non bilanciato.

In particolare, in un albero AVL, può essere necessario eseguire fino a $O(\log n)$ ristrutturazioni per ripristinare l'equilibrio dell'intero albero dopo una cancellazione problematica.

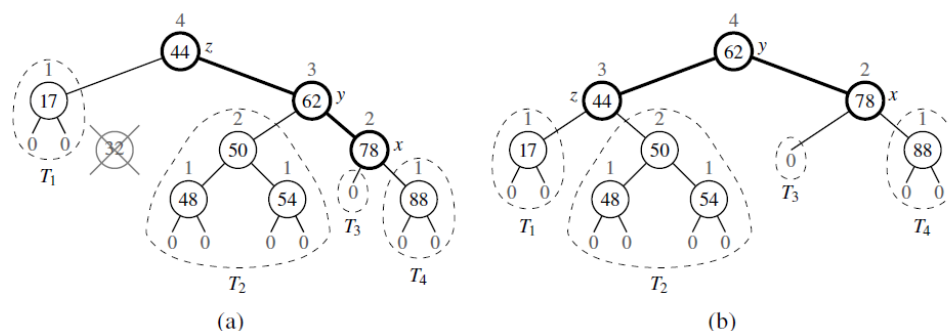


Figure 11.14: Deletion of the item with key 32 from the AVL tree of Figure 11.12b: (a) after removing the node storing key 32, the root becomes unbalanced; (b) a (single) rotation restores the height-balance property.

3.7 Analisi delle prestazioni

Operation	Running Time
k in T	$O(\log n)$
$T[k] = v$	$O(\log n)$
$T.delete(p)$, del $T[k]$	$O(\log n)$
$T.find_position(k)$	$O(\log n)$
$T.first()$, $T.last()$, $T.find_min()$, $T.find_max()$	$O(\log n)$
$T.before(p)$, $T.after(p)$	$O(\log n)$
$T.find_lt(k)$, $T.find_le(k)$, $T.find_gt(k)$, $T.find_ge(k)$	$O(\log n)$
$T.find_range(start, stop)$	$O(s + \log n)$
$iter(T)$, $reversed(T)$	$O(n)$

Table 11.2: Worst-case running times of operations for an n -item sorted map realized as an AVL tree T , with s denoting the number of items reported by `find_range`.

L'altezza di un albero AVL con n elementi è garantita essere $O(\log n)$. Poiché l'operazione standard di ricerca binaria su albero aveva tempi di esecuzione limitati dall'altezza e poiché il lavoro aggiuntivo per mantenere i fattori di bilanciamento e ristrutturare un albero AVL può essere limitato dalla lunghezza di un percorso nell'albero, le operazioni di mappatura tradizionali vengono eseguite nel caso peggiore in tempo logaritmico con un albero AVL.

Capitolo 4

Multi-Way Search Tree (MWST)

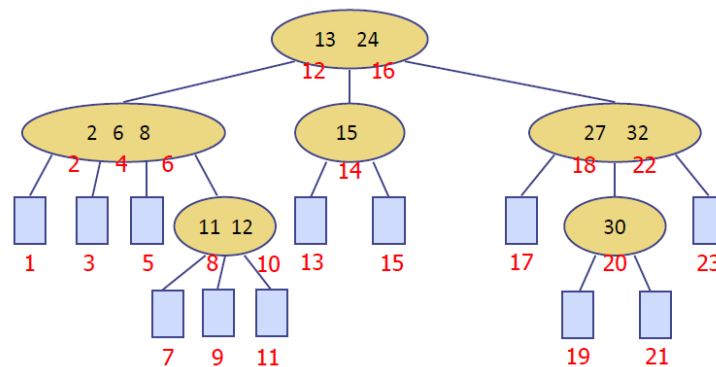
Definizione: Un **Multi-Way Search Tree (MWST)** è un albero ordinato in cui:

- Ogni nodo dell'albero (d -nodo) ha $d \geq 2$ figli e contiene $d - 1$ elementi chiave valore (k_i, v_i) ordinati in modo crescente per chiave.
- Sia w un nodo con figli $w_1, w_2, \dots, w_d$ e con chiavi $k_1, k_2, \dots, k_{d-1}$. Allora:
 - Tutte le chiavi nel sottoalbero radicato in w_1 sono minori di k_1 .
 - Tutte le chiavi nel sottoalbero radicato in w_i sono comprese tra k_{i-1} e k_i ($i = 2, \dots, d - 1$).
 - Tutte le chiavi nel sottoalbero radicato in w_d sono maggiori di k_{d-1} .
- I nodi foglia (None) non contengono elementi.

Si osservi come un MWST contenente n elementi abbia $n + 1$ nodi foglia (None).

MWST: In-Order Visit

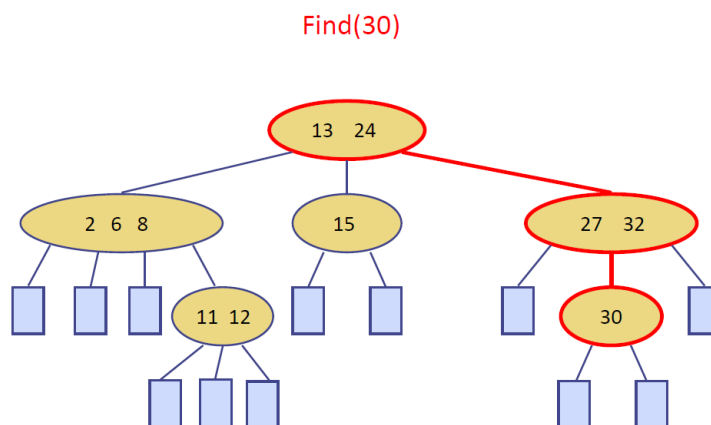
- Item (k_i, v_i) in node w is visited after the subtree rooted in w_i and before the subtree rooted in w_{i+1}



MWST: Search

- Looking for key k
 - Consider an internal node w with children $w_1 w_2 \dots w_d$ with keys $k_1 k_2 \dots k_{d-1}$
 - If $k = k_i$ ($i = 1, 2, \dots, d-1$), search ends successfully
 - If $k < k_1$ search in the subtree rooted in w_1
 - If $k_{i-1} < k < k_i$ search in the subtree rooted in w_i ($i = 2, \dots, d-1$)
 - If $k > k_{d-1}$ search in the subtree rooted in w_d
 - If we reach a leaf, search ends unsuccessfully

MWST Search: Example



4.1 (a, b)-Tree

Definizione: Gli (a, b)-Tree sono dei MWST che soddisfano le seguenti proprietà:

- $2 \leq a \leq \lceil \frac{b-1}{2} \rceil$
- **Root Property:** La radice ha almeno 2 figli e al più b figli.
- **Node-Size Property:** Ogni nodo diverso dalla radice ha almeno a figli e al più b figli.
- **Depth Property:** Tutti i nodi foglia (None) sono allo stesso livello.

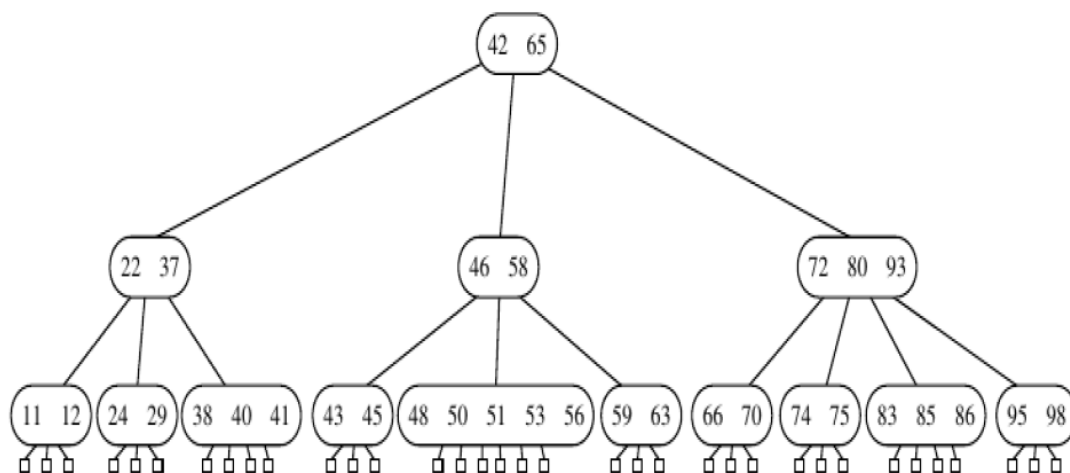


Figura 4.1: Esempio di un (a, b) -Tree con $a = 3$ e $b = 6$.

Altezza di un (a, b) -Tree e Ricerca di un elemento

L'altezza di un (a, b) -Tree con n elementi è:

- $\Omega(\log n / \log b)$ [== $\Omega(\log_b n)$].
- $O(\log n / \log a)$ [== $O(\log_a n)$].

Per ogni nodo, è necessario eseguire una ricerca tra tutti i suoi elementi. Un nodo è una *Map* di al più $b - 1$ elementi, per cui possiamo denotare con $f(b)$ il costo della ricerca in un nodo. Se l'elemento non è stato trovato è necessario scendere in uno dei figli, per cui il costo totale della ricerca in un (a, b) -Tree è:

$$O(f(b) \cdot \log n / \log a)$$

Se $f(b)$ è costante, il costo della ricerca è migliore di $O(\log n)$.

4.2 Insert

Per inserire un elemento (k, v) in un (a, b) -Tree, si procede come segue:

1. Si esegue una ricerca per trovare la posizione corretta in cui inserire l'elemento: supponendo che l'albero non presenti già un elemento con chiave k , la ricerca termina senza successo restituendo il nodo foglia (None) z . Sia w il genitore di z , inseriamo il nuovo elemento in w e aggiungiamo una nuova foglia (None) y come figlio di w .
2. A questo punto, se w ha meno di $b - 1$ elementi (cioè ha meno di b figli), l'inserimento è terminato.
3. Altrimenti, se w ha già $b - 1$ elementi (cioè b figli), si verifica un **overflow** e dobbiamo eseguire un **split** di w :
 - Consideriamo il nodo w con i suoi $b - 1$ elementi, più il nuovo elemento appena inserito.
 - Si esegue uno **split** di w in tre parti: gli elementi minori di k' , l'elemento mediano k' e gli elementi maggiori di k' .
 - Si crea un nuovo nodo che conterrà gli elementi minori di k' . (Questo nodo è automaticamente valido perchè contiene almeno $\lceil (b - 1)/2 \rceil - 1 \geq a - 1$ elementi e sicuramente meno di $b - 1$ elementi).
 - Vale lo stesso per il nodo che conterrà gli elementi maggiori di k' .
 - L'elemento mediano k' viene promosso al genitore p di w e diventa un nuovo elemento di p , con i due nodi risultanti dallo split come suoi figli.
 - Se p ha ora b elementi, si ripete la procedura di split su p .
 - Altrimenti, l'inserimento è terminato.
 - Se il nodo splittato era la radice, si crea una nuova radice che contiene solo l'elemento mediano m e ha come figli i due nodi risultanti dallo split.

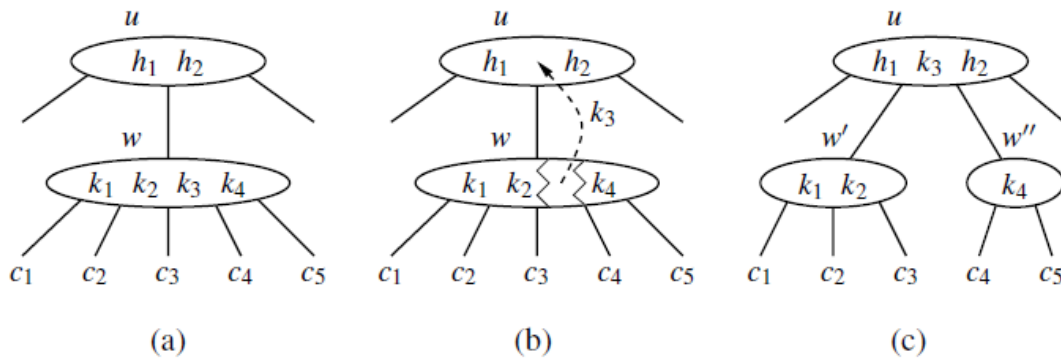


Figura 4.2: Esempio di inserimento con overflow in un (a, b) -Tree con $a = 2$ e $b = 4$. (a) overflow in un 5-nodo w [il massimo è un 4-nodo]; (b) e (c) split di w .

4.3 Delete

Per eliminare un elemento (k, v) in un (a, b) -Tree, si procede come segue:

1. Si esegue una ricerca per trovare il nodo w contenente l'elemento da eliminare k . Se il nodo non esiste, l'operazione termina.
2. Se il figlio a sinistra o a destra di k non è vuoto, oppure w è la radice, allora:
 - Si trova il predecessore o il successore di k (cioè l'elemento più grande del sottoalbero sinistro o l'elemento più piccolo del sottoalbero destro) e lo indichiamo con k' .
 - Si scambia l'elemento con chiave k con l'elemento con chiave k' .
 - Si elimina l'elemento con chiave k' dal sottoalbero (l'elemento originario con chiave k').
3. Altrimenti, se entrambi i figli di k sono vuoti, e:
 - il nodo w ha $> a - 1$ elementi, eliminiamo l'elemento con chiave k da w e l'operazione termina.
 - il nodo w ha esattamente il numero minimo di $a - 1$ elementi, si verifica un **underflow**.
4. Gestione dell'underflow:
 - Se il nodo x fratello sinistro di w (con lo stesso genitore p) ha più di $a - 1$ elementi, si esegue un **transfer**:
 - Sia k' la chiave salvata nel genitore p "che sta tra il puntatore a x e il puntatore a w ".
 - Sia k'' la chiave più grande nel nodo x (ricordiamo che x è il fratello a sinistra di w).
 - Cancella k da w , cancella k'' da x e sostituisci k' con k'' in p , e aggiungi k' in w .
 - Se il nodo x fratello destro di w (con lo stesso genitore p) ha più di $a - 1$ elementi, si esegue un **transfer**:
 - L'opposto del caso precedente: si prende la chiave più piccola dal fratello destro x e la si sposta in w , aggiornando di conseguenza il genitore p .
 - Altrimenti, se entrambi i fratelli di w hanno esattamente $a - 1$ elementi, si esegue una **fusion**:
 - Sia x un fratello di w (a sinistra o a destra) con lo stesso genitore p .
 - Sia k' la chiave salvata nel genitore p "che sta tra il puntatore a x e il puntatore a w ".
 - Si crea un nuovo nodo che contiene tutti gli elementi di w eccetto l'elemento con chiave k , tutti gli elementi di x e l'elemento con chiave k' .

* È un nodo valido perchè contiene un numero di elementi pari a:

$$a - 1 \leq (a - 2) + (a - 1) + 1 = 2a - 2 \leq b - 1$$

– Si elimina k' da p .

* Se p è la radice e k' è il suo unico elemento, il nuovo nodo creato diventa la radice.

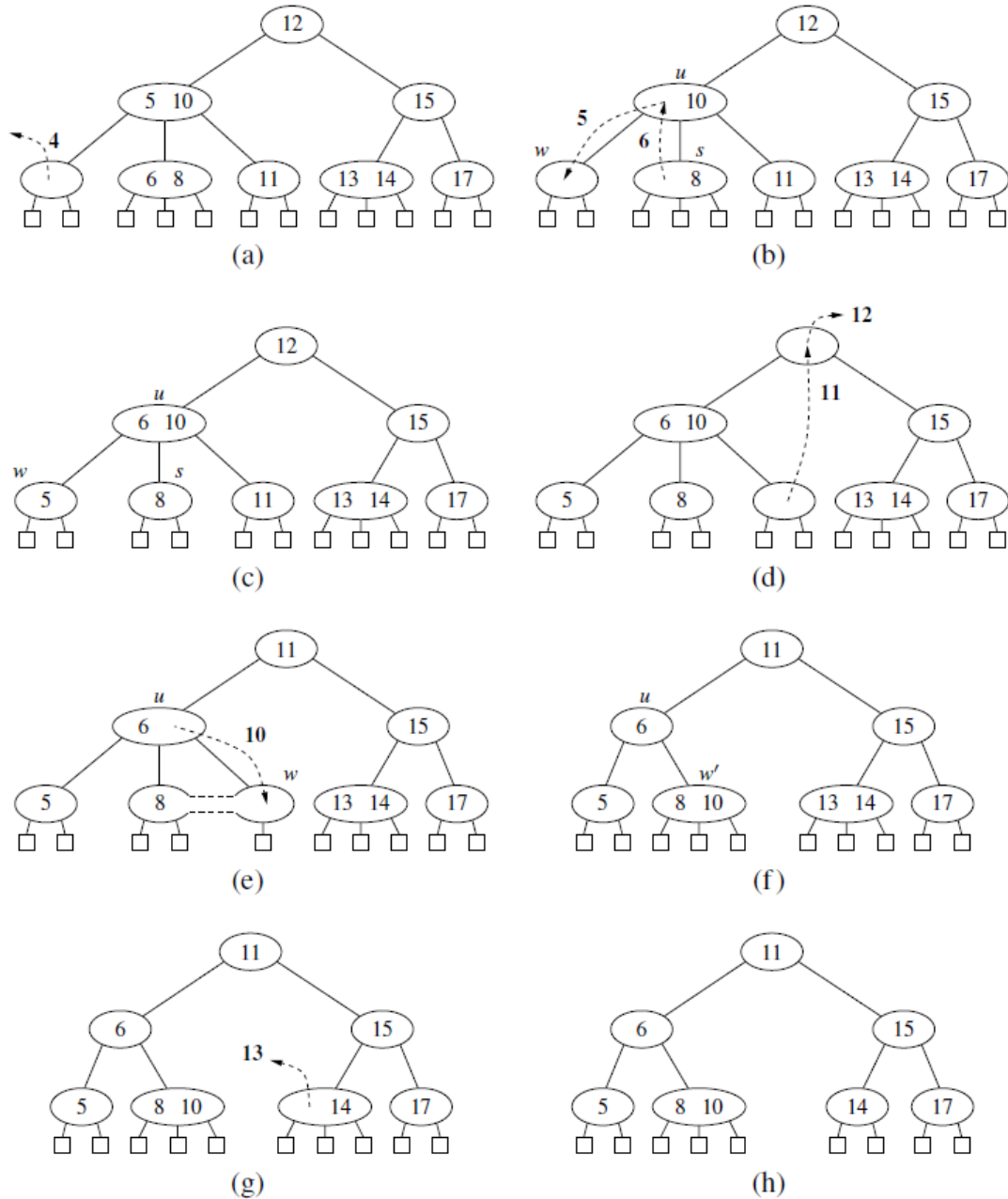


Figura 4.3: Una sequenza di rimozioni in un (a, b)-Tree con $a = 2$ e $b = 4$. (a) rimozione di 4, che causa un underflow; (b) un'operazione di transfer; (c) dopo l'operazione di transfer; (d) rimozione di 12, che causa un underflow; (e) un'operazione di fusion; (f) dopo l'operazione di fusion; (g) rimozione di 13; (h) dopo la rimozione di 13.

Complessità delle operazioni di inserimento e cancellazione

Si tenga presente che la ricerca di un nodo, come visto prima, impiega un tempo $O(f(b) \cdot \log n / \log a)$. Supponendo che la gestione di un overflow/underflow richieda al più $g(b)$ [$g(b)$ dipende dall'implementazione del nodo], e considerando che potrebbe essere necessario ripetere al più tali operazioni dal livello $h - 1$ fino alla radice. Quindi, il costo totale dell'inserimento in un (a, b)-Tree è:

$$O((f(b) + g(b)) \log n / \log a)$$

Come scegliere a e b?

Se i nodi sono troppo piccoli, l'albero sarà essenzialmente simile ad un albero binario di ricerca bilanciato. Nonostante ciò, vedremo che i (2, 4)-Tree sono particolarmente importanti perchè possono essere trasformati in degli alberi particolari chiamati **Red-Black Tree**.

Se i nodi sono troppo grandi, l'albero diventa meno efficiente in termini di spazio e di tempo per le operazioni di ricerca, inserimento e cancellazione. Dipende da come è implementata la Map all'interno del nodo.

- Se si tratta di Hash Tables, la ricerca e gli aggiornamenti sono $O(1)$, ma il calcolo della mediana è $O(b)$.
- Se si tratta di BST bilanciati, la ricerca e gli aggiornamenti sono $O(\log b)$ e il calcolo della mediana è $O(b)$.
- Se si tratta di vettori ordinati, la ricerca è $O(\log b)$, gli aggiornamenti sono $O(b)$ e il calcolo della mediana è $O(1)$.

Idealmente, vorremmo che tutte queste operazioni vadano come $O(1)$, ma ciò non è possibile. In pratica, si cerca di minimizzare b in modo che le operazioni siano efficienti, ma abbastanza grande da ridurre l'altezza dell'albero.

Se a e b sono troppo distanti tra loro, la complessità delle operazioni interne al nodo cancellano il vantaggio dato dalla riduzione dell'altezza [$f(b)$ e $g(b)$ crescono troppo rapidamente fino a diventare di gran lunga più pesanti di $\log a$].

4.4 B-Tree

Un **B-Tree** è un (a, b)-Tree con $a = \lceil (b - 1)/2 \rceil$ e $b = d$.

Per le considerazioni fatte sugli (a, b)-Tree, si ha che:

- La ricerca ha una complessità di $O(f(d) \cdot \log n / (\log d - 1))$.
- L'inserimento e la cancellazione hanno una complessità di $O(g(d) \cdot \log n / (\log d - 1))$.

I/O Complexity

Consideriamo il problema della gestione di una grande raccolta di elementi che non rientrano nella memoria principale, come un tipico database. In questo contesto, ci riferiamo ai blocchi di memoria secondaria come *disk blocks*. Allo stesso modo, ci riferiamo al trasferimento di un blocco tra la memoria secondaria e la memoria primaria come *disk transfer*. Ricordando la grande differenza di tempo che esiste tra gli accessi alla memoria principale e gli accessi al disco, l'obiettivo principale della gestione di tale raccolta nella memoria esterna è quello di ridurre al minimo il numero di trasferimenti su disco necessari per eseguire una query o un aggiornamento. Ci riferiamo a questo numero come **I/O complexity** dell'algoritmo coinvolto.

Il modo migliore di ridurre al minimo il numero di trasferimenti su disco è quello di massimizzare il numero di elementi che possono essere memorizzati in un singolo blocco. Supponiamo che ogni blocco di disco possa contenere B elementi. Nel caso di un B-Tree, scegliamo il suo *ordine* d (il numero massimo di figli per nodo) il più grande possibile, in modo tale che un nodo – contenente $O(d)$ elementi e puntatori – occupi al massimo un singolo blocco di disco. Si ottiene così una relazione diretta tra d e B , ovvero $d = \Theta(B)$. Di conseguenza, ogni accesso a un nodo del B-Tree corrisponde a un singolo trasferimento su disco. Scegliendo d così grande, si massimizza il fattore di diramazione (branching factor) e si minimizza l'altezza dell'albero. Poiché l'altezza di un B-Tree con n elementi è $O(\log_d n)$, e dato che $d = \Theta(B)$, la **I/O complexity** per la **ricerca** è $O(\log_B n)$. Anche l'inserimento e la cancellazione mantengono questa complessità, poiché, oltre alla ricerca iniziale, richiedono un numero di operazioni di modifica (come divisioni o fusioni di nodi) proporzionale all'altezza dell'albero. In conclusione, i B-Tree sono una struttura dati molto efficiente per la gestione di grandi raccolte di elementi nella memoria esterna.

Capitolo 5

Red-Black Trees e (2, 4) Trees

Abbiamo già parlato di alberi bilanciati nel Capitolo 3, in particolare degli alberi AVL. Questo tipo di alberi consentiva di mantenere l'altezza dell'albero logaritmica rispetto al numero di nodi presenti, garantendo così operazioni di ricerca, inserimento e cancellazione efficienti. Tuttavia, una cancellazione in un albero AVL poteva richiedere molte rotazioni per mantenere l'equilibrio, rendendo l'operazione più costosa in termini di tempo.

5.1 (2, 4) Trees

I (2, 4) Trees sono semplicemente degli (a, b) Trees con $a = 2$ e $b = 4$. Ciò significa che ereditano tutte le proprietà degli (a, b) Trees discusse nel Capitolo 4, e dal punto di vista di un'analisi asintotica, le performance di un (2, 4) Tree sono equivalenti a quelle di un albero AVL. Per un (2, 4) Tree:

- L'altezza è $O(\log n)$.
- Le operazioni di split, transfer e fusion hanno una complessità di $O(1)$.
- Le operazioni di ricerca, inserimento e cancellazione hanno una complessità di $O(\log n)$.

All'interno di un (2, 4) Tree, ogni nodo può contenere da 1 a 3 chiavi e può avere da 2 a 4 figli. Questo consente di distinguere tra 2-nodi, 3-nodi e 4-nodi, sulla base del numero di figli (numero di chiavi + 1). Questo tipo di alberi è particolarmente interessante per la sua relazione che ha con un tipo particolare di alberi, i **Red-Black Trees**.

5.2 Red-Black Trees

Definizione: Un Red-Black Tree è un albero binario di ricerca in cui ogni nodo ha un colore: rosso o nero, e l'albero soddisfa le seguenti proprietà:

- **Root Property:** La radice deve essere nera.
- **External Property:** Tutti i nodi foglia (None) sono neri.
- **Internal Property:** I figli di un nodo rosso sono neri.
- **Depth Property:** Tutti i nodi foglia (None) hanno la stessa *black-depth*, ovvero il numero di antenati neri.

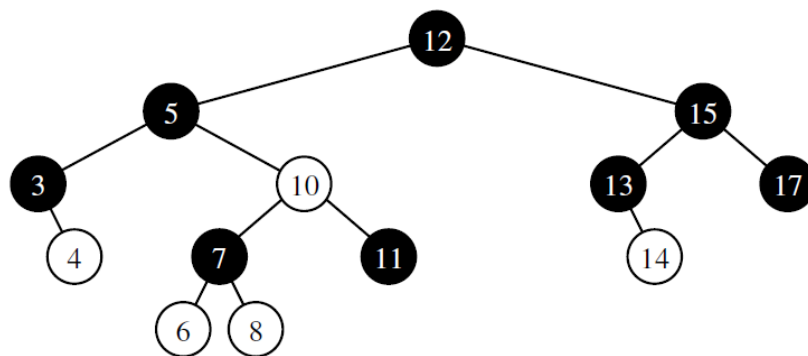


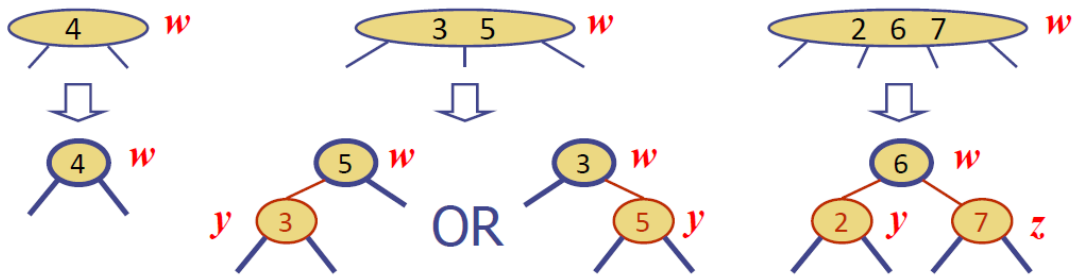
Figura 5.1: Esempio di Red-Black Tree, con i nodi rossi disegnati in bianco. La black-depth di ogni nodo foglia è 3. Da notare che non sono disegnati i nodi foglia (None), che sono tutti neri.

Come abbiamo detto, i Red-Black Trees sono strettamente correlati ai (2, 4) Trees. Infatti, ogni (2, 4) Tree può essere rappresentato come un Red-Black Tree e viceversa. Un Red-Black Tree può essere visto come una rappresentazione binaria di un (2, 4) Tree, dove i nodi rossi rappresentano i nodi con più di una chiave nel (2, 4) Tree. Proprio per questo motivo, i Red-Black Trees mantengono le stesse performance dei (2, 4)-Trees, con il beneficio aggiuntivo di una implementazione più semplice e di una maggiore efficienza nelle operazioni di inserimento e cancellazione, che richiedono al massimo una o due rotazioni per mantenere l'equilibrio dell'albero.

5.3 Dal (2, 4)-Tree al Red-Black Tree

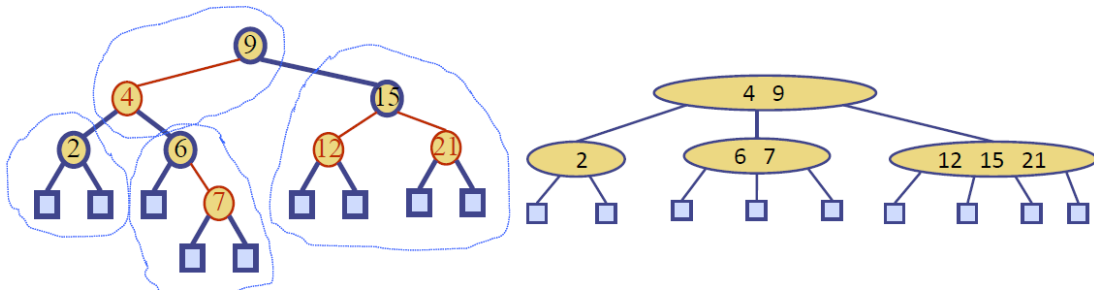
- Colora di nero tutti i nodi del (2, 4) Tree.
- Per ogni nodo w :
 - Se w è un 2-nodo, mantieni i figli (neri) di w così come sono.
 - Se w è un 3-nodo, crea un nuovo nodo rosso y , figlio destro (o sinistro) di w , e fai in modo che gli ultimi due (o i primi due) figli di w diventino figli di y , e il primo figlio (o l'ultimo) di w rimanga figlio di w .
 - Se w è un 4-nodo con chiavi k_1, k_2 e k_3 , rappresentalo come un nodo nero con due figli rossi contenenti le chiavi k_2 e k_3 .

Da notare che seguendo l'algoritmo sopra descritto, un nodo rosso avrà sempre un genitore nero.



5.4 Dal Red-Black Tree al (2, 4)-Tree

- Ogni nodo rosso w viene unito con il suo genitore nero p per formare un unico nodo del (2, 4) Tree.
 - L'elemento in w viene aggiunto alle chiavi in p .
 - I figli di w diventano figli di p .



5.5 Altezza di un Red-Black Tree

L'altezza di un Red-Black Tree con n nodi è $O(\log n)$.

Dimostrazione Sia T un Red-Black Tree con n nodi interni e altezza h , allora vale la seguente disuguaglianza:

$$\log(n+1) - 1 \leq h \leq 2 \log(n+1) - 2$$

Sia d la black-depth di T . Sia T' il (2, 4)-Tree associato a T , e sia h' l'altezza di T' . Per via della corrispondenza tra T e T' , vale la relazione $h' = d$. Quindi, si ha che $d = h' \leq \log(n+1) - 1$, da cui si ricava che $h \leq 2d \leq 2 \log(n+1)$. Sappiamo inoltre che vale la seguente proprietà: $h' \leq 2d$. Quindi, otteniamo $h \leq 2 \log(n+1) - 2$. L'altra disuguaglianza, $\log(n+1) - 1 \leq h$ deriva dalle proprietà di un qualsiasi albero binario.

5.6 Insert

Per inserire un nuovo nodo in un Red-Black Tree si procede come in un normale BST:

- Se il nuovo nodo z è la radice, coloralo di nero.
- Altrimenti, inseriscilo come un nodo rosso.

L'inserimento eseguito in questo modo mantiene di già le Root, External, e Depth Properties. Tuttavia, potrebbe violare la Internal Property in alcuni casi:

- Se il genitore di z è nero, anche la Internal Property è mantenuta.
- Se il genitore di z è rosso, la Internal Property viene violata in quanto si ottiene una sequenza di due nodi rossi, un **double red**. In questo caso, bisogna ristrutturare l'albero per ripristinare le proprietà dei Red-Black Trees.

5.6.1 Come risolvere un double red

Siano z e il suo genitore v entrambi rossi, e sia w il fratello di v (lo zio di z).

- Se w è nero (o None), il double-red corrisponde ad una trasformazione sbagliata di un 4-nodo nel (2, 4)-Tree corrispondente.
 - Si esegue una **ristrutturazione** (Three-node restructuring, singola o doppia rotazione) su z .
 - Dopo la ristrutturazione, il nodo che diventa la radice della porzione ristrutturata viene colorato di nero, mentre i suoi due figli vengono colorati di rosso.
 - Una sola ristrutturazione è sufficiente per risolvere il problema del double-red.
- Se w è rosso, il double-red corrisponde ad un overflow e quindi in un 5-nodo nel (2, 4)-Tree corrispondente.
 - Si esegue una **ricolorazione** (recoloring) colorando v e w di nero, mentre il loro genitore u (genitore di v e w , e nonno di z) viene colorato di rosso (se u non è la radice).
 - In questo caso il double-red può propagarsi verso l'alto, quindi potrebbe essere necessario ripetere la procedura sul nodo u e il suo genitore.

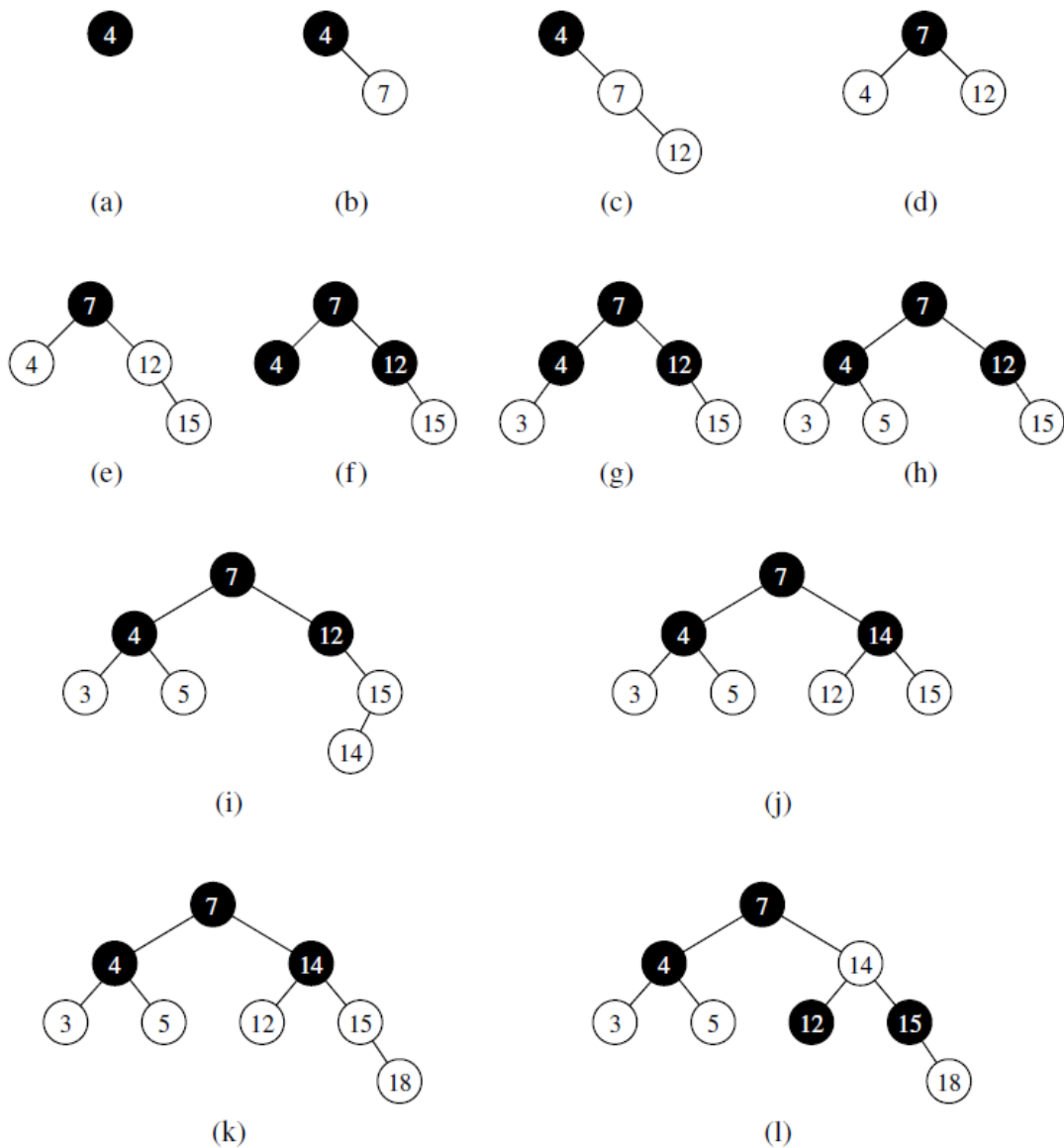


Figura 5.2: Una sequenza di inserimenti in un albero rosso-nero: (a) albero iniziale; (b) inserimento di 7; (c) inserimento di 12, che causa un double red; (d) dopo la ristrutturazione; (e) inserimento di 15, che causa un double red; (f) dopo la ricolorazione (la radice rimane nera); (g) inserimento di 3; (h) inserimento di 5; (i) inserimento di 14, che causa un double red; (j) dopo la ristrutturazione; (k) inserimento di 18, che causa un double red; (l) dopo la ricolorazione.

5.6.2 Complessità dell'inserimento

Come abbiamo visto, l'inserimento in un Red-Black Tree richiede una prima operazione di ricerca di $O(\log n)$ per trovare la posizione corretta del nuovo nodo, la creazione di un nuovo nodo in $O(1)$, e infine possono essere necessarie al massimo $O(\log n)$ ricolorazioni (ciascuna impiega $O(1)$) e al più una sola ristrutturazione ($O(1)$) per mantenere le proprietà dell'albero.

Pertanto, la complessità totale dell'inserimento in un Red-Black Tree è $O(\log n)$.

5.7 Delete

Per eliminare un nuovo nodo con chiave k in un Red-Black Tree, si segue lo stesso procedimento di un normale BST:

- Ciò vuol dire che eliminiamo sempre un nodo con al più un solo figlio. Il nodo eliminato contiene una chiave k o il suo predecessore/successore (in base all'implementazione) in ordine. Il nodo figlio di quello eliminato (se esiste) viene promosso a figlio del genitore del nodo eliminato.

5.7.1 Caso 1: Delete di un nodo rosso

Se il nodo eliminato è rosso, tutte le proprietà dei Red-Black Trees rimangono valide, poiché la rimozione di un nodo rosso non altera la black depth di alcun percorso dalla radice a una foglia, e poiché questa operazione non può introdurre un double red.

Nel corrispondente (2, 4)-Tree, la rimozione di un nodo rosso equivale alla rimozione di una chiave da un 3-nodo o da un 4-nodo, il che è sempre consentito senza ulteriori modifiche.

5.7.2 Caso 2: Delete di un nodo nero con un solo figlio (rosso)

Ricordiamo che stiamo trattando la rimozione di nodi con al più un figlio (per via della delete in un BST). Se il nodo da eliminare è nero e ha un figlio, questo sarà sicuramente rosso (altrimenti la black depth property non sarebbe soddisfatta e non si avrebbe un Red-Black Tree valido). In questo caso, possiamo semplicemente rimuovere il nodo nero e promuovere il figlio rosso al suo posto, colorandolo di nero, ristabilendo tutte le proprietà dei Red-Black Trees.

Nel corrispondente (2, 4)-Tree, questa operazione equivale alla rimozione del nodo nero da un 3-nodo.

Infine, consideriamo il caso più complesso, in cui il nodo da eliminare è un nodo nero senza figli.

5.7.3 Caso 3: Delete di un nodo nero senza figli

Il caso più complesso si verifica quando il nodo da eliminare è un nodo nero senza figli. Nel corrispondente (2, 4)-Tree, questa situazione equivale alla rimozione di una chiave da un 2-nodo. Senza un ribilanciamento, una modifica del genere comporta un deficit di uno per la black depth lungo il percorso che porta al nodo eliminato, violando così la Depth Property dei Red-Black Trees.

Per rimediare a questo scenario, consideriamo un contesto più generale con un nodo z che è noto per avere due sottoalberi, T_{heavy} e T_{light} , tale che la radice di T_{light} (se presente) è nera e tale che la black depth di T_{heavy} è esattamente uno in più rispetto a quella di T_{light} , come illustrato in Figura 5.3. Nel caso di una foglia nera rimossa, z è il genitore di quella foglia e T_{light} è banalmente il sottoalbero vuoto che rimane dopo la cancellazione. Descriviamo il caso più generale di un deficit perché il nostro algoritmo per il ribilanciamento dell'albero, in alcuni casi, spingerà il deficit più in alto nell'albero (proprio come la risoluzione di una cancellazione in un (2,4) tree a volte si propaga verso l'alto). Indichiamo con y la radice di T_{heavy} (Un tale nodo esiste perché T_{heavy} ha altezza nera almeno uno).

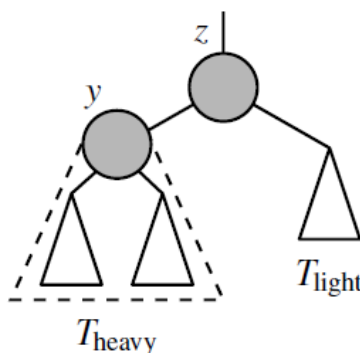


Figura 5.3: Illustrazione di un deficit tra le altezze nere dei sottoalberi del nodo z . Il colore grigio nell'illustrare y e z denota il fatto che questi nodi possono essere colorati sia di nero che di rosso.

Ricapitolando:

- z è un nodo con due sottoalberi T_{heavy} e T_{light} , tali che:
 - T_{heavy} ha altezza nera d .
 - T_{light} ha altezza nera $d - 1$.
- y è la radice di T_{heavy} , la quale esiste sempre.
- z e y possono essere sia rossi che neri.

Distinguiamo tre possibili casi:

- Nodo y nero con (almeno) un figlio rosso x .
- Nodo y nero e entrambi i figli di y sono neri (o None).
- Nodo y rosso.

Caso 3.1: Nodo y nero con (almeno) un figlio rosso x

Soluzione: Si esegue una *ristrutturazione* $restructure(x)$ sui tre nodi x , il suo genitore y , e il nonno z , rinominati temporaneamente come a , b , e c in ordine di chiave. Sostituiamo z con il nodo etichettato b , rendendolo il genitore degli altri due. Coloriamo a e c di nero, e diamo a b il colore precedente di z .

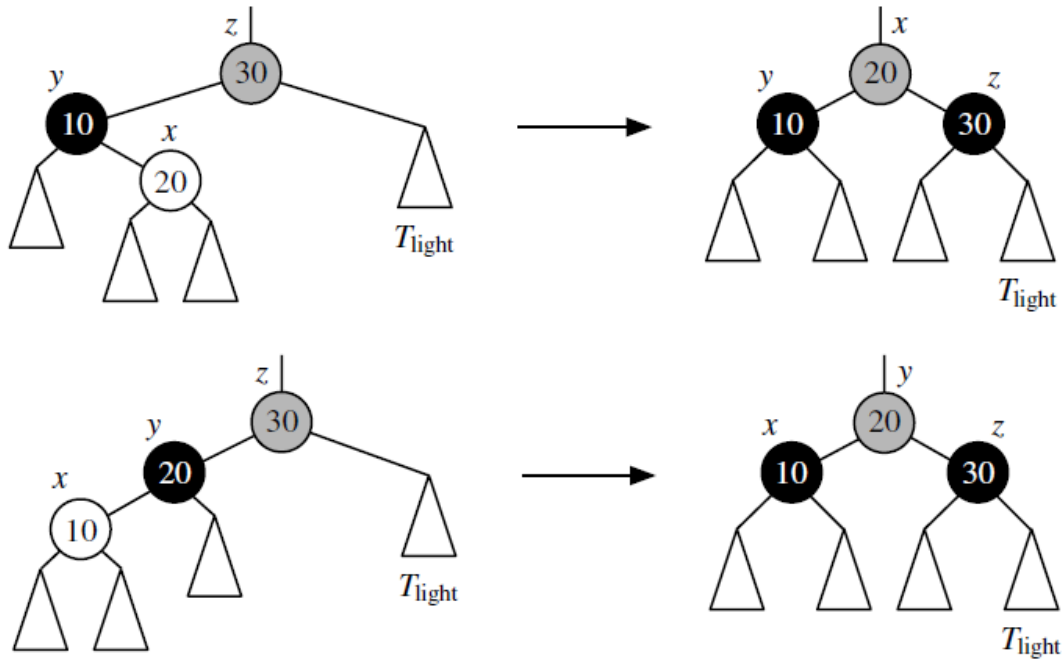


Figura 5.4: Risoluzione di un deficit nero in T_{light} attraverso una ristrutturazione su tre nodi $restructure(x)$. Sono mostrate due possibili configurazioni (le altre due sono simmetriche). Il colore grigio di z nelle figure a sinistra denota il fatto che questo nodo può essere colorato sia di rosso che di nero. La radice della porzione ristrutturata assume lo stesso colore, mentre i figli di quel nodo sono entrambi colorati di nero nel risultato.

Nel caso in cui y abbia entrambi i figli rossi, possiamo scegliere arbitrariamente uno dei due come x . Altrimenti, scegliamo l'unico figlio rosso di y come x . Da notare che il percorso verso T_{light} include un nodo nero aggiuntivo dopo la ristrutturazione, risolvendo così il suo deficit. Al contrario, il numero di nodi neri sui percorsi verso ciascuno degli altri tre sottoalberi illustrati in Figura 5.4 rimane invariato.

Risolvere questo caso corrisponde a un'operazione di *transfer* nell'albero $(2,4) T'$ tra i due figli del nodo con z . Il fatto che y abbia un figlio rosso ci assicura che rappresenta o un 3-nodo o un 4-nodo. In effetti, l'elemento precedentemente memorizzato in z viene declassato per diventare un nuovo 2-nodo per risolvere la carenza, mentre un elemento memorizzato in y o nel suo figlio viene promosso per prendere il posto dell'elemento precedentemente memorizzato in z .

Caso 3.2: Nodo y nero e entrambi i figli di y sono neri (o None)

Soluzione: Si esegue una *ricolorazione*, per cui coloriamo y di rosso e, se z è rosso, lo coloriamo di nero.

- Se z era originariamente rosso, questa ricolorazione risolve il deficit.
- Se z era originariamente nero, la ricolorazione non risolve il deficit, ma lo propaga più in alto nell'albero; dobbiamo ripetere la considerazione di tutti e tre i casi sul genitore di z come rimedio.

Questa ricolorazione non introduce alcun double-red, poiché y non ha figli rossi.

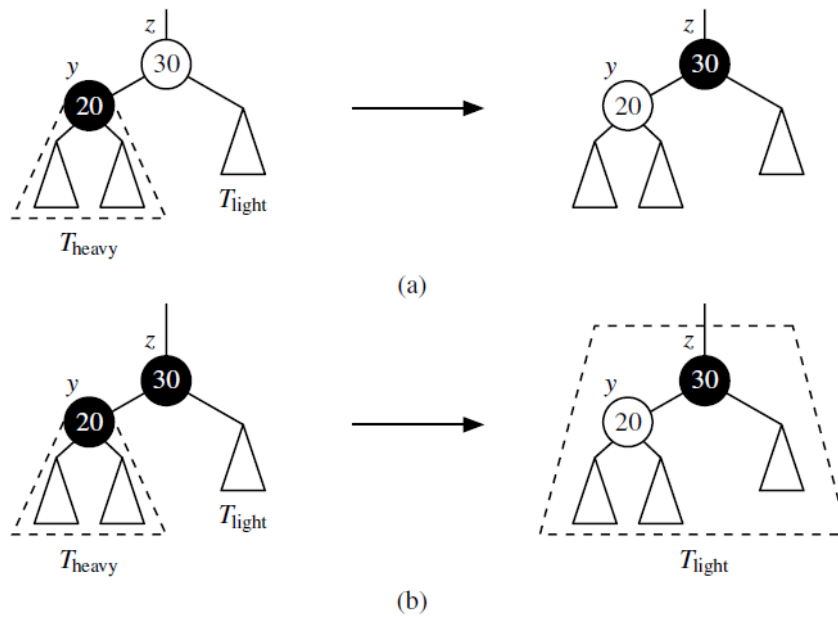


Figura 5.5: Risoluzione di un deficit nero in T_{light} tramite una ricolorazione. (a) quando z è originariamente rosso, si invertono i colori di y e z per risolvere il deficit nero in T_{light} , terminando il processo; (b) quando z è originariamente nero, la ricolorazione di y causa un deficit nero nell'intero sottoalbero di z , trasportando il problema ad un livello superiore che andrà risolto seguendo uno dei tre casi descritti.

La soluzione in questo caso corrisponde all'operazione di *fusion* nell'albero $(2, 4) T'$, poiché y deve rappresentare un 2-nodo. Nel caso in cui z era originariamente rosso, e quindi il genitore nel corrispondente albero $(2, 4)$ è un 3-nodo o un 4-nodo, questa ricolorazione risolve il deficit. (Vedi Figura 5.5a.) Il percorso che porta a T_{light} include un nodo nero aggiuntivo nel risultato, mentre la ricolorazione non ha influenzato il numero di nodi neri sul percorso verso i sottoalberi di T_{heavy} . Nel caso in cui z fosse originariamente nero, e quindi il genitore nel corrispondente albero $(2, 4)$ è un 2-nodo, la ricolorazione non ha aumentato il numero di nodi neri sul percorso verso T_{light} ; in effetti, ha ridotto il numero di nodi neri sul percorso verso T_{heavy} . (Vedi Figura 5.5b.) Dopo questo passaggio, i due figli di z avranno la stessa altezza nera. Tuttavia, l'intero albero radicato in z è diventato carente, propagando così il problema più in alto nell'albero; dobbiamo ripetere la considerazione di tutti e tre i casi sul genitore di z come rimedio.

Caso 3.3: Nodo y rosso

Soluzione: Si esegue una *rotazione* su y e z , seguita da una ricolorazione di y in nero e di z in rosso. Inoltre, poichè y era originariamente rosso, il nuovo sottoalbero di z deve avere una radice nera y' e deve avere un'altezza nera uguale a quella originale di T_{heavy} . Pertanto, un deficit nero rimane nel nodo z dopo la trasformazione, e quindi riappliciamo l'algoritmo per risolvere il deficit in z , sapendo che il nuovo figlio y' , che è la radice di T_{heavy} è ora nero, e quindi che si applica o il Caso 3.1 o il Caso 3.2. Inoltre, la prossima applicazione sarà l'ultima, perché il Caso 3.1 è sempre terminale e il Caso 3.2 sarà terminale dato che z è rosso.

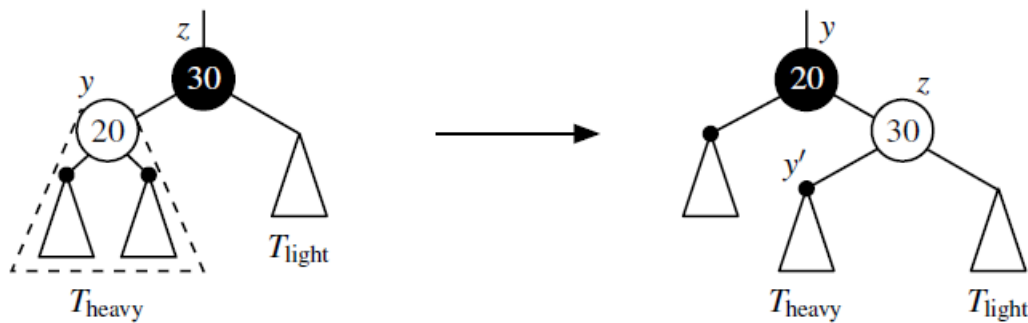


Figura 5.6: Una rotazione e una ricolorazione su un nodo rosso y e un nodo nero z , assumendo un deficit nero in z . Questo equivale a un cambiamento di orientamento nel corrispondente 3-nodo di un albero (2,4). Questa operazione non influisce sull'altezza nera di alcun percorso attraverso questa porzione dell'albero. Inoltre, poiché y era originariamente rosso, il nuovo sottoalbero di z deve avere una radice nera y' e deve avere un'altezza nera uguale a quella originale di T_{heavy} . Pertanto, un deficit nero rimane nel nodo z dopo la trasformazione.

Da notare che inizialmente y è rosso e T_{heavy} ha altezza nera almeno 1, z deve essere nero e i due sottoalberi di y devono avere ciascuno una radice nera e un'altezza nera uguale a quella di T_{heavy} .

Le prime operazioni di rotazione e ricolorazione denotano una riorientazione di un 3-nodo nel corrispondente albero (2,4) T' .

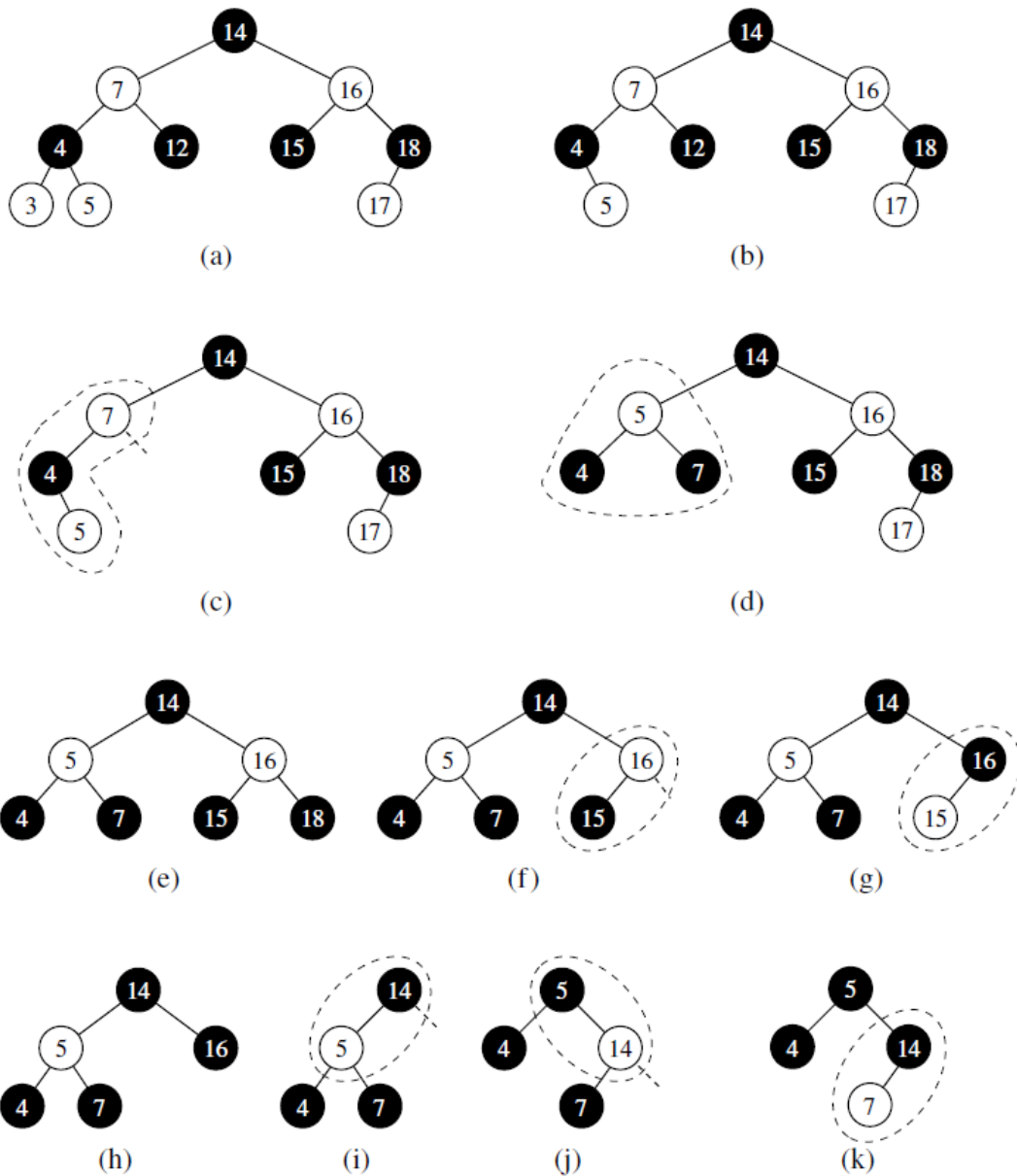


Figura 5.7: Una sequenza di cancellazioni da un Red-Black Tree: (a) albero iniziale; (b) rimozione di 3; (c) rimozione di 12, che causa un deficit nero a destra di 7 (risolto tramite ristrutturazione); (d) dopo la ristrutturazione; (e) rimozione di 17; (f) rimozione di 18, che causa un deficit nero a destra di 16 (risolto tramite ricolorazione); (g) dopo la ricolorazione; (h) rimozione di 15; (i) rimozione di 16, che causa un deficit nero a destra di 14 (risolto inizialmente tramite una rotazione); (j) dopo la rotazione il deficit nero deve essere risolto tramite una ricolorazione; (k) dopo la ricolorazione.

5.7.4 Complessità della cancellazione

L'algoritmo per eliminare un elemento da un Red-Black Tree con n elementi richiede $O(\log n)$ tempo e esegue $O(\log n)$ ricolorazioni e al massimo due operazioni di ristrutturazione.

Riepilogo Delete

Insert		Solving a double red	
RB-Tree	(2, 4)-Tree	result	
Restructuring	Change the representation of the 4-node	Double red solved	
Recoloring	Split	Double red solved or propagated	

Delete		Rebalancing the BD	
RB-Tree	(2, 4)-Tree	result	
Restructuring	transfer	BD balanced	
Recoloring	fusion	BD balanced or unbalancing propagated	
Rotation	Change the representation of the 3-node	Needs a recoloring or a restructuring	

Figura 5.8: Riepilogo dei casi di delete in un Red-Black Tree.

Capitolo 6

Hash Tables

Come visto finora, le possibili implementazioni dell'ADT *Map* sono molteplici, da una semplice lista ordinata o non ordinata, a strutture dati più complesse come tutte le tipologie di alberi presentate nei capitoli precedenti. Tuttavia, queste implementazioni hanno tutte in comune il fatto che le operazioni di ricerca, inserimento e cancellazione hanno una complessità temporale di almeno $O(\log n)$ nel caso migliore (per alberi bilanciati) e $O(n)$ nel caso peggiore (per semplici liste). Ciascuna implementazione ha i suoi pro e contro, e la scelta di quale utilizzare dipende da vari fattori, tra cui la frequenza delle operazioni di inserimento, cancellazione e ricerca, la necessità di ordinare le chiavi, ecc. In questo capitolo introduciamo una delle strutture dati più popolari per l'implementazione di una mappa, e quella utilizzata dalla stessa implementazione di Python per la classe `dict`. Questa struttura è nota come **hash table** (tabella hash).

Intuitivamente, una mappa M supporta l'astrazione di utilizzare le chiavi come indici con una sintassi del tipo $M[k]$. Consideriamo inizialmente un contesto ristretto in cui una mappa con n elementi utilizza chiavi che si presuppone essere interi in un intervallo da 0 a $N - 1$ per qualche $N \geq n$. In questo caso, possiamo rappresentare la mappa utilizzando una *lookup table* di lunghezza N , come illustrato in Figura 6.1.

0	1	2	3	4	5	6	7	8	9	10
	D		Z			C	Q			

Figura 6.1: Una lookup table di lunghezza $N = 11$ per una mappa contenente $n = 4$ elementi: $(1,D)$, $(3,Z)$, $(6,C)$, $(7,Q)$.

Ci sono due aspetti che vogliamo estendere rispetto a questa semplice rappresentazione per arrivare alla definizione di *hash table*:

- Come prima cosa, potremmo non voler utilizzare un array di lunghezza N nel caso in cui $N \gg n$.
- In secondo luogo, non è necessario che le chiavi di una mappa siano interi.

Una tabella hash, per un dato tipo di chiave, consiste in:

- Un array di bucket di lunghezza N .
- Una funzione di hash h .
- Un metodo per gestire le collisioni all'interno di ciascun bucket.

6.1 Hash Functions

Una Hash Table utilizza una **funzione di hash** per mappare una chiave generica in un indice della tabella. Idealmente, le chiavi saranno ben distribuite nell'intervallo da 0 a $N - 1$ dalla funzione di hash, ma in pratica potrebbero esserci due o più chiavi distinte che vengono mappate allo stesso indice. Di conseguenza, concettualizzeremo la nostra tabella come un array di bucket, come mostrato in Figura 6.2, in cui ogni bucket può gestire una collezione di elementi che vengono inviati a uno specifico indice dalla funzione di hash.

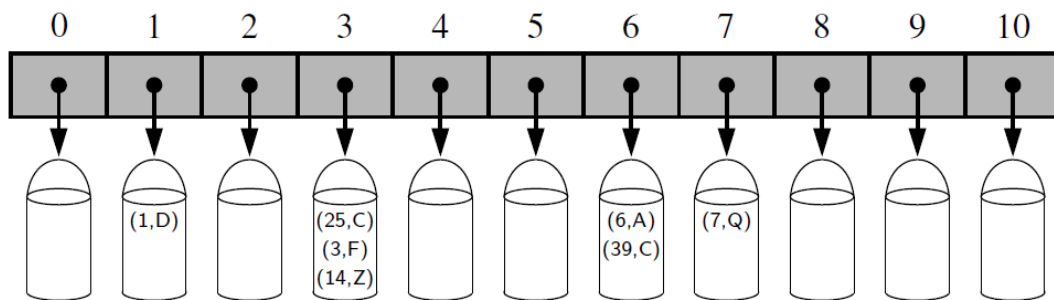


Figura 6.2: Un bucket array di lunghezza $N = 11$. Ogni bucket può contenere zero, uno, o più elementi.

Formalmente, una funzione di hash è una funzione $h : K \rightarrow \{0, 1, \dots, N - 1\}$ che mappa ogni chiave $k \in K$ in un indice della tabella. La scelta di una buona funzione di hash è cruciale per le prestazioni della hash table, poiché una cattiva distribuzione delle chiavi può portare a un numero elevato di collisioni, ovvero situazioni in cui più chiavi vengono mappate allo stesso indice, quindi allo stesso bucket.

L'idea principale è quella di usare la funzione di hash h per determinare il valore $h(k)$, che useremo come indice nel bucket array, A , al posto del valore di chiave k stesso (come abbiamo detto k non è necessariamente un intero, per cui potrebbe essere poco adatto come indice). Quindi, memorizziamo l'elemento formato dalla coppia chiave-valore (k, v) nel bucket $A[h(k)]$.

Se ci sono due o più chiavi con lo stesso valore di hash, allora due elementi diversi verranno mappati allo stesso bucket in A . In questo caso, diciamo che si è verificata una **collisione**. Ci sono modi per gestire le collisioni, di cui discuteremo più avanti, ma la strategia migliore è cercare di evitarle in primo luogo. Diciamo che una funzione di hash è "buona" se mappa le chiavi nella nostra mappa in modo da minimizzare sufficientemente le collisioni.

È comune pensare ad una funzione di hash come la composizione di due diverse funzioni:

- Hash code: una funzione $h_1 : K \rightarrow \mathbb{Z}$ che mappa una chiave k in un intero (positivo o negativo).
- Compression function: una funzione $h_2 : \mathbb{Z} \rightarrow \{0, 1, \dots, N - 1\}$ che mappa l'intero risultante in un indice della tabella hash.

$$h(x) = h_2(h_1(x))$$

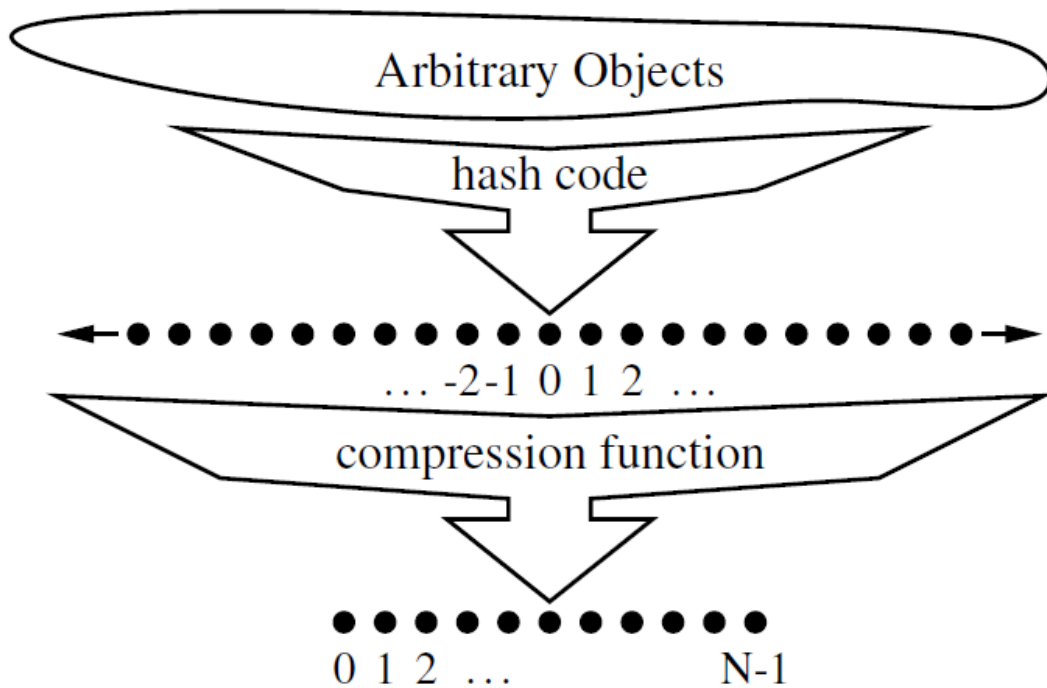


Figura 6.3: Le due parti di una funzione hash: hash code e funzione di compressione.

La separazione di queste due funzioni consente l'utilizzo di hash codes che sono validi per uno specifico insieme di chiavi, indipendentemente dalla dimensione della tabella hash (la cui dimensione può variare dinamicamente nel tempo).

6.1.1 Esempi di Hash Code

Come abbiamo detto la funzione di hash code mappa una generica chiave in un intero. Una *collisione* si verifica quando due chiavi distinte producono lo stesso hash code, per cui la funzione di compressione non può fare nulla per evitarla e memorizzerà entrambe le chiavi nello stesso bucket. Di conseguenza, è importante scegliere una buona funzione di hash code che minimizzi le collisioni per il tipo di chiavi che ci aspettiamo di utilizzare nella nostra mappa.

Presentiamo alcuni tra i metodi più comuni per il calcolo dell'hash code:

- **Memory address:** Si utilizza l'indirizzo di memoria (un intero) della chiave come hash code.
- **Rappresentazione in bit:** Si interpreta la rappresentazione in bit della chiave come un intero.
- **Composition:** Si partizionano i bit della chiave in gruppi di dimensione fissa, si sommano ignorando l'overflow (XOR).
- **Polynomial accumulation:** A differenza dei metodi precedenti che funzionano bene per chiavi numeriche, questo metodo è adatto per chiavi di tipo stringa, o più in generale per oggetti di lunghezza variabile in cui l'ordine degli elementi è importante. Si partizionano i bit della chiave in gruppi di dimensione fissa $(a_0, a_1, \dots, a_{k-1})$ e si calcola il polinomio $p(z) = a_0 + a_1z + a_2z^2 + \dots + a_{k-1}z^{k-1}$ per un opportuno valore di z . L'hash code è quindi il valore di $p(z)$ calcolato in un intero di dimensione fissa, ignorando l'overflow.

6.1.2 Esempi di Funzione di Compressione

La funzione di compressione mappa un intero (l'hash code) in un indice della tabella hash, ovvero in un intero nell'intervallo $[0, N - 1]$, dove N è la dimensione della tabella hash. Una buona funzione di compressione deve riuscire a minimizzare il numero di collisioni a partire da hash codes differenti (poiché se due hash codes sono uguali, non c'è modo di evitare la collisione).

- **Modulo:**
 - $h_2(y) = y \bmod N$
 - Questa funzione può portare a molte collisioni se N ha fattori comuni con gli hash codes, per cui N è tipicamente un numero primo.
- **MAD (Multiply-Add-Divide):**
 - $h_2(y) = [(ay + b) \bmod p] \bmod N$
 - $p > N$ è un numero primo, e a e b sono interi scelti casualmente in $[0, p - 1]$ con $a > 0$.
 - La probabilità che due hash codes distinti y_1 e y_2 causino una collisione è al più $1/N$.

6.2 Gestione delle Collisioni

L'esistenza delle collisioni ci impedisce di memorizzare semplicemente un elemento (k, v) nel bucket $A[h(k)]$. Inoltre complica in modo significativo le operazioni di ricerca, inserimento e cancellazione. In questa sezione presentiamo due approcci comuni per gestire le collisioni: *separate chaining* e *open addressing*.

6.2.1 Separate Chaining

Una soluzione semplice ed efficiente per gestire le collisioni è quella di far sì che ogni bucket $A[j]$ memorizzi un riferimento ad una struttura dati secondaria, che contiene gli elementi (k, v) tali che $h(k) = j$. Una scelta naturale per la struttura dati secondaria è una semplice linked list. Questa regola di risoluzione delle collisioni è nota come **separate chaining**, ed è illustrata in Figura 6.4.

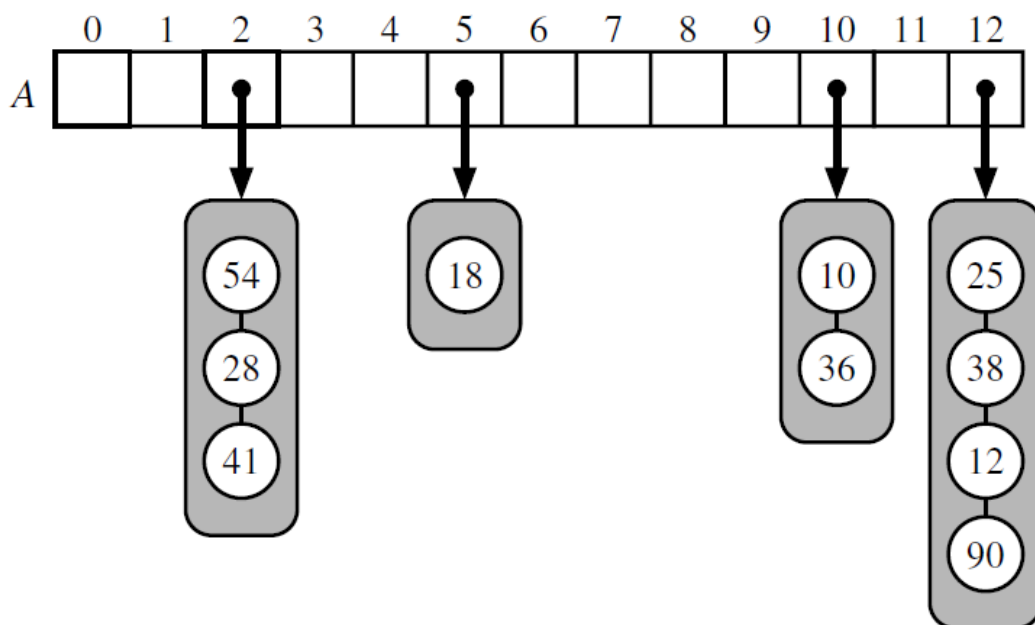


Figura 6.4: Hash Table di dimensione 13, che memorizza 10 elementi con chiavi intere, con collisioni risolte tramite separate chaining. La funzione di compressione è $h(k) = k \bmod 13$. Per semplicità, non mostriamo i valori associati alle chiavi.

Nel caso peggiore, le operazioni su di uno specifico bucket impiegano tempo proporzionale alla dimensione del bucket stesso. Assumendo di utilizzare una buona funzione di hash per indicizzare gli n elementi della nostra mappa in un bucket array di capacità N , la dimensione attesa di ciascun bucket è n/N . Dunque, con una buona funzione di hash, le operazioni principali sulla mappa richiedono un tempo atteso di $O(n/N)$. Il rapporto $\lambda = n/N$, chiamato **load factor** (fattore di carico) della tabella hash, dovrebbe essere limitato da una piccola costante, preferibilmente inferiore a 1. Finché λ è $O(1)$, le operazioni principali sulla tabella hash vengono eseguite in tempo atteso $O(1)$.

6.2.2 Open Addressing

La tecnica di *separate chaining* è efficiente e facile da implementare, ma richiede inevitabilmente spazio aggiuntivo per le strutture dati ausiliarie. Se si vuole evitare di utilizzare spazio aggiuntivo per gestire le collisioni, l'unico approccio possibile è quello di cercare uno slot alternativo, sempre all'interno del nostro array, in cui memorizzare l'elemento che causa la collisione.

Ci sono diverse varianti che utilizzano questo approccio, noti come tecniche di **open addressing**. Queste tecniche richiedono che il load factor $\lambda = n/N$ sia sempre inferiore a 1, ovvero che $n < N$, poiché ogni elemento deve essere memorizzato in uno slot dell'array.

Linear Probing

Un metodo semplice per gestire le collisioni con l'open addressing è il *linear probing*. Con questo approccio, se si tenta di inserire un elemento (k, v) in un bucket già occupato, si cerca il successivo bucket libero nell'array, procedendo in modo circolare, "sondando" linearmente uno slot per volta. Da qui il nome di *linear probing* (probe = sondare).

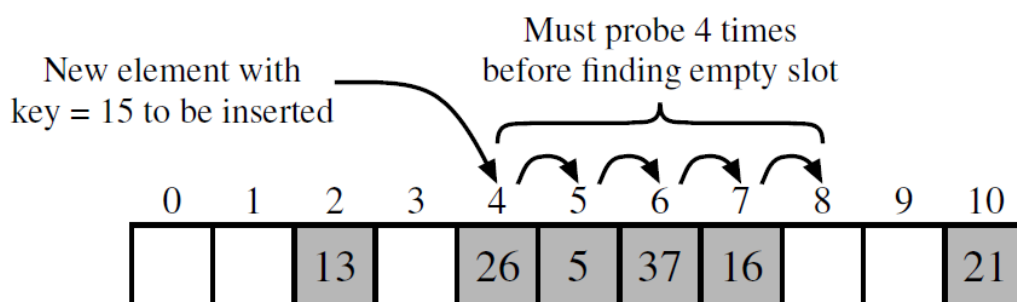


Figura 6.5: Inserimento di elementi con chiavi intere utilizzando linear probing per gestire le collisioni. La funzione di compressione è $h(k) = k \bmod 11$. Per semplicità, non mostriamo i valori associati alle chiavi.

Un problema può sorgere quando, facendo riferimento alla Figura 6.5, si elimina l'elemento con chiave 37 e successivamente si tenta di ricercare l'elemento con chiave 15. Poiché l'elemento con chiave 15 è stato memorizzato, a seguito di una collisione, dopo l'elemento 37, la ricerca di 15 fallirà se ci si ferma al primo slot vuoto incontrato (quello lasciato libero da 37).

Per risolvere questo problema, quando si elimina un elemento in una tabella hash che utilizza linear probing, si può marcare lo slot come "AVAILABLE" invece di lasciarlo vuoto. In questo modo, durante la ricerca, si continuerà a sondare gli slot successivi anche se si incontra uno slot marcato, garantendo così che tutti gli elementi possano essere trovati correttamente, mentre durante l'inserimento, gli slot marcati possono essere riutilizzati per nuovi elementi. Di fatto, uno slot marcato come "AVAILABLE" differenzia uno slot precedentemente occupato da uno slot mai usato prima, e viene trattato come occupato durante la ricerca, ma come vuoto durante l'inserimento.

Il problema del Clustering causato da Linear Probing

Un problema noto con il linear probing è il fenomeno del **clustering lineare**. Quando si verifica una collisione e si utilizza il linear probing per trovare uno slot libero, gli elementi tendono a raggrupparsi in cluster contigui all'interno dell'array. Questi cluster possono crescere nel tempo, aumentando la probabilità di ulteriori collisioni e rallentando le operazioni di ricerca, inserimento e cancellazione. Questo perché, quando si cerca uno slot libero, si potrebbe dover sondare attraverso un intero cluster, aumentando il tempo necessario per completare l'operazione, specialmente quando il load factor λ inizia ad essere maggiore di $1/2$.

Esistono altre tecniche di open addressing che cercano di mitigare il problema del clustering lineare:

- **Quadratic Probing:** Evita il clustering lineare utilizzando una funzione di probing quadratica per trovare lo slot successivo, ma crea cluster di forma più complessa (clustering secondario). Funziona bene solo se il load factor λ è mantenuto al di sotto di $1/2$.
- **Double Hashing:** Si utilizza una seconda funzione di hash per calcolare l'offset in caso di collisione. Questo metodo riduce significativamente il clustering e offre prestazioni migliori rispetto al linear e quadratic probing, specialmente a load factor più elevati.

Double Hashing

La tecnica del *double hashing* utilizza una seconda funzione di hash $d(k)$ per ricalcolare l'offset in caso di collisione. Invece di sondare linearmente o quadraticamente, si calcola un offset basato sulla chiave stessa, il che aiuta a distribuire gli elementi in modo più uniforme nell'array e riduce il clustering.

La procedura di inserimento con double hashing funziona come segue:

- $(h(k) + j \cdot d(k)) \bmod N$, per $j = 0, 1, \dots, N - 1$
- Si tenta di inserire l'elemento nel bucket calcolato. Se il bucket è occupato, si incrementa j e si ricalcola l'indice fino a trovare uno slot libero.
- Se si esauriscono tutti gli N tentativi senza trovare uno slot libero, la tabella è piena e l'inserimento fallisce.
- La funzione di offset $d(k)$ non può restituire zero.
- N dovrebbe essere un numero primo per garantire che tutti gli slot vengano sondati.
- Una scelta comune per $d(k)$ è $d(k) = q - (k \bmod q)$, dove q è un numero primo più piccolo di N .

k	$h(k)$	$d(k)$	Probes	
18	5	3	5	
41	2	1	2	
22	9	6	9	
44	5	5	5	10
59	7	4	7	
32	6	3	6	
31	5	4	5	9 0
73	8	4	8	

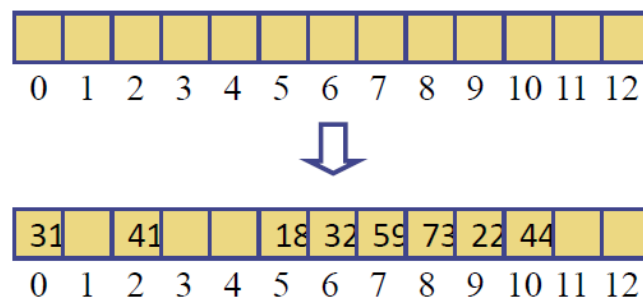


Figura 6.6: Inserimento di elementi con chiavi intere: 18, 41, 22, 59, 32, 31, 73. $N = 13$, $h(k) = k \bmod 13$, $d(k) = 7 - (k \bmod 7)$. Per semplicità, non mostriamo i valori associati alle chiavi.

6.3 Load Factor

Abbiamo visto come il **load factor** (fattore di carico) di una tabella hash è definito come il rapporto tra il numero di elementi memorizzati nella tabella n e la capacità totale della tabella N , $\lambda = n/N$. Questo parametro condiziona fortemente le performance di una tabella hash. Secondo la letteratura, per mantenere prestazioni ottimali, il load factor dovrebbe essere mantenuto sempre al di sotto di una certa soglia, in base al tipo di gestione delle collisioni utilizzata:

- *separate chaining*: $\lambda \leq 0.9$.
- *open addressing*: $\lambda \leq 0.5$ (per linear probing e quadratic probing), $\lambda \leq 0.7$ (per double hashing).

Quando il numero di elementi inseriti fa sì che il load factor λ superi le soglie critiche sopra indicate, la probabilità di collisioni aumenta drasticamente. Questo comporta una degradazione della complessità temporale media delle operazioni di ricerca e inserimento, che tendono a spostarsi da $O(1)$ verso il caso pessimo $O(n)$.

Per ripristinare l'efficienza operativa, è necessario eseguire una procedura di ridimensionamento dinamico nota come **Rehashing**.

Il processo di Rehashing

La procedura operativa segue questi passi:

1. **Allocazione**: Viene creata una nuova tabella di dimensione N' , dove solitamente $N' \approx 2N$. È preferibile che N' sia un numero primo per ottimizzare la distribuzione.
2. **Rimappatura**: Si scorre l'intera tabella originale. Per ogni elemento, viene ricalcolato l'hash in funzione della nuova capacità N' .
3. **Inserimento**: Gli elementi vengono inseriti nelle nuove posizioni calcolate.
4. **Deallocazione**: La vecchia memoria viene liberata.

Sebbene il rehashing abbia un costo immediato di $O(n)$, l'analisi ammortizzata dimostra che, raddoppiando la capacità ad ogni espansione, il costo medio per operazione di inserimento rimane $O(1)$.

6.4 Analisi delle prestazioni

- **Worst case**: Il caso peggiore si ha quando tutte le chiavi collidono, per cui si ha un tempo $O(n)$ per tutte le operazioni principali (ricerca, inserimento, cancellazione). Se i valori di hash sono distribuiti uniformemente, il numero di probe necessarie per trovare uno slot libero è in media $1/(1 - \lambda)$.
- **Average case**: Il tempo medio per tutte le operazioni principali è $O(1)$. In pratica, con una buona funzione di hash, una hash table è estremamente efficiente se il load factor è molto minore di 1.

Capitolo 7

Priority Queues

Una **coda con priorità** (*Priority Queue*) è una struttura dati astratta fondamentale, simile a una coda standard (*Queue*), ma con una differenza cruciale nel criterio di estrazione. Mentre una coda standard opera secondo la logica **FIFO** (First-In, First-Out), rimuovendo l'elemento che è in attesa da più tempo, una coda a priorità rimuove sempre l'elemento con la **priorità** più alta (o più bassa, a seconda di quale logica si vuole utilizzare), indipendentemente dall'ordine di inserimento.

È importante non confondere una coda a priorità con una mappa (Map) o dizionario. Lo scopo di una **Map** è l'associazione e la ricerca rapida: memorizza coppie $\langle \text{Chiave}, \text{Valore} \rangle$ e risponde efficientemente alla domanda: "Qual è il valore associato a questa specifica chiave?". Al contrario, lo scopo di una **Priority Queue** è l'estrazione efficiente dell'elemento più importante. Sebbene gli elementi in una coda a priorità siano spesso implementati come coppie, ad esempio $\langle \text{Priorità}, \text{Dato} \rangle$ (a cui spesso ci si riferisce comunque come (chiave, valore)), questa coppia ha una funzione diversa: la *Priorità* non serve per cercare il *Dato* (come farebbe una chiave in una mappa), ma serve solo alla struttura interna della coda (spesso uno *Heap*) per determinare l'ordine di estrazione. In sintesi, si usa una mappa per *trovare* un elemento tramite un identificatore unico (la chiave), mentre si usa una coda a priorità per *estrarre* l'elemento con il grado di urgenza massimo o minimo.

Definizione: Una **Priority Queue** è una raccolta di elementi con priorità che consente l'inserimento arbitrario di elementi e la rimozione dell'elemento con priorità primaria. Quando un elemento viene aggiunto in una Priority Queue questo assume una determinata priorità rappresentata dalla chiave ad esso associata. L'elemento con la chiave minima sarà il successivo a essere rimosso dalla coda (quindi, a un elemento con chiave 1 verrà data priorità su un elemento con chiave 2).

7.1 Chiavi (Priorità)

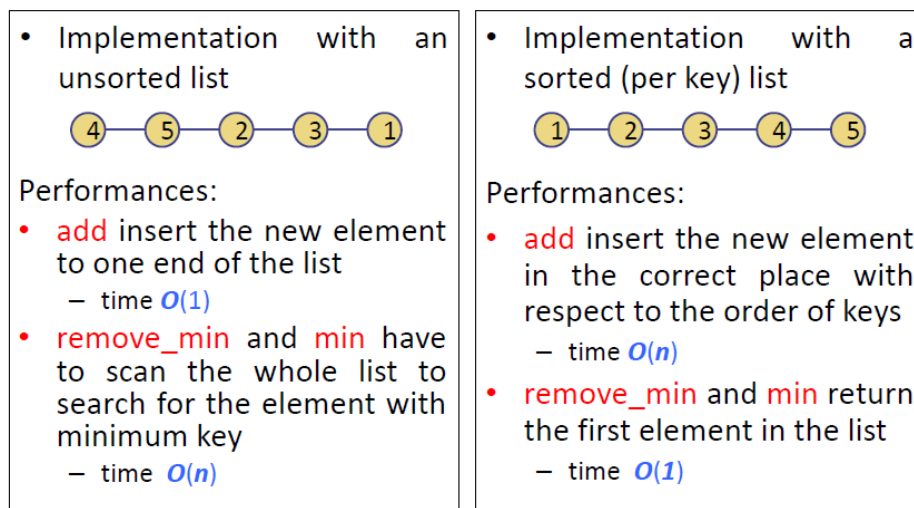
In una Priority Queue, ogni elemento è associato a una chiave che determina la sua priorità. Le chiavi in una Priority Queue possono essere oggetti di qualunque tipo tale che sia possibile

definire un ordinamento totale su di esse. Con tale generalità, le applicazioni possono sviluppare la propria nozione di priorità per ciascun elemento.

A marcare la differenza con la struttura dati Map, in una Priority Queue **le chiavi non sono necessariamente uniche**. È possibile avere più elementi con la stessa chiave (priorità). In questi casi, la politica di estrazione per gli elementi con chiavi identiche può variare a seconda dell'implementazione specifica della Priority Queue. Alcune implementazioni potrebbero adottare una politica FIFO per gli elementi con la stessa priorità, mentre altre potrebbero non garantire alcun ordine specifico tra di essi.

7.2 Possibili implementazioni di Priority Queue

Esistono diverse strategie per implementare una Priority Queue. Due approcci semplici utilizzano liste (array o liste collegate) per memorizzare gli elementi, con differenze significative nelle prestazioni delle operazioni di inserimento e rimozione.



Le due strategie per implementare una Priority Queue ADT dimostrano un interessante compromesso. Quando si utilizza una lista non ordinata per memorizzare gli elementi, possiamo eseguire inserimenti in tempo $O(1)$, ma trovare o rimuovere un elemento con chiave minima richiede un ciclo in tempo $O(n)$ attraverso l'intera collezione. Al contrario, se si utilizza una lista ordinata, possiamo banalmente trovare o rimuovere l'elemento minimo in tempo $O(1)$, ma aggiungere un nuovo elemento alla coda potrebbe richiedere tempo $O(n)$ per ripristinare l'ordinamento.

Esistono però implementazioni più efficienti, seppur più complesse, che consentono di eseguire sia inserimenti che rimozioni in tempo logaritmico. Una di queste implementazioni si basa su una struttura dati chiamata **heap binario**, la quale utilizza la struttura di un albero binario per trovare un compromesso tra elementi completamente non ordinati e perfettamente ordinati.

7.3 Heap

Definizione: Un **Heap** è un albero binario che soddisfa le seguenti proprietà:

- **Albero binario completo:** L'albero è un albero binario completo (vedi pag.6), cioè tutti i livelli dell'albero sono completamente riempiti, tranne eventualmente l'ultimo livello, che è riempito da sinistra a destra.
 - Sia h l'altezza dell'albero, per $i = 0, 1, \dots, h - 1$, il livello i -esimo contiene esattamente 2^i nodi. Se il livello $h - 1$ (cioè nel livello più basso) non è pieno, tutti i suoi nodi sono riempiti da sinistra a destra.
- **Heap-Order:** Per ogni posizione (nodo) p , la chiave di p è minore o uguale alle chiavi dei suoi figli.
 - Questo implica che la chiave minima si trova sempre nella radice dell'albero.

Dal momento che un Heap è un albero binario completo, la sua altezza è sempre logaritmica rispetto al numero di nodi nell'albero. Questo fatto è cruciale per garantire che le operazioni di inserimento e rimozione possano essere eseguite in tempo $O(\log n)$. Si ha quindi:

$$h = \lfloor \log_2(n) \rfloor$$

7.4 Implementazione di una Priority Queue attraverso Heap

Un Heap può essere efficacemente utilizzato per implementare una Priority Queue, sfruttando le sue proprietà di albero binario completo e heap-order. Le operazioni principali di una Priority Queue, ovvero l'inserimento di un elemento e la rimozione dell'elemento con priorità minima, possono essere eseguite in tempo logaritmico grazie alla struttura dell'Heap.

- In ogni nodo dello Heap viene memorizzata una coppia chiave-valore, dove la chiave rappresenta la priorità dell'elemento.
 - N.B.: è possibile avere più nodi con la stessa chiave, in quanto si tratta di un'implementazione di una Priority Queue.
- Manteniamo un riferimento all'ultimo nodo dello Heap, cioè il nodo più a destra sull'ultimo livello dell'albero.

Inserimento $\text{add}(k, v)$

Si vuole inserire una nuova coppia chiave-valore (k, v) nell'Heap. Per mantenere la proprietà di albero binario completo, dobbiamo inserire il nuovo nodo nella posizione corretta, ovvero appena oltre il nodo più a destra al livello più basso dell'albero, o come posizione più a sinistra di un nuovo livello, se il livello più basso è già pieno (o se l'heap è vuoto).

La posizione del nodo nel quale inserire il nuovo elemento può essere trovato in un tempo $O(\log n)$ a partire dal riferimento all'ultimo nodo:

- Si parte dal riferimento all'ultimo nodo.
- Si risale l'albero fino alla radice o ad un nodo che è figlio sinistro del suo genitore.
- Se il nodo in cui si è giunti è il figlio sinistro del suo genitore, vai al nodo fratello (cioè il figlio destro del genitore).
- Infine, scendi sempre a sinistra fino a raggiungere un nodo foglia (None). Questa sarà la posizione in cui inserire il nuovo nodo.

Up-Heap Bubbling dopo l'inserimento

Dopo aver inserito il nuovo nodo nella posizione corretta al fine di mantenere la proprietà di albero binario completo, è possibile che la proprietà di heap-order venga violata, poiché la chiave del nuovo nodo potrebbe essere minore della chiave del suo genitore. Per ripristinare la proprietà di heap-order, si esegue un processo chiamato **up-heap bubbling**.

- Partendo dal nuovo nodo con chiave k , e risalendo l'albero verso la radice, si scambia il nodo corrente con il suo genitore fino a quando la heap-order property non è ristabilita.
- L'algoritmo termina quando la chiave k raggiunge la radice dell'albero o quando la chiave k si trova in un nodo il cui padre ha una chiave minore o uguale a k .

Dal momento che l'altezza dell'Heap è $O(\log n)$, l'operazione di up-heap bubbling richiede un tempo $O(\log n)$ nel caso peggiore.

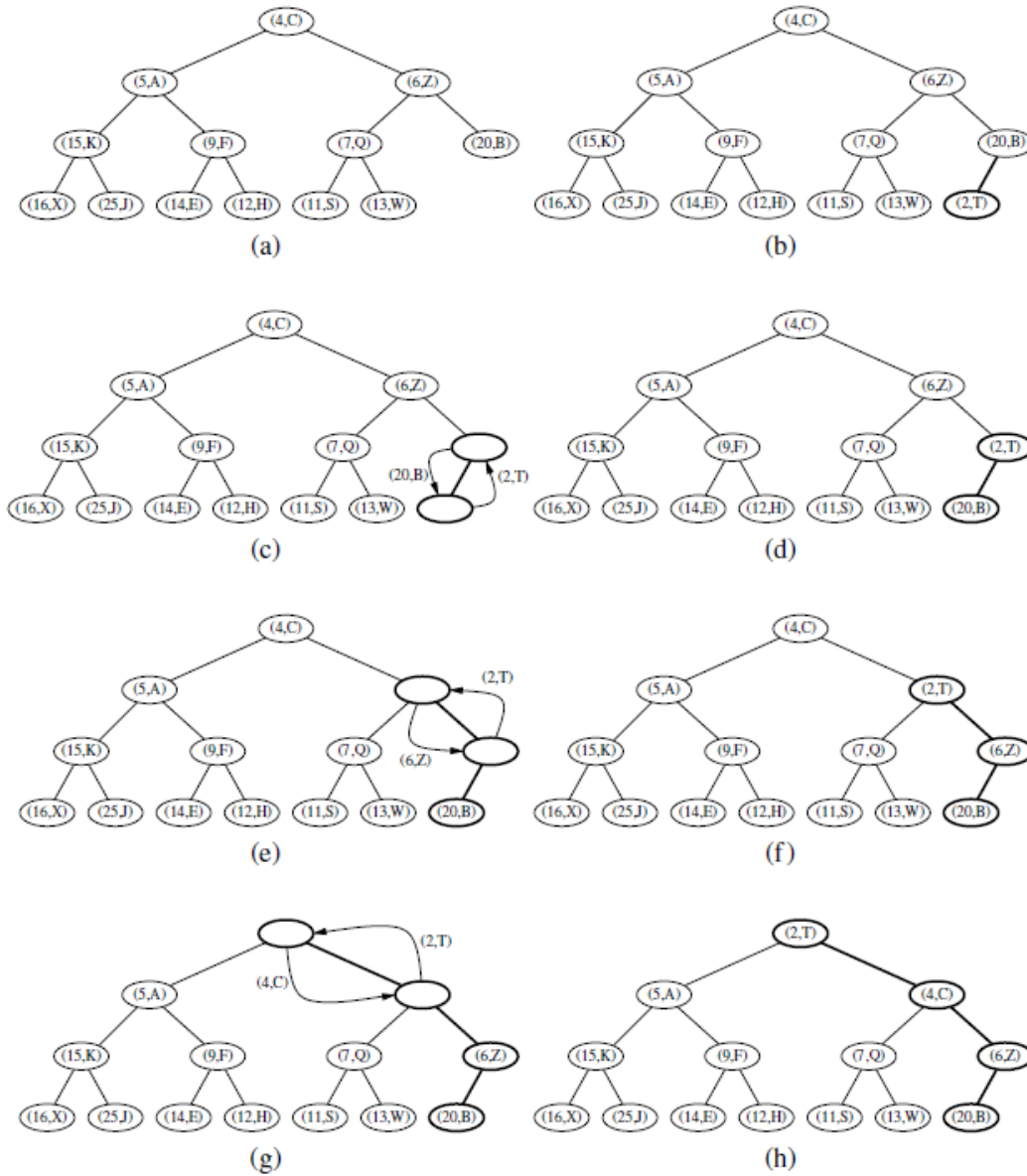


Figure 9.2: Insertion of a new entry with key 2 into the heap of Figure 9.1: (a) initial heap; (b) after performing operation add; (c and d) swap to locally restore the partial order property; (e and f) another swap; (g and h) final swap.

Cancellazione `remove_min()`

Il metodo `remove_min()` rimuove e restituisce l'elemento con la chiave minima dalla Priority Queue. In un Heap, l'elemento con la chiave minima si trova sempre nella radice dell'albero. Per mantenere la proprietà di albero binario completo, dobbiamo sostituire la radice con l'ultimo nodo dell'Heap (cioè il nodo più a destra nell'ultimo livello dell'albero) e poi rimuovere l'ultimo nodo.

Dopo aver rimosso il nodo, è fondamentale aggiornare il riferimento all'ultimo nodo dell'Heap affinché punti al predecessore del nodo appena eliminato (secondo l'ordine di livello). La posizione del nuovo ultimo nodo può essere trovata in un tempo $O(\log n)$ a partire dal riferimento all'ultimo nodo rimosso:

- Si parte dalla posizione del nodo che è stato appena rimosso (il vecchio ultimo nodo).
- Si risale l'albero fino alla radice o ad un nodo che è figlio destro del suo genitore.
- Se il nodo in cui si è giunti è il figlio destro del suo genitore, vai al nodo fratello (cioè il figlio sinistro del genitore).
- Infine, scendi sempre a destra fino a raggiungere un nodo foglia. Questo sarà il nuovo riferimento all'ultimo nodo.

Down-Heap Bubbling dopo la cancellazione

Dopo aver effettuato la sostituzione della radice con l'ultimo nodo, è possibile che la proprietà di heap-order venga violata, poiché la chiave del nuovo nodo radice potrebbe essere maggiore della chiave di uno o entrambi i suoi figli. Per ripristinare la proprietà di heap-order, si esegue un processo chiamato **down-heap bubbling**.

- Partendo dalla radice, si scambia il nodo corrente con il figlio che ha la chiave minima, a condizione che la chiave del figlio sia minore della chiave del nodo corrente.
- Questo processo continua fino a quando la chiave del nodo corrente è minore o uguale alle chiavi dei suoi figli, o fino a quando il nodo corrente diventa un nodo foglia (None), per cui la heap-order property è ristabilita.

Dal momento che l'altezza dell'Heap è $O(\log n)$, l'operazione di down-heap bubbling richiede un tempo $O(\log n)$ nel caso peggiore.

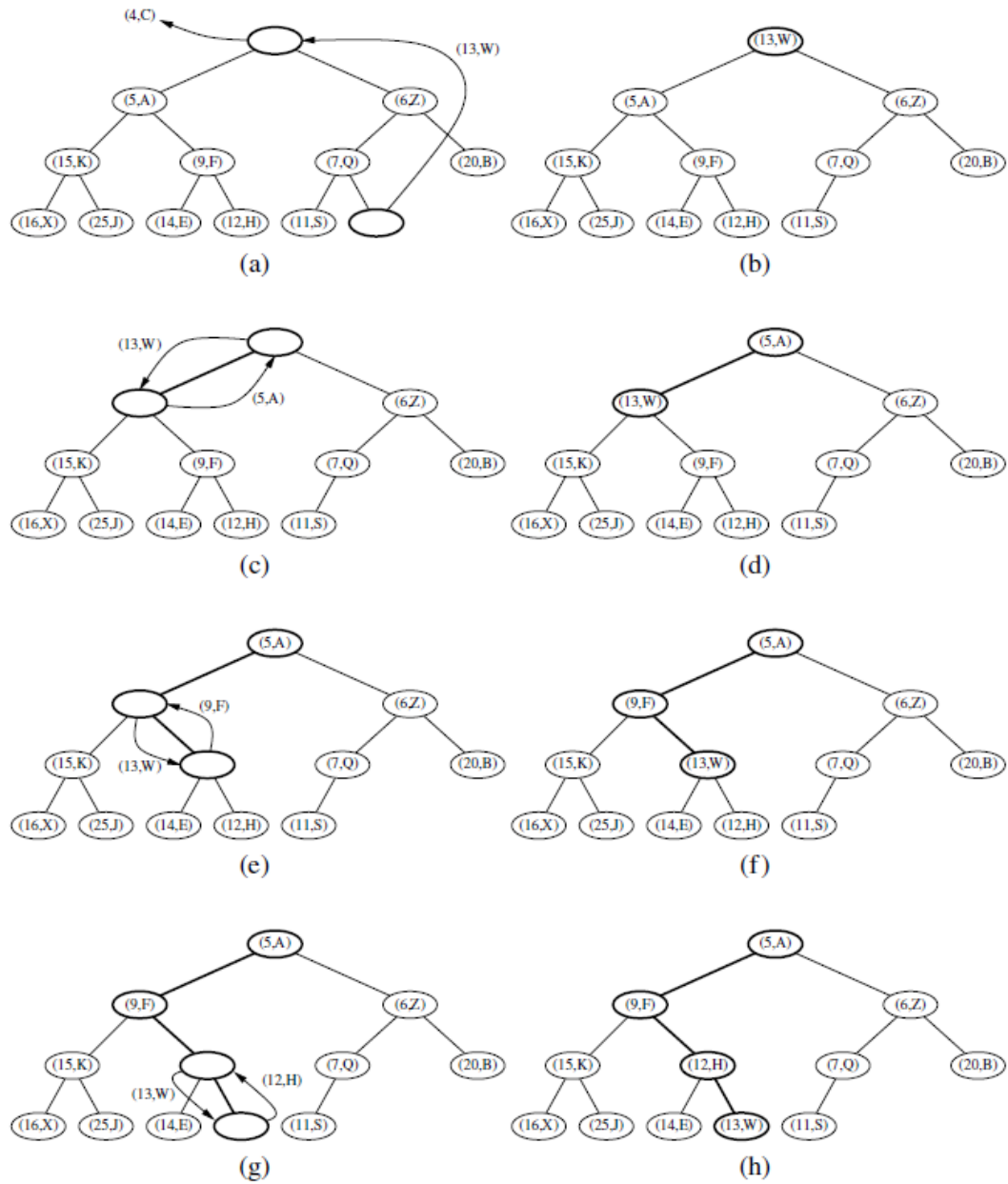


Figure 9.3: Removal of the entry with the smallest key from a heap: (a and b) deletion of the last node, whose entry gets stored into the root; (c and d) swap to locally restore the heap-order property; (e and f) another swap; (g and h) final swap.

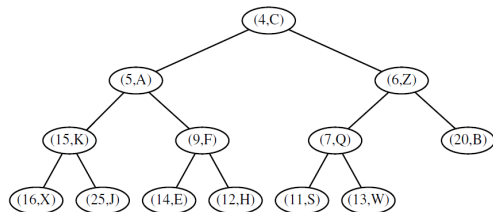
7.5 Implementazione Array-based di un Heap

È sempre possibile rappresentare un albero binario utilizzando un array, sfruttando la relazione tra gli indici dei nodi genitori e figli. In generale questa rappresentazione è meno efficiente in termini di spazio rispetto a una rappresentazione basata su nodi collegati, poiché richiede spazio per tutti i nodi, compresi quelli vuoti.

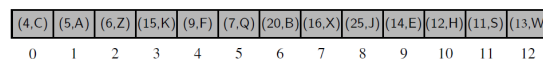
Tuttavia, per un albero binario *completo* come un Heap, questa rappresentazione è particolarmente efficiente, poiché non ci sono nodi vuoti tra i nodi effettivamente presenti nell'albero. Sia n il numero di nodi nell'albero, nel caso di un albero binario qualsiasi l'array può avere nel caso peggiore $N = 2^n - 1$ elementi, mentre nel caso di albero binario completo l'array avrà esattamente $N = n$ elementi.

Questo procedimento vale per ogni albero binario, ma è particolarmente efficiente per un Heap, poiché l'albero è completo. La mappatura tra i nodi dell'albero e gli indici dell'array avviene seguendo queste regole:

- Il nodo radice dell'albero viene memorizzato all'indice 0 dell'array.
- Per un nodo situato all'indice i nell'array:
 - Il figlio sinistro del nodo si trova all'indice $2i + 1$.
 - Il figlio destro del nodo si trova all'indice $2i + 2$.
 - Il genitore del nodo si trova all'indice $\lfloor (i - 1)/2 \rfloor$, a condizione che $i > 0$.



(a) Esempio di uno Heap con 13 elementi. L'ultima posizione è occupata dal nodo con chiave 13.



(b) Sua rappresentazione come array.

7.6 Analisi delle prestazioni

Operation	Running Time
$\text{len}(P)$, $P.\text{is_empty}()$	$O(1)$
$P.\text{min}()$	$O(1)$
$P.\text{add}()$	$O(\log n)^*$
$P.\text{remove_min}()$	$O(\log n)^*$

*amortized, if array-based

In figura sono riassunte le complessità temporali delle operazioni principali di una Priority Queue implementata tramite un Heap binario, assumendo che due chiavi possano essere confrontate in tempo costante $O(1)$, e che l'Heap sia implementato come array-based o linked-based tree. L'analisi è basata sulle seguenti considerazioni:

- L'Heap ha n nodi, ognuno dei quali contiene una coppia chiave-valore.
- L'altezza dell'Heap è $O(\log n)$ [In particolare, $h = \lfloor \log_2 n \rfloor$], dal momento che si tratta di un albero binario completo.
- L'operazione $\text{min}()$ viene eseguita in $O(1)$ perché la radice dell'albero contiene tale elemento.
- Nel caso peggiore, up-heap e down-heap eseguono un numero di scambi pari all'altezza dell'Heap.

Concludiamo che la struttura dati Heap è una realizzazione molto efficiente dell'ADT Priority Queue, indipendentemente dal fatto che l'heap sia implementato come array-based o linked-based tree. L'implementazione basata su heap raggiunge tempi di esecuzione rapidi sia per l'inserimento che per la rimozione, a differenza delle implementazioni basate sull'utilizzo di una sorted-list o unsorted-list.

7.7 Costruzione di un Heap da una lista di elementi: Heapify

A partire da un Heap vuoto, n inserimenti successivi al suo interno richiederebbero un tempo complessivo di $O(n \cdot \log n)$, poiché ogni inserimento richiede un tempo $O(\log n)$. Tuttavia, se le n coppie chiave-valore sono note a prescindere, esiste un algoritmo più efficiente chiamato **Heapify** che consente di costruire un Heap a partire da una lista di n elementi in tempo lineare $O(n)$.

Fusione di due Heap

Supponiamo di avere due Heap della stessa altezza h , e un nuovo elemento con chiave k . Vogliamo creare un nuovo Heap di altezza $h + 1$ che contenga tutti gli elementi dei due Heap e l'elemento con chiave k . Per fare ciò, possiamo seguire questi passaggi:

- Creiamo un nuovo nodo radice con chiave k .
- Assegniamo i due Heap esistenti come figli sinistro e destro della nuova radice.
- Eseguiamo l'operazione di down-heap bubbling a partire dalla radice per ripristinare la proprietà di heap-order.

Heapify

Per semplicità ipotizziamo che il numero di elementi n sia un intero tale che $n = 2^{h+1} - 1$ per qualche intero $h \geq 0$, in modo che l'Heap risultante sia un albero binario completo, con anche l'ultimo livello completamente pieno, per cui l'Heap ha altezza $h = \log_2(n + 1) - 1$. L'algoritmo Heapify può essere visto come una sequenza di $h + 1 = \log_2(n + 1)$ step [dato dal fatto che in un albero di altezza h ci sono $h + 1$ livelli, numerati da 0 a h]:

1. Nel primo step (Figura 9.5b), costruiamo $(n + 1)/2$ Heap di altezza 0 (cioè nodi singoli).
2. Nel secondo step (Figura 9.5c-d), costruiamo $(n + 1)/4$ Heap di altezza 1, ciascuno contenente tre nodi, unendo coppie di Heap elementari (ottenuti al passo precedente) e aggiungendo un nuovo elemento. Il nuovo nodo viene posizionato alla radice e potrebbe dover essere scambiato con un elemento figlio per preservare la heap-order property.
3. Nel terzo step (Figura 9.5e-f), costruiamo $(n + 1)/8$ Heap di altezza 2, ciascuno contenente sette nodi, unendo coppie di Heap (ottenuti al passo precedente) e aggiungendo un nuovo elemento. Il nuovo nodo viene posizionato alla radice e potrebbe dover essere scambiato attraverso un down-heap bubbling per preservare la heap-order property.
- ⋮
- i. Nel generico i -esimo step, con $2 \leq i \leq h$, costruiamo $(n + 1)/2^i$ Heap di altezza $i - 1$, ciascuno contenente $2^i - 1$ nodi, unendo coppie di Heap (ottenuti al passo precedente) e aggiungendo un nuovo elemento. Il nuovo nodo viene posizionato alla radice e potrebbe dover essere scambiato attraverso un down-heap bubbling per preservare la heap-order property.
- ⋮
- $h + 1$. Nell'ultimo step (Figura 9.5g-h), costruiamo un singolo Heap di altezza h , contenente tutti e n gli elementi, unendo gli ultimi due Heap che risultano essere di $(n - 1)/2$ elementi e di altezza $h - 1$ (ottenuti al passo precedente) e aggiungendo un nuovo elemento. Il nuovo nodo viene posizionato alla radice e potrebbe dover essere scambiato attraverso un down-heap bubbling per preservare la heap-order property.

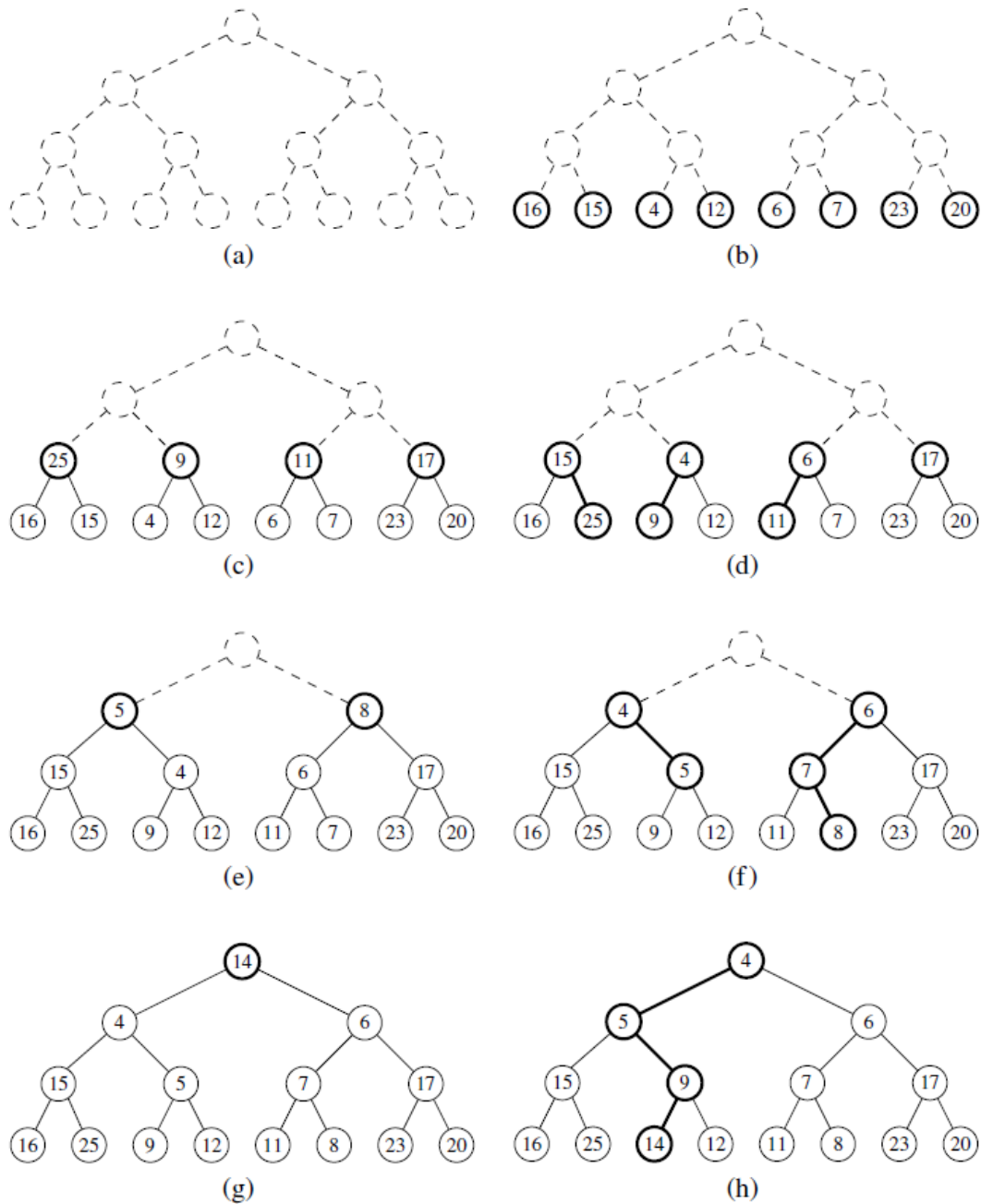


Figura 7.2: Costruzione bottom-up di un Heap con 15 elementi: (a, b) iniziamo costruendo Heap elementari di un solo elemento; (c, d) combiniamo questi Heap in Heap di 3 elementi, e poi (e, f) in Heap di 7 elementi, fino a (g, h) dove otteniamo l'Heap finale di 15 elementi. Il path del down-heap bubbling è evidenziato in (d, f, h). Per semplicità è mostrata solamente la chiave in ogni nodo anziché la coppia chiave-valore.

7.8 Heap-Sort

L'algoritmo **Heap-Sort** sfrutta la struttura dati Heap: data la lista di partenza, inserisco tutti gli elementi in un **Min-Heap** (al fine di avere gli elementi in ordine crescente alla fine dell'algoritmo) e poi estraggo ripetutamente l'elemento massimo (la radice del Min-Heap) per costruire la lista ordinata. Questo algoritmo viene eseguito già in tempo $O(n \cdot \log n)$, poiché l'inserimento di n elementi in un Heap richiede un tempo $O(n)$ (grazie all'algoritmo Heapify) e ogni estrazione dell'elemento massimo richiede un tempo $O(\log n)$, per un totale di n estrazioni.

L'idea per un ordinamento in-place

L'algoritmo che presentiamo di seguito è una versione più raffinata dello stesso Heap-sort che anziché utilizzare memoria aggiuntiva permette di avere un **ordinamento in-place** (cioè occupa al più memoria aggiuntiva $O(1)$) dell'Heap-sort, e utilizza invece un **Max-Heap** per costruire l'array ordinato in senso crescente.

L'idea chiave è dividere l'array C in due parti contigue:

1. **La parte sinistra (Heap):** Da $C[0]$ a $C[i - 1]$.
2. **La parte destra (Sequenza):** Da $C[i]$ a $C[n - 1]$.

Durante l'algoritmo, il confine i tra queste due parti si sposta.

Fase 1: costruzione di un Max Heap

Obiettivo: Trasformare l'array C in un unico Max-Heap.

In questa fase, si parte da un heap vuoto e si sposta il confine i **da sinistra verso destra**, da 1 a n . La parte sinistra (l'Heap) cresce, mentre la parte destra (la Sequenza di input) si riduce.

Per ogni passo i , da $i = 1$ fino a n :

1. **Azione (Espansione):** Il confine si sposta. L'elemento $C[i - 1]$ (il primo della Sequenza) viene aggiunto all'Heap, diventandone l'ultima foglia.
2. **Aggiustamento (Up-Heap Bubbling):** L'aggiunta di $C[i - 1]$ potrebbe violare la proprietà del Max-Heap. Si esegue quindi un **Up-Heap Bubbling** a partire da $C[i - 1]$. Questo elemento viene scambiato con il suo genitore finché non è più piccolo del genitore o non raggiunge la radice $C[0]$.

Alla fine della Fase 1, $i = n$. La parte Heap occupa l'intero array $C[0 \dots n - 1]$ e la Sequenza è vuota. L'intero array è ora un Max-Heap valido.

Fase 2: estrazione degli elementi in ordine

Obiettivo: Estrarre gli elementi dall'Heap in ordine decrescente per costruire l'array ordinato.

In questa fase, si parte con tutti gli elementi nell'Heap e la Sequenza vuota. Si sposta il confine **da destra verso sinistra**.

Per ogni passo i , da $i = 1$ fino a n :

1. **Azione (Estrazione):** L'elemento massimo dell'Heap si trova sempre alla radice, $C[0]$. Questo elemento deve essere spostato nella sua posizione ordinata finale, che in questo passo è $C[n - i]$ (la prima posizione libera a sinistra della Sequenza ordinata). Si **scambiano** $C[0]$ e $C[n - i]$.
2. **Aggiustamento (Down-Heap Bubbling):** Dopo lo scambio, $C[n - i]$ contiene il massimo ed è ora "bloccato" (fa parte della Sequenza ordinata). L'Heap si è ridotto (ora va da $C[0]$ a $C[n - i - 1]$). L'elemento che è finito in $C[0]$ (quello che era $C[n - i]$) è probabilmente fuori posto e viola la proprietà del max-Heap. Si esegue quindi un **Down-Heap Bubbling** a partire dalla radice $C[0]$. Questo elemento "scende" scambiandosi con il suo figlio *maggiore*, finché non è più grande di entrambi i figli o non raggiunge una foglia dell'Heap.

Alla fine della Fase 2, $i = n$. La parte "Heap" è vuota e la "Sequenza" (ora ordinata in senso crescente) occupa l'intero array.

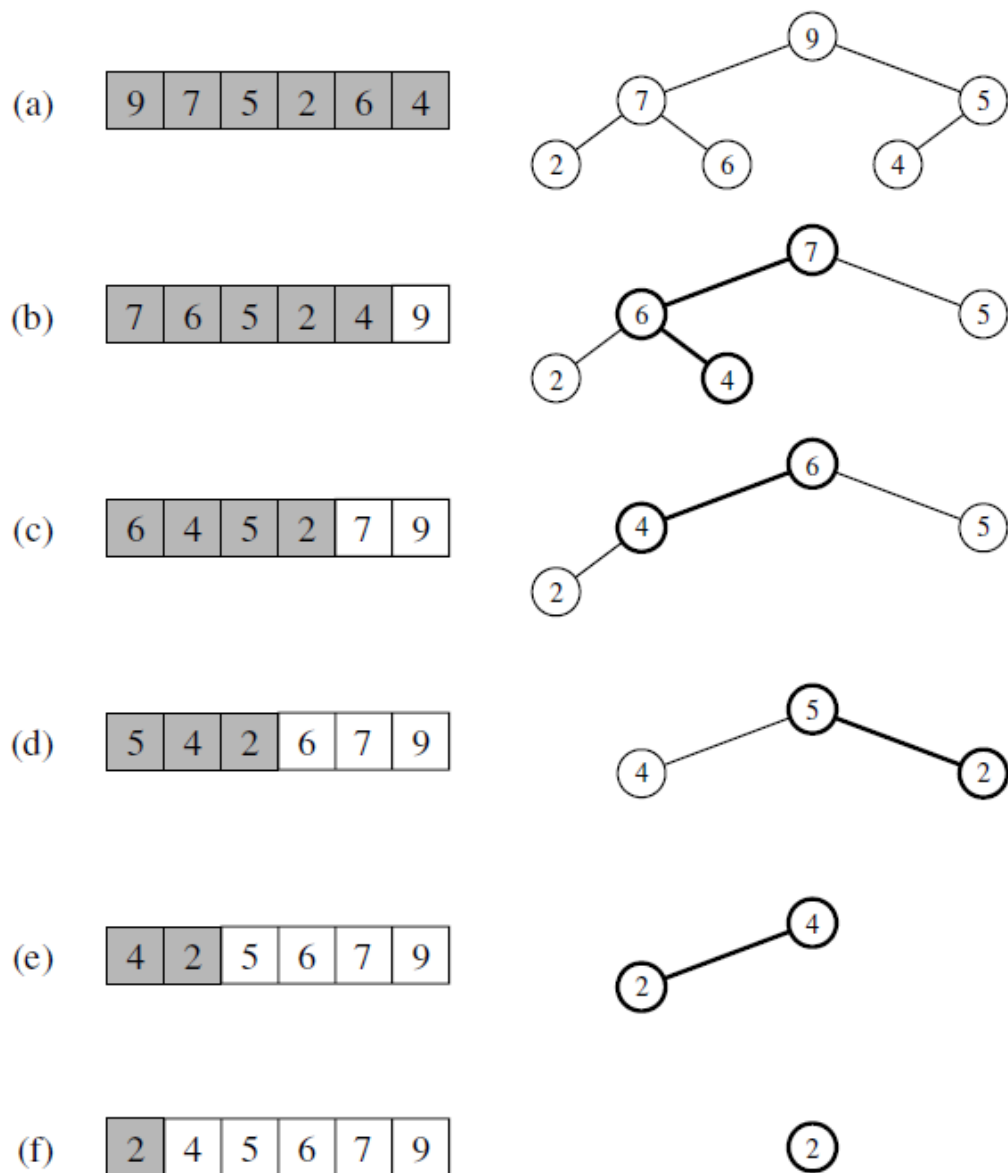


Figura 7.3: Fase 2 di un Heap-Sort in-place. La porzione Heap è evidenziata in grigio all'interno dell'array, per ogni iterazione dell'algoritmo. L'albero binario equivalente alla porzione Heap per ogni iterazione è rappresentato graficamente con il percorso più recente di Down-Heap Bubbling evidenziato.

7.9 Adaptable Priority Queue

L'ADT Priority Queue è sufficiente per molte applicazioni, ma in alcuni casi è indispensabile avere delle funzionalità aggiuntive per gestire in modo più flessibile gli elementi all'interno della coda, per esempio modificare la chiave (priorità) di un elemento esistente o rimuovere un elemento specifico (non necessariamente quello con priorità minima). Per questo motivo, presentiamo una variante chiamata **Adaptable Priority Queue** che estende l'ADT Priority Queue con queste funzionalità aggiuntive.

Locators

Per poter implementare in modo efficiente le nuove operazioni di modifica e cancellazione, è necessario trovare un meccanismo che ci permetta di trovare uno specifico elemento all'interno della coda senza dover scorrere l'intera struttura dati. Per questo motivo, quando un elemento viene inserito nella coda, viene restituito uno speciale oggetto chiamato **locator** al chiamante. Di conseguenza, ogni volta che si desidera modificare o rimuovere un elemento specifico di una coda a priorità P è necessario utilizzare il locator associato a quell'elemento per accedervi direttamente.

$P.update(loc, k, v)$: Sostituisce la chiave k e il valore v dell'elemento associato al locator loc nella coda a priorità P .

$P.remove(loc)$: Rimuove l'elemento associato al locator loc dalla coda a priorità P e lo restituisce.

L'astrazione del *locator* è in qualche modo simile all'astrazione della *position*. Tuttavia, facciamo una distinzione tra un locator e una position perché un locator per una coda prioritaria non rappresenta una collocazione tangibile di un elemento all'interno della struttura. Nella nostra coda prioritaria, un elemento può essere ricollocato all'interno della nostra struttura dati durante un'operazione che non sembra direttamente rilevante per quell'elemento. Un locator per un elemento rimarrà valido fintanto che quell'elemento rimarrà da qualche parte nella coda.

Implementazione di un Adaptable Priority Queue

L'implementazione della classe Locator estende la classe `_Item` dell'ADT Priority Queue per includere un campo aggiuntivo che tiene traccia della posizione corrente dell'elemento all'interno della rappresentazione basata su array del nostro Heap. Questo campo aggiuntivo consente di accedere rapidamente alla posizione dell'elemento nell'Heap, facilitando le operazioni di aggiornamento e rimozione, come mostrato Nella Figura 7.4.

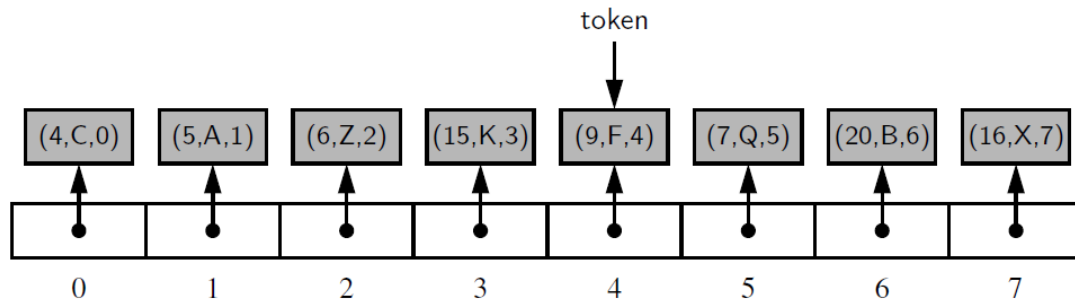


Figura 7.4: Rappresentazione di un Heap utilizzando una sequenza di Locator. Il terzo elemento di ciascuna istanza di Locator corrisponde all'indice dell'elemento all'interno dell'array. Si presume che l'identificatore "token" sia un riferimento al localizzatore nello scope dell'utente.

La lista è una sequenza di riferimenti a istanze di Locator, ognuna delle quali memorizza una chiave, un valore, e l'indice corrente dell'elemento all'interno dell'array che rappresenta l'Heap. All'utente verrà fornito un riferimento al Locator corrispondente per ciascun elemento inserito, come illustrato dall'identificatore "token" nella Figura 7.4.

Quando eseguiamo delle operazioni sulla Priority Queue che potrebbero alterare la posizione di un elemento all'interno dell'Heap (come l'inserimento che comporta un Up-Heap Bubbling, la rimozione che comporta un Down-Heap Bubbling o l'aggiornamento della chiave), dobbiamo assicurarci di aggiornare il campo dell'indice all'interno del Locator corrispondente. Questo garantisce che il Locator rimanga valido e punti sempre alla posizione corretta dell'elemento nell'Heap.

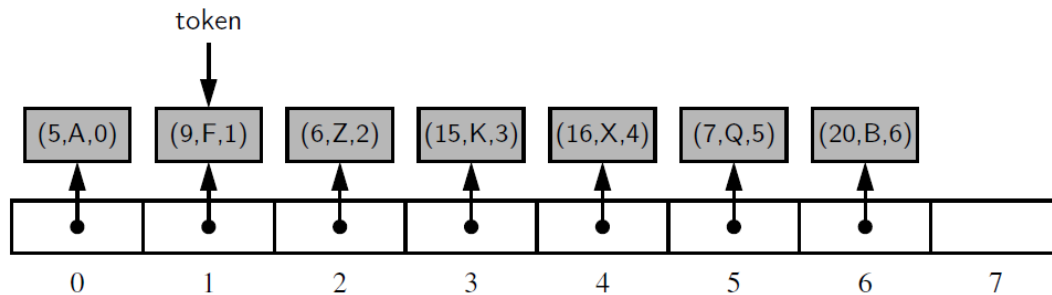


Figura 7.5: È possibile osservare lo stato dell'Heap dopo aver eseguito una `remove_min()`. Questa operazione causa un Down-Heap Bubbling che ricolloca gli elementi all'interno dell'Heap, e di conseguenza aggiorna gli indici nei Locator associati agli elementi coinvolti nel bubbling.

Operation	Running Time
<code>len(P)</code> , <code>P.is_empty()</code> , <code>P.min()</code>	$O(1)$
<code>P.add(k,v)</code>	$O(\log n)^*$
<code>P.update(loc, k, v)</code>	$O(\log n)$
<code>P.remove(loc)</code>	$O(\log n)^*$
<code>P.remove_min()</code>	$O(\log n)^*$

*amortized with dynamic array

Capitolo 8

Pattern Matching

Introduciamo di seguito la terminologia di base:

- Σ : l'alfabeto, ovvero l'insieme di caratteri possibili.
- $|\Sigma|$: la dimensione dell'alfabeto.
- Stringa S : una sequenza finita di caratteri appartenenti all'alfabeto Σ , di lunghezza m .
- $S[i]$: il carattere alla posizione i della stringa S .
 - In Python: `S[i]`
- $S[i..j]$: la sottostringa di S che va dall'indice i all'indice j .
 - In Python: `S[i:j+1]`
- $S[0..k]$: prefisso di lunghezza $k + 1$ della stringa S .
 - In Python: `S[:k+1]`
- $S[j..m - 1]$: suffisso di lunghezza $m - j$ della stringa S .
 - In Python: `S[j:]`

Convenzioni Python:

- La sottostringa $S[j:j+m]$ ha lunghezza m .
- La sottostringa $S[j:j]$ è la stringa nulla, di lunghezza 0.
- La sottostringa $S[j:k]$ è la stringa nulla quando $k < j$.

Nel classico problema di pattern matching, ci viene data una stringa di testo T di lunghezza n e una stringa di pattern P di lunghezza m , e vogliamo scoprire se P è una sottostringa di T . In tal caso, potremmo voler trovare l'indice più basso j all'interno di T in cui inizia P , in modo che $T[j..j + m - 1]$ sia uguale a P , o forse trovare tutti gli indici di T in cui inizia il pattern P .

8.1 Brute Force

Il metodo più semplice per risolvere il problema del pattern matching è il metodo *brute force*. L'idea alla base di questo metodo è di confrontare il pattern P con ogni possibile sottostringa di T di lunghezza m . In particolare, per ogni indice i da 0 a $n - m$, confrontiamo la sottostringa $T[i..i + m - 1]$ con il pattern P . Se troviamo una corrispondenza, restituiamo l'indice i .

```
1 def find_brute(T, P):
2     """Return the lowest index of T at which substring P begins (or else
3         -1)."""
4     n, m = len(T), len(P) # introduce convenient notations
5     for i in range(n-m+1): # try every potential starting index within T
6         k = 0 # an index into pattern P
7         while k < m and T[i + k] == P[k]: # kth character of P matches
8             k += 1
9         if k == m: # if we reached the end of pattern,
10            return i # substring T[i:i+m] matches P
11    return -1 # failed to find a match starting with any i
```

Performance

L'algoritmo consiste in due cicli annidati, con il ciclo esterno che scorre tutti i possibili indici iniziali del pattern nel testo T , e il ciclo interno che scorre ogni carattere del pattern P , confrontandolo con il suo potenziale carattere corrispondente nel testo. Pertanto, la correttezza dell'algoritmo deriva direttamente da questo approccio di ricerca esaustiva.

Il tempo di esecuzione del pattern matching tramite *Brute Force* nel caso peggiore non è buono poiché per ogni indice candidato in T , possiamo eseguire fino a m confronti di caratteri per scoprire che P non corrisponde a T all'indice corrente. Dal blocco di codice si può osservare che il ciclo *for* esterno viene eseguito al massimo $n - m + 1$ volte e il ciclo *while* interno viene eseguito al massimo m volte. Pertanto, il tempo di esecuzione nel caso peggiore è $O(n \cdot m)$.

Esempio

Supponiamo di avere un testo

$T = \text{"abacaabaccabacabaabb"}$

e un pattern

$P = \text{"abacab"}$

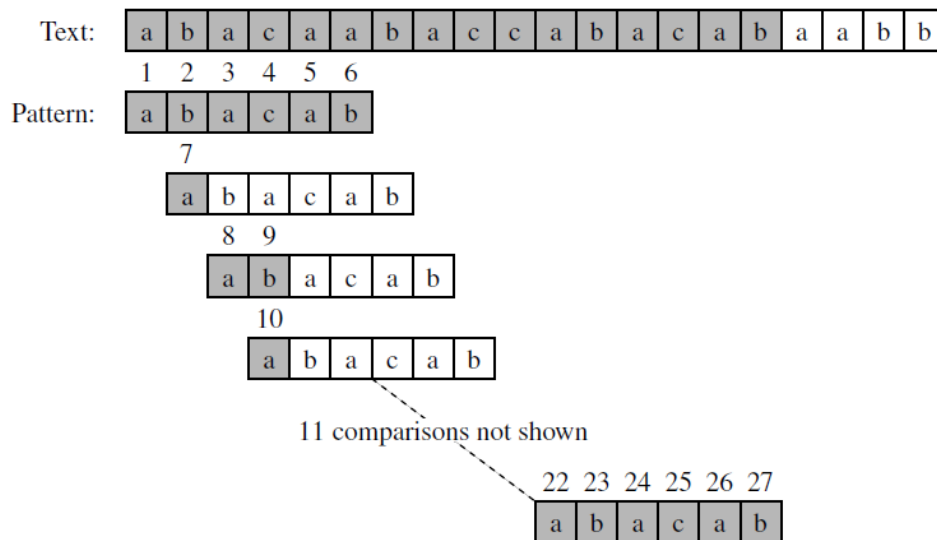


Figura 8.1: Esempio di Pattern Matching con algoritmo Brute Force. L'algoritmo esegue 27 confronti tra caratteri, numerati in figura.

8.2 L'algoritmo di Boyer-Moore

Come vedremo tra poco, non è sempre necessario confrontare ogni carattere del pattern con il testo. L'algoritmo di *Boyer-Moore* sfrutta questa osservazione per saltare alcune posizioni nel testo, riducendo così il numero di confronti necessari.

L'idea principale dell'algoritmo di *Boyer-Moore* è di migliorare l'efficienza dell'algoritmo *Brute Force* utilizzando due tecniche (euristiche) principali:

- **Looking-Glass Heuristic:** Quando si confrontano i caratteri del pattern con il testo, si inizia dal carattere più a destra del pattern e si procede verso sinistra.
- **Character-Jump Heuristic:** Durante la verifica di un possibile piazzamento di P in T , un mismatch tra $T[i] = c$ e $P[k]$ viene gestito come segue:

Supponiamo che $T[i] \neq P[k]$ e $T[i] = c$.

- Se c non appare in P , P può essere "spostato" completamente oltre $T[i]$ ($P[0]$ viene allineato con $T[i + 1]$).
- Altrimenti, $T[i]$ viene allineato con l'ultima occorrenza di c in P .

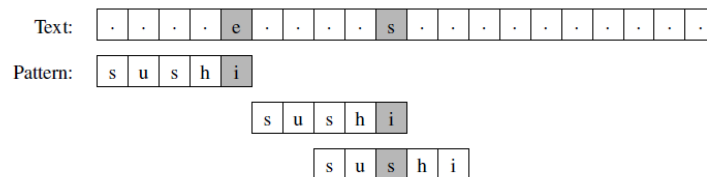


Figura 8.2: Una semplice dimostrazione dell'algoritmo di Boyer-Moore. Nel primo confronto si ha $T[4] \neq P[4]$ con $T[4] = 'e'$ che non è presente in P , per cui spostiamo P oltre $T[4]$. Nel secondo confronto si ha $T[9] \neq P[4]$ con $T[9] = 's'$ che è presente in P , in particolare l'ultima occorrenza di $'s'$ è in $P[2]$, per cui allineiamo $P[2]$ con $T[9]$.

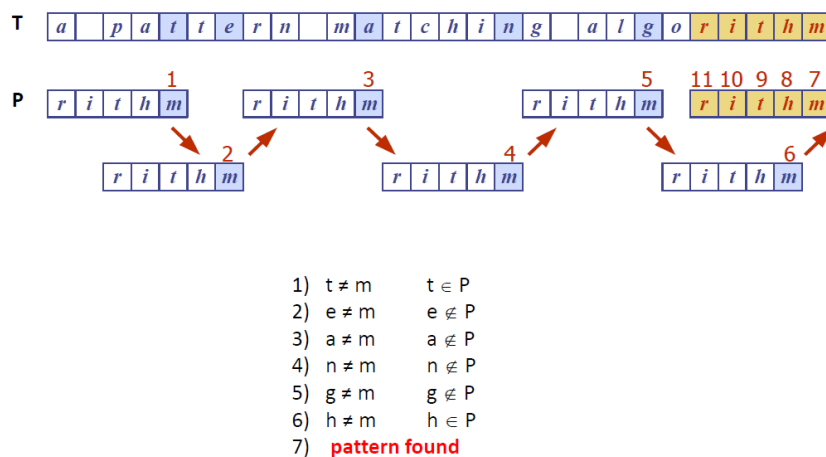


Figura 8.3: Esempio completo che mostra il numero di confronti.

Per formalizzare l'algoritmo di *Boyer-Moore* possiamo generalizzare il funzionamento come di seguito:

- Quando viene trovata una corrispondenza (a partire dall'ultimo carattere del pattern), l'algoritmo continua cercando di estendere la corrispondenza con il penultimo carattere del pattern nel suo allineamento corrente. Questo processo continua fino a quando tutti i caratteri del pattern sono stati confrontati con esito positivo o fino a quando si verifica un mismatch.
- Quando si verifica un mismatch, e il carattere del testo che ha causato il mismatch non è presente nel pattern, il pattern viene spostato completamente oltre quel carattere del testo. Se il carattere del testo è presente da qualche altra parte nel pattern, dobbiamo considerare due diversi casi a seconda che la sua ultima occorrenza (nel pattern) sia (a) precedente o (b) successiva al carattere del pattern che ha causato il mismatch.

Questi due casi sono rappresentati in figura 8.4 e approfonditi di seguito:

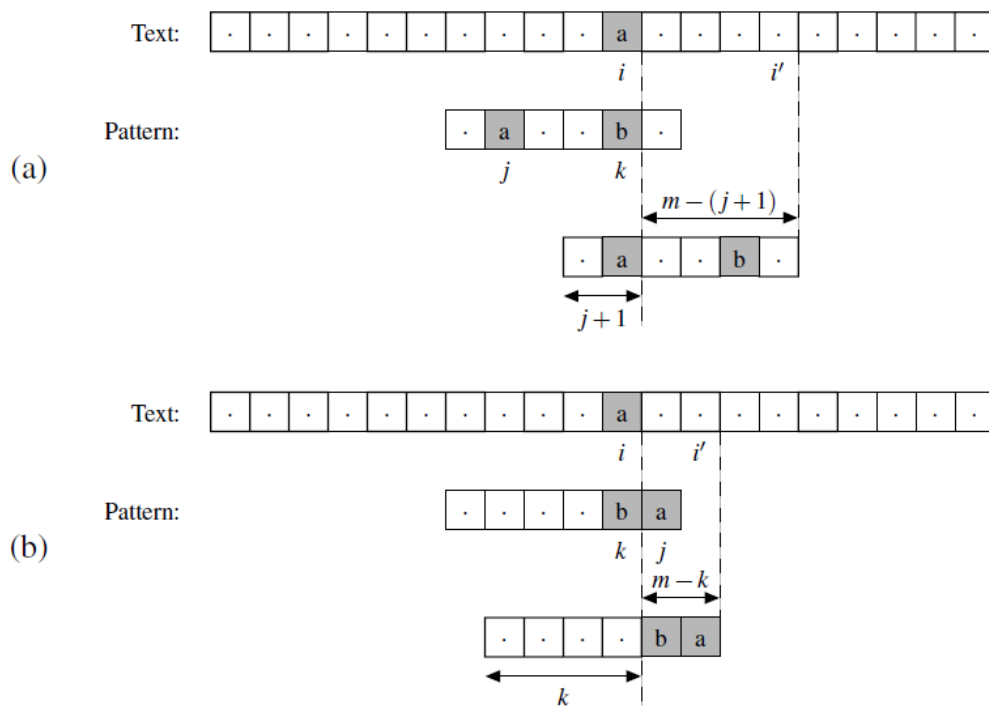


Figura 8.4: Indichiamo con i l'indice del carattere non corrispondente nel testo, con k l'indice corrispondente nel pattern e con j l'indice dell'ultima occorrenza di $T[i]$ all'interno del pattern. Distinguiamo due casi: **(a)** $j < k$, nel qual caso spostiamo il pattern di $k - j$ unità, e quindi l'indice i avanza di $m - (j + 1)$ unità; **(b)** $j > k$, nel qual caso spostiamo il pattern di un'unità, e l'indice i avanza di $m - k$ unità. N.B. m è la lunghezza del pattern.

Nel caso di 8.4(b), il pattern viene spostato di una sola unità a destra. Sarebbe sicuramente più produttivo spostare il pattern a destra fino a quando l'ultima occorrenza di $T[i]$ nel pattern non è allineata con $T[i]$, ma questo richiederebbe un ulteriore calcolo per la ricerca della nuova occorrenza.

L'efficienza dell'algoritmo di *Boyer-Moore* risiede nell'utilizzo di una funzione di pre-elaborazione, $L(c)$, chiamata **last occurrence function** che consente di determinare rapidamente l'ultima occorrenza di un carattere nel pattern. Questa funzione viene calcolata una sola volta prima dell'inizio del processo di ricerca e viene utilizzata ogni volta che si verifica un mismatch.

$$\forall c \in \Sigma, \quad L(c) = \begin{cases} \max\{i \mid P[i] = c\} & \text{se } c \in P \\ -1 & \text{se } c \notin P \end{cases}$$

Per esempio, se abbiamo $\Sigma = \{'a', 'b', 'c', 'd'\}$ e $P = \text{"abacab"}$:

c	'a'	'b'	'c'	'd'
$L(c)$	4	5	3	-1

Possiamo modellare L come una *Map* che ha per chiavi i caratteri dell'alfabeto e per valori gli indici delle loro ultime occorrenze nel pattern (ad esempio, in Python possiamo usare un dizionario). La Map L può essere costruita in tempo $O(m + |\Sigma|)$ dove m è la lunghezza del pattern e $|\Sigma|$ è la dimensione dell'alfabeto.

```

1 def last_occurrence(p, sigma):
2     """Return the last-occurrence map L for pattern p over alphabet
      sigma."""
3     L = {c:-1 for c in sigma} # initialize all characters to -1
4     for i in range(len(p)):
5         L[p[i]] = i # update with last occurrence of character p[i]
6     return L

```



```

1 def find_boyer_moore(T, P):
2     """Restituisce l'indice più basso in T dove inizia P (altrimenti -1)."""
3     n, m = len(T), len(P) # Notazioni comode per le lunghezze
4     if m == 0:
5         return 0 # Ricerca banale per pattern vuoto
6
7     # last occurrence function
8     last = {} # Costruisce la mappa delle ultime occorrenze
9     for k in range(m):
10         last[P[k]] = k # Le occorrenze successive sovrascrivono le
11                         # precedenti
12
13     # Allinea la fine del pattern all'indice m-1 del testo
14     i = m-1 # Indice corrente nel Testo (T)
15     k = m-1 # Indice corrente nel Pattern (P)
16
17     while i < n:
18         if T[i] == P[k]: # Carattere corrispondente trovato
19             if k == 0:
20                 return i # Trovato! Il pattern inizia all'indice i del
21                         # testo
22             else:
23                 i -= 1 # Estendi la corrispondenza a sinistra
24                 k -= 1 # Sposta entrambi gli indici
25             else:
26                 # Mismatch: gestione del salto
27                 j = last.get(T[i], -1) # Indice dell'ultima occorrenza di T[i]
28                                     # nel pattern
29
30                 # Calcolo del salto (shift) combinando i casi:
31                 # min(k, j+1) gestisce la logica dei Casi A e B:
32                 # - Se j < k (Caso A): allinea l'occorrenza j con il testo.
33                 # - Se j > k (Caso B): evita shift negativo, sposta il pattern
34                 #   di 1 posizione.
35                 i += m - min(k, j+1)
36
37         k = m-1 # Riavvia il confronto dalla fine del pattern
38
39     return -1

```

Performance

La versione semplificata dell'algoritmo di Boyer-Moore presentata qui ha un tempo di esecuzione di $O(n \cdot m + |\Sigma|)$, in quanto la last-occurrence function richiede $O(m + |\Sigma|)$ tempo per essere costruita e la ricerca del pattern richiede $O(n \cdot m)$.

Il caso peggiore si verifica quando si ha una coppia del tipo:

$$T = \text{"aaa...aaa"} \quad P = \text{"baa...aaa"}$$

In questo caso, l'algoritmo di Boyer-Moore si comporta come l'algoritmo Brute Force, eseguendo $O(n \cdot m)$ confronti tra caratteri.

Tuttavia, l'algoritmo originale di Boyer-Moore utilizza euristiche più avanzate ed efficienti che consentono di ottenere un tempo di esecuzione medio di $O(n + m + |\Sigma|)$.

Esempio

Supponiamo di avere un testo

$$T = \text{"abacaabadcabacabaabb"}$$

e un pattern

$$P = \text{"abacab"}$$

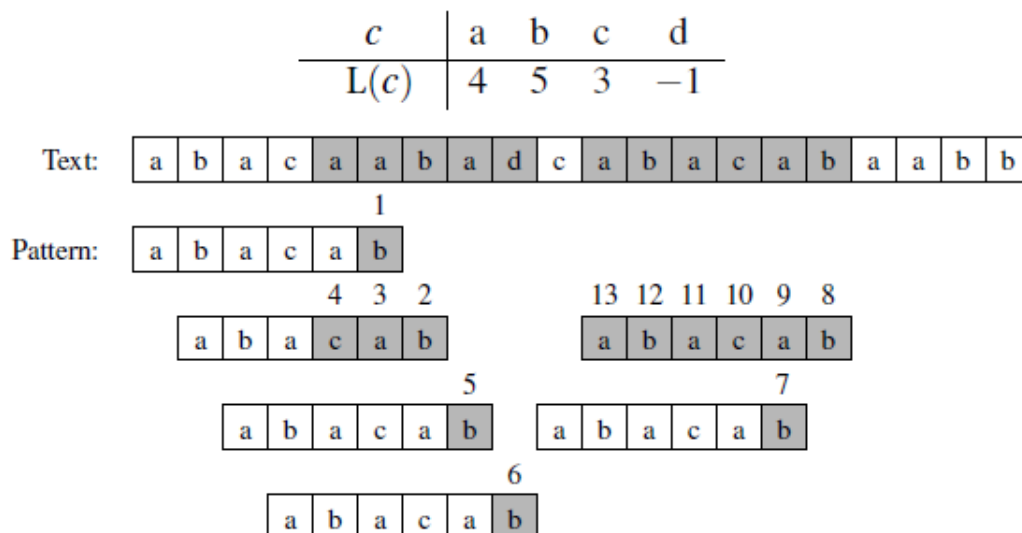


Figura 8.5: Esempio di Pattern Matching con algoritmo di Boyer-Moore, con anche la tabella delle last occurrence. L'algoritmo esegue 13 confronti tra caratteri, numerati in figura.

8.3 L'algoritmo di Knuth-Morris-Pratt

Analizzando le prestazioni nel caso peggiore degli algoritmi di pattern-matching *brute-force* e *Boyer-Moore* su istanze specifiche del problema possiamo notare una notevole inefficienza. Per un certo allineamento del pattern, se troviamo diversi caratteri corrispondenti ma poi rileviamo un mismatch, ignoriamo tutte le informazioni ottenute dai confronti andati a buon fine.

L'algoritmo di *Knuth-Morris-Pratt* (o "KMP") evita questo spreco di informazioni e, facendo così, raggiunge un tempo di esecuzione di $O(n + m)$: $O(m)$ per il pre-processing del pattern, e $O(n)$ per i confronti tra testo e pattern. Nel caso peggiore qualsiasi algoritmo di pattern-matching dovrà esaminare tutti i caratteri del testo e tutti i caratteri del pattern almeno una volta. L'idea principale dell'algoritmo KMP è quella di pre-calcolare le sovrapposizioni del pattern su se stesso. In questo modo, quando si verifica un mismatch in una certa posizione, sappiamo immediatamente qual è lo spostamento massimo che possiamo applicare al pattern prima di continuare la ricerca.

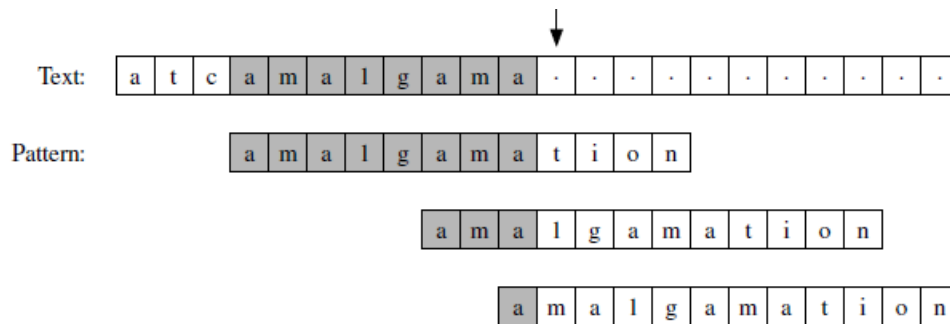


Figura 8.6: KMP non ripete i confronti sui primi tre caratteri del pattern dal momento che si ripetono e sono già stati confrontati con successo.

8.3.1 Failure-Function

L'algoritmo KMP si basa sul calcolo della **failure function** f , che indica il corretto spostamento di P in caso di confronto fallito. In particolare, la funzione di fallimento $f(k)$ è definita come la **lunghezza del più lungo prefisso di P che è anche un suffisso di $P[1 : k + 1]$** ¹. Intuitivamente, se troviamo un mismatch sul carattere $P[k + 1]$, la funzione $f(k)$ ci dice quanti dei caratteri immediatamente precedenti possono essere riutilizzati per riavviare il pattern.

Operativamente, la funzione viene utilizzata nel seguente modo: se durante il confronto si verifica un *mismatch* all'indice j del pattern (cioè $P[j] \neq T[i]$), significa che i caratteri precedenti $P[0 \dots j - 1]$ corrispondevano al testo. Invece di ripartire da zero, l'algoritmo consulta il valore $f(j - 1)$, il quale ci indica quanti caratteri del prefisso possiamo "salvare". Il confronto riprenderà quindi confrontando il carattere del testo $T[i]$ con il nuovo indice del pattern $j' = f(j - 1)$.

¹ $P[1:k+1]$ comprende i caratteri da indice 1 a indice k , $k + 1$ non è incluso. Nota che non abbiamo incluso $P[0]$ qui, dal momento che una stringa è suffisso di sè stesso; così facendo imponiamo che sia un **suffisso proprio**, cioè diverso dalla stringa stessa

Per esempio, consideriamo il pattern $P = \text{"amalgamation"}$. La failure function f per questo pattern è mostrata nella tabella seguente:

k	0	1	2	3	4	5	6	7	8	9	10	11
$P[k]$	a	m	a	l	g	a	m	a	t	i	o	n
$f(k)$	0	0	1	0	0	1	2	3	0	0	0	0

Implementazione

L'implementazione dell'algoritmo KMP si basa su una funzione di utilità, `compute_kmp_fail`, per calcolare efficientemente la funzione di fallimento. La parte principale dell'algoritmo KMP è il suo ciclo `while`, che per ogni iterazione esegue un confronto tra il carattere $T[j]$ e il carattere $P[k]$. Se l'esito di questo confronto è una corrispondenza, l'algoritmo procede ai caratteri successivi sia in T che in P (o segnala una corrispondenza completa se si raggiunge la fine del pattern). Se il confronto fallisce, l'algoritmo consulta la funzione di fallimento per un nuovo carattere candidato in P , o ricomincia con il prossimo indice in T se si fallisce sul primo carattere del pattern (dato che nulla può essere riutilizzato).

```

1 def find_kmp(T, P):
2     """Return the lowest index of T at which substring P begins (or else
3         -1)."""
4     n, m = len(T), len(P)    # introduce convenient notations
5     if m == 0:
6         return 0             # trivial search of empty pattern
7
8     fail = compute_kmp_fail(P) # rely on utility to precompute fail
9                                 function
10    j = 0                      # index into text
11    k = 0                      # index into pattern
12    while j < n:
13        if T[j] == P[k]:      # P[0:k] matched thus far
14            if k == m-1:      # match is complete
15                return j - m + 1 # pattern begins at index (j-m+1) of text
16            j += 1            # try to extend match
17            k += 1
18        elif k > 0:
19            k = fail[k-1]      # reuse suffix of P[0:k-1]
20        else:
21            j += 1            # no match at P[0], try next character in T
22    return -1

```

```

1 def compute_kmp_fail(P):
2     """Utility that computes and returns KMP fail list."""
3     m = len(P)
4     fail = [0] * m      # by default, presume overlap of 0 everywhere
5     j = 1
6     k = 0
7     while j < m:        # compute f(j) during this pass, if nonzero
8         if P[j] == P[k]: # k+1 characters match thus far
9             fail[j] = k + 1
10            j += 1
11            k += 1
12        elif k > 0:      # k follows a matching prefix
13            k = fail[k-1]
14        else:            # no match found starting at j
15            j += 1
16    return fail

```

Performance

Tralasciando momentaneamente il calcolo della failure function, il tempo di esecuzione dell'algoritmo KMP è chiaramente proporzionale al numero di iterazioni del ciclo while. Ai fini dell'analisi, definiamo $s = j - k$ come la quantità totale di spostamento del pattern P rispetto al testo T . Notiamo che durante l'esecuzione dell'algoritmo, abbiamo sempre $s \leq n$. Ad ogni iterazione del ciclo si verifica uno dei seguenti tre casi:

- Se $T[j] = P[k]$, allora sia j che k aumentano di 1, e quindi s non cambia.
- Se $T[j] \neq P[k]$ e $k > 0$, allora j non cambia e s aumenta di almeno 1, poiché in questo caso s cambia da $j - k$ a $j - f(k - 1)$, che è un'aggiunta pari a $k - f(k - 1)$, che è positiva perché $f(k - 1) < k$.
- Se $T[j] \neq P[k]$ e $k = 0$, allora j aumenta di 1 e s aumenta di 1, poiché k non cambia.

Quindi, in ogni iterazione del ciclo, o j o s vengono incrementati di almeno 1 (eventualmente entrambi); pertanto, il numero totale di iterazioni del ciclo while nell'algoritmo KMP è al massimo $2n$. Per rendere tutto ciò possibile si presuppone che la failure function di P sia già stata precedentemente calcolata.

L'algoritmo per il calcolo della failure function ha una complessità di $O(m)$. La sua analisi è analoga a quella del principale algoritmo KMP, ma esegue dei confronti tra il pattern di lunghezza m con se stesso.

Dunque, combinando entrambi i risultati, otteniamo che l'algoritmo KMP ha una complessità temporale complessiva di $O(n + m)$. La correttezza dell'algoritmo KMP deriva direttamente dalla definizione della failure function. Qualsiasi confronto che viene saltato è in realtà superfluo, poiché la funzione di fallimento garantisce che tutti i confronti ignorati siano ridondanti.

Esempio

Supponiamo di avere un testo

T = "abacaabaccabacabaabb"

e un pattern

P = "abacab"

k	0	1	2	3	4	5
P[k]	a	b	a	c	a	b
f(k)	0	0	1	0	1	2

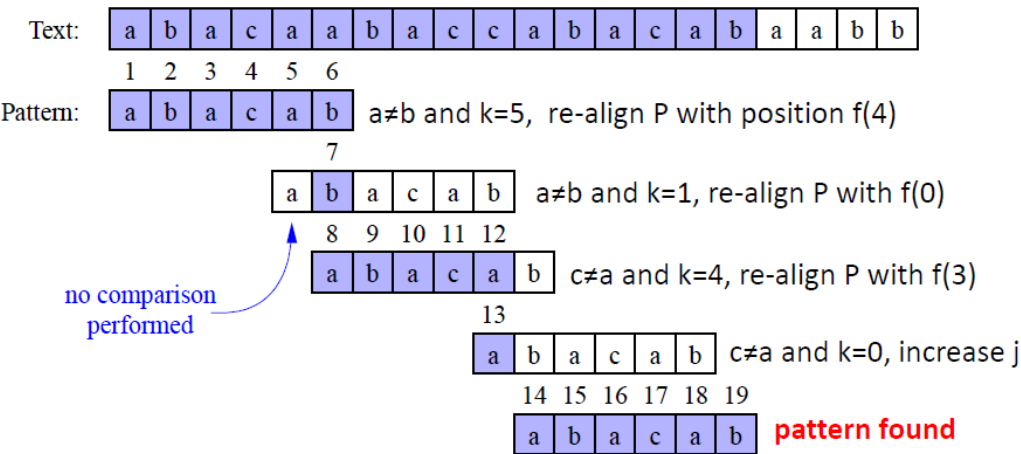


Figura 8.7: Esempio di Pattern Matching con algoritmo KMP, con anche la tabella della failure function. L'algoritmo esegue 19 confronti tra caratteri, numerati in figura (sono necessari ulteriori confronti per il calcolo della failure function non presenti in figura).

Capitolo 9

Tries

Il problema del *pattern matching* presentato nel capitolo 8 velocizza la ricerca in un testo effettuando una pre-elaborazione del pattern (per calcolare la funzione di fallimento nell'algoritmo di Knuth-Morris-Pratt o la funzione last nell'algoritmo di Boyer-Moore). In questa sezione, adottiamo un approccio complementare, presentando algoritmi di ricerca di stringhe che effettuano una pre-elaborazione del testo. Questo approccio è adatto per applicazioni in cui viene eseguita una serie di query su un testo fisso, in modo che il costo iniziale della pre-elaborazione del testo sia compensato da un'accelerazione in ogni query successiva. A questo proposito introduciamo i *tries* (pronunciato "try"), una struttura dati basata su alberi per memorizzare stringhe al fine di supportare un rapido pattern matching. La principale applicazione dei tries è nel recupero delle informazioni, da cui il nome "trie" che deriva dalla parola "retrieval".

9.1 Standard Tries

Sia S un insieme di s stringhe su un alfabeto Σ tale che nessuna stringa in S sia prefisso di un'altra stringa. Uno **Standard Trie** per S è un albero ordinato T con le seguenti proprietà:

- Ogni nodo di T , eccetto la radice, è etichettato con un carattere di Σ .
- I figli di un nodo interno di T hanno etichette distinte.
- T ha s foglie, ciascuna associata a una stringa di S , tale che la concatenazione delle etichette dei nodi sul percorso dalla radice a una foglia v di T dia la stringa di S associata a v .

Dunque, un trie T rappresenta le stringhe di S tramite i percorsi dalla radice alle foglie di T . È importante assumere che nessuna stringa in S sia prefisso di un'altra stringa poichè ciò garantisce che ogni stringa di S sia univocamente associata a una foglia di T . Possiamo sempre soddisfare questa assunzione aggiungendo un carattere speciale che non è nell'alfabeto originale Σ alla fine di ogni stringa.

Uno standard trie che memorizza una collezione S di s stringhe di lunghezza totale n da un alfabeto Σ ha le seguenti proprietà:

- L'altezza di T è uguale alla lunghezza della stringa più lunga in S .
- Ogni nodo interno di T può avere da 1 a $|\Sigma|$ figli.
- T ha s foglie.
- Il numero di nodi di T è al più $n + 1$.
 - Infatti, il caso peggiore per il numero di nodi di un trie si verifica quando nessuna coppia di stringhe condivide un prefisso non vuoto; cioè, ad eccezione della radice, tutti i nodi interni hanno un solo figlio.

Ricerca

Un trie T per un insieme S di stringhe permette di implementare una mappa in cui le chiavi sono le stringhe stesse; la ricerca di una stringa X avviene tracciando dalla radice il percorso indicato dai caratteri di X e, se tale percorso termina in un nodo foglia, la stringa è presente nella mappa, mentre se il percorso si interrompe o termina in un nodo interno la stringa non è una chiave valida.

Il tempo di esecuzione per cercare una stringa di lunghezza m è limitato superiormente da $O(m \cdot |\Sigma|)$, in quanto possiamo visitare al più $m + 1$ nodi di T e spendiamo $O(|\Sigma|)$ tempo in ogni nodo per determinare il figlio che ha come etichetta il carattere successivo. Tuttavia, possiamo migliorare il tempo speso in un nodo a $O(\log |\Sigma|)$ o atteso $O(1)$, mappando i caratteri ai figli utilizzando una tabella hash in ogni nodo, oppure utilizzando una lookup table diretta di dimensione $|\Sigma|$ in ogni nodo, se $|\Sigma|$ è sufficientemente piccolo (come nel caso delle stringhe di DNA). Per questi motivi, ci aspettiamo tipicamente che una ricerca per una stringa di lunghezza m venga eseguita in tempo $O(m)$.

Word Matching

Grazie a queste caratteristiche, il trie è adatto per il **word matching esatto** e per le query sui prefissi (per via dell'operazione di ricerca appena descritta), ma non per il pattern matching di sottostringhe arbitrarie. Il *word matching* è un caso particolare di pattern matching in cui vogliamo determinare se un dato pattern corrisponde esattamente a una delle parole del testo. Il word matching si differenzia dal pattern matching standard analizzato nel capitolo 8 perché in questo caso il pattern non può corrispondere a una sottostringa arbitraria del testo, ma solo a una delle sue parole.

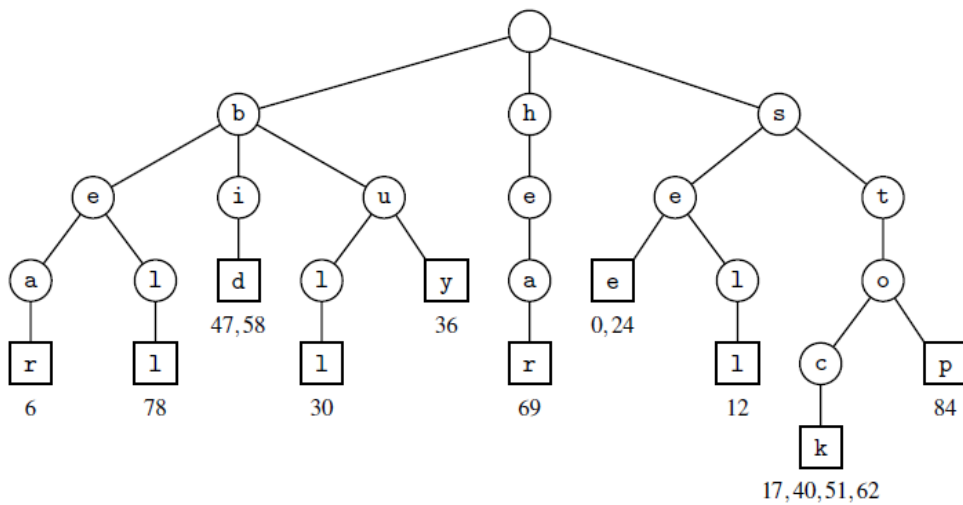
Costruzione di uno Standard Trie

Per costruire uno standard trie per un insieme S di stringhe, possiamo utilizzare un algoritmo incrementale che inserisce le stringhe una alla volta. Ricordiamo l'assunzione che nessuna stringa di S sia prefisso di un'altra stringa. Per inserire una nuova stringa X nel trie, seguiamo il percorso dei suoi caratteri finché esistono nodi corrispondenti. Nel punto in cui il percorso esistente si interrompe (ovvero non troviamo il carattere successivo), creiamo una nuova sequenza

di nodi per tutti i caratteri restanti di X . Il tempo di esecuzione per inserire X di lunghezza m è simile a una ricerca, con prestazioni nel caso peggiore di $O(m \cdot |\Sigma|)$, o attese $O(m)$ se si utilizzano tabelle hash secondarie in ogni nodo. Pertanto, la costruzione dell'intero trie per l'insieme S richiede un tempo atteso di $O(n)$, dove n è la lunghezza totale delle stringhe di S .

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
s	e	e		a		b	e	a	r	?		s	e	l	l		s	t	o	c	k	!
23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45
	s	e	e		a		b	u	l	l	?		b	u	y		s	t	o	c	k	!
46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68
	b	i	d		s	t	o	c	k	!		b	i	d		s	t	o	c	k	!	
69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88			
h	e	a	r		t	h	e		b	e	l	l	?		s	t	o	p	!			

(a)



(b)

Figura 9.1: Word Matching tramite uno Standard Trie: (a) testo da cercare (articoli e preposizioni, noti anche come stop words, esclusi); (b) standard trie per le parole nel testo, con le foglie che mantengono l'informazione relativa all'indice in cui la data parola inizia nel testo. Ad esempio, la foglia per la parola *stock* indica che la parola inizia agli indici 17, 40, 51 e 62 del testo.

Come si può notare anche dall'esempio in Figura 9.1, c'è una potenziale inefficienza di spazio nello standard trie dovuta alla presenza di molti nodi che hanno un solo figlio, e l'esistenza di tali nodi rappresenta uno spreco. La ricerca di una soluzione a questo problema ha portato allo sviluppo del *Compressed Trie*, o *Patricia Trie*.

9.2 Compressed Tries

Un **Compressed Trie** è una variante dello standard trie che garantisce che ogni nodo interno del trie abbia almeno due figli. Questa regola viene applicata comprimendo le catene di nodi con un solo figlio.

Sia T un trie standard. Diciamo che un nodo interno v di T è **ridondante** se v ha un solo figlio e non è la radice. Diciamo anche che una *catena* di $k \geq 2$ archi,

$$(v_0, v_1)(v_1, v_2) \cdots (v_{k-1}, v_k)$$

è **ridondante** se:

- v_i è ridondante per ogni $i = 1, \dots, k-1$.
- v_0 e v_k non sono ridondanti.

Si può trasformare T in un *Compressed Trie* sostituendo ogni catena ridondante $(v_0, v_1) \cdots (v_{k-1}, v_k)$ di $k \geq 2$ archi con un singolo arco (v_0, v_k) , rietichettando v_k con la concatenazione delle etichette dei nodi $v_1, \dots, v_k$.

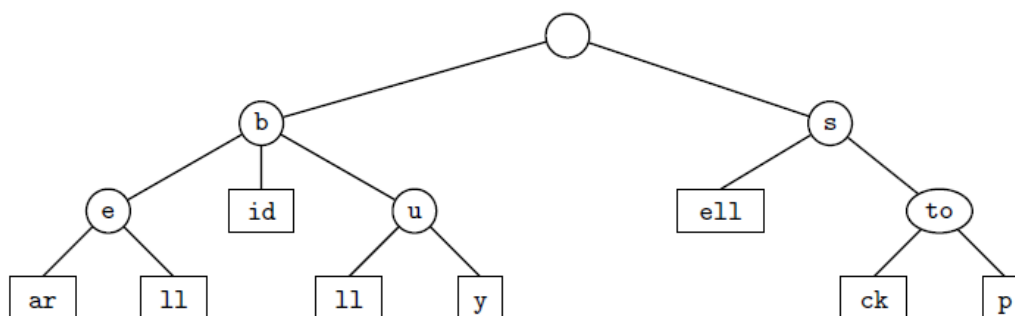


Figura 9.2: Compressed Trie per le stringhe {bear, bell, bid, bull, buy, sell, stock, stop}. (Confronta questo con lo standard trie mostrato in Figura 9.1.) Oltre alla compressione ai nodi foglia, nota il nodo interno con etichetta "to" condivisa dalle parole "stock" e "stop".

I nodi di un compressed trie sono etichettati con delle stringhe, che sono sottostringhe delle stringhe nella collezione, piuttosto che con singoli caratteri. Il vantaggio di un compressed trie rispetto a uno standard trie è che il numero di nodi del compressed trie è proporzionale al numero di stringhe e non alla loro lunghezza totale.

Un compressed trie che memorizza una collezione S di s stringhe in un alfabeto di dimensione d ha le seguenti proprietà:

- Ciascun nodo interno di T ha almeno due figli e al massimo d figli.
- T ha s nodi foglia.
- Il numero di nodi di T è $O(s)$.

Il *Compressed Trie* offre un reale vantaggio quando viene utilizzato come struttura di indicizzazione ausiliaria per una collezione di stringhe già memorizzate in una struttura primaria.

Supponiamo, ad esempio, che la collezione S di stringhe sia un array di stringhe $S[0], S[1], \dots, S[s-1]$. Invece di memorizzare esplicitamente l'etichetta X di un nodo, la rappresentiamo implicitamente mediante una combinazione di tre interi $(i, j : k)$, tali che $X = S[i][j : k]$; cioè, X è la porzione di $S[i]$ costituita dai caratteri dalla posizione j fino a, ma non includendo, la posizione k .

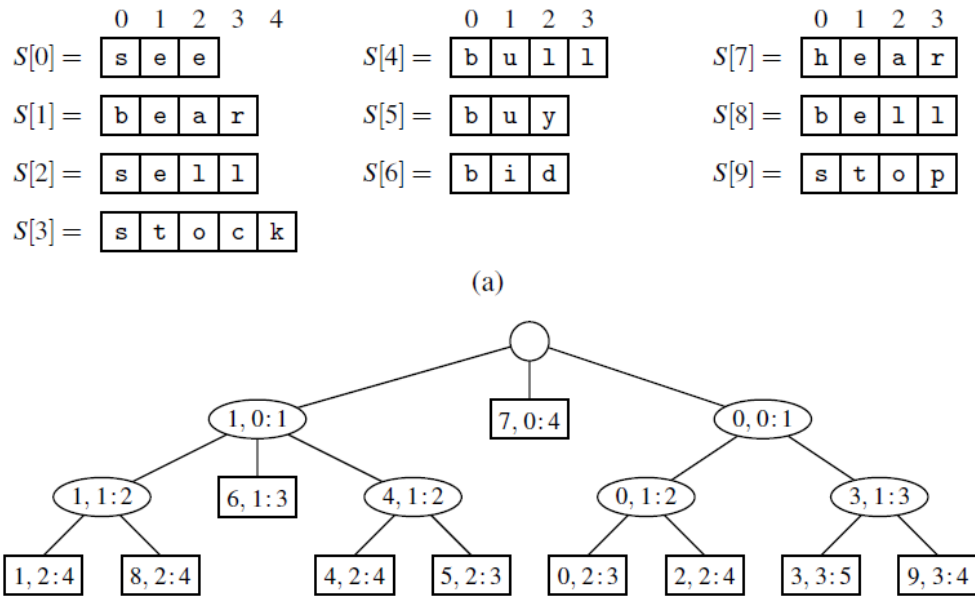


Figura 9.3: (a) Collezione S di stringhe memorizzata in un array. (b) Rappresentazione compatta del compressed trie per S .

In questo modo è possibile ridurre lo spazio totale per il trie stesso da $O(n)$ per lo standard trie a $O(s)$ per il compressed trie, dove n è la lunghezza totale delle stringhe in S e s è il numero di stringhe in S . Naturalmente, dobbiamo comunque memorizzare le diverse stringhe in S , ma riduciamo comunque lo spazio per il trie.

La ricerca in un compressed trie non è necessariamente più veloce che in un albero standard, poiché è ancora necessario confrontare ogni carattere del pattern desiderato con le etichette, potenzialmente multi-carattere, durante la traversata dei percorsi nel trie.

9.3 Suffix Tries

Un **Suffix Trie** (o *Suffix Tree*) per una stringa X è un trie che contiene tutti i suffissi di X .

Per un Suffix Trie, la rappresentazione compatta presentata nella sezione precedente può essere ulteriormente semplificata. In particolare, l'etichetta di ogni vertice è una coppia (j, k) che indica la stringa $X[j : k]$. Al fine di soddisfare la regola che nessun suffisso di X sia prefisso di un altro suffisso, possiamo aggiungere un carattere speciale, denotato con $\$$ (che non è nell'alfabeto originale Σ) alla fine di X (e quindi a ogni suffisso).

Al netto di tutto ciò, se la stringa X ha lunghezza n , costruiamo un trie per l'insieme di n stringhe $X[j : n]$, per $j = 0, \dots, n - 1$.

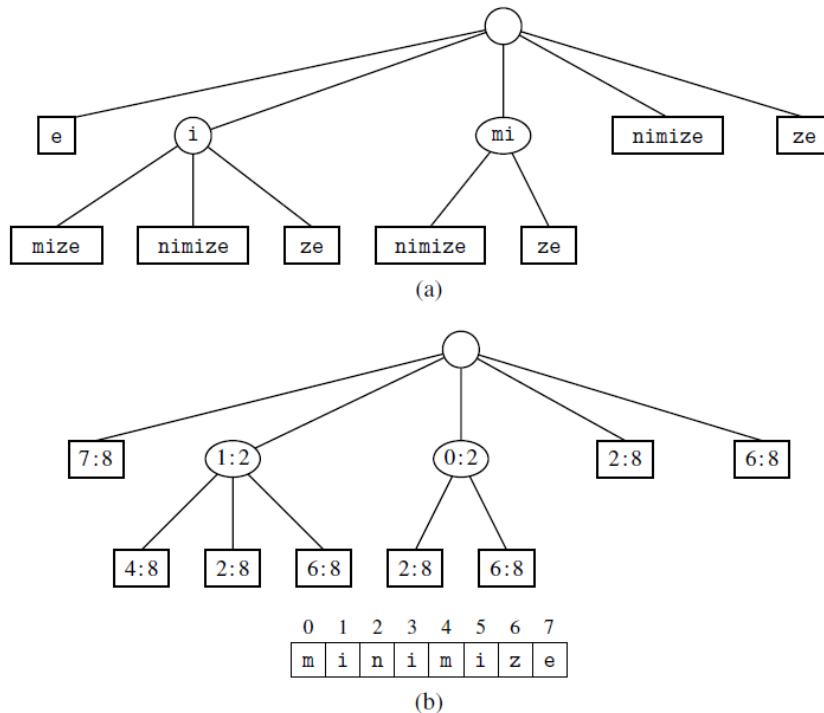


Figura 9.4: (a) Suffix trie T per la stringa $X = \text{"minimize"}$. (b) Rappresentazione compatta di T , dove la coppia $(j : k)$ denota la porzione $X[j : k]$ nella stringa di riferimento.

Per costruire un Suffix Trie possiamo procedere in modo simile alla costruzione di uno Standard Trie (compresso). In breve, si segue il cammino esistente finché coincide; dove il cammino finisce o diverge si crea una diramazione.

La lunghezza totale dei suffissi di una stringa X di lunghezza n è pari a:

$$1 + 2 + \dots + n = \frac{n(n+1)}{2} = O(n^2)$$

Nonostante ciò, il suffix trie può essere costruito in spazio $O(n)$ poiché molte porzioni dei suffissi condividono prefissi comuni. La costruzione di un Suffix Trie richiede un tempo $O(n^2)$, anche se in realtà esiste un algoritmo più complesso (che non approfondiremo) che costruisce un Suffix Trie in tempo $O(n)$.

9.4 Pattern Matching con Suffix Tries

Il Suffix Trie T per una stringa X può essere utilizzato per eseguire efficientemente query di pattern-matching sul testo X .

Supponiamo di voler cercare un pattern P di lunghezza m e sia D un testo di lunghezza n (di solito $n \gg m$). Siamo in grado di trovare un'occorrenza di P in D con $O(n)$ pre-processing del testo, $O(m)$ per la ricerca e uno spazio utilizzato $O(n)$.

- Se dobbiamo ricercare un nuovo pattern P' in D , abbiamo bisogno solamente di un tempo $O(m')$, dove m' è la lunghezza di P' .
- Questo metodo è più efficiente di *BM* o *KMP* quando il testo è fisso e disponibile per essere pre-elaborato in anticipo.

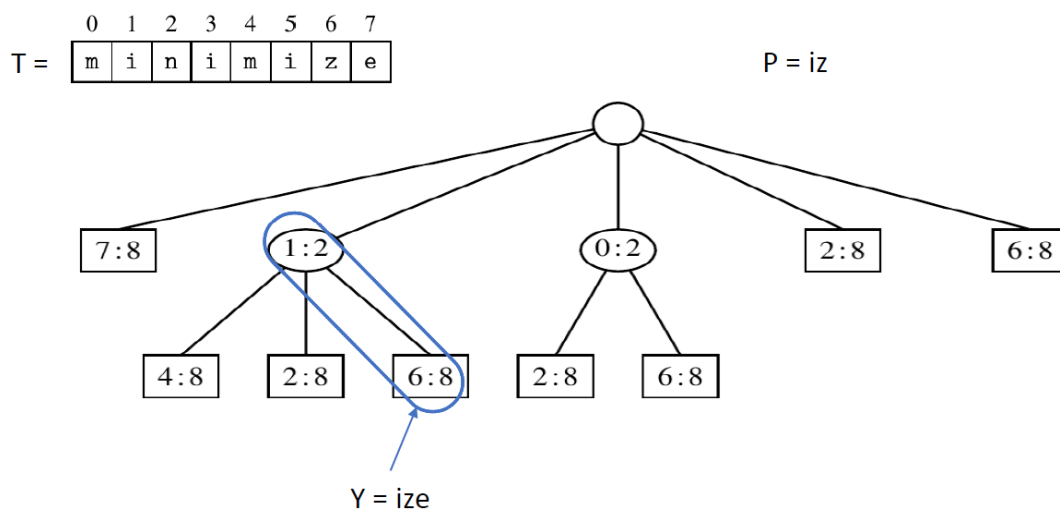


Figura 9.5: Il pattern viene trovato in posizione: $8 - \text{len}(Y) = 5$

Eseguiamo il pattern matching come segue:

- Costruiamo il Suffix Trie T per il testo D , il che può essere fatto in $O(n)$ con uno spazio $O(n)$.
- Confronta i caratteri di P lungo l'unico percorso corrispondente in T fino a quando:
 - Tutti i caratteri di P sono stati confrontati con successo (in questo caso, P si trova in D).
 - Non sono possibili ulteriori match (in questo caso, P non si trova in D).
- Se P è stato trovato, possiamo recuperare la posizione di P all'interno di D nel modo seguente:
 - Sia v il nodo in cui termina il confronto, con etichetta (j, k) .
 - Sia Y la stringa di lunghezza y corrispondente al percorso dalla radice a v .
 - L'indice del primo carattere di P in D è dato da $k - y$.

9.5 The Substring Problem

In questa tipologia di problema abbiamo a disposizione un insieme noto e fissato di stringhe che chiameremo **database dei pattern**. Successivamente ci viene fornita una stringa S per cui vogliamo trovare tutti i pattern nel database che contengono S come sottostringa¹.

Osservando attentamente questo problema notiamo che si tratta in effetti del problema duale del *pattern matching*, in cui abbiamo un testo fisso e cerchiamo un pattern variabile. Qui, invece, abbiamo un insieme fisso di pattern e cerchiamo una sottostringa che è variabile.

Supponiamo che la lunghezza totale delle stringhe nel database sia pari a m . Siccome normalmente questo valore m può essere molto grande, vogliamo limitare lo spazio utilizzato per memorizzare il database, limitare il tempo necessario per il pre-processing e limitare la fase di ricerca il più possibile.

Soluzione

La soluzione a questo problema sfrutta proprio i **Suffix Tries**, che come abbiamo visto sono capaci di rispettare i vincoli di spazio e tempo richiesti.

- Costruiamo un Suffix Trie T per l'insieme dei pattern nel database, che quindi conterrà tutti i suffissi di tutti i pattern. Questo richiede un tempo $O(m)$ e uno spazio $O(m)$.
- Ogni nodo v in T mantiene una lista di tutti i pattern che contengono la stringa rappresentata dal percorso dalla radice a v .
- Per cercare una stringa S di lunghezza n , seguiamo il percorso in T indicato dai caratteri di S . Se la ricerca termina con successo in un nodo v , allora tutti i pattern nella lista associata a v contengono S come sottostringa. Se il percorso si interrompe prima di raggiungere un nodo, allora nessun pattern nel database contiene S come sottostringa.
 - Se la ricerca ha successo, e v è un nodo foglia, la stringa S è proprio un pattern del database.
 - Se la ricerca ha successo, e v è un nodo interno, la lista associata a v contiene tutti i pattern che contengono S come sottostringa.
 - Se la ricerca fallisce in un nodo f , il percorso per cui si ha avuto un match (parziale) fino a quel punto rappresenta il prefisso più lungo di S che è anche una sottostringa di almeno un pattern nel database.
- La ricerca richiede un tempo $O(n)$, dove n è la lunghezza di S .

¹Un'applicazione di questo problema riguarda l'autocompletamento all'interno di un IDE. Nel database rientrano tutti i nomi delle classi, metodi e variabili definite dal programmatore, mentre la stringa S rappresenta quello che si sta digitando (es. "List"). Come risultato l'IDE mostra istantaneamente non solo ciò che inizia con "List", ma tutto ciò che lo contiene (es. ArrayList, LinkedList, ListListener).

9.6 Longest Common Substring: two strings

Ricordiamo che una sottostringa $S[i : j]$ di una stringa S è data dai caratteri $S[i], \dots, S[j - 1]$. Una stringa S è una **sottostringa comune** di due stringhe S_1 e S_2 se ci esistono degli indici i, j, i', j' tali che $S = S_1[i : j] = S_2[i' : j']$.

Il problema della *longest common substring* (LCS) consiste nel trovare la sottostringa comune più lunga, in questo caso tra due sole stringhe.

Soluzione

Siano m_1 e m_2 le lunghezze delle due stringhe S_1 e S_2 , e sia $m = m_1 + m_2$. La sottostringa comune più lunga tra due stringhe può essere trovata in tempo $O(m)$ e spazio $O(m)$ utilizzando un Suffix Trie, come segue:

- Costruiamo un Suffix Trie T che contiene entrambe le stringhe (e i loro suffissi) e memorizziamo in ogni nodo un'etichetta con uno dei valori $\{1, 2, 1/2\}$ a seconda che il suffisso provenga da S_1 , S_2 o entrambi.
 - Questo richiede un tempo $O(m)$ e uno spazio $O(m)$.
- Cerchiamo un nodo v di T con etichetta $1/2$ tale che la lunghezza della stringa corrispondente al percorso dalla radice a v sia massima.
 - Richiede di visitare al più tutti i nodi di T , quindi richiede un tempo $O(m)$.
- La stringa corrispondente al percorso dalla radice a v è la sottostringa comune più lunga tra S_1 e S_2 .

• $S_1 = \text{"alive"}$ $S_2 = \text{"cali"}$

• $D = \{e\$, ve\$, ive\$, live\$, alive\$, i\$, li\$, ali\$, cali\$\}$

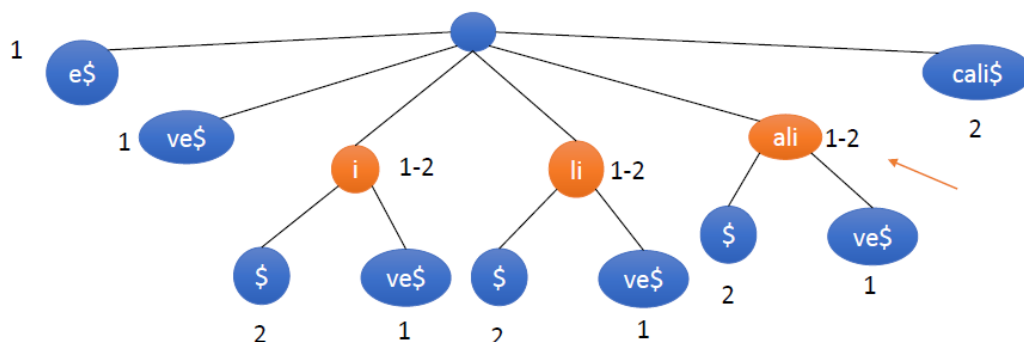


Figura 9.6: Il simbolo \$ viene utilizzato per fare in modo che nessuna stringa sia prefisso di un'altra all'interno di D .

9.7 Longest Common Substring: many strings

Generalizziamo ora il problema della *longest common substring* (LCS) a più di due stringhe. Supponiamo di avere K stringhe la cui lunghezza totale è m . Per ogni k compreso tra 2 e K , definiamo $l(k)$ come la lunghezza della sottostringa comune più lunga per almeno k delle K stringhe. Il problema della LCS per più stringhe consiste nel calcolare i valori $l(2), l(3), \dots, l(K)$ e le stringhe corrispondenti.

Esempio: Sia $S = \{ \text{sandollar, sandlot, handler, grand, pantry} \}$. In questo caso, $l(2) = 4$ (la sottostringa comune più lunga tra almeno due stringhe è *sand*), $l(3) = 3$ (la sottostringa comune più lunga tra almeno tre stringhe è *and*), $l(4) = 3$ (è ancora *and*) e $l(5) = 2$ (*an*).

Soluzione

- Costruiamo un **Suffix Tree generalizzato**² T per le K stringhe.
 - Ogni nodo foglia di T rappresenta un suffisso di una delle K stringhe e viene etichettato con l'indice della stringa a cui appartiene.
 - * Anche in questo caso, per garantire che nessuna stringa sia prefisso di un'altra, aggiungiamo un carattere speciale unico alla fine di ogni stringa.
 - Per ogni nodo interno v di T , calcoliamo $C(v)$ come il numero di identificatori di stringa distinti che compaiono alle foglie del sottoalbero radicato in v .
 - * Questa operazione è il **collo di bottiglia della soluzione**³ e può essere eseguita in tempo $O(K \cdot m)$.
 - Per ogni nodo v di T , calcoliamo la lunghezza della stringa corrispondente al percorso dalla radice a v .
- I valori di $l(k)$ possono essere calcolati facilmente visitando l'albero in tempo lineare:
 - Durante questa visita, costruiamo un vettore V in cui, per ogni valore di k da 2 a K , la cella $V(k)$ memorizza la massima string-depth trovata tra i nodi che hanno $C(v) = k$ (cioè i nodi condivisi da esattamente k stringhe).
 - Infine, calcoliamo i valori finali partendo da $k = K$ e andando a ritroso: impostiamo $l(k)$ uguale a $V(k)$ se questo è maggiore del valore successivo, altrimenti $l(k)$ eredita il valore di $l(k + 1)$ (questo serve a propagare la soluzione: se una stringa è condivisa da K elementi, vale anche come stringa condivisa da $K - 1$).

$$l(k) = \max(V(k), l(k + 1))$$

²Un *Suffix Tree generalizzato* rappresenta tutti i suffissi di un insieme di stringhe, anziché di una singola stringa; Nota: è stato utilizzato nelle soluzioni dei paragrafi precedenti ai problemi proposti.

³La soluzione proposta viene eseguita in tempo $O(K \cdot m)$, ma esiste una soluzione più avanzata in grado di raggiungere tempo lineare $O(n)$.

Capitolo 10

Greedy Algorithms

L'*approccio Greedy* è un paradigma di progettazione di algoritmi utilizzato per risolvere **problemi di ottimizzazione**. Un algoritmo greedy funziona bene quando una soluzione ottimale può essere raggiunta attraverso una *serie di scelte locali ottimali*. A partire da una *configurazione iniziale* (soluzione parziale), l'algoritmo effettua una scelta (la migliore possibile localmente) in una classe di possibili opzioni, e ripete lo stesso procedimento aggiornando di volta in volta la configurazione corrente, fino a raggiungere una soluzione completa.

In particolare, non possiamo utilizzare questo approccio per tutti i problemi di ottimizzazioni. Diciamo che un problema di ottimizzazione ammette una **soluzione greedy** se il problema soddisfa la proprietà:

- **Greedy-choice property**: la soluzione completa ottimale può sempre essere raggiunta effettuando una serie di progressi, che rappresentano delle scelte locali ottimali, a partire da una configurazione iniziale.

Ad ogni passo, la "scelta" deve essere:

- **Realizzabile**: deve soddisfare i vincoli dettati dal problema.
- **Localmente ottima**: deve essere la scelta migliore tra tutte le scelte possibili (e realizzabili) in quel momento.
 - Non è necessario che questa scelta sia ottima rispetto alla soluzione globale.
 - Da notare che non è sempre detto che effettuare scelte localmente ottimali porti ad una soluzione globale ottimale. In questi casi, l'algoritmo greedy non funziona correttamente.
- **Irrevocabile**: una volta effettuata una scelta, questa non può essere modificata in futuro.

Un algoritmo greedy costruisce una soluzione in piccoli passi successivi, scegliendo ad ogni passo una decisione che riguarda esclusivamente la configurazione corrente. Spesso si possono progettare molti algoritmi greedy diversi per lo stesso problema, ognuno dei quali ottimizza localmente e in modo incrementale qualche misura diversa nel suo cammino verso una soluzione. È facile inventare algoritmi greedy per quasi tutti i problemi; trovare i casi in cui funzionano bene, e dimostrare che effettivamente funzionano bene, è la sfida interessante.

10.1 Modello generale

Indichiamo con S la soluzione parziale corrente, e con P il sotto problema che rimane da risolvere. Inizialmente, S è vuota e P coincide con il problema originale. Ad ogni passo, l'algoritmo greedy:

-
1. Generate all candidate choices as list L for current sub-problem P .
 2. **While** (L is not empty **or** other finish condition is not met)
 3. Compute the feasible value of each choice in L ;
 4. Modify S and P by taking choice with the highest feasible value;
 5. Update L according to S and P ;
 6. **Endwhile**
 7. **Return** the resulting complete solution.
-

Sia A l'insieme di tutti i possibili elementi del problema. Ad ogni step, l'algoritmo greedy mantiene una partizione $\langle X, Y, W \rangle$ di A , dove:

- X : insieme degli elementi selezionati fino a quel momento (soluzione parziale corrente).
- Y : insieme degli elementi valutati ma non ancora selezionati.
- W : insieme degli elementi non ancora valutati.

Inizialmente, $W = A$ e $X = Y = \emptyset$. Alla fine dell'algoritmo, X conterrà la soluzione completa, $Y = A/X$ contiene tutti gli elementi di A che non sono stati selezionati, e $W = \emptyset$.

Gli algoritmi greedy sono spesso **estremamente intuitivi** al tal punto da rappresentare la soluzione più semplice e naturale per molti problemi. Sono anche molto **efficienti**, con complessità temporali che vanno da $O(n \log n)$ a $O(n)$, dove n è la dimensione dell'input. Tuttavia, la loro efficienza dipende fortemente dalla natura del problema e dalla struttura dei dati utilizzati per implementare l'algoritmo.

La parte più complicata nella progettazione di un algoritmo greedy è la **dimostrazione della correttezza** dell'algoritmo, che spesso richiede tecniche di dimostrazione specifiche per ogni problema. Le tecniche più comuni sono:

- **Greedy stays ahead**: si dimostra che, ad ogni passo dell'algoritmo, la soluzione parziale costruita dall'algoritmo greedy è almeno altrettanto buona quanto qualsiasi altra soluzione parziale possibile.
- **Exchange**: si dimostra che qualsiasi soluzione ottimale può essere trasformata nella soluzione prodotta dall'algoritmo greedy attraverso una serie di scambi di elementi, senza peggiorare la qualità della soluzione.

10.2 Coin Change: The Cashier Algorithm

Il problema del *Coin Change* (cambio di monete) consiste nel trovare il numero minimo di monete necessarie per rappresentare un dato importo di denaro, utilizzando un insieme predefinito di tagli di monete.

Il problema prende in input un importo R (in centesimi di euro) e richiede in output il numero minimo di monete necessarie per rappresentare tale importo, utilizzando solo, ad esempio, monete di taglio 2€, 1€, 50c, 20c, 10c, 5c, 2c e 1c. Si assume di avere a disposizione un numero illimitato di monete per ogni taglio.

```
Sort coins denominations by value:  $c[1] > c[2] > \dots > c[k]$ 
1.  $S = \{\}$  // inizializza soluzione parziale (insieme vuoto)
2. while ( $x \neq 0$ ) { // finché l'importo da cambiare non è zero
3.   let  $k$  be the largest integer such that  $c[k] \leq x$ 
4.   if ( $k = 0$ )
5.     return "no solution found"
6.    $x = x - c[k]$  // riduci l'importo rimanente
7.    $S = \{S, c[k]\}$  // aggiungi  $c[k]$  alla soluzione
8. }
9. return  $S$  // restituisci la soluzione completa
```

```
1 def coin_change(amount_rem):
2     coin_combinations = [50, 20, 10, 5, 2, 1] # Valori in centesimi
3     result = []
4
5     for coin in coin_combinations:
6         coin_count = amount_rem // coin # Divisione intera per ottenere il
7             numero massimo di monete di questo taglio
8         result += [coin] * coin_count # Aggiungi le monete alla soluzione
9         amount_rem -= coin * coin_count # Aggiorna l'importo rimanente
10        if amount_rem == 0:
11            return result # Restituisci la soluzione completa
12
13    if amount_rem > 0: # Se non e' stato possibile coprire l'importo
14        return "No solution found"
```

In generale, questo algoritmo restituisce sempre una soluzione (non necessariamente ottimale) se $c[k] = 1$ (cioè se esiste una moneta di taglio unitario). Tuttavia, l'algoritmo è ottimale solo per alcuni sistemi di monete specifici, come quello europeo, i cosiddetti **sistemi canonici**.

Un insieme di monete è un **sistema canonico** se l'algoritmo del cassiere fornisce la soluzione ottimale per ogni importo R da cambiare. In generale, non tutti i sistemi di monete sono canonici e determinare se un sistema di monete è canonico può essere un problema complesso.

10.3 Scheduling

Il problema dello *scheduling* riguarda la pianificazione di un insieme di attività o compiti su risorse limitate, come macchine, lavoratori o tempo. L'obiettivo è ottimizzare l'uso delle risorse per massimizzare l'efficienza, minimizzare i tempi di completamento o soddisfare altre metriche di performance. Le attività possono avere vincoli di tempo, priorità diverse e requisiti specifici, rendendo il problema complesso e variegato.

In generale, dato un insieme di n attività ognuna con un *tempo di inizio* s_i e un *tempo di fine* f_i (con $s_i < f_i$), si chiede di:

- **Task Scheduling:** Minimizzare il numero di macchine (in generale risorse) necessarie per completare tutte le attività senza sovrapposizioni.
- **Interval Scheduling:** Massimizzare il numero di attività che possono essere completate senza sovrapposizioni, utilizzando una singola macchina.

10.3.1 Task Scheduling

In questa tipologia di problemi, abbiamo delle risorse identiche limitate (useremo il termine "macchine" per semplicità) e desideriamo assegnare un insieme di attività a queste macchine in modo tale che nessuna attività si sovrapponga temporalmente su una stessa macchina. L'obiettivo è minimizzare il numero di macchine utilizzate per completare tutte le attività.

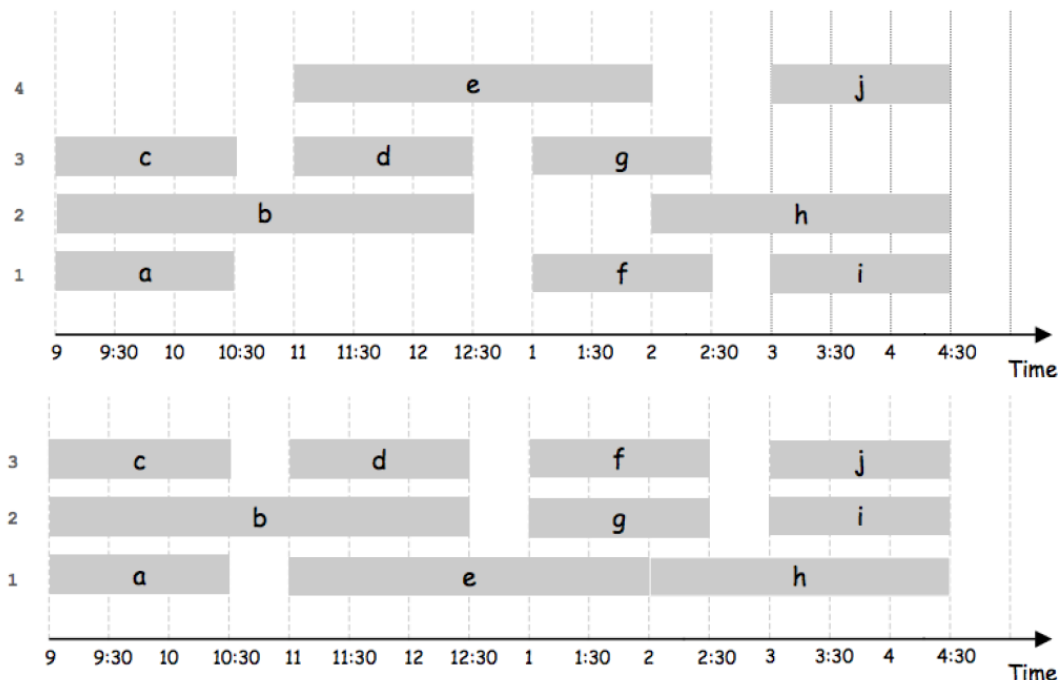


Figura 10.1: (a) Una istanza del problema di Task Scheduling, in cui le risorse sono delle aule e le attività sono lezioni da assegnare. (b) Una soluzione in cui tutte le attività sono pianificate utilizzando tre risorse: ogni riga rappresenta un insieme di attività che possono essere tutte pianificate su una singola risorsa.

Soluzione Greedy

Una soluzione greedy molto intuitiva per questo problema consiste nell'ordinare le attività in ordine crescente di tempo di inizio. Successivamente, si itera attraverso l'elenco delle attività e si assegna ciascuna attività alla prima macchina disponibile che non abbia conflitti di orario con le attività già assegnate. Se nessuna macchina è disponibile, si aggiunge una nuova macchina.

```
Sort intervals by starting time so that  $s[1] \leq s[2] \leq \dots \leq s[n]$ 
1.  $d = 0$  // inizializza il numero di macchine
2. for  $j = 1$  to  $n$  {
3.     if (task  $j$  is compatible with some machine  $k$ )
4.         schedule task  $j$  on machine  $k$ 
5.     else
6.         allocate a new machine  $d + 1$ 
7.         schedule task  $j$  on machine  $d + 1$ 
8.          $d = d + 1$ 
9. }
10. return  $d$  // restituisci il numero di macchine utilizzate
```

L'algoritmo ha complessità complessiva $O(n \log n)$, dominata dall'ordinamento iniziale delle attività per tempo di inizio.

Nella fase di assegnazione si utilizza una *Min-Priority Queue* (min-heap) contenente, per ciascuna macchina, il tempo di fine dell'ultimo task eseguito. In questo modo la radice del heap rappresenta sempre la macchina che si libera per prima.

Per ogni attività j , processata in ordine crescente di $s[j]$, è sufficiente confrontare $s[j]$ con il minimo del heap:

- se $s[j] \geq f_{\min}$, la macchina si è liberata: si estrae il minimo e si inserisce il nuovo tempo di fine ($O(\log n)$);
- altrimenti, tutte le macchine sono occupate: si alloca una nuova macchina e si inserisce il suo tempo di fine nel heap ($O(\log n)$).

Poiché ogni attività effettua al più un'estrazione e un'inserzione nel heap, la fase di scansione richiede $O(n \log n)$, in linea con il costo dell'ordinamento.

10.3.2 Interval Scheduling

In questa tipologia di problemi, abbiamo una singola macchina e desideriamo selezionare un sottoinsieme di attività da eseguire su questa macchina in modo tale che nessuna attività si sovrapponga con un'altra. L'obiettivo è massimizzare il numero di attività completate senza sovrapposizioni.

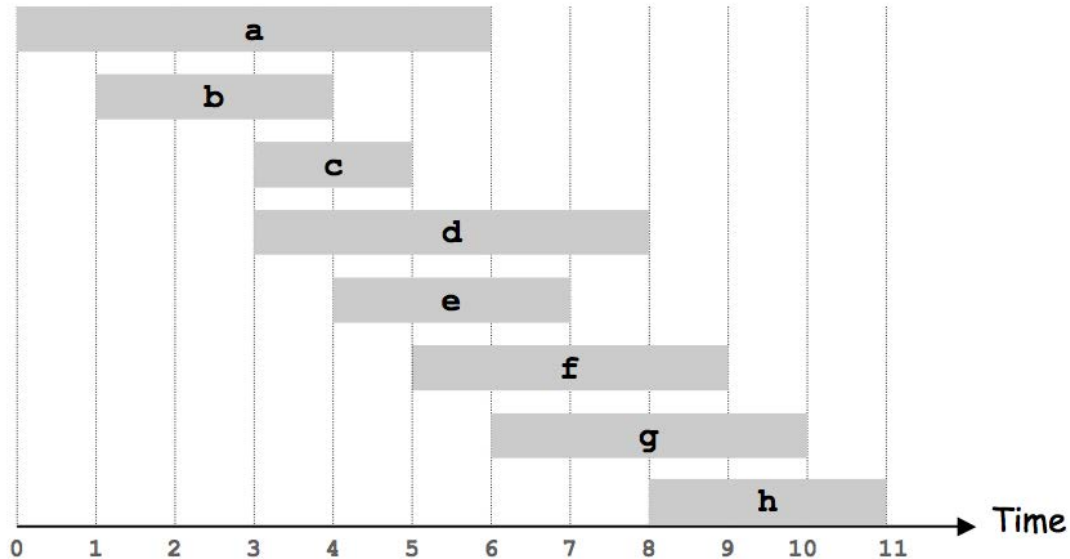


Figura 10.2: Una istanza del problema di Interval Scheduling in cui si hanno 8 attività da collocare su una singola macchina.

Soluzione Greedy

Nel cercare una soluzione ottimale per questo problema, possiamo considerare diverse strategie greedy intuitive, e selezionare la prima attività compatibile con quelle già scelte. Alcune possibili strategie includono:

- Ordinamento per tempo di inizio crescente. Seleziono le attività compatibili in ordine di inizio $s[i]$.
- **Ordinamento per tempo di fine crescente.** Seleziono le attività compatibili in ordine di fine $f[i]$.
- Ordinamento per durata crescente. Seleziono le attività compatibili in ordine di durata $f[i] - s[i]$.
- Ordinamento per numero di conflitti crescente. Per ogni attività i , conto il numero c_i di altre attività che non sono compatibili con i (cioè che si sovrappongono temporalmente con i). Seleziono le attività compatibili in ordine crescente di c_i .



Figura 10.3: Controesempi per le strategie greedy di Interval Scheduling basate su (a) tempo di inizio crescente, (b) durata crescente, (c) numero di conflitti crescente.

Tra queste strategie, solo l'**ordinamento per tempo di fine crescente** garantisce una **soluzione ottimale** per il problema di *Interval Scheduling*. Le altre strategie possono portare a soluzioni subottimali, come mostrato nei controesempi della Figura 10.3. Dal momento che per confutare una strategia greedy è sufficiente trovare un singolo controesempio, possiamo concludere che l'unica strategia ottimale tra quelle elencate è quella basata sul tempo di fine crescente (ci siamo limitati a confutare le strategie mostrate in Figura 10.3, ma per essere certi che la strategia basata sul tempo di fine crescente sia ottimale, sarebbe necessario dimostrarne la correttezza).

```

Sort intervals by finishing time so that  $f[1] < f[2] < \dots < f[n]$ 
1.  $n = s.length$  // number of activities
2.  $A = \{a[1]\}$  // initialize solution with first activity
3.  $k = 1$  // index of last activity added to A
4. for  $m = 2$  to  $n$  {
5.     if ( $s[m] \geq f[k]$ )
6.          $A = \{A, a[m]\}$  // add activity  $a[m]$  to A
7.          $k = m$  // update index of last activity added to A
8. }
9. return A // return the set of accepted activities

```

L'algoritmo ha complessità complessiva $O(n \log n)$, dominata dall'ordinamento iniziale delle attività per tempo di fine. L'algoritmo sfrutta una semplice *scansione lineare* delle attività ordinate, selezionando ogni volta la prima attività compatibile con l'ultima selezionata.

10.4 Fractional Knapsack

Il problema del *Fractional Knapsack* consiste nel massimizzare il valore totale degli elementi inseriti in uno "zaino" (knapsack) con una capacità limitata, permettendo di prendere frazioni degli oggetti.

Data una sequenza S di n elementi, ognuno dei quali ha un **peso** p_i e un **valore** v_i (entrambi interi positivi), e data una **capacità massima** P , l'obiettivo è selezionare una combinazione di elementi di S (o frazioni di essi) in modo tale che il peso totale non superi P e il valore totale sia massimizzato.

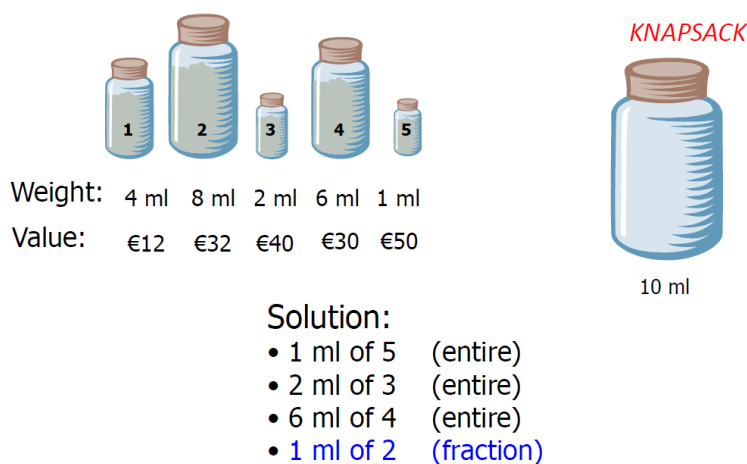


Figura 10.4: Esempio di soluzione ottimale per un'istanza del problema di Fractional Knapsack con capacità $P = 10\text{ml}$.

Soluzione Greedy

Una soluzione greedy per questo problema consiste nell'ordinare gli elementi in ordine decrescente di *valore per unità di peso*, dato dal rapporto v_i/p_i . Questo valore rappresenta il **profitto** ottenuto per ogni unità di peso dell'elemento i . Successivamente, gli elementi vengono inseriti per intero nello zaino in questo ordine, fino a quando non si raggiunge un elemento j , detto *elemento critico*, che ha un peso maggiore della capacità massima residua dello zaino. A questo punto, si inserisce solo la frazione di j che può essere contenuta nello zaino, e l'algoritmo termina, restituendo la soluzione completa.

Input: Set S of items i with weight p_i and value v_i all positive

Knapsack capacity $P > 0$

Output: Amount x_i of i that maximizes the total benefit without exceeding in the capacity

```
1. FractionalKnapsack( $S, P$ ):
2.   for each  $i$  in  $S$  do
3.      $x_i = 0$            // initially the knapsack is empty
4.      $b_i = v_i / w_i$        // the benefit of item  $i$ 
5.      $p = P$                // remaining capacity in knapsack (initially  $p = P$ )
6.   while  $p > 0$  do
7.     remove from  $S$  an item of a maximal benefit    // greedy choice
8.      $x_i = \min(p_i, p)$     // can't carry more than  $p$ 
9.      $p = p - x_i$           // update remaining capacity
```

L'algoritmo ha una complessità complessiva di $O(n \log n)$, dominata dall'ordinamento iniziale degli elementi in base al loro valore per unità di peso. La fase di selezione degli elementi richiede invece $O(n)$, poiché ogni elemento viene considerato una sola volta.

Come già accennato, progettare un algoritmo greedy è spesso semplice ed intuitivo; d'altra parte bisogna essere consapevoli che non tutti i problemi di ottimizzazione ammettono una soluzione *ottimale* basata su questo approccio.

Ad esempio, il problema del *0-1 Knapsack*, in cui gli elementi non possono essere frazionati (cioè devono essere presi per intero o lasciati fuori dallo zaino), non ammette una soluzione ottimale basata su un algoritmo greedy. In questo caso, è necessario utilizzare tecniche più complesse, come la programmazione dinamica, per trovare la soluzione ottimale.

I casi in cui un algoritmo greedy fallisce nel trovare la soluzione ottimale sono spesso caratterizzati dalla presenza di **scelte locali che non portano a una soluzione globale ottimale**, e trovare delle particolari istanze del problema che dimostrino questo fatto non è sempre banale. Inoltre, dimostrare che un algoritmo greedy è effettivamente ottimale per un dato problema può richiedere tecniche di dimostrazione specifiche e non sempre intuitive, anche se spesso si basano su idee semplici come *greedy stays ahead* e *exchange* già menzionate in precedenza.

10.5 Text Compression

La *compressione del testo* è un'operazione che mira a ridurre la quantità di spazio necessario per memorizzare o trasmettere dati testuali. Questo processo è fondamentale in molte applicazioni, come l'archiviazione di file, la trasmissione di dati su reti a larghezza di banda limitata e l'ottimizzazione delle prestazioni dei sistemi informatici. Una tecnica comune per la compressione del testo è l'uso di algoritmi di codifica che sfruttano la ridondanza nei dati per rappresentarli in modo più efficiente.

Data una stringa X , vogliamo codificare in modo efficiente X in una stringa binaria Y più piccola (usando solo i caratteri 0 e 1). Uno dei modi possibili per risolvere questo problema è utilizzare la **codifica di Huffman** (Huffman code).

10.5.1 Codifica di Huffman

Questa soluzione si basa sul calcolo della frequenza f_c di ogni carattere c che compare in X . L'idea è di assegnare codici binari più brevi ai caratteri che appaiono più frequentemente e codici più lunghi ai caratteri meno frequenti. In questo modo, la lunghezza totale della stringa codificata Y sarà minimizzata.

- Una **code map** (mappa di codifica) associa ad ogni carattere un corrispondente codice binario, detto *code-word*.
- Un **prefix code** (codice prefisso) è un sistema di codifica binario in cui nessuna code-word è il prefisso di un'altra code-word. Questo garantisce che la decodifica sia univoca e non ambigua.
- Un codice prefisso può essere rappresentato da un **encoding tree** (albero di codifica).
- Un *encoding tree* è un albero binario in cui:
 - Ogni foglia rappresenta un carattere.
 - La code-word associata ad un carattere è data dal percorso dalla radice alla foglia corrispondente che memorizza tale carattere.
 - Per ciascun nodo, il figlio sinistro rappresenta il bit 0 e il figlio destro rappresenta il bit 1.

Data una stringa X , vogliamo cercare un codice prefisso (e quindi un encoding tree) per i caratteri di X tale che la codifica binaria di X sia la più corta possibile.

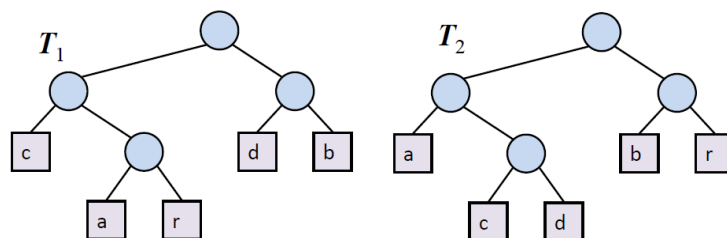


Figura 10.5: Esempio con $X = \text{"abracadabra"}$. L'encoding tree T_1 codifica X in 29 bit, mentre l'encoding tree T_2 codifica X in 24 bit, per cui la codifica di Huffman restituisce T_2 come soluzione ottimale.

Soluzione Greedy: l'algoritmo di Huffman

L'algoritmo di codifica di Huffman inizia con ciascuno dei d caratteri distinti della stringa X da codificare ponendoli come nodo radice di un albero binario a nodo singolo. L'algoritmo procede in una serie di passaggi. In ogni passaggio, l'algoritmo prende i due alberi binari con le frequenze più piccole e li unisce in un unico albero binario. Ripete questo processo fino a quando non rimane un solo albero.

Ogni iterazione del ciclo `while` nell'algoritmo di Huffman può essere implementata in tempo $O(\log d)$ utilizzando una *coda di priorità* Q rappresentata con un *heap*. Inoltre, ogni iterazione estrae due nodi da Q e ne aggiunge uno, un processo che verrà ripetuto $d - 1$ volte prima che rimanga esattamente un nodo in Q . Pertanto, questo algoritmo funziona in tempo $O(n + d \log d)$, dove n è la lunghezza della stringa X e d è il numero di caratteri distinti in X , e considerando che il calcolo delle frequenze richiede $O(n)$ tempo.

Sebbene una giustificazione completa della correttezza di questo algoritmo sia al di fuori del nostro ambito qui, notiamo che la sua intuizione deriva da una semplice idea: qualsiasi codice ottimale può essere convertito in un codice ottimale in cui le code-word per i due caratteri a frequenza più bassa, a e b , differiscono solo nel loro ultimo bit. Reiterando il ragionamento per una sequenza in cui a e b sono sostituiti da un unico carattere c , si ottiene quanto segue:

Input: String X of length n with d distinct characters

Output: Encoding tree for X

```
1. Huffman( $X$ ):
2.   Compute the frequency  $f(c)$  of each character  $c$  of  $X$ 
3.   Initialize a priority queue  $Q$ 
4.   for each character  $c$  in  $X$  do
5.       Create a single-node binary tree  $T$  storing  $c$ 
6.       Insert  $T$  into  $Q$  with key  $f(c)$ 
7.   while  $\text{len}(Q) > 1$  do
8.        $(f_1, T_1) = Q.\text{remove\_min}()$ 
9.        $(f_2, T_2) = Q.\text{remove\_min}()$ 
10.      Create a new binary tree  $T$  with left subtree  $T_1$  and
        right subtree  $T_2$ 
11.      Insert  $T$  into  $Q$  with key  $f_1 + f_2$ 
12.   $(f, T) = Q.\text{remove\_min}()$ 
13.  return tree  $T$ 
```

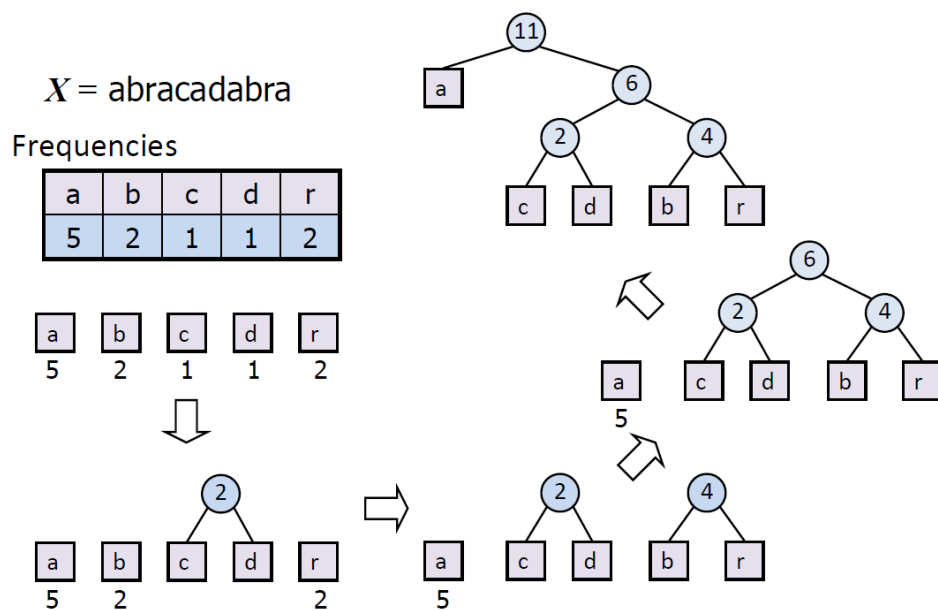


Figura 10.6: Esempio di esecuzione dell'algoritmo di Huffman per la stringa $X = \text{"abracadabra"}$. Ipotizzando che ciascun carattere sia rappresentato con 8 bit, la codifica standard richiederebbe $11 \times 8 = 88$ bit. La codifica di Huffman riduce questo numero a 23 bit, risparmiando così il 73.86% di spazio.

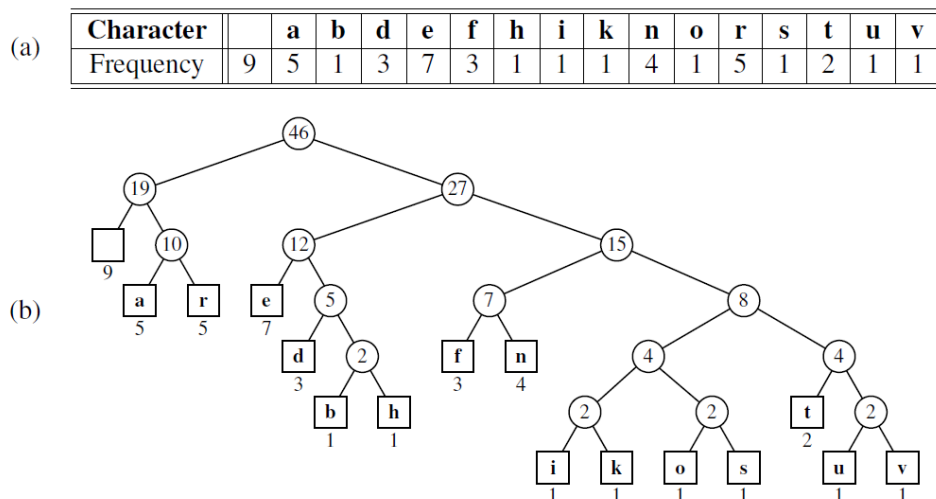


Figura 10.7: Esempio di esecuzione dell'algoritmo di Huffman per la stringa $X = \text{"a fast runner need never be afraid of the dark"}$. Ipotizzando che ciascun carattere (compreso lo spazio vuoto) sia rappresentato con 8 bit, la codifica standard richiederebbe $46 \times 8 = 368$ bit. La codifica di Huffman riduce questo numero a 165 bit, risparmiando così il 55.16% di spazio.

Capitolo 11

Dynamic Programming

La *programmazione dinamica* è una tecnica di progettazione di algoritmi per la risoluzione di problemi di ottimizzazione (così come l'approccio Greedy discusso nel capitolo precedente).

Questa tecnica è simile alla tecnica *divide-et-impera*, e proprio per questo motivo può essere applicata a una vasta gamma di problemi differenti. La programmazione dinamica può spesso essere utilizzata per trasformare problemi che sembrano richiedere un tempo esponenziale in algoritmi che li risolvono in tempo polinomiale. Inoltre, gli algoritmi che risultano dall'applicazione della tecnica di programmazione dinamica sono solitamente piuttosto semplici a livello concettuale.

11.1 Modello generale

Quando si progetta un algoritmo di programmazione dinamica è importante seguire in ordine i seguenti passi:

1. Definire i **sotto-problemi**.
2. Definire come la soluzione ottimale può essere ottenuta dalle soluzioni ottimali dei sotto-problemi ... e, ricorsivamente, come la soluzione ottimale di un sotto-problema può essere ottenuta dalla soluzione ottimale dei suoi sotto-problemi.
3. Descrivere la soluzione ottimale attraverso un'**equazione caratteristica**.
 - Questa relazione lega la soluzione globale con le soluzioni dei sotto-problemi, e definisce inoltre i *casi base* (**boundary conditions**), ovvero quei sotto-problemi le cui soluzioni sono banali e immediate, fondamentali per risolvere i sotto-problemi più grandi.

È fondamentale scegliere i sotto-problemi in modo "intelligente", così come vedremo nei prossimi esempi, in modo da facilitare la definizione dell'equazione caratteristica e la risoluzione dei sotto-problemi stessi.

La programmazione dinamica è per alcuni aspetti simile alla tecnica *divide-et-impera*. Sebbene entrambe le tecniche suddividano il problema originale in porzioni più piccole, la differenza sostanziale risiede nella relazione tra questi sotto-problemi. Nel paradigma *divide-et-impera*, i sotto-problemi sono **indipendenti** (o disgiunti): la soluzione di un ramo non influenza né è necessaria per la risoluzione degli altri. Al contrario, la *programmazione dinamica* è applicabile quando i sotto-problemi sono **sovrapposti** (*overlapping subproblems*), ovvero quando condividono a loro volta dei sotto-problemi comuni.

Mentre un approccio *divide-et-impera* puro ricalcolerebbe più volte la soluzione per lo stesso sotto-problema condiviso (portando spesso a una complessità esponenziale), la programmazione dinamica risolve ogni sotto-problema una sola volta e ne memorizza il risultato (*memoization* o tabulazione) per riutilizzarlo in futuro, garantendo così l'efficienza polinomiale.

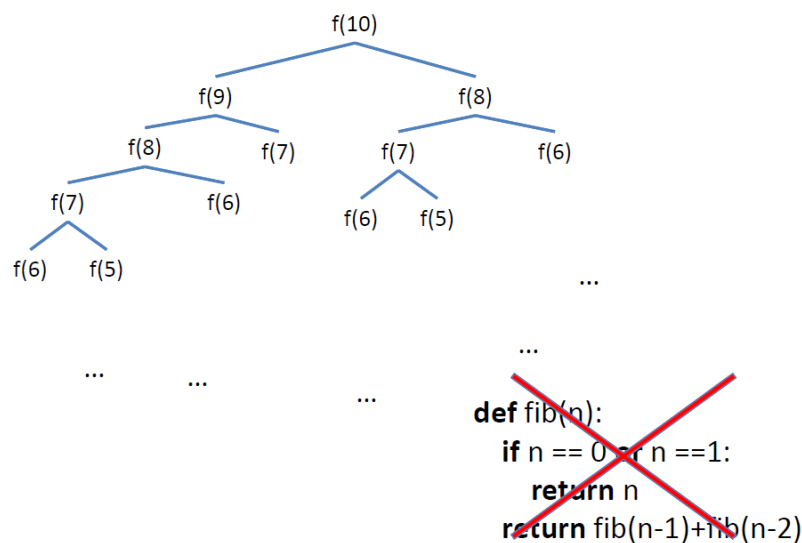


Figura 11.1: Esempio di risoluzione del problema della *sequenza di Fibonacci* con divide-et-impera. Questa soluzione non è ottimale dal punto di vista computazionale, in quanto uno stesso sotto-problema viene risolto più volte (ad esempio, in figura $f(6)$ viene calcolato 4 volte).

Una possibile soluzione che sfrutta la programmazione dinamica:

```

1 def fibonacci(n):
2     # Gestione caso base immediato
3     if n <= 1:
4         return n
5     # Creazione della tabella (array) per memorizzare i risultati
6     # Inizializzata a 0, dimensione n+1 per ospitare l'indice n
7     table = [0] * (n + 1)
8     # Impostazione dei casi base noti
9     table[0] = 0
10    table[1] = 1
11    # Riempimento della tabella dal basso verso l'alto
12    for i in range(2, n + 1):
13        table[i] = table[i-1] + table[i-2]
14    return table[n]

```

11.2 Longest Common Subsequence (LCS)

Una *sottosequenza* di una stringa $x_0x_1x_2 \dots x_{n-1}$ è una stringa $x_{i_0}x_{i_1} \dots x_{i_k}$, dove $i_j < i_{j+1}$. In altre parole, una sottosequenza si ottiene eliminando alcuni caratteri dalla stringa originale senza cambiare l'ordine dei caratteri rimanenti.

Da notare che è diverso dal concetto di *sottostringa*: una sottostringa è una sequenza contigua di caratteri all'interno della stringa originale, mentre una sottosequenza può essere formata da caratteri non contigui. Per cui, una sottostringa è sempre una sottosequenza, ma non viceversa.

Esempio : Data la stringa "AGGTAB", alcune delle sue sottosequenze sono "GTA", "ATAB", "GAB", mentre alcune delle sue sottostringhe sono "AGG", "GGTA", "TAB".

Uno dei problemi classici che può essere risolto in modo efficiente tramite la programmazione dinamica è il problema della *Longest Common Subsequence* (LCS), ovvero la ricerca della sottosequenza comune più lunga tra due sequenze date.

- Date due stringhe $X = x_0x_1 \dots x_{n-1}$ e $Y = y_0y_1 \dots y_{m-1}$ su di un alfabeto Σ , trovare la stringa più lunga che è una sottosequenza di entrambe le stringhe.

Soluzione Brute-Force

Tutte le possibili sottosequenze di una stringa X di lunghezza n sono 2^n . Quindi, un approccio brute-force per risolvere il problema LCS sarebbe generare tutte le sottosequenze di X , e per ognuna di esse verificare se è anche una sottosequenza di Y , tenendo traccia della più lunga trovata. Questo approccio ha una complessità temporale esponenziale di $O(2^n \cdot m)$, dove m è la lunghezza della stringa Y .

Soluzione Dynamic Programming

Sia $L_{n,m}$ la lunghezza della LCS tra le stringhe $X = x_0x_1 \dots x_{n-1}$ e $Y = y_0y_1 \dots y_{m-1}$. $L_{n,m}$ rappresenta quindi la soluzione ottimale del problema.

Scomposizione in sotto-problemi

Sia $L_{j,k}$ la lunghezza della LCS tra i prefissi $x_0x_1 \dots x_{j-1}$ e $y_0y_1 \dots y_{k-1}$, con $0 \leq j \leq n$ e $0 \leq k \leq m$. $L_{j,k}$ rappresenta quindi la soluzione ottimale al sotto-problema che considera solo i primi j caratteri di X e i primi k caratteri di Y .

Osserviamo l'ultimo carattere di entrambe le stringhe considerate nello specifico sotto-problema, e da qui ricaviamo le due **equazioni caratteristiche** che ci permettono di esprimere $L_{j,k}$ in funzione dei sotto-problemi più piccoli:

- Se $x_{j-1} = y_{k-1}$, allora questo carattere fa parte della LCS, e possiamo scrivere:

$$L_{j,k} = 1 + L_{j-1,k-1}$$

- Se $x_{j-1} \neq y_{k-1}$, allora l'ultimo carattere di almeno una delle due stringhe non fa parte della LCS, e possiamo scrivere:

$$L_{j,k} = \max(L_{j-1,k}, L_{j,k-1})$$

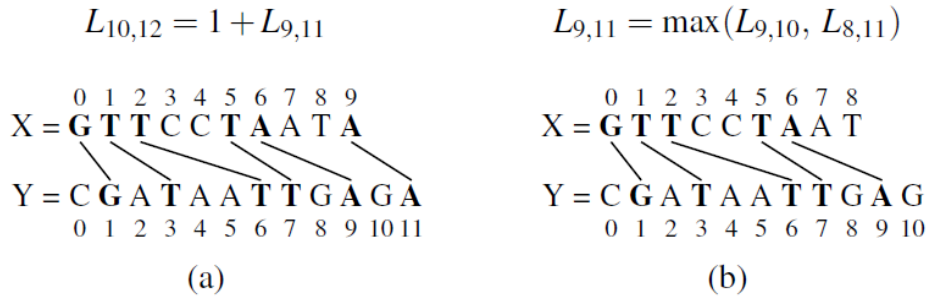


Figura 11.2: Rappresentazione grafica delle equazioni caratteristiche per il calcolo di $L_{j,k}$.

Boundary Conditions

Il caso base si verifica quando una delle due stringhe è vuota, ovvero quando $j = 0$ o $k = 0$. In questi casi, la LCS è anch'essa vuota, quindi:

$$L_{0,k} = 0 \quad \text{per } 0 \leq k \leq m$$

$$L_{j,0} = 0 \quad \text{per } 0 \leq j \leq n$$

Notiamo che la soluzione $L_{j,k}$ appare nella computazione di:

$$L_{j+1,k} \quad L_{j,k+1} \quad L_{j+1,k+1}$$

per cui i sotto-problemi si sovrappongono, rendendo la programmazione dinamica una tecnica adatta per risolvere questo problema. Quindi, anziché utilizzare un approccio ricorsivo che ricalcola più volte gli stessi sotto-problemi (divide-et-impera), possiamo utilizzare una tabella bidimensionale per memorizzare i risultati dei sotto-problemi già calcolati.

Calcolo della tabella

La tabella avrà dimensioni $(n + 1) \times (m + 1)$, dove l'elemento nella cella (j, k) conterrà il valore di $L_{j,k}$, ovvero la LCS tra i primi j caratteri di X e i primi k caratteri di Y . Inizializziamo la prima riga e la prima colonna della tabella con i valori dei casi base, e poi riempiamo la tabella utilizzando le equazioni caratteristiche definite sopra.

Anche l'ordine in cui viene riempita la tabella è importante: in questo caso (e in molti altri, ma dipende dal problema specifico) possiamo procedere per righe, in quanto la computazione di $L_{j,k}$ dipende solo dai valori presenti nelle equazioni caratteristiche, ovvero:

- $L_{j-1,k-1}$ (riga e colonna precedenti).
- $L_{j-1,k}$ (riga precedente e stessa colonna).
- $L_{j,k-1}$ (stessa riga e colonna precedente).

```
1 def LCS(X, Y):
2     """ Returns table such that L[j][k] is length of LCS for X[0:j] and
3         Y[0:k] """
4     n, m = len(X), len(Y)
5     L = [[0] * (m + 1) for k in range(n + 1)] # (n+1) x (m+1) table
6     for j in range(1, n + 1): # j from 1 to n
7         for k in range(1, m + 1): # k from 1 to m
8             if X[j-1] == Y[k-1]: # match
9                 L[j][k] = 1 + L[j-1][k-1]
10            else: # no match
11                L[j][k] = max(L[j-1][k], L[j][k-1])
12     return L
```

L'algoritmo che si occupa di calcolare la tabella L impiega due cicli annidati, iterando rispettivamente su j e k . All'interno del ciclo più interno, viene eseguita una semplice operazione di confronto e un'assegnazione, entrambe con complessità $O(1)$, quindi la complessità totale dell'algoritmo è $O(n \cdot m)$, dove n e m sono le lunghezze delle stringhe X e Y rispettivamente.

Estrazione della soluzione

Come già detto, la tabella L contiene le lunghezze delle LCS per tutti i sotto-problemi, ma non restituisce direttamente la LCS stessa. Tuttavia, è possibile ricostruire la LCS a partire dalla tabella L . La soluzione può essere ricavata, a partire dalla cella $L[n][m]$ nel modo seguente:

- Considerando una generica cella $L[j][k]$:
 - Se $c = x_{j-1} = y_{k-1}$, significa che questo carattere comune c ha contribuito alla lunghezza della sottosequenza basandosi sul valore precedente $L_{j-1,k-1}$. Possiamo quindi registrare x_{j-1} come parte della soluzione e proseguire l'analisi dalla cella $L_{j-1,k-1}$.
 - Se $x_{j-1} \neq y_{k-1}$, allora la lunghezza della LCS dipende dal massimo tra $L[j][k-1]$ e $L[j-1][k]$. In questo caso, dobbiamo spostarci nella direzione del massimo valore per continuare la ricerca della LCS.
 - Continuiamo questo processo fino a raggiungere una cella $L[j][k] = 0$.

```

1 def LCS_solution(X, Y, L):
2     """ Returns the LCS of X and Y, given LCS table L """
3     solution = []
4     j, k = len(X), len(Y)
5     while L[j][k] > 0:          # common characters remain
6         if X[j-1] == Y[k-1]:    # match
7             solution.append(X[j-1]) # add to solution
8             j -= 1
9             k -= 1
10        elif L[j-1][k] >= L[j][k-1]: # no match
11            j -= 1
12        else:
13            k -= 1
14    return ''.join(reversed(solution)) # return left-to-right LCS

```

L'algoritmo per ricostruire la LCS ha una complessità temporale di $O(n + m)$, poiché in ogni iterazione del ciclo while si decrementa almeno uno tra j e k , e il ciclo termina quando uno dei due raggiunge zero.

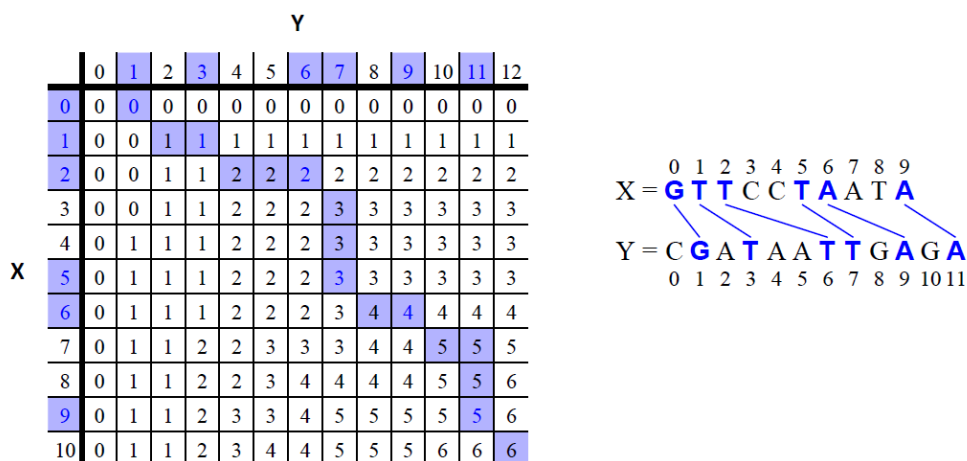


Figura 11.3: Illustrazione dell'algoritmo per la costruzione di una longest common subsequence a partire dall'array L . Un passo diagonale da $L_{j,k}$ a $L_{j-1,k-1}$ sul percorso evidenziato rappresenta l'uso di un carattere comune, ovvero il carattere $c = x_{j-1} = y_{k-1}$.

11.3 Edit Distance: Levenshtein Distance

Il problema della *Edit Distance* consiste nel trovare il numero minimo di operazioni necessarie per trasformare una stringa in un'altra stringa.

- Date due stringhe $A = a_0a_1 \dots a_{n-1}$ e $B = b_0b_1 \dots b_{m-1}$ su di un alfabeto Σ , trovare il numero minimo di operazioni necessarie per trasformare A in B . Le operazioni consentite sono: Inserimento(Insert), Cancellazione>Delete), Sostituzione(Replace).

L'*Edit Transcript* di due stringhe A e B è una stringa T sull'alfabeto $\{I, D, R, M\}$ (Insert, Delete, Replace, Match) che descrive una sequenza di operazioni per trasformare A in B . Se in T sostituiamo tutte le I con D , e tutte le D con I , otteniamo l'edit transcript T' che trasforma B in A .

Soluzione Dynamic Programming

Sia $D(n, m)$ la distanza di edit tra le stringhe $A = a_0a_1 \dots a_{n-1}$ e $B = b_0b_1 \dots b_{m-1}$. $D_{n,m}$ rappresenta quindi la soluzione ottimale del problema.

Scomposizione in sotto-problemi

Sia $D(i, j)$ la distanza di edit tra i prefissi $a_0a_1 \dots a_{i-1}$ e $b_0b_1 \dots b_{j-1}$, con $0 \leq i \leq n$ e $0 \leq j \leq m$. $D(i, j)$ rappresenta quindi la soluzione ottimale al sotto-problema che considera solo i primi i caratteri di A e i primi j caratteri di B .

Anche in questo caso, osserviamo l'ultimo carattere di entrambe le stringhe, e da qui ricaviamo le **equazioni caratteristiche** che ci permettono di esprimere $D(i, j)$ in funzione dei sotto-problemi più piccoli:

- Se $a_{i-1} = b_{j-1}$ (**match**), allora non è necessaria alcuna operazione, e possiamo scrivere:

$$D(i, j) = D(i-1, j-1) = D(i-1, j-1) + t(a_{i-1}, b_{j-1})$$

- Se c'è una **replace** da a_{i-1} a b_{j-1} :

$$D(i, j) = D(i-1, j-1) + 1 = D(i-1, j-1) + t(a_{i-1}, b_{j-1})$$

- se c'è una **delete** di a_{i-1} :

$$D(i, j) = D(i-1, j) + 1$$

- se c'è una **insert** di b_{j-1} dopo il carattere a_{i-1} :

$$D(i, j) = D(i, j-1) + 1$$

Dove la funzione di costo $t(a, b)$ è definita come:

$$t(a, b) = \begin{cases} 0 & \text{se } a = b \\ 1 & \text{se } a \neq b \end{cases}$$

Si è in grado quindi di riassumere le equazioni caratteristiche in un'unica espressione:

$$D(i, j) = \min \begin{cases} D(i-1, j-1) + t(a_{i-1}, b_{j-1}) \\ D(i-1, j) + 1 \\ D(i, j-1) + 1 \end{cases}$$

Boundary Conditions

È immediato riconoscere i casi base:

- $D(0, 0) = 0$, in quanto la distanza di edit tra due stringhe vuote è zero.
- Se una delle due stringhe è vuota, la distanza di edit è pari alla lunghezza dell'altra stringa, in quanto sono necessarie tante operazioni di inserimento o cancellazione quanti sono i caratteri della stringa non vuota. Quindi:

$$D(0, j) = j \quad \text{per } 0 \leq j \leq m$$

$$D(i, 0) = i \quad \text{per } 0 \leq i \leq n$$

Calcolo della tabella

La tabella avrà dimensioni $(n+1) \times (m+1)$, dove l'elemento nella cella (i, j) conterrà il valore di $D(i, j)$, ovvero la distanza di edit tra i primi i caratteri di A e i primi j caratteri di B . Inizializziamo la prima riga e la prima colonna della tabella con i valori dei casi base, e poi riempiamo la tabella utilizzando l'equazione caratteristica definita sopra.

Notiamo inoltre che la soluzione $D(i, j)$ appare nella computazione di:

$D(i+1, j)$ $D(i, j+1)$ $D(i+1, j+1)$ per cui riempiamo la tabella riga per riga (come LCS).

EditDistance(A, B)

```

1. int  $n = \text{len}(A)$ 
2. int  $m = \text{len}(B)$ 
3. int  $D[n+1][m+1]$ 
4. for  $i$  from 0 to  $n$  do
5.      $D[i][0] = i$ 
6. for  $j$  from 0 to  $m$  do
7.      $D[0][j] = j$ 
8. for  $i$  from 1 to  $n$  do
9.     for  $j$  from 1 to  $m$  do
10.          $cost = 0$  if  $A[i-1] == B[j-1]$  else 1
11.          $D[i][j] = \min(D[i-1][j-1] + cost,$ 
12.                        $D[i-1][j] + 1,$ 
13.                        $D[i][j-1] + 1)$ 
14. return  $D[n][m]$ 
```

L'algoritmo che si occupa di calcolare la tabella D impiega due cicli annidati (oltre ai cicli di inizializzazione), iterando rispettivamente su i e j . All'interno del ciclo più interno, viene eseguita una semplice operazione di confronto, una selezione condizionale e un'assegnazione, tutte con complessità $O(1)$, quindi la complessità totale dell'algoritmo è $O(n \cdot m)$, dove n e m sono le lunghezze delle stringhe A e B rispettivamente.

Estrazione della soluzione

Anche in questo caso, la tabella D contiene le distanze di edit per tutti i sotto-problemi, ma non restituisce direttamente la sequenza di operazioni necessarie per trasformare A in B . Tuttavia, è possibile ricostruire questa sequenza a partire dalla tabella D in tempo $O(n + m)$.

- Vogliamo trovare il percorso ottimale, ovvero quello che porta dalla cella $D(n, m)$ alla cella $D(0, 0)$.
- Per fare ciò, per ciascuna cella $D(i, j)$ memorizziamo come il suo valore è stato calcolato.
 - Per esempio, se $D(i, j) = D(i, j - 1) + 1$, allora sappiamo il valore della cella $D(i, j)$ è stato ottenuto dalla cella $D(i, j - 1)$ tramite inserimento del carattere b_{j-1} .

		w i n t e r s							
		0	1	2	3	4	5	6	7
v i n t n e r	0	0	←1	←2	←3	←4	←5	←6	←7
	1	↑1	↖1	↖2	↖3	↖4	↖5	↖6	↖7
	2	↑2	↑↖2	↖2	↖2	←3	←4	←5	←6
	3	↑3	↑↖3	↑↖3	↑↖3	↖3	↖4	↖5	↖6
	4	↑4	↑↖4	↑↖4	↑↖4	↖3	↖4	↖5	↖6
	5	↑5	↑↖5	↑↖5	↑↖5	↑4	↖4	↖5	↖6
	6	↑6	↑↖6	↑↖6	↑↖6	↑5	↖4	↖5	↖6
	7	↑7	↑↖7	↖6	↖↖7	↑6	↑5	↖4	↖5

Figura 11.4: Esempio di tabella per il calcolo della Edit Distance dalla stringa "vintner" alla stringa "winters". Le frecce indicano il percorso ottimale per trasformare la stringa A (vintner) nella stringa B (winters).

La freccia orizzontale ($\leftarrow$, da $D(i, j)$ a $D(i, j - 1)$) rappresenta un'operazione di **inserimento** (Insert) del carattere b_{j-1} in A .

La freccia verticale ($\uparrow$, da $D(i, j)$ a $D(i - 1, j)$) rappresenta un'operazione di **cancellazione** (Delete) del carattere a_{i-1} da A .

La freccia diagonale ($\nwarrow$, da $D(i, j)$ a $D(i - 1, j - 1)$) rappresenta un'operazione di **match** (Match) se $a_{i-1} = b_{j-1}$ o di **sostituzione** (Replace) se $a_{i-1} \neq b_{j-1}$.

In riferimento all'esempio in Figura 11.4, per capire *quali* operazioni abbiamo fatto, dobbiamo partire dalla fine (la soluzione ottimale) e ripercorrere i passi all'indietro fino all'inizio.

Gli Indici Per prima cosa, fissiamo la notazione per gli indici:

- **La Cella corrente** $D(i, j)$: È la casella della tabella in cui ci troviamo adesso.
- i e j : Sono le **lunghezze** dei prefissi. Se ad esempio siamo in $D(3, 2)$, stiamo guardando una sottostringa di A lunga 3 e una di B lunga 2, per cui gli indici reali degli ultimi caratteri sono 2 e 1 rispettivamente.
- $i - 1$ e $j - 1$: Sono gli **indici reali** dei caratteri nelle stringhe (poiché gli array partono da 0).
 - Il carattere “sotto esame” della stringa A è a_{i-1} .
 - Il carattere “sotto esame” della stringa B è b_{j-1} .

Il Procedimento: Camminare all'Indietro Partiamo dalla cella in basso a destra $D(n, m)$ e ci chiediamo: “Da quale vicino sono arrivato qui pagando il costo minore?”

1. **Controllo Diagonale (Match o Replace):** Guardiamo la cella $D(i - 1, j - 1)$.
 - Se $a_{i-1} == b_{j-1}$ (i caratteri sono uguali) e il costo non è cambiato, allora c'è stato un **Match**. Decrementiamo sia i che j .
 - Se i caratteri sono diversi e $D(i, j) = D(i - 1, j - 1) + 1$, allora c'è stata una **Sostituzione** (Replace). Decrementiamo sia i che j .
2. **Controllo Sinistra (Insert):** Guardiamo la cella $D(i, j - 1)$. Se $D(i, j) = D(i, j - 1) + 1$, significa che siamo arrivati qui aggiungendo un carattere. L'operazione è stata una **Inserimento** del carattere b_{j-1} dopo il carattere a_{i-1} .
 - Ci spostiamo a sinistra: j diventa $j - 1$, ma i resta uguale (perché non abbiamo consumato nulla da A).
3. **Controllo Sopra (Delete):** Guardiamo la cella $D(i - 1, j)$. Se $D(i, j) = D(i - 1, j) + 1$, significa che siamo arrivati qui rimuovendo un carattere superfluo. L'operazione è stata una **Cancellazione** del carattere a_{i-1} .
 - Ci spostiamo in alto: i diventa $i - 1$, ma j resta uguale (perché non abbiamo avanzato nella costruzione di B).

Ripetiamo questi controlli finché non arriviamo alla cella $D(0, 0)$. La sequenza di operazioni trovata (letta al contrario) è la soluzione.

11.4 Sequence Alignment

Un altro modo per misurare la somiglianza tra due stringhe è attraverso il *sequence alignment*.

- Siano date due stringhe $X = x_1x_2 \dots x_m$ e $Y = y_1y_2 \dots y_n$.
- Siano $S = \{1, 2, \dots, m\}$ e $T = \{1, 2, \dots, n\}$ gli insiemi degli indici rispettivamente di X e Y .
- Un **matching** tra X e Y consiste in un insieme M di coppie (x, y) in cui:
 - $x \in S$ e $y \in T$.
 - Ogni indice di S compare al più una volta in M .
 - Ogni indice di T compare al più una volta in M .
- Un **alignment** di X e Y è un *matching* M di queste due stringhe, senza "crossing pairs" (coppie incrociate).
 - In altre parole, se $(i, j), (i', j') \in M$ e $i < i'$, allora deve essere $j < j'$.
- Un *alignment* ci dice quali coppie di posizioni allineate tra loro.

stop-
-tops

l'alignment corrispondente è: $\{(2, 1), (3, 2), (4, 3)\}$.

- Sia M un dato alignment tra X e Y .
 - Per ogni posizione di X e Y che non è matchata in M (un gap) si ha un **gap penalty** di $\delta > 0$.
 - Per ogni coppia $(i, j) \in M$ tale che $x_i = y_j$, non abbiamo alcun penalty.
 - Per ogni coppia $(i, j) \in M$ tale che $x_i = p, y_j = q$ e $p \neq q$, abbiamo un **mismatch penalty** di $\alpha_{pq} > 0$.
- L'**alignment cost** di M è la somma dei suoi *gap* e *mismatch* penalties. I valori di δ e α_{pq} sono specificati come parte del problema e dipendono dalla specifica applicazione.

o-currance
occurrence

Alignment costs $\delta + \alpha_{ae}$

Cerchiamo un alignment tra X e Y che minimizzi il costo totale con qualsiasi δ e α_{pq} .

o-curr-ance
occurre-nce

Alignment costs 3δ

Soluzione Dynamic Programming

Nell'alignment ottimale M , la coppia (m, n) può essere presente o meno in M . Inoltre, gli ultimi caratteri delle due stringhe possono essere matchati tra loro oppure no.

- Se $(m, n) \notin M$, allora o la posizione m di X o la posizione n di Y non è matchata in M .
 - Altrimenti, ci sarebbero (m, j) e (i, n) in M con $j < n$ e $i < m$.
 - Impossibile, poiché sarebbero coppie incrociate.
- Quindi, in un alignment ottimale M , almeno una delle seguenti condizioni è vera:
 - $(m, n) \in M$ e paghiamo α_{pq} dove $p = x_m$ e $q = y_n$ (assumendo che $\alpha_{pp} = 0$).
 - La posizione m di X non è matchata e paghiamo δ .
 - La posizione n di Y non è matchata e paghiamo δ .

Denotiamo con $\text{OPT}(i, j)$ il costo minimo di alignment tra $x_1x_2 \dots x_i$ e $y_1y_2 \dots y_j$.

- Se scegliamo di inserire (i, j) nell'alignment, allora dobbiamo pagare α_{pq} , dove $p = x_i$, e $q = y_j$, più il costo del miglior alignment tra $x_1x_2 \dots x_{i-1}$ e $y_1y_2 \dots y_{j-1}$.

$$\text{OPT}(i, j) = \alpha_{pq} + \text{OPT}(i - 1, j - 1)$$

- Se scegliamo di non matchare x_i , allora dobbiamo pagare un gap cost di δ più il costo del miglior alignment tra $x_1x_2 \dots x_{i-1}$ e $y_1y_2 \dots y_j$.

$$\text{OPT}(i, j) = \delta + \text{OPT}(i - 1, j)$$

- Se scegliamo di non matchare y_j , allora dobbiamo pagare un gap cost di δ più il costo del miglior alignment tra $x_1x_2 \dots x_i$ e $y_1y_2 \dots y_{j-1}$.

$$\text{OPT}(i, j) = \delta + \text{OPT}(i, j - 1)$$

Tra tutte queste possibilità, scegliamo quella che minimizza il costo totale:

$$\text{OPT}(i, j) = \min \begin{cases} \alpha_{pq} + \text{OPT}(i - 1, j - 1) \\ \delta + \text{OPT}(i - 1, j) \\ \delta + \text{OPT}(i, j - 1) \end{cases}$$

Le **boundary conditions** sono date dal fatto che l'allineamento di una stringa vuota con una stringa di lunghezza i richiede i gap, ovvero i volte il costo di gap δ :

$$\text{OPT}(i, 0) = \text{OPT}(0, i) = i \cdot \delta$$

A questo punto non ci resta che mettere tutto insieme:

- Costruiamo una tabella contenente i valori di $\text{OPT}(i, j)$ per $0 \leq i \leq m$ e $0 \leq j \leq n$.
 - Il tempo impiegato per questa operazione è $O(m \cdot n)$, dato che la tabella ha $O(m \cdot n)$ celle, e ciascuna può essere calcolata in tempo $O(1)$.
- Il valore di $\text{OPT}(m, n)$ rappresenta il costo minimo di alignment tra le due stringhe X e Y , e quindi la soluzione al problema originale.
- Come già visto nei casi precedenti, possiamo tracciare il percorso attraverso la tabella per ricostruire l'alignment ottimale stesso.
 - Il tempo impiegato per questa operazione è $O(m + n)$, poiché in ogni passo si decrementa almeno uno tra i e j , e il processo termina quando uno dei due raggiunge zero.

11.5 Coin Change per sistemi non canonici

Il problema del *Coin Change*, così come già visto nel Capitolo 10, consiste nel trovare il numero minimo di monete necessarie per eguagliare una certa somma, dato un insieme di tagli di monete disponibili. La soluzione greedy funziona correttamente solo per sistemi di monete canonici, ovvero quei sistemi in cui la scelta della moneta di taglio più grande possibile in ogni passo porta sempre alla soluzione ottimale. Tuttavia, esistono sistemi di monete non canonici in cui l'approccio greedy non garantisce la soluzione ottimale.

Soluzione Dynamic Programming: 1° approccio

Per risolvere il problema del Coin Change in sistemi non canonici, possiamo utilizzare un approccio di programmazione dinamica.

- Sia dato un insieme di n tagli di monete $1 = c_1 < c_2 < \dots < c_n$.
- Definiamo $C[r]$ come il numero minimo di monete necessarie per eguagliare il resto r .

La soluzione ottimale può essere espressa in termini di soluzioni ottimali a sotto-problemi più piccoli. L'idea di base è che se in un dato step la soluzione ottimale utilizza una moneta di valore c_i , allora $C[r] = 1 + C[r - c_i]$. Quindi, possiamo esprimere $C[r]$ come:

$$C[r] = \begin{cases} \infty & \text{se } r < 0 \\ 0 & \text{se } r = 0 \\ 1 + \min_{1 \leq i \leq n} \{C[r - c_i]\} & \text{se } r \geq 1 \end{cases}$$

Questo è un possibile approccio per scomporre il problema in sotto-problemi più piccoli. Sviluppando questo ragionamento, avremmo una soluzione che impiega un tempo $O(n \cdot r)$. A differenza degli esempi precedenti, in questo caso non c'è una tabella bidimensionale, ma un array monodimensionale C di lunghezza $r + 1$, dove ogni elemento $C[j]$ rappresenta il numero minimo di monete necessarie per eguagliare il resto j . Questo approccio utilizza la programmazione dinamica per salvare i risultati intermedi, evitando di dover ricalcolare più volte la stessa soluzione. L'unica boundary condition è banalmente $C[0] = 0$, ovvero non è necessaria alcuna moneta per eguagliare il resto zero; tutti gli altri valori $C[j]$ per $j \geq 1$ vengono derivati calcolando il minimo tra le opzioni disponibili.

Soluzione Dynamic Programming: 2° approccio

Riprendiamo le assunzioni fatte prima ma in maniera leggermente diversa:

- Sia dato un insieme di n tagli di monete $1 = c_1 < c_2 < \dots < c_n$.
- Definiamo $M(i, r)$ come il numero minimo di monete $(c_1, c_2, \dots, c_i)$, con $i \leq n$, necessarie per eguagliare il resto r .

L'idea di base è che in un dato step possiamo scegliere se includere o meno la moneta di taglio c_i nella soluzione ottimale. Quindi, possiamo esprimere $M(i, r)$ come:

$$M(i, r) = \begin{cases} M(i-1, r) & \text{se non inseriamo } c_i \text{ nella soluzione} \\ M(i, r - c_i) & \text{se inseriamo } c_i \text{ nella soluzione} \end{cases}$$

Le boundary conditions sono le seguenti:

- $M(i, 0) = 0$, in quanto per eguagliare il resto zero non è necessaria alcuna moneta.
- $M(1, r) = r$, se abbiamo solo monete di valore unitario $c_1 = 1$ l'unica soluzione possibile è utilizzare r monete.
- $M(i, r) = 1$ se $c_i = r$, in quanto possiamo eguagliare il resto r con una sola moneta di valore c_i .
- $M(i, r) = M(i-1, r)$ se $c_i > r$, in quanto non possiamo utilizzare la moneta di taglio c_i se è maggiore del resto r .

```
1 def dp_coin_change(amount, coins=[0, 1, 2, 5, 10, 20, 50]):
2     # coins[0] (che è 0) non viene usato, serve solo a scalare gli indici
3     nc = len(coins)
4     # Crea una matrice di nc righe. Nota: m[0] sarà la riga inutilizzata
5     m = [[0]*(amount+1) for _ in range(nc)]
6     # Caso base: gestiamo la prima moneta reale (coins[1] = 1)
7     # Scriviamo sulla riga m[1]
8     for r in range(amount+1):
9         m[1][r] = r
10
11     # Ciclo principale: partiamo dalla seconda moneta reale (indice 2)
12     for i in range(2, nc):
13         for r in range(1, amount+1):
14             if coins[i] == r:
15                 m[i][r] = 1
16             elif coins[i] > r:
17                 m[i][r] = m[i-1][r]
18             else:
19                 m[i][r] = min(m[i-1][r], m[i][r-coins[i]] + 1)
20
21     return m, m[-1][-1] # ritorna la matrice e il risultato finale
```

Una volta popolata la matrice M , la sequenza esatta di monete che costituisce la soluzione ottimale si ricava mediante una procedura di *backtracking*. Si parte dalla cella finale $M[n][R]$ (dove n è il numero di tipi di monete e R l'importo totale) e si procede a ritroso fino a raggiungere un resto nullo ($r = 0$). Ad ogni passo, trovandosi nella cella (i, r) , si confronta il valore corrente con quello della riga immediatamente superiore, $M[i - 1][r]$:

- Se $M[i][r] = M[i - 1][r]$, significa che la moneta di taglio c_i non è stata utilizzata per ottenere l'ottimo locale (la soluzione migliore era già stata trovata con le monete precedenti). In questo caso, ci si sposta semplicemente alla riga superiore: $(i - 1, r)$.
- Se invece $M[i][r] < M[i - 1][r]$, implica che la moneta c_i è stata utilizzata, facendo diminuire il numero di monete necessarie ($M[i][r] < M[i - 1][r]$). Si aggiunge c_i all'elenco delle monete scelte e si sottrae il suo valore dal resto residuo, spostandosi alla cella $(i, r - c_i)$. Si rimane sulla riga i poiché, nel problema con ripetizioni, la stessa moneta potrebbe essere stata usata più volte.

11.6 0-1 Knapsack

Il problema dello *0-1 Knapsack* si differenzia dal problema del *Fractional Knapsack* (Capitolo 10) per il fatto che ogni oggetto può essere inserito nello zaino solo per intero e non in frazioni. In altre parole, ogni oggetto può essere inserito nello zaino solo per intero o non essere inserito affatto.

- Dati n oggetti, ciascuno con un volume v_i e un costo/valore c_i , e un limite di volume B , trovare un sottoinsieme $S \subseteq \{1, \dots, n\}$ di oggetti con volume totale al più B tale che il costo totale sia massimizzato.

Come già accennato in precedenza, il problema dello 0-1 Knapsack non può essere risolto in modo ottimale utilizzando un approccio greedy, poiché la scelta dell'oggetto con il miglior rapporto costo/volume non garantisce una soluzione ottimale globale, ma questa soluzione funzionava solo per il problema frazionario. Tuttavia, possiamo utilizzare un approccio di programmazione dinamica per risolvere questo problema in modo efficiente.

Soluzione Dynamic Programming

Assumendo che i volumi v_i siano interi positivi, definiamo i sotto-problemi come segue:

- Consideriamo solamente i primi i oggetti $x_1, x_2, \dots, x_i$, ed un volume massimo j dello zaino, con $0 \leq i \leq n$ e $0 \leq j \leq B$.
- Definiamo $V(i, j)$ come il valore massimo ottenibile utilizzando i primi i oggetti con un volume massimo j dello zaino.

Il valore di $V(i, j)$ dipende dalla scelta di includere o meno l'oggetto x_i nello zaino:

- $V(i, j) = V(i - 1, j)$, se scegliamo di non inserire l'oggetto x_i nello zaino.
- $V(i, j) = c_i + V(i - 1, j - v_i)$, se scegliamo di inserire l'oggetto x_i nello zaino (a condizione che $v_i \leq j$).

Dal momento che vogliamo massimizzare il valore totale, l'**equazione caratteristica** diventa:

$$V(i, j) = \begin{cases} \max(V(i - 1, j), & c_i + V(i - 1, j - v_i)) & \text{se } v_i \leq j \\ V(i - 1, j) & \text{se } v_i > j \end{cases}$$

Le boundary conditions sono le seguenti:

- $V(i, 0) = 0$ per ogni $1 \leq i \leq n$, poiché se il volume dello zaino è zero, non possiamo inserire alcun oggetto.
- $V(0, j) = 0$ per ogni $1 \leq j \leq B$, poiché se non abbiamo oggetti a disposizione, il valore massimo ottenibile è zero.
- $V(1, j) = v_1$ se $v_1 \leq j$, altrimenti $V(1, j) = 0$, poiché con un solo oggetto possiamo inserirlo nello zaino solo se il suo volume è minore o uguale al volume massimo dello zaino.

Un algoritmo iterativo può riempire la matrice calcolando ogni voce in tempo costante. Alla fine, il costo della soluzione ottimale è riportato nella cella $V(n, B)$. Il tempo necessario da un algoritmo per riempire la tabella è $O(nB)$.

Per costruire una soluzione ottimale, possiamo anche costruire una matrice booleana C della stessa dimensione $n \times (B + 1)$. Per ogni $1 \leq i \leq n$ e $0 \leq j \leq B$, $C(i, j) = \text{True}$ se e solo se esiste una soluzione ottimale che inserisce un sottoinsieme dei primi i oggetti nel volume j in modo che l'oggetto i sia incluso nella soluzione. Utilizzando la matrice C possiamo lavorare a ritroso per ricostruire gli elementi presenti in una soluzione ottimale.

Item	1	2	3	4	5	6	7	8	9
Cost	2	3	3	4	4	5	7	8	8
Volume	3	5	7	4	3	9	2	11	5

j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$i = 9$	0	0	7	7	7	11	11	15	15	15	19	19	19	21	23	23
$i = 8$	0	0	7	7	7	11	11	11	13	15	15	15	17	17	18	18
$i = 7$	0	0	7	7	7	11	11	11	13	15	15	15	17	17	18	18
$i = 6$	0	0	0	4	4	4	6	8	8	8	10	10	11	11	11	13
$i = 5$	0	0	0	4	4	4	6	8	8	8	10	10	11	11	11	13
$i = 4$	0	0	0	2	4	4	4	6	6	7	7	7	9	9	9	9
$i = 3$	0	0	0	2	2	3	3	3	5	5	5	5	6	6	6	8
$i = 2$	0	0	0	2	2	3	3	3	5	5	5	5	5	5	5	5
$i = 1$	0	0	0	2	2	2	2	2	2	2	2	2	2	2	2	2

j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$i = 9$	0	0	0	0	0	0	0	1	1	0	1	1	1	1	1	1
$i = 8$	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
$i = 7$	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1
$i = 6$	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
$i = 5$	0	0	0	1	0	0	1	1	1	1	1	1	1	1	1	1
$i = 4$	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1
$i = 3$	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1
$i = 2$	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1
$i = 1$	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1

Figura 11.5: Esempio con $n = 9$ oggetti e limite di volume $B = 15$. La tabella V presenta le righe stampate in ordine decrescente, ma questo non è un problema ma riguarda solo una scelta di impaginazione. La soluzione ottimale ha valore $V(9, 15) = 23$, ottenuta selezionando gli oggetti: 9, 7, 5, 4

Input:

v : array dei volumi ($v[1] \dots v[n]$)

c : array dei costi ($c[1] \dots c[n]$)

B : capacità massima

Output:

V : Tabella delle soluzioni ottime

C : Tabella delle scelte (inclusione oggetto)

```

ALGORITMO Knapsack01( $v, c, B$ )
1.  $n = \text{len}(v)$ 
2. for  $j = 0$  to  $B$  do // Inizializzazione riga 0 (nessun oggetto)
3.    $V[0, j] = 0$ 
4. for  $i = 0$  to  $n$  do // Inizializzazione colonna 0 (capacità 0)
5.    $V[i, 0] = 0$ 
   // Ciclo principale:  $i$  scorre gli oggetti,  $j$  le capacità
6. for  $i = 1$  to  $n$  do
7.   for  $j = 1$  to  $B$  do

       // 1. L'oggetto  $i$  ha volume maggiore della capacità attuale  $j$ ?
       // Se sì, non entra fisicamente nello zaino.
8.     if  $v[i] > j$  then
9.        $V[i, j] = V[i-1, j]$ 
10.       $C[i, j] = \text{False}$ 

       // 2. L'oggetto entra, ma conviene prenderlo?
       // Se il valore SENZA l'oggetto ( $V[i-1, j]$ ) è MAGGIORE
       // del valore CON l'oggetto, allora NON lo prendiamo.
11.     else if  $V[i-1, j] > c[i] + V[i-1, j - v[i]]$  then
12.        $V[i, j] = V[i-1, j]$ 
13.        $C[i, j] = \text{False}$ 

       // 3. L'oggetto entra e conviene prenderlo.
       // (Il valore CON l'oggetto è maggiore o uguale al precedente).
14.     else
15.        $V[i, j] = c[i] + V[i-1, j - v[i]]$ 
16.        $C[i, j] = \text{True}$ 

17. return  $V, C$ 

```

11.7 Matrix Chain-Product

Supponiamo di avere una collezione di n matrici $A_1, A_2, \dots, A_n$ e vogliamo calcolare il loro prodotto A . La matrice A_i ha dimensioni $d_i \times d_{i+1}$, per $i = 0, 1, 2, \dots, n - 1$.

Ricordiamo la definizione formale di moltiplicazione tra due matrici B di dimensioni $d \times e$ e C di dimensioni $e \times f$:

$$A[i][j] = \sum_{k=0}^{e-1} B[i][k] \cdot C[k][j] \quad \text{richiede: } d \cdot e \cdot f \text{ moltiplicazioni scalari}$$

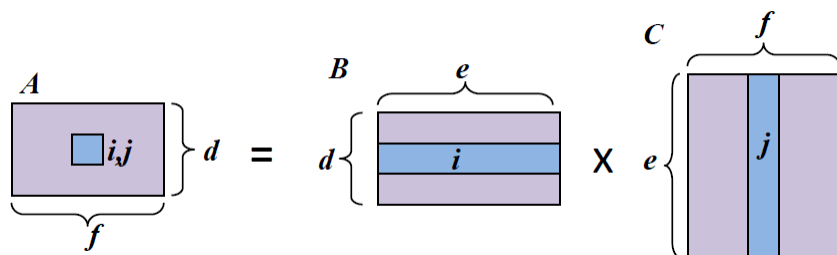


Figura 11.6: Esempio di moltiplicazione tra tre matrici B e C .

La moltiplicazione di matrici è un'operazione associativa, il che significa che $B \cdot (C \cdot D) = (B \cdot C) \cdot D$. Pertanto, possiamo inserire parentesi nell'espressione in qualsiasi modo desideriamo e otterremo lo stesso risultato. Tuttavia, modificando l'ordine delle parentesi possiamo influenzare il numero di moltiplicazioni scalari necessarie per calcolare il prodotto finale, come illustrato nell'esempio seguente.

Esempio: Sia B una matrice di dimensioni 2×10 , C una matrice di dimensioni 10×50 e D una matrice di dimensioni 50×20 . Calcolare il prodotto $B \cdot (C \cdot D)$ richiede $2 \cdot 10 \cdot 20 + 10 \cdot 50 \cdot 20 = 10400$ moltiplicazioni, mentre calcolare il prodotto $(B \cdot C) \cdot D$ richiede $2 \cdot 10 \cdot 50 + 2 \cdot 50 \cdot 20 = 3000$ moltiplicazioni.

Il problema del **Matrix Chain-Product** consiste nel determinare l'ordine delle moltiplicazioni tra matrici (tramite inserimento di opportune parentesi) che minimizza il numero totale di moltiplicazioni scalari necessarie per calcolare il prodotto finale.

Soluzione Dynamic Programming

Per risolvere il problema del Matrix Chain-Product, definiamo i sotto-problemi come segue:

- Consideriamo una catena di matrici da $A_i \times A_{i+1} \times \cdots \times A_j$, con $0 \leq i \leq j \leq n - 1$.
- Definiamo $N(i, j)$ come il numero minimo di moltiplicazioni scalari necessarie per calcolare questo sotto-problema.
 - La soluzione al problema originale richiederà $N(0, n - 1)$ moltiplicazioni scalari.

L'idea di base è che per qualsiasi ordine di moltiplicazione possibile, esiste un punto in cui la catena viene divisa in due sottocatene, ovvero il punto in cui verrà eseguito il **prodotto finale** tra due matrici risultanti. Supponiamo che questo prodotto finale si trovi all'indice i , per cui avremo che:

$$i: (A_0 \times \cdots \times A_i) \times (A_{i+1} \times \cdots \times A_{n-1})$$

Il costo della soluzione ottimale $N(0, n - 1)$ è la somma del costo di due sotto-problemi ottimali $N(0, i)$ e $N(i + 1, n - 1)$, più il costo del prodotto finale tra le due matrici risultanti.

Quindi esistono diversi indici in cui possiamo dividere la catena, e ogni scelta ha un costo differente. Possiamo calcolare $N(i, j)$ valutando ogni possibile punto di divisione k tra i e $j - 1$, e scegliendo quello che minimizza il costo totale, per cui l'**equazione caratteristica** per $N(i, j)$ diventa:

$$N(i, j) = \min_{i \leq k < j} \{N(i, k) + N(k + 1, j) + d_i \cdot d_{k+1} \cdot d_{j+1}\}$$

dove $d_i \cdot d_{k+1} \cdot d_{j+1}$ rappresenta il costo del prodotto finale tra le due matrici risultanti dalle sottocatene.

Possiamo, tuttavia, usare l'equazione per $N(i, j)$ per derivare un algoritmo efficiente calcolando i valori di $N(i, j)$ in modo bottom-up, memorizzando le soluzioni intermedie in una tabella di valori $N(i, j)$. Possiamo iniziare assegnando $N(i, i) = 0$, che rappresenta il caso base, per $i = 0, 1, \dots, n - 1$. Possiamo quindi applicare l'equazione generale per calcolare i valori $N(i, i + 1)$, poiché dipendono solo dai valori $N(i, i)$ e $N(i + 1, i + 1)$ che sono già disponibili. Dati i valori $N(i, i + 1)$, possiamo quindi calcolare i valori $N(i, i + 2)$, e così via. Pertanto, possiamo costruire i valori $N(i, j)$ a partire dai valori precedentemente calcolati fino a poter finalmente calcolare il valore di $N(0, n - 1)$, che è il numero che stiamo cercando.

L'algoritmo riportato di seguito calcola la tabella N in tempo $O(n^3)$, poiché ci sono $O(n^2)$ celle nella tabella e il calcolo di ciascuna cella richiede tempo $O(n)$ per valutare tutti i possibili punti di divisione k . Lo spazio richiesto è $O(n^2)$ per memorizzare la tabella N . Per ricostruire l'ordine ottimale delle moltiplicazioni, è necessario salvare per ogni cella l'indice del punto di divisione scelto per il calcolo del costo minimo per quella stessa cella.

```

1 def matrix_chain(d):
2     """
3     d is a list of n+1 numbers such that size of kth matrix is
4     d[k]-by-d[k+1].
5     Return an n-by-n table such that N[i][j] represents the minimum number
6     of multiplications needed to compute the product of Ai through Aj
7     inclusive. """
8     n = len(d) - 1 # number of matrices
9     N = [[0] * n for i in range(n)] # initialize n-by-n result to zero
10
11     for b in range(1, n): # number of products in subchain
12         for i in range(n - b): # start of subchain
13             j = i + b # end of subchain
14             N[i][j] = float('inf') # initialize to infinity
15
16             # Find the split point k that minimizes cost
17             for k in range(i, j):
18                 # Cost q = cost(left) + cost(right) + cost(multiplication)
19                 q = N[i][k] + N[k+1][j] + d[i] * d[k+1] * d[j+1]
20                 if q < N[i][j]:
21                     N[i][j] = q
22
23     return N

```

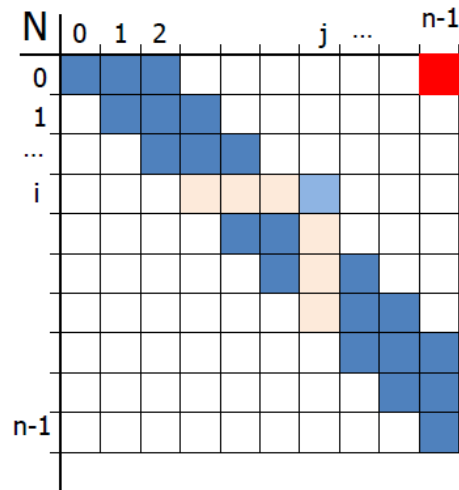


Figura 11.7: La matrice triangolare superiore mostra il processo di calcolo: la diagonale principale (blu scuro) rappresenta i casi base di costo zero. Una cella specifica (i, j) è evidenziata in azzurro, indicando il sottoproblema corrente. Le celle beige lungo la riga i e la colonna j evidenziano i sottoproblemi precedentemente risolti necessari per calcolare il valore corrente. La cella rossa nell'angolo in alto a destra è $N(0, n - 1)$, e indica il risultato finale dell'algoritmo.

Capitolo 12

Local Search

La **Local Search** (Ricerca Locale) rappresenta una delle tecniche fondamentali per la risoluzione di problemi di ottimizzazione complessi. A differenza degli algoritmi visti in precedenza, come l'approccio *Greedy* o la *Programmazione Dinamica*, che costruiscono la soluzione da zero, la ricerca locale opera su soluzioni complete. L'idea centrale è partire da una soluzione completa iniziale (spesso generata casualmente o ricavata da semplici euristiche) e migliorarla iterativamente esplorando un "intorno" locale di soluzioni simili.

L'ottimizzazione di un problema può essere rappresentata graficamente come una curva o superficie in cui ogni punto corrisponde ad una soluzione del problema e la sua "altezza" rappresenta il costo associato a quella soluzione. Il problema diventa quello di individuare il punto più basso possibile, ovvero il minimo costo per quel problema di ottimizzazione.

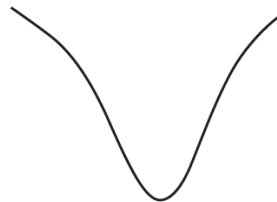


Figura 12.1: Rappresentazione grafica della curva delle soluzioni in un problema di ottimizzazione.

Formalmente, consideriamo un problema di ottimizzazione definito da:

- C : l'insieme delle soluzioni ammissibili.
- c : una funzione di costo che associa ad ogni soluzione $S \in C$ un valore reale $c(S)$.
- N : una funzione che definisce l'intorno di una soluzione, ovvero l'insieme delle soluzioni "vicine" a S , denotato come $N(S) \subseteq C$.

Graficamente, possiamo immaginare lo spazio delle soluzioni come un paesaggio: le soluzioni sono le coordinate e il costo è l'altitudine. L'algoritmo cerca di scendere verso il punto più basso (la valle più profonda).

A questo proposito, una distinzione cruciale è quella tra ottimi locali e globali:

- Un **Minimo Globale** è una soluzione S^* con costo minimo assoluto su tutto C .
- Un **Minimo Locale** è una soluzione S tale che $c(S) \leq c(S')$ per ogni vicino $S' \in N(S)$.
- Il problema principale della ricerca locale è che l'algoritmo può rimanere intrappolato in un minimo locale, incapace di "vedere" una soluzione migliore che si trova oltre una "collina" di costi crescenti.

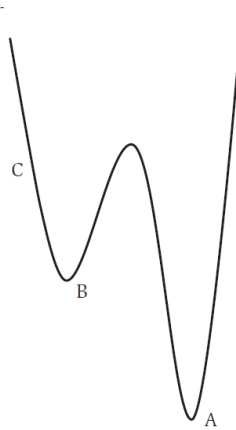


Figura 12.2: La ricerca locale esplora l'intorno delle soluzioni per trovare minimi locali e globali.

Ad ogni iterazione, l'algoritmo di ricerca mantiene una soluzione corrente $S \in C$. Ad ogni step, sceglie un vicino S' di S , dichiara S' come nuova soluzione corrente se $c(S') \leq c(S)$, e itera. Durante l'esecuzione dell'algoritmo, ricorda la soluzione a costo minimo che ha visto finora, S^* ; quindi, man mano che procede, trova soluzioni sempre migliori e aggiorna S^* di conseguenza. L'algoritmo termina quando non riesce più a trovare un vicino S' migliore della soluzione corrente S , restituendo S^* come soluzione finale.

Il *punto cruciale* di un algoritmo di ricerca locale risiede nella scelta della **neighbor relation** (relazione di vicinato) e nella progettazione della regola per scegliere una soluzione vicina ad ogni passo. Da un lato, la neighbor relation deve essere sufficientemente ampia da permettere all'algoritmo di uscire da minimi locali indesiderati; dall'altro, deve essere sufficientemente ristretta da mantenere l'efficienza computazionale dell'algoritmo.

A differenza dell'approccio Greedy, la ricerca locale permette di rivalutare le scelte fatte in precedenza (come già detto, si parte da una soluzione iniziale completa ma non necessariamente buona) e di esplorare soluzioni alternative. Questo rende la ricerca locale particolarmente adatta per problemi complessi dove le soluzioni ottimali sono difficili da trovare direttamente. Il vantaggio dunque è che con Greedy si può facilmente incappare in qualche minimo locale senza possibilità di uscirne, mentre con la ricerca locale si ha la possibilità di esplorare l'intorno delle soluzioni e potenzialmente trovare soluzioni migliori.

Va detto però che un algoritmo di ricerca locale che sia efficiente non esiste per tutti i problemi di ottimizzazione.

12.1 Problema: Vertex Cover

Consideriamo il problema del **Vertex Cover**. Siano dati un insieme V di elementi e un insieme E di coppie di elementi di V (tutte le coppie di E sono distinte, cioè differiscono almeno per un elemento). L'insieme C delle possibili soluzioni è dato da tutti i sottoinsiemi (soluzioni) S di V tali che ogni coppia in E ha almeno un elemento in S . Il costo di una soluzione S è semplicemente la sua cardinalità, cioè il numero di elementi in S : $c(S) = |S|$.

In altre parole, un **Vertex Cover** è un sottoinsieme $S \subseteq V$ che “copre” tutte le coppie in E : questo significa che, per ogni coppia presente in E , **almeno uno dei due elementi** che la compongono deve appartenere alla soluzione S .

Esempio: $V = \{a,b,c,d,e\}$ e $E = \{(a,b), (a,c), (b,d), (c,d), (d,e)\}$. Dei possibili Vertex Cover sono $S = \{b,c,e\}$, poiché ogni coppia in E contiene almeno uno degli elementi in S , e il costo di questa soluzione è $c(S) = 3$. Un altro possibile Vertex Cover è $S' = \{a,d\}$, con costo $c(S') = 2$, che è migliore della soluzione precedente. Se prendiamo $S = \{b,c\}$, questa non è una soluzione valida perché la coppia (d,e) non è coperta.

Per applicare la ricerca locale al problema del Vertex Cover, dobbiamo definire una *neighbor relation* efficace. Una semplice relazione può essere considerare come vicini due soluzioni che differiscono per l'aggiunta o la rimozione di un singolo elemento da V . In questo modo, ciascuna soluzione S ha n vicini, dove n è la dimensione di V .

- È semplice notare questa proprietà considerando un esempio: se $V = \{a,b,c,d,e\}$ e $S = \{b,c,e\}$, i vicini di S sono: $\{a, b, c, e\}$, $\{b, c, d, e\}$, $\{c, e\}$, $\{b, e\}$, $\{b, c\}$.

12.1.1 Gradient Descent per Vertex Cover

L'algoritmo di ricerca locale più semplice possibile è il **Gradient Descent** (Discesa del Gradiente). In questo approccio, ad ogni iterazione, abbiamo una soluzione corrente S e l'algoritmo esplora tutti i suoi vicini per poi scegliere il vicino S' con il costo più basso tra quelli che migliorano la soluzione corrente (cioè quelli con $c(S') < c(S)$). Se non esistono vicini migliori, l'algoritmo termina e restituisce la soluzione corrente come risultato finale.

Questo algoritmo è semplice ma può facilmente rimanere intrappolato in minimi locali, poiché esplora solo i vicini che migliorano la soluzione corrente. Nel contesto del problema Vertex Cover, consideriamo un insieme V di elementi e un insieme E di coppie. L'algoritmo Gradient Descent viene inizializzato con $S = V$ e procede rimuovendo un elemento per volta fintanto che la condizione di copertura delle coppie rimane soddisfatta.

Per valutare l'efficacia dell'algoritmo, analizziamo il suo comportamento in tre scenari distinti. Mentre l'approccio funziona correttamente in casi banali, mostra evidenti limiti e tendenza a bloccarsi in minimi locali non appena la struttura delle coppie diventa più complessa:

- **Caso banale (Insieme E vuoto):** Se non ci sono coppie da coprire, l'algoritmo rimuove correttamente tutti gli elementi fino a raggiungere l'insieme vuoto, che rappresenta il minimo globale.
- **Caso dell'Elemento "Centro":** Supponiamo che esista un elemento speciale $c \in V$ tale che c è contenuto in ogni coppia di E .
 - La soluzione ottima è data dal singleton $\{c\}$.
 - Se l'algoritmo rimuove c nelle prime iterazioni, sarà costretto a mantenere in S tutti gli altri elementi accoppiati con c per garantire la copertura. Si raggiunge così un *minimo locale* molto più costoso dell'ottimo globale, da cui non è possibile uscire poiché l'algoritmo non permette di reinserire elementi (poiché l'algoritmo aggiorna la soluzione solo se il costo diminuisce; il costo è dato dal numero di elementi in S che quindi può solo diminuire).
- **Caso delle Coppie Sequenziali:** Consideriamo $V = \{v_1, \dots, v_n\}$ con coppie definite come $E = \{(v_1, v_2), (v_2, v_3), \dots, (v_{n-1}, v_n)\}$.
 - La soluzione ottima consiste nel selezionare elementi alternati (es. $\{v_2, v_4, \dots\}$), ottenendo una cardinalità di circa $(n - 1)/2$.
 - L'algoritmo può invece convergere su minimi locali inefficienti, come ad esempio la configurazione $\{v_1, v_2, v_4, v_5, \dots\}$. Questa soluzione è valida e non riducibile (rimuovere un elemento scoprirebbe una coppia), ma ha una cardinalità di circa $2n/3$, risultando significativamente peggiore dell'ottimo.

12.2 Neighbor Relation: Algoritmo Metropolis

Come abbiamo visto con il Gradient Descent, l'approccio puramente "greedy" (che accetta solo miglioramenti) tende a rimanere intrappolato nei minimi locali. Per risolvere questo problema, è necessario introdurre un meccanismo che permetta all'algoritmo di accettare occasionalmente soluzioni peggiori, nella speranza che queste mosse "in salita" (uphill) permettano di scavalcare le barriere dei minimi locali e raggiungere il minimo globale.

Questa intuizione è alla base dell'**Algoritmo Metropolis**, che simula il comportamento di un sistema fisico basandosi sui principi della meccanica statistica. L'idea fondamentale è che il processo di ricerca sia globalmente orientato verso passi "in discesa" (miglioramento del costo), ma occasionalmente compia passi "in salita" per uscire dai minimi locali.

12.2.1 La Funzione di Gibbs-Boltzmann

L'algoritmo utilizza un'analogia fisica in cui ogni soluzione ammissibile S corrisponde a uno stato del sistema, e la funzione di costo $c(S)$ corrisponde all'*energia* E di quello stato. La probabilità di trovare il sistema in un determinato stato con energia E è modellata dalla **funzione di Gibbs-Boltzmann**:

$$e^{-E/(kT)}$$

- E è l'energia (il costo) dello stato corrente.
- $T > 0$ è la *temperatura* del sistema.
- k è una costante (costante di Boltzmann).
- La funzione rappresenta una probabilità, e infatti assume valori tra 0 e 1.

Questa funzione è monotona decrescente rispetto all'energia E : ciò significa che il sistema ha una probabilità maggiore di trovarsi in stati a bassa energia (basso costo) rispetto a stati ad alta energia. Il parametro T (temperatura) gioca un ruolo cruciale nel determinare il comportamento del sistema:

- **Se T è grande:** stati ad alta e bassa energia hanno approssimativamente la stessa probabilità. L'algoritmo esplora lo spazio delle soluzioni quasi casualmente.
- **Se T è piccolo:** gli stati a bassa energia sono molto più probabili. Il sistema tende a cristallizzarsi verso i minimi.

12.2.2 Funzionamento dell'Algoritmo

L'algoritmo Metropolis utilizza questa distribuzione di probabilità per decidere se accettare o meno una nuova soluzione. Fissata una temperatura T , l'algoritmo mantiene uno stato corrente S e procede come segue:

1. Viene generata una perturbazione casuale dello stato corrente S per ottenere un nuovo stato vicino $S' \in N(S)$.
2. Si calcola la variazione di energia (costo): $\Delta E = E(S') - E(S)$.
3. **Se $E(S') \leq E(S)$:** la nuova soluzione è migliore (o uguale). L'algoritmo accetta sempre il cambiamento e aggiorna lo stato corrente a S' .
4. **Altrimenti ($E(S') > E(S)$):** la nuova soluzione è peggiore ($\Delta E > 0$). L'algoritmo accetta il nuovo stato S' solo con una certa probabilità, data da:

$$\mathbb{P}(\text{accettare } S') = e^{-\frac{\Delta E}{kT}}$$

L'intuizione fondamentale è che il processo di ricerca non è casuale ma pesato: in accordo con la distribuzione di Gibbs-Boltzmann, l'algoritmo tende a soffermarsi con maggiore frequenza negli stati a basso costo. Di conseguenza, pur potendo accettare peggioramenti momentanei, il sistema privilegia statisticamente le zone migliori, aumentando la probabilità di convergere verso l'ottimo globale.

12.3 Neighbor Relation: Simulated Annealing

Il **Simulated Annealing** (Ricottura Simulata) rappresenta un'evoluzione dell'algoritmo Metropolis. È fondamentale sottolineare che il meccanismo di decisione rimane invariato: il Simulated Annealing utilizza la **stessa funzione di Gibbs-Boltzmann** vista in precedenza per calcolare la probabilità di accettare una mossa peggiorativa ($\Delta E > 0$):

$$\mathbb{P}(\text{accettare } S') = e^{-\frac{\Delta E}{kT}}$$

Mentre nell'algoritmo Metropolis la temperatura T è costante, il Simulated Annealing introduce una variazione dinamica del parametro T per bilanciare esplorazione e sfruttamento.

Il comportamento dell'algoritmo cambia drasticamente in funzione della temperatura:

- **Se T è grande:** la probabilità di accettare una mossa "in salita" (peggiorativa) è molto alta. In questa fase l'algoritmo esplora lo spazio quasi liberamente.
- **Se T è piccolo:** le mosse in salita non vengono quasi mai accettate. L'algoritmo si comporta in modo simile al Gradient Descent, raffinando la soluzione locale.

L'idea centrale è quella di utilizzare una "manopola" per controllare T durante l'esecuzione. Si definisce quindi una **Cooling Schedule** (piano di raffreddamento), ovvero una funzione $T(i)$ che determina il valore della temperatura all'iterazione i -esima, decrescendo progressivamente.

Analogia Fisica. Il nome e il funzionamento dell'algoritmo derivano direttamente dal processo fisico di ricottura (*annealing*) dei materiali cristallini:

- Se portiamo un solido ad alta temperatura, l'agitazione termica rompe i legami e non ci aspettiamo che mantenga una struttura cristallina ordinata.
- Se prendiamo un solido fuso e lo "congeliamo" molto bruscamente (raffreddamento rapido), le molecole non hanno il tempo di riorganizzarsi e non otterremo un cristallo perfetto, ma un solido amorfo o con difetti (metafora del minimo locale).
- L'**Annealing** (ricottura) consiste invece nel raffreddare il materiale gradualmente partendo da alte temperature. Questo permette al sistema di raggiungere l'equilibrio termico attraverso una successione di stati a temperatura intermedia, permettendo agli atomi di disporsi nella configurazione a minima energia (metafora del minimo globale).

Nello stesso modo, il Simulated Annealing parte con T alta per evitare di rimanere intrappolato subito in minimi locali, e abbassa T lentamente per permettere all'algoritmo di convergere verso il minimo globale.

Da sottolineare che **nessuna di queste tecniche garantisce di trovare sempre la soluzione ottima**, ma il Simulated Annealing, se opportunamente configurato (scelta della cooling schedule e del numero di iterazioni), funziona meglio rispetto all'algoritmo Metropolis o al Gradient Descent puro.

12.4 Problema: Max Cut

Un altro classico problema di ottimizzazione combinatoria che ben si presta all'approccio della ricerca locale è il **Max Cut**. A differenza del problema del Vertex Cover, dove l'obiettivo era minimizzare un costo, nel Max Cut l'obiettivo è **massimizzare** il valore della soluzione.

- Sia dato un insieme V di elementi (nodi) e un insieme E di coppie di elementi (archi). Ogni coppia $e \in E$ possiede un peso positivo $w_e > 0$.
- L'insieme C delle soluzioni ammissibili è costituito da tutte le possibili **partizioni** di V in due insiemi disgiunti (A, B) tali che $A \cup B = V$ e $A \cap B = \emptyset$.
- Una coppia $(u, v) \in E$ si dice "tagliata" (cut pair) se i due elementi appartengono a insiemi diversi della partizione (uno in A e l'altro in B).
- Il **cut value** (costo o valore) di una soluzione è la somma dei pesi delle coppie tagliate dalla partizione. L'obiettivo è trovare la partizione (A, B) che massimizzi questo costo totale.

Esempio. Consideriamo il seguente scenario:

- $V = \{a, b, c, d, e\}$
- $E = \{(a, b, 2), (a, c, 4), (b, d, 3), (c, d, 5), (d, e, 3)\}$

Analizziamo alcune partizioni per capire come calcolare il costo e distinguere soluzioni valide da quelle non valide:

- **Una soluzione valida (massimo locale):** $(\{a, b\}, \{c, d, e\})$. Costo = $4 + 3 = 7$.
- **Una soluzione non valida:** $(\{a, b\}, \{c, d\})$. Questa non è una soluzione ammissibile perché il nodo e non è stato assegnato a nessuno dei due insiemi (l'unione non è V).
- **La soluzione ottima (massimo globale):** $(\{a, d\}, \{b, c, e\})$. Costo Ottimo = $2 + 4 + 3 + 5 + 3 = 17$.

È doveroso fare una precisazione sulla complessità computazionale del problema. Sebbene in linea teorica sia sempre possibile determinare la soluzione ottima enumerando tutte le $2^{|V|}$ possibili partizioni (approccio *Brute Force*), tale metodo diviene rapidamente inapplicabile all'aumentare del numero di nodi a causa dell'esplosione combinatoria. Dunque, per il problema del Max Cut, così come per altri particolari problemi di ottimizzazione, non esiste alcun algoritmo noto in grado di trovare la soluzione ottima in **tempo polinomiale** rispetto alla dimensione dell'input. Di conseguenza, per istanze di dimensioni significative, la ricerca della soluzione esatta è computazionalmente intrattabile, rendendo necessario l'utilizzo di algoritmi di approssimazione o euristiche (come la Local Search) che forniscano soluzioni di buona qualità, pur senza garanzia di ottimalità assoluta.

Per applicare la ricerca locale, definiamo una neighbor relation semplice ed efficace.

Single-flip Neighborhood. Data una partizione corrente (A, B) , una partizione vicina (neighbor) si ottiene spostando esattamente un nodo da un insieme all'altro. Questa operazione è chiamata "single-flip".

Algoritmo Greedy. Possiamo implementare un semplice algoritmo Greedy (o di ascesa del gradiente - Gradient Ascent -, dato che stiamo massimizzando) basato su questo vicinato:

```
1. MaxCutLocal( $G, w$ ):
2.   Inizializza una partizione casuale  $(A, B)$  di  $V$ 
3.   while esiste un nodo  $v$  che migliora il cut value:
4.     if  $v$  in  $A$ : Sposta  $v$  da  $A$  a  $B$ 
5.     else:      Sposta  $v$  da  $B$  a  $A$ 
6.   return  $(A, B)$ 
```

Anche in questo caso, l'approccio Greedy può bloccarsi in ottimi locali. Per ottenere risultati migliori, si possono applicare le euristiche viste in precedenza, come il Simulated Annealing, o definire intorni più complessi che spostano più nodi contemporaneamente.

Finora abbiamo ipotizzato di muoverci nello spazio delle soluzioni spostando un singolo nodo alla volta. Tuttavia, la scelta della dimensione e della struttura del vicinato è determinante per la qualità della soluzione finale e per il tempo di esecuzione. Più ampio è il vicinato, minore è il rischio di bloccarsi in minimi locali, ma maggiore è il costo computazionale per esplorarlo.

Possiamo distinguere tre tipologie principali di vicinato:

- **1-flip neighborhood:** È il vicinato base utilizzato nell'algoritmo Greedy sopra descritto. Due partizioni (A, B) e (A', B') sono vicine se differiscono per la posizione di **esattamente un nodo**.
- **k-flip neighborhood:** È una generalizzazione del precedente. Due partizioni sono considerate vicine se differiscono per **al più k nodi**.
 - Questo approccio permette di scavalcare minimi locali che richiederebbero lo spostamento simultaneo di più elementi.
 - Tuttavia, il numero di vicini cresce come $\Theta(n^k)$. Questo rende l'esplorazione computazionalmente proibitiva (troppo costosa) per valori di k elevati.
- **KL-neighborhood (Kernighan-Lin):** Rappresenta un approccio sofisticato che permette di esplorare configurazioni molto distanti senza il costo esponenziale del k-flip.

L'approccio Kernighan-Lin. L'intuizione di Kernighan-Lin è quella di costruire un vicinato complesso attraverso una sequenza di mosse guidate. L'idea chiave è accettare temporaneamente partizioni con un **cut value** inferiore, pur di uscire da un ottimo locale ed esplorare configurazioni che potrebbero rivelarsi migliori a lungo termine.

Per generare i vicini di una partizione iniziale (A, B) , si procede nel seguente modo:

- **Iterazione 1:** Si valuta lo spostamento di ogni singolo nodo (da A a B o viceversa). Tra tutte le possibili mosse, si sceglie quella che porta alla nuova partizione (A_1, B_1) con il **cut value** più alto tra quelle disponibili.
 - *Punto cruciale:* Si seleziona la mossa che massimizza il cut value corrente, **anche se questo valore è inferiore** rispetto a quello della configurazione di partenza (A, B) .
 - In altre parole: se tutte le mosse possibili peggiorano la soluzione, scegliamo quella che la peggiora di meno. Questo passo è fondamentale per permettere all'algoritmo di "scendere" da un massimo locale per poi risalire verso uno globale.

Il nodo appena spostato viene "marcato" (bloccato) e non potrà essere mosso nuovamente in questa fase.

- **Iterazioni successive ($i > 1$):** Si ripete il procedimento considerando esclusivamente i nodi **non ancora marcati** (ovvero quelli che non sono stati spostati nelle iterazioni precedenti). Partendo dalla configurazione corrente (A_{i-1}, B_{i-1}) , si valuta lo spostamento di ciascun nodo disponibile e si sceglie quello che risulta nel miglior **cut value** per la nuova configurazione (A_i, B_i) .
 - Anche in questo caso, la scelta ricade sulla mossa migliore possibile al momento tra i nodi residui, indipendentemente dal fatto che il cut value aumenti o diminuisca rispetto al passo precedente.
 - Il nodo scelto viene spostato e marcato.
- **Conclusione della sequenza:** Il processo continua iterativamente finché **tutti** i nodi sono stati spostati e marcati esattamente una volta. Questo genera una sequenza ordinata di partizioni $(A_1, B_1), \dots, (A_n, B_n)$. Nota che l'ultima partizione (A_n, B_n) avrà tutti i nodi scambiati rispetto all'inizio (è la partizione speculare).

Il "vicinato" KL è costituito dall'insieme di tutte queste partizioni intermedie generate. L'algoritmo di ricerca locale esamina questa sequenza e si sposta nella configurazione (A_k, B_k) che ha il valore di taglio massimo tra tutte quelle generate. In pratica, questo metodo permette di eseguire una lunga sequenza di flip in un singolo passo logico dell'algoritmo, risultando un framework estremamente potente ed efficace.

12.5 Problema: Hopfield Neural Networks

Le **Reti Neurali di Hopfield** (Hopfield Neural Networks) rappresentano un modello semplice di memoria associativa, in cui una vasta collezione di unità (neuroni) è connessa da una rete sottostante, e le unità vicine cercano di correlare i propri stati. Il problema può essere modellato come segue:

- **Input:** Un insieme V di unità (nodi) e un insieme E di coppie (archi).
- **Pesi:** Ogni coppia $e \in E$ possiede un peso intero w_e , che può essere positivo o negativo.
- **Soluzioni Ammissibili (C):** Una configurazione è un assegnamento di uno stato $s_u \in \{+1, -1\}$ ad ogni unità $u \in V$.

12.5.1 Configurazioni Stabili

L'obiettivo in una rete di Hopfield non è necessariamente massimizzare o minimizzare una funzione globale in modo diretto, ma trovare una configurazione che sia "stabile", ovvero in cui nessun nodo ha incentivo a cambiare stato dato lo stato dei suoi vicini.

L'intuizione fisica dietro ai pesi è la seguente:

- Se $w_{uv} < 0$, allora u e v "vogliono" avere lo stesso stato (entrambi $+1$ o entrambi -1).
- Se $w_{uv} > 0$, allora u e v "vogliono" avere stati diversi.

Formalizziamo questo concetto definendo la qualità dei singoli archi e nodi:

- **Arco Buono (Good Edge):** Un arco $e = (u, v)$ si dice buono rispetto a una configurazione S se il prodotto del peso per gli stati è negativo:

$$w_e s_u s_v < 0$$

Questa è una quantità negativa se si tratta di un arco buono, positiva altrimenti.

- Si ha un arco buono se $w_e < 0$ e $s_u = s_v$, oppure se $w_e > 0$ e $s_u \neq s_v$.

- **Nodo Soddisfatto (Satisfied Node):** Un nodo u è soddisfatto se il peso assoluto degli archi incidenti "buoni" è maggiore o uguale al peso degli archi incidenti "cattivi". Formalmente, la condizione è:

$$\sum_{v: e=(u,v) \in E} w_{uv} s_u s_v \leq 0$$

- **Configurazione Stabile:** Una configurazione è stabile se **tutti** i nodi della rete sono soddisfatti.

L'obiettivo del problema di ricerca è trovare una configurazione stabile, se ne esiste una.

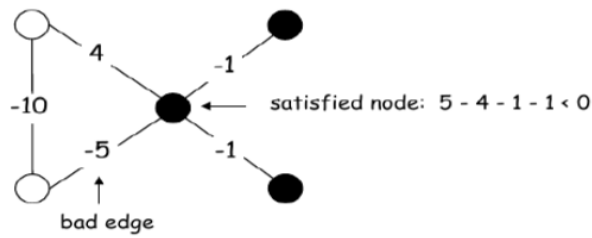


Figura 12.3: Esempio di nodo soddisfatto.

12.5.2 Algoritmo State-Flipping

Per trovare una configurazione stabile, utilizziamo un algoritmo di ricerca locale chiamato **State-flipping Algorithm**. La logica è estremamente semplice: partendo da una configurazione arbitraria, finché la rete non è stabile, si sceglie un nodo insoddisfatto e si inverte il suo stato.

-
1. HopfieldFlip(G, w):
 2. S = configurazione arbitraria
 3. **while** esiste un nodo insoddisfatto u in S :
 4. $s_u = -s_u$ # Inverti lo stato di u
 5. **return** S
-

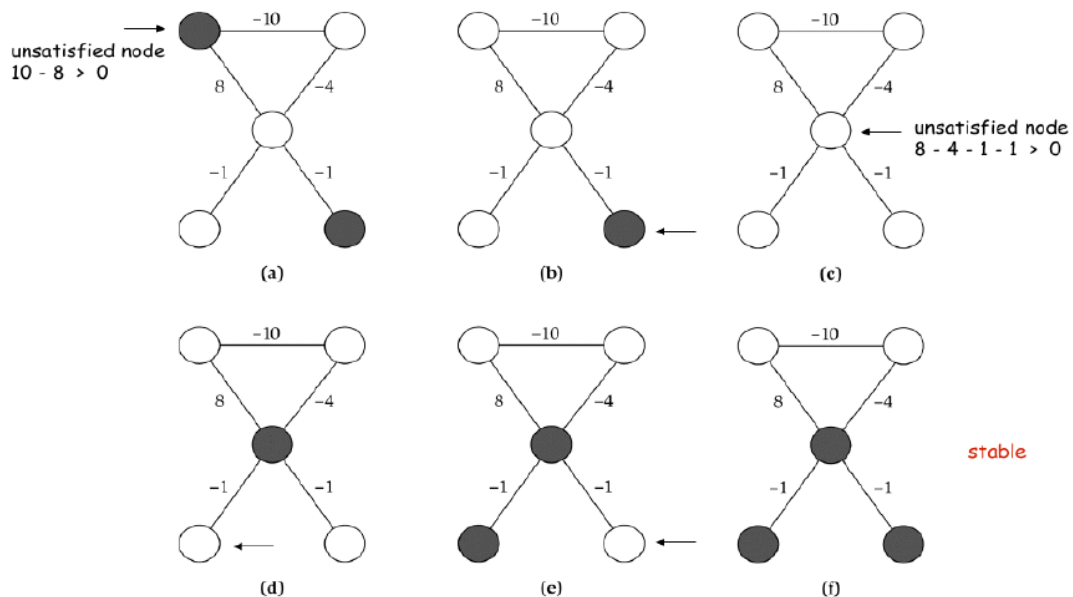


Figura 12.4: Le parti (a)-(f) mostrano i passi di un'esecuzione dell'algoritmo State-Flipping per una rete di Hopfield a cinque nodi, terminando in una configurazione stabile (i nodi sono colorati in nero o bianco per indicare il loro stato).

12.5.3 Convergenza e Complessità

L'algoritmo State-flipping termina sempre con una configurazione stabile dopo al più $W = \sum_e |w_e|$ iterazioni.

Consideriamo una funzione $\Phi(S)$ definita come la somma dei pesi assoluti di tutti gli archi "buoni" nella configurazione corrente:

$$\Phi(S) = \sum_{e \text{ good}} |w_e|$$

Chiaramente, il valore minimo di questa funzione è 0 e il massimo è W . Dobbiamo mostrare che $\Phi(S)$ aumenta di almeno 1 ad ogni flip:

- Quando un nodo insoddisfatto u cambia stato:
 - Tutti gli archi "buoni" incidenti a u diventano "cattivi".
 - Tutti gli archi "cattivi" incidenti a u diventano "buoni".
 - Gli archi non incidenti a u non cambiano.
- Poiché u era insoddisfatto, per definizione il peso degli archi cattivi incidenti era maggiore di quelli buoni.
- Di conseguenza, scambiando i due insiemi, il guadagno in termini di peso degli archi buoni supera la perdita:

$$\Phi(S') \geq 1 + \Phi(S)$$

Poiché la funzione Φ è limitata superiormente e cresce strettamente ad ogni passo (cioè fino a quando ci sono nodi insoddisfatti da flippare), l'algoritmo deve terminare al più in un numero finito di passi W .

Capitolo 13

Graphs

Un **grafo** è una struttura matematica che può essere utilizzata per rappresentare un insieme di relazioni binarie tra coppie di oggetti in una collezione: gli oggetti sono chiamati **vertici** (o nodi) e le relazioni tra di essi sono chiamate **archi**.

Definizione: Un grafo G è una coppia ordinata (V, E) dove $V = \{v_1, v_2, \dots, v_n\}$ è un insieme non vuoto di vertici e $E = \{u, v\}$ è una collezione di coppie di vertici (archi) di V .

Un arco può essere *orientato* o *non orientato*.

- Un arco (u, v) è orientato se la coppia (u, v) è ordinata, con u che precede v . Il primo vertice, u , è chiamato *sorgente* e il secondo vertice, v , è chiamato *destinazione*. Rappresenta una relazione unidirezionale (asimmetrica) da u a v .
- Un arco (u, v) è non orientato se la coppia (u, v) non è ordinata. Entrambi i vertici sono chiamati *estremi* dell'arco. Rappresenta una relazione bidirezionale (simmetrica) tra u e v .

Se tutti gli archi di un grafo sono orientati, allora si dice che il grafo è un *grafo orientato*. Allo stesso modo, un *grafo non orientato* è un grafo i cui archi sono tutti non orientati. Un grafo che ha sia archi orientati che non orientati è chiamato *grafo misto*.

La teoria dei grafi rappresenta una pietra miliare dell'informatica teorica e applicata, offrendo un linguaggio formale per modellare le relazioni tra oggetti. Questa struttura è onnipresente: dai *social network*, dove i nodi rappresentano gli utenti e gli archi le amicizie, alle reti di calcolatori e ai sistemi di navigazione satellitare. Algoritmi fondamentali, come quello di **Dijkstra** per il calcolo del cammino minimo o l'algoritmo **PageRank** di Google, si basano interamente sulle proprietà topologiche dei grafi. Senza l'astrazione fornita dai grafi, la risoluzione efficiente di problemi complessi di connettività, flusso e ottimizzazione sarebbe pressoché impossibile.

Terminologia sui Grafi: Con riferimento al grafo mostrato in Figura 13.1:

- **Estremi di un arco:** Gli estremi di un arco sono i vertici collegati dallo stesso arco. u e v sono gli estremi dell'arco a .
- **Vertici adiacenti:** Due vertici sono adiacenti se esiste un arco che li collega. u e v sono vertici adiacenti.
- **Archi incidenti in un vertice:** Gli archi incidenti in un vertice sono gli archi che hanno quel vertice come estremo. a, b, d sono archi incidenti in v .
- **Grado di un vertice:** Il grado di un vertice v è il numero di archi incidenti in v . x ha grado 5.
- **Archi paralleli:** Due archi sono paralleli se collegano gli stessi vertici. h e i sono archi paralleli.

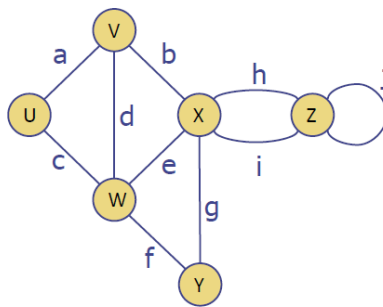


Figura 13.1

Terminologia sui Grafi Orientati: Con riferimento al grafo mostrato in Figura 13.2:

- **Archi entranti in un vertice:** Gli archi entranti in un vertice sono gli archi che hanno quel vertice come destinazione. b, e, h sono archi entranti in x .
- **Archi uscenti da un vertice:** Gli archi uscenti da un vertice sono gli archi che hanno quel vertice come sorgente. g, i , sono archi uscenti da x .
- **In-degree:** L'in-degree di un vertice è il numero di archi entranti in esso. x ha in-degree 3.
- **Out-degree:** L'out-degree di un vertice è il numero di archi uscenti da esso. x ha out-degree 2.

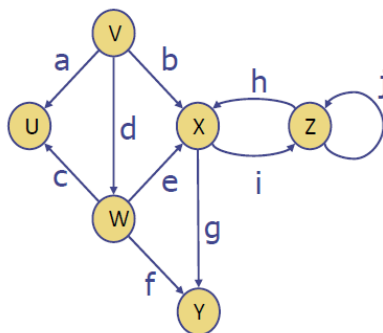


Figura 13.2

Un **cammino** (o percorso, path) in un grafo è una sequenza di vertici collegati da archi, ovvero una sequenza di vertici $(v_1, v_2, \dots, v_k)$ tale che (v_i, v_{i+1}) è un arco del grafo, per $i = 1, 2, \dots, k - 1$. È anche possibile vedere un cammino come una sequenza di archi $(e_1, e_2, \dots, e_{k-1})$ tale che l'estremo di destinazione di e_i è l'estremo sorgente di e_{i+1} , per $i = 1, 2, \dots, k - 1$. Un **cammino semplice** non visita uno stesso vertice più di una volta.

Un **cammino orientato** in un grafo orientato è una sequenza di vertici $(v_1, v_2, \dots, v_k)$ tale che ogni arco (v_i, v_{i+1}) è un arco orientato del grafo, per $i = 1, 2, \dots, k - 1$.

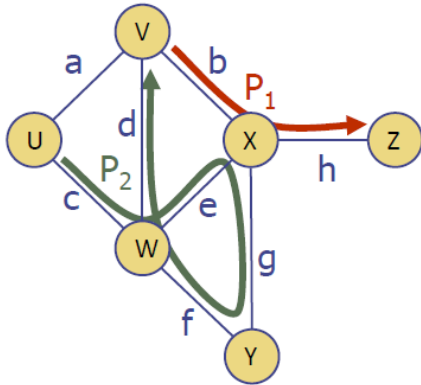


Figura 13.3: $P_1 = (v, x, z)$ è un cammino semplice; $P_2 = (u, w, x, y, w, v)$ è un cammino ma non semplice.

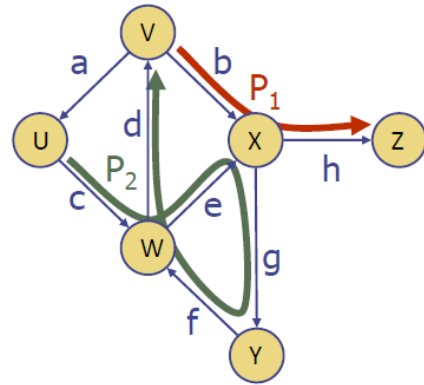


Figura 13.4: $P_1 = (v, x, z)$ è un cammino orientato semplice; $P_2 = (u, w, x, y, w, v)$ è un cammino orientato ma non semplice.

Un **ciclo** in un grafo è un cammino che inizia e termina nello stesso vertice, con almeno un arco. Un **ciclo semplice** non visita uno stesso vertice più di una volta, ad eccezione del vertice iniziale/finale. Un **auto-ciclo** è un ciclo che consiste in un singolo arco che collega un vertice a se stesso. Un **grafo semplice** è un grafo che non contiene *auto-cicli* e non presenta più archi tra la stessa coppia di vertici (archi paralleli).

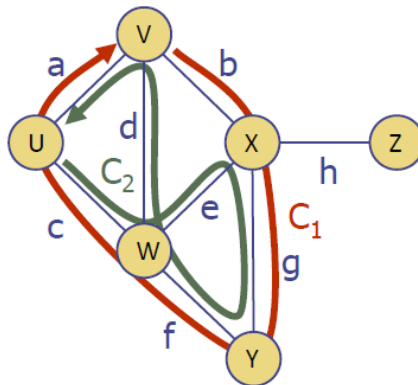


Figura 13.5: $C_1 = (v, x, y, w, u, v)$ è un ciclo semplice; $C_2 = (u, w, x, y, w, v, u)$ è un ciclo ma non semplice.

Proprietà di un Grafo non orientato

Siano n il numero di vertici e m il numero di archi di un grafo non orientato (Figura 13.6).

1. $\sum_{v \in V} \deg(v) = 2m$, dove la somma è calcolata su tutti i vertici v del grafo.
 - Questo perché ogni arco contribuisce a incrementare il grado di due vertici.
2. In un grafo non orientato senza archi paralleli e auto-cicli (ovvero un *grafo semplice*), il numero massimo di archi è dato da $m \leq \frac{n(n-1)}{2}$.
 - Per via delle ipotesi di cui sopra, ciascun nodo può avere grado massimo $(\leq)n - 1$. Riprendendo la proprietà precedente, si ha quindi che $2m \leq n(n - 1)$.

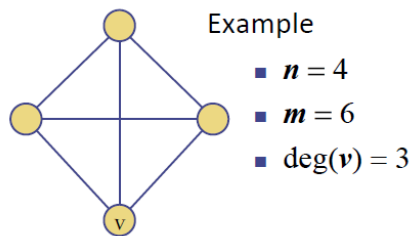


Figura 13.6

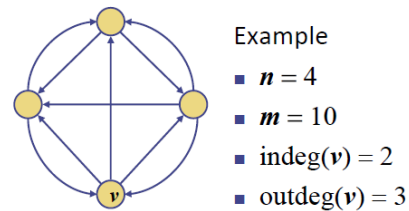


Figura 13.7

Proprietà di un Grafo orientato

Siano n il numero di vertici e m il numero di archi di un grafo orientato (Figura 13.7).

1. $\sum_{v \in V} \text{in-deg}(v) = \sum_{v \in V} \text{out-deg}(v) = m$, dove la somma è calcolata su tutti i vertici del grafo.
 - Questo perché ogni arco contribuisce di uno all'in-degree di un vertice e di uno all'out-degree di un altro vertice.
2. In un grafo orientato senza archi paralleli e auto-cicli, il numero massimo di archi è dato da $m \leq n(n - 1)$.
 - Per via delle ipotesi di cui sopra, ciascun nodo può avere grado massimo $(\leq)n - 1$. Riprendendo la proprietà precedente, si ha quindi che $m \leq n(n - 1)$.

Due vertici u, v di un grafo (orientato) sono **connessi** se esiste un cammino (orientato) da u a v . In un grafo non orientato, la *connettività* è una relazione simmetrica. In un grafo orientato, invece, la connettività non è necessariamente simmetrica: potrebbe esistere un cammino da u a v ma non da v a u .

Un grafo non orientato si dice **connesso** se, per ogni coppia di vertici, esiste un cammino tra di essi. Un grafo orientato è **debolmente connesso** se, ignorando l'orientamento degli archi, il grafo non orientato risultante è connesso. Un grafo orientato è **fortemente connesso** se per ogni coppia di vertici (u, v) , u raggiunge v e v raggiunge u .

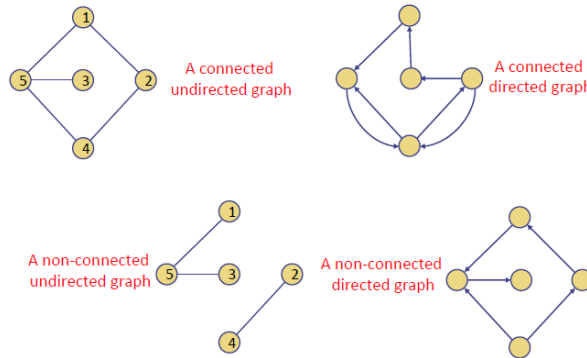


Figura 13.8: Esempi di grafi connessi e non connessi.

Una **forest** è un grafo non orientato e aciclico (ovvero privo di cicli). Un **tree** è una forest connessa. In altre parole, un tree è un grafo non orientato, aciclico e connesso.

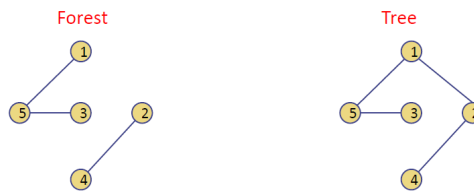


Figura 13.9: Esempi di tree e forest.

Un **sottografo** H di un grafo $G = (V, E)$ è un grafo $H = (V', E')$ tale che $V' \subseteq V$ e $E' \subseteq E$. In altre parole, un sottografo è ottenuto rimuovendo vertici e/o archi da G . Uno **spanning subgraph** (la traduzione corretta è "sottografo ricoprente") di un grafo $G = (V, E)$ è un sottografo $H = (V', E')$ tale che $V' = V$. In altre parole, uno spanning subgraph contiene tutti i vertici di G , ma potrebbe non contenere tutti gli archi di G .

Uno **spanning tree** di un grafo è uno spanning subgraph che è anche un tree (ricordiamo, tree = grafo non orientato, aciclico e connesso).

Dato un grafo non orientato G con n vertici e m archi:

- Se G è connesso, allora $m \geq n - 1$.
- Se G è una forest, allora $m \leq n - 1$.
- Se G è un tree, allora $m = n - 1$.

13.1 Rappresentazioni dell'ADT Grafo

Esistono diverse rappresentazioni possibili di un grafo. In ognuna di esse, manteniamo una collezione per memorizzare i vertici di un grafo. Tuttavia, le rappresentazioni differiscono notevolmente nel modo in cui organizzano gli archi.

- **Edge List:** in un edge list, manteniamo una lista non ordinata di tutti gli archi. Questo è il minimo indispensabile, ma non esiste un modo efficiente per localizzare un particolare arco (u,v) , o l'insieme di tutti gli archi incidenti a un vertice v .
- **Adjacency List:** in un adjacency list, manteniamo, per ogni vertice, una lista separata contenente quegli archi che sono incidenti al vertice. L'insieme completo degli archi può essere determinato prendendo l'unione dei set più piccoli, mentre l'organizzazione consente di trovare in modo più efficiente tutti gli archi incidenti a un dato vertice.
- **Adjacency Map:** un adjacency map è molto simile a un adjacency list, ma il contenitore secondario di tutti gli archi incidenti a un vertice è organizzato come una mappa, anziché come una lista, con il vertice adiacente che funge da chiave. Ciò consente l'accesso a un arco specifico (u,v) in tempo $O(1)$ atteso.
- **Adjacency Matrix:** un adjacency matrix fornisce un accesso nel caso peggiore $O(1)$ a un arco specifico (u,v) mantenendo una matrice $n \times n$, per un grafo con n vertici. Ogni voce è dedicata a memorizzare un riferimento all'arco (u,v) per una particolare coppia di vertici u e v ; se non esiste tale arco, la voce sarà None.

Operation	Edge List	Adj. List	Adj. Map	Adj. Matrix
vertex_count()	$O(1)$	$O(1)$	$O(1)$	$O(1)$
edge_count()	$O(1)$	$O(1)$	$O(1)$	$O(1)$
vertices()	$O(n)$	$O(n)$	$O(n)$	$O(n)$
edges()	$O(m)$	$O(m)$	$O(m)$	$O(m)$
get_edge(u,v)	$O(m)$	$O(\min(d_u, d_v))$	$O(1)$ exp.	$O(1)$
degree(v)	$O(m)$	$O(1)$	$O(1)$	$O(n)$
incident_edges(v)	$O(m)$	$O(d_v)$	$O(d_v)$	$O(n)$
insert_vertex(x)	$O(1)$	$O(1)$	$O(1)$	$O(n^2)$
remove_vertex(v)	$O(m)$	$O(d_v)$	$O(d_v)$	$O(n^2)$
insert_edge(u,v,x)	$O(1)$	$O(1)$	$O(1)$ exp.	$O(1)$
remove_edge(e)	$O(1)$	$O(1)$	$O(1)$ exp.	$O(1)$

Figura 13.10: Un riepilogo dei tempi di esecuzione per i metodi dell'ADT grafo, utilizzando le rappresentazioni del grafo discusse in questa sezione. Indichiamo con n il numero di vertici, m il numero di archi, e d_v il grado del vertice v . Si noti che la matrice di adiacenza utilizza uno spazio $O(n^2)$, mentre tutte le altre strutture utilizzano uno spazio $O(n + m)$.

13.2 Rappresentazione mediante Edge List

L'*edge list* è la rappresentazione più semplice possibile di un grafo, anche se non la più efficiente. Tutti gli oggetti vertice sono memorizzati in una lista non ordinata V , e tutti gli oggetti arco sono memorizzati in una lista non ordinata E .

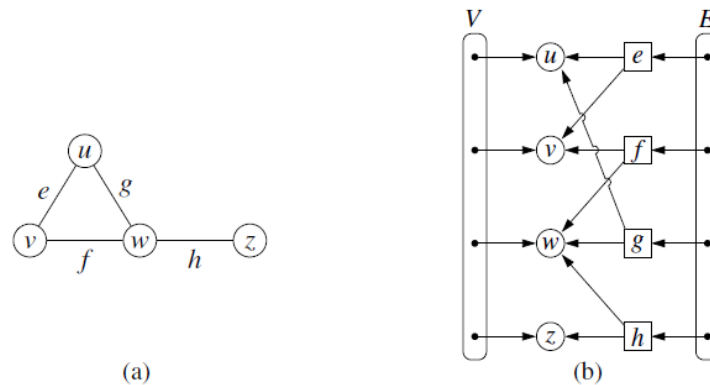


Figura 13.11: (a) Un grafo G ; (b) rappresentazione schematica della struttura edge list per G . Si noti che un oggetto arco fa riferimento ai due oggetti vertice che corrispondono ai suoi estremi, ma che i vertici non fanno riferimento agli archi incidenti per cui non mantengono questa informazione.

Per supportare i metodi dell'ADT Grafo, assumiamo le seguenti caratteristiche aggiuntive di una rappresentazione edge list. Le collezioni V ed E sono rappresentate come doubly linked list.

- Oggetto **Vertex**: rappresenta un vertice del grafo. Mantiene un riferimento al suo elemento (ad esempio, una stringa o un numero) e un riferimento alla sua posizione nella lista V del grafo.
- Oggetto **Edge**: rappresenta un arco del grafo. Mantiene un riferimento al suo elemento (ad esempio, un peso o un'etichetta) e riferimenti ai due vertici estremi dell'arco. Inoltre, mantiene un riferimento alla sua posizione nella lista E del grafo.

Space: $O(n+m)$

- $O(n)$ for V and $O(m)$ for E

Time:

- $O(1)$ for `vertex_count()` e `edge_count()`, $O(n)$ for `vertices()` e $O(m)$ for `edges()`
 - Based on methods of PositionalList
- $O(m)$ for `get_edge(u, v)`, `degree(v)` and `incident_edges(v)`
 - We have to scan the whole list of edges
- $O(1)$ for `insert_vertex(x)`, `insert_edge(u, v, x)` and `remove_edge(e)`
 - Insert in a PositionalList
 - `remove_edge(e)` takes time $O(1)$ since Edge contains a reference to its position in the list of edges
- $O(m)$ for `remove_vertex(v)`
 - We have to scan the whole list of edges and remove all edges incident on v

Figura 13.12: Riepilogo tempi di esecuzione ADT Grafo con rappresentazione Edge List.

13.3 Rappresentazione mediante Adjacency List

Diversamente dall'edge list, l'*adjacency list* è una rappresentazione di un grafo che organizza gli archi in modo più strutturato, raggruppandoli per vertice. Ciascun vertice mantiene una collezione separata di archi incidenti, consentendo un accesso più efficiente agli archi associati a un particolare vertice. Più di preciso, ogni vertice v del grafo mantiene una collezione $I(v)$, chiamata *incidence collection* di v . (Nel caso di un grafo orientato, gli archi in uscita e in entrata possono essere memorizzati rispettivamente in due collezioni separate, $I_{out}(v)$ e $I_{in}(v)$.) Tradizionalmente, la collezione di incidenza $I(v)$ per un vertice v è una lista, motivo per cui chiamiamo questo modo di rappresentare un grafo la struttura *adjacency list*.

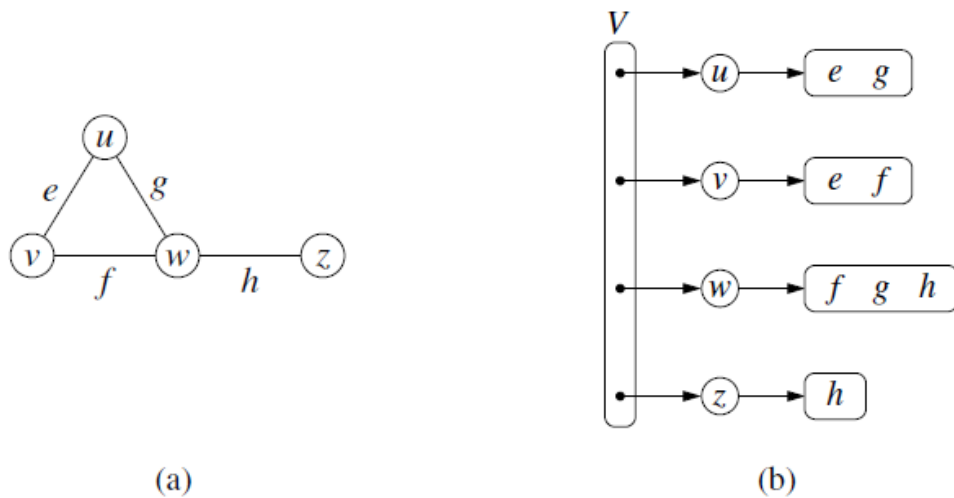


Figura 13.13: (a) Un grafo non orientato G ; (b) una rappresentazione schematica della struttura adjacency list per G . La collezione V è la lista primaria di vertici, e ogni vertice ha una lista associata di archi incidenti. Sebbene non sia mostrato in figura, presumiamo che ogni arco del grafo sia rappresentato con un'istanza di Edge unica che mantiene riferimenti ai suoi vertici estremi.

Richiediamo che la struttura primaria per un adjacency list mantenga la collezione V di vertici in modo tale da poter localizzare la struttura secondaria $I(v)$ per un dato vertice v in tempo $O(1)$. Ciò potrebbe essere fatto utilizzando una positional list per rappresentare V , con ogni istanza di Vertex che mantiene un riferimento diretto alla sua collezione di incidenza $I(v)$. Se i vertici possono essere numerati in modo univoco da 0 a $n - 1$, potremmo invece utilizzare una struttura primaria basata su array per accedere alle liste secondarie appropriate. Il vantaggio principale di un adjacency list è che la collezione $I(v)$ contiene esattamente quegli archi che dovrebbero essere riportati dal metodo `incident_edges(v)`. Pertanto, possiamo implementare questo metodo iterando sugli archi di $I(v)$ in tempo $O(\deg(v))$, dove $\deg(v)$ è il grado del vertice v . Questo è il miglior risultato possibile per qualsiasi rappresentazione di grafo, poiché ci sono $\deg(v)$ archi da riportare.

L'utilizzo primario di questa rappresentazione riguarda i **grafi sparsi**¹, tipici di reti reali (web, social network, mappe stradali), dove la maggior parte delle coppie di nodi non è collegata. Risulta inoltre ottimale per algoritmi che richiedono l'esplorazione sistematica dei vicini di un nodo. Lo svantaggio riguarda la verifica dell'esistenza di uno specifico arco tra due nodi: tale operazione non è istantanea, ma richiede una ricerca all'interno della collezione di adiacenza del vertice, con un tempo di esecuzione che cresce all'aumentare del numero di vicini del nodo stesso.

Space: $O(n+m)$

- $O(n)$ for V and $O(m)$ for all the adjacency lists ($\sum_v \deg(v) = 2m$)

Time:

- $O(1)$ for `vertex_count()` and `degree(v)`, $O(n)$ for `vertices()`, $O(n+m)$ `edges()`
 - Based on methods of PositionalList
 - $O(1)$ for `edge_count()` if we maintain an auxiliary list of all the edges
- $O(\deg(v))$ for `get_edge(u, v)` and `incident_edges(v)`
 - We have to scan only elements in an adjacency list
- $O(1)$ per `insert_vertex(x)`, `insert_edge(u, v, x)` e `remove_edge(e)`
 - Insert in a PositionalList
 - $O(1)$ for `remove_edge(e)` since Edge has a reference to its position in the lists of its adjacent vertices
- $O(\deg(v))$ for `remove_vertex(v)`
 - Edge has a reference to its position in the lists of its adjacent vertices

Figura 13.14: Riepilogo tempi di esecuzione ADT Grafo con rappresentazione Adjacency List.

In una rappresentazione di tipo adjacency list, assumiamo che la struttura dati secondaria utilizzata per la adjacency list sia una linked list non ordinata. Tale collezione $I(v)$ utilizza uno spazio proporzionale a $O(\deg(v))$, consente di aggiungere o rimuovere un arco in tempo $O(1)$ e consente di iterare su tutti gli archi incidenti al vertice v in tempo $O(\deg(v))$. Tuttavia, la migliore implementazione del metodo `get_edge(u, v)` richiede un tempo di $O(\min(\deg(u), \deg(v)))$, poiché dobbiamo cercare all'interno di $I(u)$ o $I(v)$.

¹Un **grafo sparso** è un grafo che ha un numero di archi (collegamenti) significativamente inferiore rispetto al numero massimo possibile, ovvero quando il numero di archi m è dell'ordine di $O(n)$ (lineare) rispetto al numero di vertici n , mentre un **grafo denso** ha $m \approx O(n^2)$ (quadratico). In pratica, i grafi sparsi hanno poche connessioni tra i nodi, rendendo le rappresentazioni come le liste di adiacenza più efficienti in termini di spazio rispetto alle matrici di adiacenza.

13.4 Rappresentazione mediante Adjacency Map

Possiamo migliorare le prestazioni viste finora utilizzando una mappa basata su hash per implementare $I(v)$ (incidence collection) per ogni vertice v . In particolare, lasciamo che l'estremo opposto di ogni arco incidente serva come chiave nella mappa, con la struttura dell'arco che funge da valore. Chiamiamo tale rappresentazione del grafo una *adjacency map*. L'uso dello spazio per un adjacency map rimane $O(n + m)$, poiché $I(v)$ utilizza uno spazio $O(\deg(v))$ per ogni vertice v , come nell'adjacency list. Il vantaggio dell'adjacency map, rispetto all'adjacency list, è che il metodo `get_edge(u, v)` può essere implementato in tempo atteso $O(1)$ cercando il vertice u come chiave in $I(v)$, o viceversa. Ciò fornisce un probabile miglioramento rispetto all'adjacency list, pur mantenendo il limite nel caso peggiore di $O(\min(\deg(u), \deg(v)))$. Nel confrontare le prestazioni dell'adjacency map con altre rappresentazioni, scopriamo che esso raggiunge essenzialmente tempi di esecuzione ottimali per tutti i metodi, rendendolo un'ottima scelta in generale come rappresentazione del grafo.

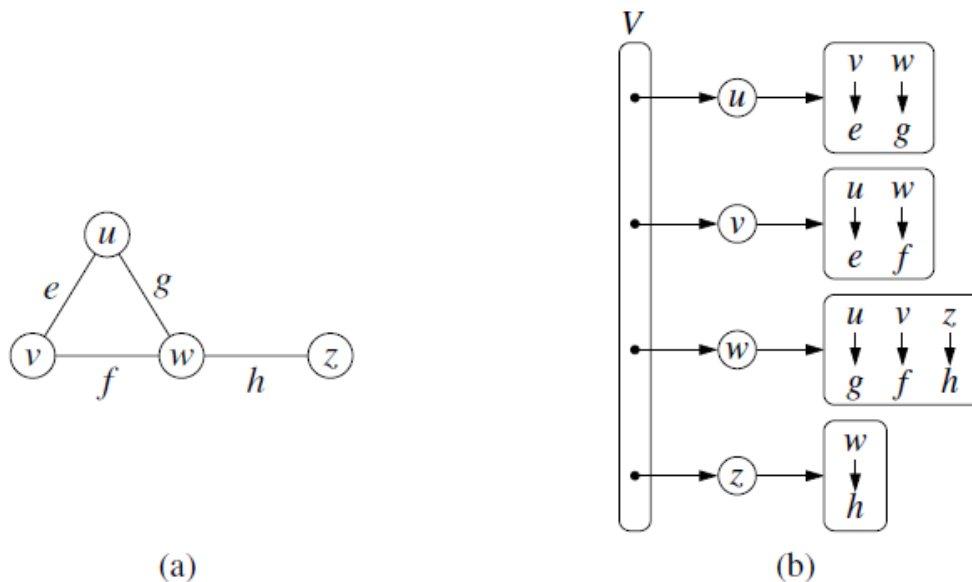


Figura 13.15: (a) un grafo non orientato G ; (b) una rappresentazione schematica della struttura adjacency map per G . Ogni vertice mantiene una mappa secondaria in cui i vertici adiacenti fungono da chiavi, con gli archi di collegamento come valori associati. Sebbene non sia rappresentato come tale, presumiamo che esista un'istanza Edge unica per ogni arco del grafo, e che essa mantenga riferimenti ai suoi vertici estremi.

Questa struttura è frequentemente adottata in contesti che richiedono alta dinamicità e prestazioni bilanciate, come nelle librerie di manipolazione grafi general-purpose. Il vantaggio distintivo è l'utilizzo di una struttura di hashing per i collegamenti uscenti, che permette di accedere a uno specifico arco e di verificarne l'esistenza con un tempo atteso costante $O(1)$, pur mantenendo un consumo di memoria lineare rispetto al contenuto del grafo. Ciò facilita operazioni veloci di inserimento e rimozione degli archi. Di contro, la gestione interna delle tabelle hash introduce un certo *overhead* di memoria e complessità computazionale per la gestione delle collisioni, rendendo la struttura leggermente più pesante per grafi di dimensioni molto ridotte o banali.

Space: $O(n+m)$

- $O(n)$ for V and $O(m)$ for all the adjacency maps ($\sum_v \deg(v) = 2m$)

Time:

- $O(1)$ for `vertex_count()`, `edge_count()` and `degree(v)`, $O(n)$ for `vertices()`, $O(n+m)$ for `edges()`
 - As for Adjacency List
- Expected $O(1)$ for `get_edge(u, v)`, `insert_edge(u, v, x)` and `remove_edge(e)`; $O(1)$ for `insert_vertex(x)`
 - Based on methods of the Adjacency Map implemented with an HashTable
- $O(\deg(v))$ for `remove_vertex(v)` and `incident_edges(v)`
 - Iterates over all the elements of the Adjacency Map

Figura 13.16: Riepilogo tempi di esecuzione ADT Grafo con rappresentazione Adjacency Map.

13.5 Rappresentazione mediante Adjacency Matrix

La struttura *adjacency matrix* per un grafo G arricchisce la struttura edge list con una matrice A , che ci consente di localizzare un arco tra una data coppia di vertici in tempo costante nel caso peggiore. Nella rappresentazione adjacency matrix, consideriamo i vertici come gli interi nell'insieme $\{0, 1, \dots, n-1\}$ e gli archi come coppie di tali interi. Ciò ci consente di memorizzare riferimenti agli archi nelle celle di un array bidimensionale $n \times n$ A . In particolare, la cella $A[i, j]$ contiene un riferimento all'arco (u, v) , se esiste, dove u è il vertice con indice i e v è il vertice con indice j . Se non esiste tale arco, allora $A[i, j] = \text{None}$. Notiamo che l'array A è simmetrico se il grafo G è non orientato, poiché $A[i, j] = A[j, i]$ per tutte le coppie i e j . Il vantaggio più significativo di una matrice di adiacenza è che qualsiasi arco (u, v) può essere visitato in tempo $O(1)$. Tuttavia, diverse operazioni sono meno efficienti con una matrice di adiacenza. Ad esempio, per trovare gli archi incidenti al vertice v , dobbiamo presumibilmente esaminare tutte le n voci nella riga associata a v ; ricordiamo che un adjacency list o map può localizzare quegli archi in tempo ottimale $O(\deg(v))$. Aggiungere o rimuovere vertici da un grafo è problematico, poiché la matrice deve essere ridimensionata. Inoltre, l'uso dello spazio $O(n^2)$ di una matrice di adiacenza è tipicamente molto peggiore rispetto allo spazio $O(n + m)$ richiesto dalle altre rappresentazioni. Sebbene, nel caso peggiore, il numero di archi in un grafo denso sia proporzionale a n^2 , la maggior parte dei grafi del mondo reale sono sparsi. In tali casi, l'uso di una matrice di adiacenza è inefficiente. Tuttavia, se un grafo è denso, le costanti di proporzionalità di una matrice di adiacenza possono essere più piccole rispetto a quelle di una lista o mappa di adiacenza. Infatti, se gli archi non hanno dati ausiliari, una matrice di adiacenza booleana può utilizzare un bit per ogni slot di arco, in modo tale che $A[i, j] = \text{True}$ se e solo se l'arco associato (u, v) esiste.

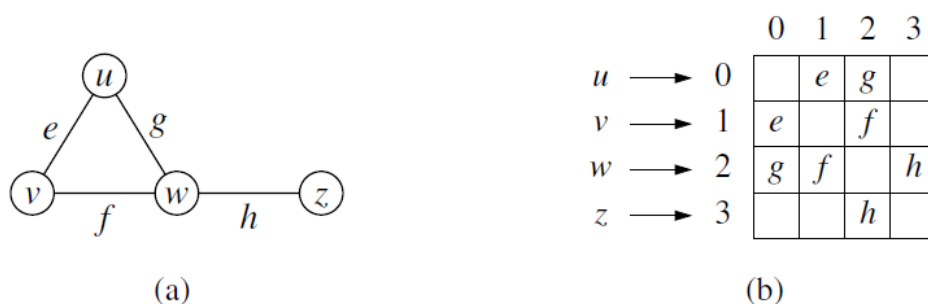


Figura 13.17: (a) un grafo non orientato G ; (b) una rappresentazione schematica della struttura di matrice di adiacenza per G , in cui n vertici sono mappati agli indici da 0 a $n-1$. Sebbene non sia mostrato in figura, presumiamo che esista un'istanza Edge unica per ogni arco, e che essa mantenga riferimenti ai suoi vertici estremi. Presumiamo inoltre che esista una lista secondaria di archi (non illustrata), per consentire al metodo `edges()` di funzionare in tempo $O(m)$, per un grafo con m archi.

Questa struttura è indicata principalmente per la gestione di **grafi densi** o di dimensioni contenute e statiche, dove il numero di archi è elevato rispetto ai vertici. Il suo vantaggio fondamentale risiede nell'accesso immediato ai dati: verificare l'esistenza di un collegamento tra due nodi specifici avviene in tempo costante $O(1)$ tramite indirizzamento diretto degli indici. Tuttavia, questa rappresentazione impone un utilizzo della memoria quadratico $O(n^2)$ indipendentemente dal numero reale di archi, comportando un significativo spreco di spazio nei casi in cui i collegamenti siano scarsi. Inoltre, le operazioni di ridimensionamento del grafo, come l'aggiunta o la rimozione di vertici, risultano computazionalmente onerose poiché richiedono la riallocazione dell'intera struttura dati.

Space: $O(n^2)$

- Good for dense graphs
- For sake of efficiency, it maintains also the edge list

Time:

- $O(1)$ for `vertex_count()`, `get_edge(u, v)`, `insert_edge(u, v, x)` and `remove_edge(e)`
 - Directly access to an element of the matrix
- $O(n)$ for `vertices()`, $O(m)$ for `edges()`
 - Uses an auxiliary edge list
- $O(n)$ for `degree(v)` and `incident_edges(v)`
 - Scan a row of the matrix
- $O(n^2)$ for `insert_vertex(x)`, `remove_vertex(v)`
 - We must add/remove a row and a column to the matrix

Figura 13.18: Riepilogo tempi di esecuzione ADT Grafo con rappresentazione Adjacency Matrix.

13.6 Implementazione Python (Adjacency Map)

In questa sezione, forniamo un'implementazione dell'ADT Grafo. La nostra implementazione supporterà grafi orientati o non orientati, ma per facilità di spiegazione, la descriviamo prima nel contesto di un grafo non orientato.

Utilizziamo una variante della rappresentazione **adjacency map**. Per ogni vertice v , utilizziamo un dizionario Python per rappresentare la mappa di incidenza secondaria $I(v)$. Tuttavia, non manteniamo esplicitamente le liste V ed E , come originariamente descritto nella rappresentazione edge list. La lista V è sostituita da un dizionario di primo livello D che mappa ogni vertice v alla sua mappa di incidenza $I(v)$; si noti che possiamo iterare su tutti i vertici generando l'insieme delle chiavi per il dizionario D . Utilizzando tale dizionario D per mappare i vertici alle mappe di incidenza secondarie, non è necessario mantenere riferimenti a tali mappe di incidenza come parte delle strutture dei vertici. Inoltre, un vertice non ha bisogno di mantenere esplicitamente un riferimento alla sua posizione in D , poiché può essere determinato in tempo atteso $O(1)$. Ciò semplifica notevolmente la nostra implementazione. Tuttavia, una conseguenza del nostro design è che alcuni dei limiti di tempo di esecuzione nel caso peggiore per le operazioni dell'ADT grafo, dati nella Tabella 13.10, **diventano limiti attesi**. Piuttosto che mantenere la lista E , ci accontentiamo di prendere l'unione degli archi trovati nelle varie mappe di incidenza; tecnicamente, ciò richiede un tempo $O(n + m)$ anziché strettamente $O(m)$, poiché il dizionario D ha n chiavi, anche se alcune mappe di incidenza sono vuote.

Da notare che abbiamo i metodi `__hash__` per entrambe le classi *Vertex* e *Edge* in modo che tali istanze possano essere utilizzate come chiavi negli insiemi e nei dizionari basati su hash di Python. I grafi sono non orientati per impostazione predefinita, ma possono essere dichiarati come orientati con un parametro opzionale al costruttore.

Internamente, gestiamo il caso orientato avendo due diverse istanze di dizionario di primo livello, `_outgoing` e `_incoming`, in modo tale che `_outgoing[v]` mappi a un altro dizionario che rappresenta $I_{out}(v)$, e `_incoming[v]` mappi a una rappresentazione di $I_{in}(v)$. Per unificare il nostro trattamento dei grafi orientati e non orientati, continuiamo a utilizzare gli identificatori `_outgoing` e `_incoming` nel caso non orientato, ma come alias dello stesso dizionario. Per comodità, definiamo un'utilità denominata `is_directed` per consentirci di distinguere tra i due casi.

Per i metodi `degree` e `incident_edges`, che accettano ciascuno un parametro opzionale per differenziare tra le orientazioni in uscita e in entrata, scegliamo la mappa appropriata prima di procedere. Per il metodo `insert_vertex`, inizializziamo sempre `_outgoing[v]` a un dizionario vuoto per il nuovo vertice v . Nel caso orientato, inizializziamo indipendentemente anche `_incoming[v]`. Per il caso non orientato, tale passaggio non è necessario poiché `_outgoing` e `_incoming` sono alias.

```

1 #----- nested Vertex class -----
2 class Vertex:
3     """Lightweight vertex structure for a graph."""
4     __slots__ = '_element'
5
6     def __init__(self, x):
7         """Do not call constructor directly. Use Graph's
8             insert_vertex(x)."""
9         self._element = x
10
11     def element(self):
12         """Return element associated with this vertex."""
13         return self._element
14
15     def __hash__(self):    # will allow vertex to be a map/set key
16         return hash(id(self))
17
18 #----- nested Edge class -----
19 class Edge:
20     """Lightweight edge structure for a graph."""
21     __slots__ = '_origin', '_destination', '_element'
22
23     def __init__(self, u, v, x):
24         """Do not call constructor directly. Use Graph's
25             insert_edge(u,v,x)."""
26         self._origin = u
27         self._destination = v
28         self._element = x
29
30     def endpoints(self):
31         """Return (u,v) tuple for vertices u and v."""
32         return (self._origin, self._destination)
33
34     def opposite(self, v):
35         """Return the vertex that is opposite v on this edge."""
36         return self._destination if v is self._origin else self._origin
37
38     def element(self):
39         """Return element associated with this edge."""
40         return self._element
41
42     def __hash__(self):    # will allow edge to be a map/set key
43         return hash((self._origin, self._destination))

```

Listing 13.1: Classi Vertex ed Edge (da annidare all'interno della classe Graph).

Listing 13.2: Classe Graph con rappresentazione Adjacency Map.

```

1 class Graph:
2     """Representation of a simple graph using an adjacency map."""
3
4     def __init__(self, directed=False):
5         """Create an empty graph (undirected, by default).
6
7         Graph is directed if optional parameter is set to True.
8         """
9         self._outgoing = {}
10        # only create second map for directed graph; use alias for
11        # undirected
12        self._incoming = {} if directed else self._outgoing
13
14    def is_directed(self):
15        """Return True if this is a directed graph; False if undirected.
16
17        Property is based on the original declaration of the graph, not its
18        contents.
19        """
20        return self._incoming is not self._outgoing # directed if maps are
21        distinct
22
23    def vertex_count(self):
24        """Return the number of vertices in the graph."""
25        return len(self._outgoing)
26
27    def vertices(self):
28        """Return an iteration of all vertices of the graph."""
29        return self._outgoing.keys()
30
31    def edge_count(self):
32        """Return the number of edges in the graph."""
33        total = 0
34        for v in self._outgoing:
35            total += len(self._outgoing[v])
36        # for undirected graphs, make sure not to double-count edges
37        return total // 2 if not self.is_directed() else total
38
39    def edges(self):
40        """Return a set of all edges of the graph."""
41        result = set() # avoid double-reporting edges of undirected
42        graph
43        for secondary_map in self._outgoing.values():
44            result.update(secondary_map.values()) # add edges to resulting
45            set
46        return result

```

```

42
43 def get_edge(self, u, v):
44     """Return the edge from u to v, or None if not adjacent."""
45     return self._outgoing[u].get(v) # returns None if v not adjacent
46
47 def degree(self, v, outgoing=True):
48     """Return number of (outgoing) edges incident to vertex v in the
49         graph.
50
51         If graph is directed, optional parameter used to count incoming
52         edges.
53     """
54     adj = self._outgoing if outgoing else self._incoming
55     return len(adj[v])
56
57 def incident_edges(self, v, outgoing=True):
58     """Return all (outgoing) edges incident to vertex v in the graph.
59
60         If graph is directed, optional parameter used to request incoming
61         edges.
62     """
63     adj = self._outgoing if outgoing else self._incoming
64     for edge in adj[v].values():
65         yield edge
66
67 def insert_vertex(self, x=None):
68     """Insert and return a new Vertex with element x."""
69     v = self.Vertex(x)
70     self._outgoing[v] = {}
71     if self.is_directed():
72         self._incoming[v] = {} # need distinct map for incoming edges
73     return v
74
75 def insert_edge(self, u, v, x=None):
76     """Insert and return a new Edge from u to v with auxiliary element
77         x."""
78     e = self.Edge(u, v, x)
79     self._outgoing[u][v] = e
80     self._incoming[v][u] = e

```

Listing 13.3: Metodi di cancellazione per la classe Graph.

```
76
77 def remove_vertex(self, v):
78     """Remove vertex v and all its incident edges from the graph."""
79     # Phase 1: Remove references to v from the adjacency maps of its
80     neighbors.
81
82     # For every neighbor w connected by an outgoing edge (v -> w),
83     # remove the link back to v from w's incoming map.
84     for w in list(self._outgoing[v]):
85         del self._incoming[w][v]
86
87     # If the graph is directed, we also need to handle incoming edges
88     (u -> v).
89     # We must remove the link to v from u's outgoing map.
90     # (In an undirected graph, _incoming is _outgoing, so the loop
91     above covered this).
92     if self.is_directed():
93         for u in list(self._incoming[v]):
94             del self._outgoing[u][v]
95
96     # Phase 2: Remove v from the graph's internal dictionaries.
97     del self._outgoing[v]
98     if self.is_directed():
99         del self._incoming[v]
100
101 def remove_edge(self, e):
102     """Remove edge e from the graph."""
103     u, v = e.endpoints()
104
105     # Remove the edge from u's outgoing map
106     del self._outgoing[u][v]
107
108     # Remove the edge from v's incoming map
109     del self._incoming[v][u]
```


Capitolo 14

Graphs Traversal

Un **graph traversal algorithm** (algoritmo di attraversamento dei grafi) è una procedura sistematica per esplorare un grafo esaminando tutti i suoi vertici e archi. Un algoritmo di graph traversal è efficiente se visita tutti i vertici e archi in un tempo proporzionale al loro numero, cioè in tempo lineare.

Gli algoritmi di graph traversal sono fondamentali per rispondere a molte domande fondamentali sui grafi che coinvolgono la nozione di *raggiungibilità*, cioè nel determinare come viaggiare da un vertice all'altro seguendo i percorsi di un grafo.

Gli algoritmi di graph traversal più utilizzati sono la **depth-first search** (DFS, ricerca in profondità) e la **breadth-first search** (BFS, ricerca in ampiezza). Entrambi gli algoritmi iniziano da un vertice di partenza e visitano sistematicamente tutti i vertici raggiungibili da quel vertice, ma lo fanno in modi diversi come vedremo più avanti.

Alcuni problemi interessanti che trattano la raggiungibilità in un grafo non orientato G includono i seguenti:

- Calcolare un percorso dal vertice u al vertice v , o segnalare che non esiste tale percorso.
- Dato un vertice di partenza s di G , calcolare, per ogni vertice v di G , un percorso con il numero minimo di archi tra s e v , o segnalare che non esiste tale percorso.
- Verificare se G è connesso.
- Calcolare un albero di copertura (spanning tree) di G , se G è connesso.

Alcuni riguardano in modo specifico i grafi orientati.

- Calcolare un percorso diretto dal vertice u al vertice v , o segnalare che non esiste tale percorso.
- Trovare tutti i vertici di G raggiungibili da un dato vertice s .
- Determinare se G è aciclico.
- Determinare se G è fortemente connesso.

14.1 Depth-First Search (DFS)

La **Depth-First Search** (DFS) è una tecnica generale per l'attraversamento dei grafi che esplora "in profondità": parte da un nodo radice e avanza lungo ogni ramo il più possibile prima di tornare indietro (backtracking).

L'algoritmo DFS applicato ad un grafo G con n vertici e m archi visita tutti i vertici e archi del grafo in tempo $O(n + m)$. In particolare, la DFS può essere usata per verificare se G è connesso, calcolare le *connected components*¹ di G e calcolare uno *spanning forest*² di G .

- **DFS su un singolo vertice in un grafo non connesso:** visita solamente i vertici della *connected component* contenente il vertice di partenza e costruisce uno *spanning tree* di tale componente. Non permette di visitare tutti i vertici del grafo, né di verificare la connettività globale o calcolare tutte le *connected components*.
- **DFS su tutti i vertici non ancora visitati in un grafo non connesso:** eseguendo DFS partendo da ciascun vertice non visitato si visitano tutti i vertici del grafo. Questo procedimento permette di calcolare tutte le *connected components* e di costruire uno *spanning forest* del grafo.
- **DFS su un singolo vertice in un grafo connesso:** in questo caso l'esecuzione di DFS sul vertice di partenza visita tutti i vertici del grafo, costruendo uno *spanning tree* dell'intero grafo e permettendo di verificare la connettività globale.

L'idea di base dell'algoritmo DFS è la seguente:

- DFS inizia da un vertice sorgente s e marca s come visitato.
- Quando DFS arriva in un vertice u non ancora visitato:
 - Se esiste un arco (u, v) che conduce a un vertice v non ancora visitato, DFS prende tale arco, marca v come visitato e applica ricorsivamente lo stesso algoritmo su v .
 - Se non ci sono archi che conducono a vertici non ancora visitati, DFS torna indietro lungo il percorso fino a raggiungere un vertice che ha dei vicini non ancora visitati.
- Quando una chiamata DFS termina:
 - Se ci sono vertici non ancora visitati, se ne sceglie uno e si ripete il processo. Ogni chiamata DFS completa esplora una singola *connected component*.
 - Se tutti i vertici sono stati visitati, l'algoritmo termina.

¹Una **connected component** (componente connessa) è un sottografo massimale di un *grafo non orientato* in cui ogni coppia di vertici è collegata da almeno un cammino (sottografo massimale vuol dire che non può essere esteso aggiungendo altri vertici del grafo mantenendo la proprietà per cui tutti i vertici sono connessi tra loro).

²Uno **spanning tree** è un sottoinsieme di archi che connette tutti i vertici di un grafo connesso senza cicli. Uno **spanning forest** è un insieme di *spanning tree*, uno per ciascuna componente connessa di un grafo non connesso. L'algoritmo DFS genera uno *spanning tree* se il grafo è connesso; se il grafo non è connesso, eseguita su tutte le componenti, genera uno *spanning forest*.

Per implementare DFS abbiamo bisogno di etichettare i vertici e gli archi del grafo:

- Ogni vertice può essere in uno dei seguenti stati:
 - **Unexplored**: il vertice non è stato ancora visitato.
 - **Visited**: il vertice è stato già visitato.
- Ogni arco può essere in uno dei seguenti stati:
 - **Unexplored**: l'arco non è stato ancora esplorato.
 - **Discovery**: l'arco è stato esplorato per raggiungere un vertice non ancora visitato. Questo arco fa parte dello spanning tree.
 - **Back edge**: l'arco è stato esplorato e conduce a un vertice già visitato. Questo arco non fa parte dello spanning tree.

Input: graph G

Output: labeling of the edges of G as discovery or back

```
1. Algorithm DFS( $G$ ):
2.   for all  $u$  in  $G$ .vertices()
3.     setLabel( $u$ , UNEXPLORED)
4.   for all  $e$  in  $G$ .edges()
5.     setLabel( $e$ , UNEXPLORED)
6.   for all  $v$  in  $G$ .vertices()
7.     if getLabel( $v$ ) = UNEXPLORED
8.       DFS( $G$ ,  $v$ )
```

Input: graph G and a source vertex v

Output: labels for the edges in the connected component
of G containing v

```
1. Algorithm DFS( $G$ ,  $v$ ):
2.   setLabel( $v$ , VISITED)
3.   for all  $e$  in  $G$ .incidentEdges( $v$ )
4.     if getLabel( $e$ ) = UNEXPLORED
5.        $w$  = opposite( $v$ ,  $e$ )
6.       if getLabel( $w$ ) = UNEXPLORED
7.         setLabel( $e$ , DISCOVERY)
8.         DFS( $G$ ,  $w$ )
9.       else
10.        setLabel( $e$ , BACK)
```

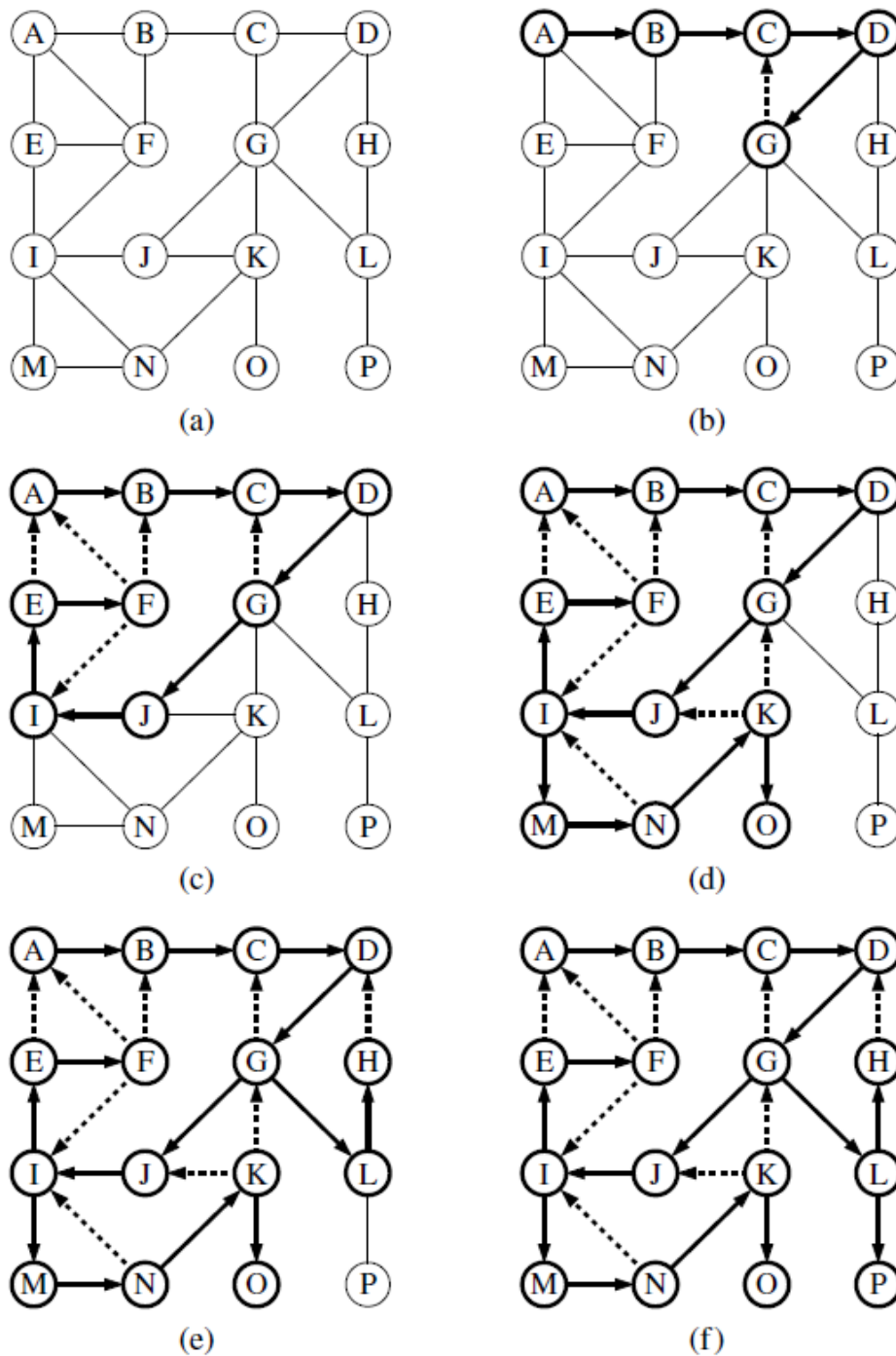


Figura 14.1: Esempio di DFS su un grafo non orientato a partire dal vertice A. Supponiamo che le adiacenze di un vertice siano considerate in ordine alfabetico. I vertici visitati e gli archi esplorati sono evidenziati, con gli archi di scoperta (discovery) disegnati come linee solide e gli archi non appartenenti all'albero (back edges) come linee tratteggiate; i vertici e gli archi non evidenziati sono tutti unexplored: (a) grafo di input; (b) percorso degli archi dell'albero, tracciato da A fino a quando viene esaminato l'arco di back (G,C); (c) raggiungimento di F, che è un vicolo cieco; (d) dopo il backtracking a I, riprendendo con l'arco (I,M), e raggiungendo un altro vicolo cieco in O; (e) dopo il backtracking a G, continuando con l'arco (G,L), e raggiungendo un altro vicolo cieco in H; (f) risultato finale.

Riportiamo alcune proprietà fondamentali dell'algoritmo DFS:

1. Sia G un **grafo non orientato** su cui è stata eseguita una DFS a partire da un vertice s . Allora la visita DFS copre tutti i vertici della componente connessa di s , e gli archi di discovery formano uno spanning tree della componente connessa di s .
 - Calcolare uno spanning tree della componente connessa di s vuol dire trovare un cammino da s a ogni altro vertice raggiungibile da s .
2. Sia G un **grafo orientato**. La DFS su G a partire da un vertice s visita tutti i vertici di G raggiungibili da s . Inoltre, l'albero DFS contiene percorsi orientati da s a ogni vertice raggiungibile da s .

14.1.1 Analisi e implementazione dell'algoritmo DFS

Nell'algoritmo di visita in profondità (Depth First Search, DFS) ogni vertice del grafo viene etichettato esattamente due volte: inizialmente come *UNEXPLORED*, e successivamente come *VISITED* nel momento in cui viene effettivamente visitato dall'algoritmo. Analogamente, ogni arco viene etichettato due volte: la prima volta come *UNEXPLORED* quando viene incontrato per la prima volta, e la seconda volta come *DISCOVERY* oppure *BACK*, a seconda che conduca rispettivamente a un vertice non ancora visitato oppure a un vertice già visitato. Le operazioni di lettura e scrittura delle etichette associate a vertici e archi richiedono tempo costante $O(1)$. Inoltre, il metodo `incidentEdges(v)`, che restituisce tutti gli archi incidenti a un vertice v , viene invocato una sola volta per ciascun vertice del grafo. Se il grafo è rappresentato mediante *Adjacency List* o *Adjacency Map*, il tempo necessario per eseguire `incidentEdges(v)` è proporzionale al grado del vertice v , cioè $O(\deg(v))$. Sommando tale costo su tutti i vertici del grafo, si ottiene $\sum_v \deg(v) = 2m$, dove m è il numero di archi del grafo. Ne consegue che il tempo totale impiegato dall'algoritmo DFS è $O(n + m)$, dove n è il numero di vertici e m il numero di archi.

Per completezza, se invece il grafo è rappresentato mediante una *edge list*, per determinare gli archi incidenti a un vertice è necessario scandire l'intera lista degli archi, operazione che richiede tempo $O(m)$ per ciascun vertice; poiché tale operazione viene eseguita per tutti i n vertici, il tempo totale dell'algoritmo diventa $O(n \cdot m)$. Infine, nel caso di una rappresentazione tramite *Adjacency Matrix*, il metodo `incidentEdges(v)` richiede la scansione dell'intera riga (o colonna) associata al vertice v , con costo $O(n)$; essendo tale operazione eseguita per ogni vertice, il tempo complessivo della DFS risulta $O(n^2)$, indipendentemente dal numero di archi presenti nel grafo.

```

1 def DFS_complete(g):
2     """Makes a DFS on the graph g and returns a dictionary where each
        vertex is mapped to the edge used to discover it (all the roots of
        the DFS trees are mapped to None)."""
3     forest = {}
4     for u in g.vertices():
5         if u not in forest:
6             forest[u] = None      # u is the root of the tree
7             DFS(g, u, forest)
8     return forest
9
10 def DFS(g, u, discovered):
11     """Makes a DFS traversal from the vertex u that has been already
        visited (discovered is a dictionary mapping each visited vertex
        with the edge used to discover it). When new vertices are
        discovered they are added to the dictionary."""
12     for e in g.incident_edges(u):    # for each edge outgoing from u
13         v = e.opposite(u)
14         if v not in discovered:      # if v is not visited
15             discovered[v] = e        # edge e is added to the dictionary
16             DFS(g, v, discovered)    # runs a DFS recursively from v

```

Listing 14.1: Implementazione Python dell'algoritmo DFS.

14.1.2 DFS su grafi orientati

L'algoritmo di DFS funziona per grafi orientati percorrendo ogni arco solo nella sua direzione.

- In una DFS orientata distinguiamo quattro tipi di archi:
 - **discovery edges** (archi di scoperta): sono gli archi (u, v) che portano a un vertice v non ancora visitato. Questi archi formano la foresta (o l'albero) di esplorazione DFS.
 - **back edges** (archi di ritorno): sono gli archi (u, v) che collegano un vertice u a un suo antenato³ v nell'albero DFS. La presenza di un arco di ritorno durante l'esecuzione della DFS indica l'esistenza di almeno un ciclo nel grafo.
 - **forward edges** (archi in avanti): sono gli archi (u, v) che collegano un vertice u a un suo discendente v nell'albero DFS (che non sia però un figlio diretto, altrimenti sarebbe classificato come *discovery edge*).
 - **cross edges** (archi trasversali): sono tutti gli altri archi. Collegano un vertice u a un vertice v tale che v non è né antenato né discendente di u . Possono collegare due rami diversi dello stesso albero DFS oppure due alberi diversi all'interno della foresta DFS.

³La visita DFS impone una struttura gerarchica "orientata" (un albero radicato) anche su grafi non orientati. Quando la visita passa da un vertice v a un vertice non visitato u , si stabilisce una relazione diretta in cui v è padre e u è figlio. Questo orienta logicamente l'arco di scoperta (discovery edge) da v verso u . In questo contesto: Un vertice v è definito **antenato** di u se esiste un percorso composto esclusivamente da discovery edges che collega v a u (seguendo la direzione padre $\rightarrow$ figlio). Viceversa, u è definito **discendente** di v se si trova nel sotto-albero che ha come radice v .

14.1.3 Estensioni della DFS

Attraverso l'utilizzo del *Template Method Pattern*, è possibile specializzare l'algoritmo di DFS per risolvere una vasta gamma di problemi sui grafi mantenendo una complessità temporale efficiente di $O(n + m)$.

Il *Template Method* è un pattern che definisce lo scheletro di un algoritmo in un metodo base, delegando l'implementazione di specifici passaggi (o “ganci”, hook) alle sottoclassi. Nel contesto della DFS, questo significa che la logica di attraversamento ricorsivo rimane invariata, ma vengono esposte delle funzioni personalizzabili che vengono invocate in momenti chiave dell'esecuzione (ad esempio: quando un vertice viene visitato per la prima volta, quando si attraversa un arco, o quando si termina la visita di un vertice). Sovrascrivendo questi metodi specifici, possiamo adattare la DFS generica per ottenere le seguenti informazioni:

- Calcolo delle **componenti connesse** di un grafo;
- Costruzione di una **spanning forest** del grafo;
- Individuazione di un **cammino** tra due vertici (se esistente);
- Rilevamento di un **ciclo** nel grafo (se esistente);
- Identificazione di tutti i vertici **raggiungibili** da un dato vertice in un grafo orientato;
- Calcolo delle **componenti fortemente connesse** in un grafo orientato.

Connectivity Test

Un grafo è connesso se ogni vertice è raggiungibile da tutti gli altri vertici del grafo. Possiamo quindi utilizzare DFS per verificare se G è connesso:

- Eseguiamo una DFS a partire da un vertice arbitrario v ;
- Se la DFS visita tutti i vertici del grafo ($\text{len}(\text{discovered}) == n$) allora G è connesso.

Strong Connectivity Test

Possiamo utilizzare DFS per verificare se un grafo orientato è *fortemente connesso*, cioè se ogni vertice è raggiungibile da tutti gli altri vertici attraverso cammini orientati.

- Eseguiamo una DFS a partire da un vertice arbitrario v ;
- Se la DFS non visita tutti i vertici del grafo, allora G non è fortemente connesso;
- Altrimenti, costruiamo il grafo trasposto G' invertendo la direzione di tutti gli archi di G ;
- Eseguiamo una DFS su G' a partire da v ;
- Se la DFS non visita tutti i vertici di G' , allora G non è fortemente connesso; altrimenti, G è fortemente connesso.

Possiamo ragionare allo stesso modo per verificare se una *componente connessa* di un grafo orientato è *fortemente connessa* (**Strongly Connected Components**).

Una componente connessa di un grafo orientato è fortemente connessa se ogni vertice della componente è raggiungibile da tutti gli altri vertici della componente attraverso cammini orientati.

Searching Paths

Possiamo utilizzare DFS per trovare un cammino tra due vertici v e z :

- Eseguiamo una DFS a partire da v ;
- Utilizziamo uno stack S per tenere traccia del cammino dal vertice di partenza v al vertice corrente (e quindi degli archi di discovery attraversati);
- Quando raggiungiamo z , restituiamo tutti i vertici nello stack.

```
1. Algorithm pathDFS( $G, v, z$ ):
2.   setLabel( $v$ , VISITED)    // Segna  $v$  come visitato
3.    $S.push(v)$               // Aggiunge  $v$  al percorso corrente
4.   if  $v = z$                 // Caso base: destinazione raggiunta
5.     return  $S.elements()$  // Restituisce il percorso trovato
6.   for all  $e$  in  $G.incidentEdges(v)$ 
7.     if getLabel( $e$ ) = UNEXPLORED
8.        $w = opposite(v, e)$ 
9.       if getLabel( $w$ ) = UNEXPLORED
10.        setLabel( $e$ , DISCOVERY)
11.         $S.push(e)$  // Aggiunge l'arco al percorso corrente
12.        pathDFS( $G, w, z$ )
13.         $S.pop(e)$  // Backtracking: strada fallimentare
14.      else
15.        setLabel( $e$ , BACK)
16.     $S.pop(v)$  // Backtracking: vicolo cieco
```

```
1 def construct_path(u, v, discovered):
2     """Return a list of vertices forming a path from u to v (the list is
3     empty if v is not reachable from u). Discovered is the dictionary
4     returned by the DFS."""
5     path = [] # when we start the path is empty
6     if v in discovered: # if v is in the connected component of u
7         # we build backward the path from v to u
8         path.append(v)
9         walk = v
10        while walk is not u:
11            e = discovered[walk] # find the edge used to discover walk
12            parent = e.opposite(walk)
13            path.append(parent) # add to the path the vertex opposite
14            # to walk
15            walk = parent
16        path.reverse() # invert the path from u to v
17    return path
```

Listing 14.2: Implementazione Python della ricerca di un path tra due vertici. Nota: discovered è il dizionario restituito dalla DFS eseguita a partire da u (e non la $DFS_complete(g)$).

Searching Cycles

Possiamo utilizzare l'algoritmo DFS per verificare l'esistenza di un ciclo semplice all'interno della componente connessa del grafo a partire da un vertice v :

- Eseguiamo una DFS a partire dal vertice v ;
- Utilizziamo uno stack S per tenere traccia del cammino attivo dal vertice di partenza al vertice corrente;
- Se durante l'esplorazione incontriamo un arco che punta a un vertice w già visitato (e presente nello stack), abbiamo individuato un arco all'indietro (*back edge*);
- Questo implica l'esistenza di un ciclo, che può essere ricostruito estraendo gli elementi dallo stack fino a ritrovare w .

```
1. Algorithm cycleDFS( $G, v$ ):
2.   setLabel( $v$ , VISITED)    // Marca  $v$  come visitato
3.    $S.push(v)$               // Aggiunge  $v$  al percorso corrente
4.   for all  $e$  in  $G.incidentEdges(v)$ 
5.     if getLabel( $e$ ) = UNEXPLORED
6.        $w = opposite(v, e)$ 
7.        $S.push(e)$           // Aggiunge l'arco corrente allo stack
8.       if getLabel( $w$ ) = UNEXPLORED
9.         setLabel( $e$ , DISCOVERY)
10.        cycleDFS( $G, w$ ) // Chiamata ricorsiva
11.         $S.pop(e)$         // Backtracking: nessun ciclo qui
12.      else                  //  $w$  è già visitato: CICLO TROVATO
13.         $T = new\ empty\ stack$ 
14.        repeat // Travasa lo stack per isolare il ciclo
15.           $o = S.pop()$ 
16.           $T.push(o)$ 
17.        until  $o = w$ 
18.        return  $T.elements()$  // Restituisce la sequenza
19.    $S.pop(v)$               // Backtracking: rimuove  $v$  (vicolo cieco)
```

14.2 Breadth-First Search (BFS)

La **Breadth-First Search** (BFS) è una tecnica generale per l'attraversamento dei grafi che esplora "in ampiezza": parte da un nodo radice e visita tutti i suoi vicini prima di procedere ai nodi di livello successivo.

L'algoritmo BFS applicato ad un grafo G con n vertici e m archi visita tutti i vertici e archi del grafo in tempo $O(n + m)$. In particolare, la BFS può essere usata, come DFS, per verificare se G è connesso, calcolare le *connected components* di G e calcolare uno *spanning forest* di G .

Una differenza sostanziale tra BFS e DFS è che l'algoritmo BFS può essere utilizzato per calcolare i percorsi più brevi (in termini di numero di archi) tra due vertici, se questo esiste. Come abbiamo visto in precedenza, DFS è in grado di trovare un percorso tra due vertici, ma non garantisce che questo sia il più breve.

L'idea di base dell'algoritmo BFS è la seguente:

- La visita procede per step e divide i vertici del grafico in livelli:
 - Comincia dal vertice sorgente s che è l'unico vertice al livello 0, e marca s come visitato.
 - Al passo 1 si marciano come visitati tutti i vicini di s e li si aggiunge al livello 1.
 - Al passo 2 si marciano come visitati tutti i vertici adiacenti ai vertici al livello 1 che non sono stati ancora visitati e li si aggiunge al livello 2.
 - Al passo i si marciano come visitati tutti i vertici adiacenti ai vertici al livello $i - 1$ che non sono stati ancora visitati e li si aggiunge al livello i .
 - Il processo si ferma quando non ci sono più vertici non visitati adiacenti a vertici visitati.

Per implementare BFS abbiamo bisogno di etichettare i vertici e gli archi del grafo, come visto per DFS, ma con una differenza nei tipi di etichette utilizzate (Back edge DFS vs Cross edge BFS):

- Ogni vertice può essere in uno dei seguenti stati:
 - **Unexplored**: il vertice non è stato ancora visitato.
 - **Visited**: il vertice è stato già visitato.
- Ogni arco può essere in uno dei seguenti stati:
 - **Unexplored**: l'arco non è stato ancora esplorato.
 - **Discovery**: l'arco è stato esplorato per raggiungere un vertice non ancora visitato. Questo arco fa parte dello spanning tree.
 - **Cross edge**: l'arco è stato esplorato per raggiungere un vertice già visitato. Questo arco non fa parte dello spanning tree.

Input: graph G

Output: labeling of the edges and partition of the vertices of G

```
1. Algorithm BFS( $G$ )
2.   for all  $u$  in  $G$ .vertices()
3.     setLabel( $u$ , UNEXPLORED)
4.   for all  $e$  in  $G$ .edges()
5.     setLabel( $e$ , UNEXPLORED)
6.   for all  $v$  in  $G$ .vertices()
7.     if getLabel( $v$ ) = UNEXPLORED
8.       BFS( $G$ ,  $v$ )
```

Input: graph G and a source vertex s

Output: labeling of the edges and vertices in the connected component of G containing s

```
1. Algorithm BFS( $G$ ,  $s$ )
2.    $L_0$  = new empty sequence
3.    $L_0$ .addLast( $s$ )
4.   setLabel( $s$ , VISITED)
5.    $i$  = 0
6.   while not  $L_i$ .isEmpty()
7.      $L_{i+1}$  = new empty sequence
8.     for all  $v$  in  $L_i$ .elements()
9.       | for all  $e$  in  $G$ .incidentEdges( $v$ )
10.      |   if getLabel( $e$ ) = UNEXPLORED
11.      |      $w$  = opposite( $v$ ,  $e$ )
12.      |     if getLabel( $w$ ) = UNEXPLORED
13.      |       setLabel( $e$ , DISCOVERY)
14.      |       setLabel( $w$ , VISITED)
15.      |        $L_{i+1}$ .addLast( $w$ )
16.      |     else
17.      |       setLabel( $e$ , CROSS)
18.      $i$  =  $i$  + 1 // move to the next level
```

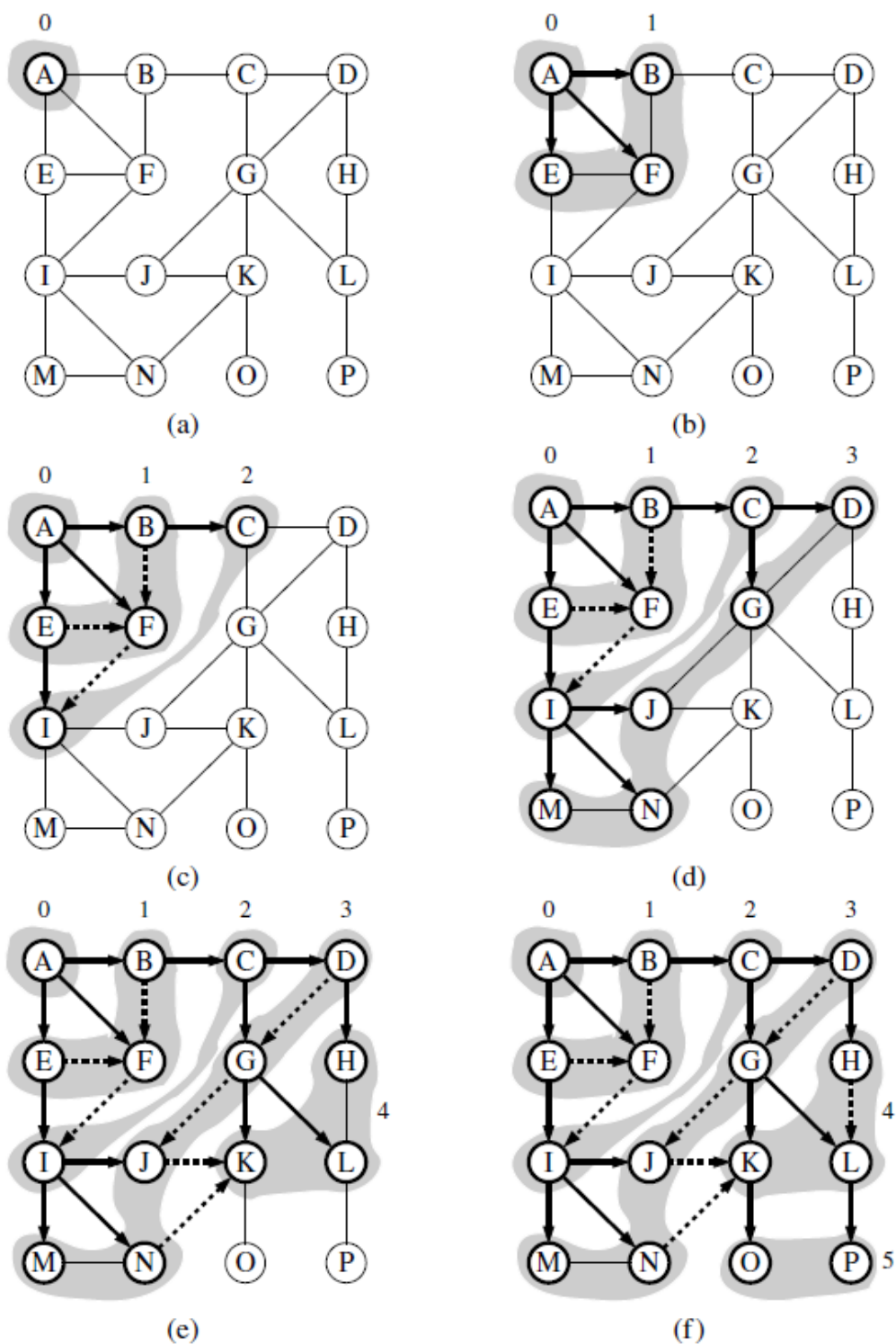


Figura 14.2: Esempio di BFS su un grafo non orientato a partire dal vertice A. Supponiamo che le adiacenze di un vertice siano considerate in ordine alfabetico. I vertici visitati e gli archi esplorati sono evidenziati, con gli archi di scoperta (discovery) disegnati come linee solide e gli archi non appartenenti all'albero (cross edges) come linee tratteggiate; i vertici e gli archi non evidenziati sono tutti unexplored: (a) inizio della ricerca in A; (b) scoperta del livello 1; (c) scoperta del livello 2; (d) scoperta del livello 3; (e) scoperta del livello 4; (f) scoperta del livello 5.

Riportiamo alcune proprietà fondamentali dell'algoritmo BFS. Sia G un grafo (orientato o non orientato) su cui è stata eseguita una BFS a partire da un vertice s , $\text{BFS}(G, s)$. Allora:

1. La visita $\text{BFS}(G, s)$ visita tutti gli archi e i vertici raggiungibili da s , ovvero tutti gli archi e i vertici della componente connessa di s , G_s .
2. Gli archi di discovery della visita $\text{BFS}(G, s)$ formano uno spanning tree T_s di G_s , chiamato **BFS tree** (albero BFS) radicato in s .
3. Per ogni vertice v al livello i , il cammino nell'albero BFS T_s tra s e v ha i archi, e ogni altro cammino in G da s a v ha almeno i archi.
4. Se (u, v) è un arco che non appartiene all'albero BFS, allora il livello del vertice v può essere, al massimo, di 1 maggiore del livello del vertice u .

14.2.1 Analisi e implementazione dell'algoritmo BFS

Nell'algoritmo di visita in ampiezza (Breadth First Search, BFS) ogni vertice del grafo viene etichettato esattamente due volte: inizialmente come *UNEXPLORED*, e successivamente come *VISITED*. Una proprietà fondamentale della procedura è che ogni vertice viene inserito in una e una sola sequenza di livello. Analogamente, ogni arco viene etichettato due volte: la prima volta come *UNEXPLORED*, e la seconda volta come *DISCOVERY* oppure *CROSS*, a seconda della relazione tra i vertici collegati. Le operazioni di lettura e scrittura delle etichette associate a vertici e archi richiedono tempo costante $O(1)$. Inoltre, il metodo `incidentEdges(v)`, che restituisce tutti gli archi incidenti a un vertice v , viene invocato una sola volta per ciascun vertice del grafo. Se il grafo è rappresentato mediante *Adjacency List* o *Adjacency Map*, il tempo necessario per eseguire `incidentEdges(v)` è proporzionale al grado del vertice v , cioè $O(\deg(v))$. Sommando tale costo su tutti i vertici del grafo, si ottiene $\sum_v \deg(v) = 2m$, dove m è il numero di archi del grafo. Ne consegue che il tempo totale impiegato dall'algoritmo BFS è $O(n + m)$, dove n è il numero di vertici e m il numero di archi.

Per completezza, se invece il grafo è rappresentato mediante una *edge list*, per determinare gli archi incidenti a un vertice è necessario scandire l'intera lista degli archi, operazione che richiede tempo $O(m)$ per ciascun vertice; poiché tale operazione viene eseguita per tutti i n vertici, il tempo totale dell'algoritmo diventa $O(n \cdot m)$. Infine, nel caso di una rappresentazione tramite *Adjacency Matrix*, il metodo `incidentEdges(v)` richiede la scansione dell'intera riga (o colonna) associata al vertice v , con costo $O(n)$; essendo tale operazione eseguita per ogni vertice, il tempo complessivo della BFS risulta $O(n^2)$, indipendentemente dal numero di archi presenti nel grafo.

```

1 def BFS_complete(g):
2     """ Makes a BFS on the graph g and returns adictionary where each
        vertex is mapped to the edgeused to discover it (all the roots of
        the BFS treesare mapped to None)"""
3     forest = {}
4     for u in g.vertices():
5         if u not in forest:
6             forest[u] = None # u is the root of the tree
7             BFS(g, u, forest)
8     return forest
9
10 def BFS(g, s, discovered):
11     """ Makes a BFS traversal from the vertex s that has been already
        visited (discovered is a dictionary mapping each visited vertex
        with the edge used to discover it). When new vertices are
        discovered they are added to the dictionary."""
12     level = [s] # level 0 includes only s
13     while len(level) > 0:
14         next_level = [] # list of the vertices in the next level
15         for u in level: # for each vertex u in the level
16             for e in g.incident_edges(u): # for each edge outgoing from u
17                 v = e.opposite(u)
18                 if v not in discovered: # if v has not been visited yet
19                     discovered[v] = e # add e to the dictionary
20                     next_level.append(v) # add v to the next level
21     level = next_level

```

Listing 14.3: Implementazione Python dell'algoritmo BFS.

14.2.2 Estensioni della BFS

Così come visto per DFS, attraverso l'utilizzo del *Template Method Pattern*, è possibile specializzare l'algoritmo di BFS per risolvere una vasta gamma di problemi sui grafi mantenendo una complessità temporale efficiente di $O(n + m)$.

- Calcolo delle **componenti connesse** di un grafo;
- Costruzione di una **spanning forest** del grafo;
- Rilevamento di un **ciclo** nel grafo tra due vertici (se esistente);
- Calcolo di un **percorso con il numero minimo di archi** tra due vertici (se esistente);
 - Non è possibile con DFS.

Capitolo 15

Transitive Closure

Come abbiamo visto, gli attraversamenti dei grafi possono essere utilizzati per rispondere a domande di base sulla raggiungibilità in un grafo orientato. In particolare, se siamo interessati a sapere se esiste un percorso dal vertice u al vertice v in un grafo, possiamo eseguire una *DFS* o *BFS* a partire da u e osservare se v viene scoperto. Se rappresentiamo un grafo con una *Adjacency list* o *Adjacency map*, possiamo rispondere alla domanda di raggiungibilità da u a v in tempo $O(n + m)$.

In alcune applicazioni, potremmo voler rispondere a molte query di raggiungibilità in modo più efficiente, e dunque potrebbe valere la pena precomputare una rappresentazione più conveniente di un grafo. Ad esempio, il primo passo per un servizio che calcola le indicazioni stradali da un'origine a una destinazione potrebbe essere quello di valutare se la destinazione è raggiungibile. Allo stesso modo, in una rete elettrica, potremmo voler essere in grado di determinare rapidamente se la corrente fluisce da un particolare vertice a un altro. Motivati da tali applicazioni, introduciamo la seguente definizione.

Definizione: La **chiusura transitiva** (transitive closure) di un **grafo orientato** $\vec{G} = (V, E)$ è a sua volta un *grafo orientato* $\vec{G}^* = (V^*, E^*)$ tale che:

- $V^* = V$
- $(u, v) \in E^*$ se e solo se v è raggiungibile da u in $\vec{G}$.

Dunque, la chiusura transitiva fornisce informazioni di raggiungibilità tra tutti i vertici in un grafo orientato, e in alcune applicazioni è più efficiente calcolare $\vec{G}^*$ piuttosto che eseguire un attraversamento (DFS o BFS) da ciascun vertice.

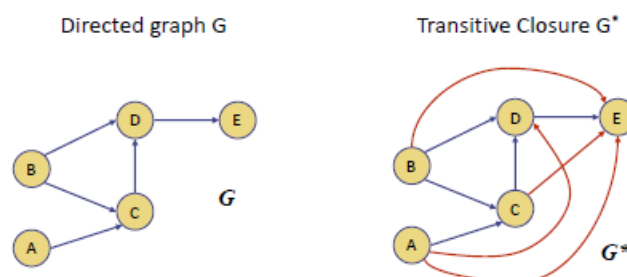


Figura 15.1: Esempio di grafo orientato (a sinistra) e la sua chiusura transitiva (a destra).

Se un grafo è rappresentato come una Adjacency List o Adjacency Map, possiamo calcolare la sua chiusura transitiva in tempo $O(n(n + m))$ facendo uso di n attraversamenti del grafo, uno da ciascun vertice di partenza. Ad esempio, una DFS che inizia al vertice u può essere utilizzata per determinare tutti i vertici raggiungibili da u , e quindi una collezione di archi che originano da u nella chiusura transitiva.

15.1 Algoritmo di Floyd-Warshall per la Chiusura Transitiva

Un'alternativa efficace per calcolare la *chiusura transitiva* di un grafo orientato sfrutta strutture che supportano la ricerca in tempo $O(1)$ per il metodo `get_edge(u, v)`, come ad esempio una *Adjacency Matrix*.

Si tratta di un algoritmo di programmazione dinamica, l'algoritmo di **Floyd-Warshall**. L'idea alla base di questo algoritmo è quella di costruire una funzione booleana $G(i, j, k) = 1$ (vero) se esiste un percorso da v_i a v_j che utilizza solo i vertici tra $v_1, v_2, \dots, v_k$ come vertici intermedi, $G(i, j, k) = 0$ (falso) altrimenti.

- La relazione di raggiungibilità soddisfa le seguenti condizioni:

- $G(i, j, k) = 1$ **if** $G(i, j, k - 1) = 1$;
- $G(i, j, k) = 1$ **if** $G(i, k, k - 1) = 1$ **AND** $G(k, j, k - 1) = 1$;
- $G(i, j, k) = 0$ **otherwise**.

- **Caso base** ($k = 0$):

$$G(i, j, 0) = \begin{cases} 1 & \text{if } (v_i, v_j) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

- **Equazione caratteristica** ($k \geq 1$):

$$G(i, j, k) = G(i, j, k - 1) \text{ OR } (G(i, k, k - 1) \text{ AND } G(k, j, k - 1)).$$

L'algoritmo di Floyd-Warshall, descritto tramite la funzione booleana $G(i, j, k)$, va interpretato come un procedimento incrementale che costruisce gradualmente tutti i cammini possibili nel grafo.

Il caso base indica che, senza alcun vertice intermedio, un cammino tra i e j esiste solo se c'è un arco diretto, fornendo il punto di partenza per cammini più complessi.

La regola ricorsiva mostra come, ad ogni passo, aggiungere un nuovo vertice v_k come possibile intermedio: un cammino tra i e j esiste se esiste già senza usare v_k , oppure se si possono concatenare due cammini più piccoli, da i a v_k e da v_k a j , che non utilizzano v_k come intermedio. In altre parole, ogni nuovo vertice intermedio permette di combinare cammini già noti per crearne di nuovi, senza introdurre ambiguità o cicli.

Considerando tutti i vertici come potenziali intermedi uno alla volta, alla fine $G(i, j, n)$ rappresenta tutti i cammini possibili tra ogni coppia di vertici, cioè la chiusura transitiva del grafo.

Per comprendere meglio l'algoritmo, è utile presentare anche una versione alternativa basata sull'approccio del grafo incrementale, che realizza la stessa logica della costruzione booleana ma opera direttamente sugli archi e mostra in maniera più intuitiva come i nuovi percorsi vengono aggiunti passo dopo passo.

Sia $\vec{G}$ un grafo orientato con n vertici e m archi. L'idea è quella di calcolare la chiusura transitiva di $\vec{G}$ in una serie di round successivi.

1. Inizializziamo $\vec{G}_0 = \vec{G}$.
2. Numeriamo arbitrariamente i vertici di $\vec{G}$ come $v_1, v_2, \dots, v_n$.
3. Per ogni round $k = 1, \dots, n$:
 - Un arco (v_i, v_j) appartiene a $\vec{G}_k$ se e solo se: (v_i, v_j) era già presente in $\vec{G}_{k-1}$ oppure (v_i, v_k) e (v_k, v_j) appartengono a $\vec{G}_{k-1}$.
 - Costruiamo $\vec{G}_k$ copiando $\vec{G}_{k-1}$ e aggiungendo tutti gli archi (v_i, v_j) che soddisfano la condizione precedente.
4. Dopo aver completato il round n , otteniamo $\vec{G}_n = \vec{G}^*$; restituiamo $\vec{G}_n$ come chiusura transitiva di $\vec{G}$.

Questo approccio incrementale è facilmente implementabile seguendo lo pseudocodice riportato di seguito.

Input: A directed graph G with n vertices

Output: The transitive closure G^* of G

```

1. Algorithm FloydWarshall( $G$ )
2.    $i = 1$ 
3.   for all  $v$  in  $G$ .vertices()
4.     denote  $v$  as  $v_i$ 
5.      $i = i + 1$ 
6.    $G_0 = G$ 
7.   for  $k = 1$  to  $n$  do                                //  $k$  è il vertice intermedio
8.      $G_k = G_{k-1}$                                        // Copia stato precedente
9.     for  $i = 1$  to  $n$  ( $i \neq k$ ) do // Nodo sorgente
10.      for  $j = 1$  to  $n$  ( $j \neq i, k$ ) do // Nodo destinazione
11.        if  $G_{k-1}$ .areAdjacent( $v_i, v_k$ ) AND
12.           $G_{k-1}$ .areAdjacent( $v_k, v_j$ )
13.          if NOT  $G_{k-1}$ .areAdjacent( $v_i, v_j$ )
14.             $G_k$ .insertDirectedEdge( $v_i, v_j, k$ )
15.   return  $G_n$ 

```

15.1.1 Analisi e implementazione dell'algoritmo

Dallo pseudocodice, possiamo facilmente analizzare il tempo di esecuzione dell'algoritmo di Floyd-Warshall assumendo che la struttura dati che rappresenta G supporti i metodi `get_edge` e `insert_edge` in tempo $O(1)$ (come *Adjacency Matrix*). Il ciclo principale viene eseguito n volte e il ciclo interno considera ciascuna delle $O(n^2)$ coppie di vertici, eseguendo un calcolo a tempo costante per ciascuna. Dunque, il tempo totale di esecuzione dell'algoritmo di Floyd-Warshall è $O(n^3)$ (IMPORTANTE: valido date le assunzioni fatte sulla struttura dati).

```
1 def floyd_warshall(g):
2     """Return a new graph that is the transitive closure of g."""
3     closure = deepcopy(g)          # imported from copy module
4     verts = list(closure.vertices()) # make indexable list
5     n = len(verts)
6     for k in range(n):
7         for i in range(n):
8             # verify that edge (i,k) exists in the partial closure
9             if i != k and closure.get_edge(verts[i],verts[k]) is not None:
10                for j in range(n):
11                    # verify that edge (k,j) exists in the partial closure
12                    if i != j != k and closure.get_edge(verts[k],verts[j])
13                       is not None:
14                        # if (i,j) not yet included, add it to the closure
15                        if closure.get_edge(verts[i],verts[j]) is None:
16                            closure.insert_edge(verts[i],verts[j])
17     return closure
```

Listing 15.1: Implementazione Python dell'algoritmo di Floyd-Warshall.

Asintoticamente, il tempo di esecuzione $O(n^3)$ dell'algoritmo di Floyd-Warshall non è migliore di quello ottenuto eseguendo ripetutamente una DFS, una volta da ciascun vertice, per calcolare la raggiungibilità (tempo $O(n(n+m))$). Tuttavia, l'algoritmo di Floyd-Warshall eguaglia i limiti asintotici della DFS ripetuta quando un grafo è denso, o quando un grafo è sparso ma rappresentato come una *Adjacency Matrix*.

L'importanza dell'algoritmo di Floyd-Warshall risiede nel fatto che è molto più semplice da implementare rispetto alla DFS, e molto più veloce nella pratica perché ci sono relativamente poche operazioni di basso livello nascoste nella notazione asintotica. L'algoritmo è particolarmente adatto per l'uso di una *Adjacency Matrix*, poiché un singolo bit può essere utilizzato per designare la raggiungibilità modellata come un arco (u, v) nella chiusura transitiva.

Tuttavia, si noti che chiamate ripetute a DFS portano a prestazioni asintotiche migliori quando il grafo è sparso e rappresentato utilizzando una *Adjacency List* o *Adjacency Map*. In tal caso, una singola DFS viene eseguita in tempo $O(n+m)$, e quindi la chiusura transitiva può essere calcolata in tempo $O(n(n+m)) = O(n^2 + nm)$, che è preferibile a $O(n^3)$.

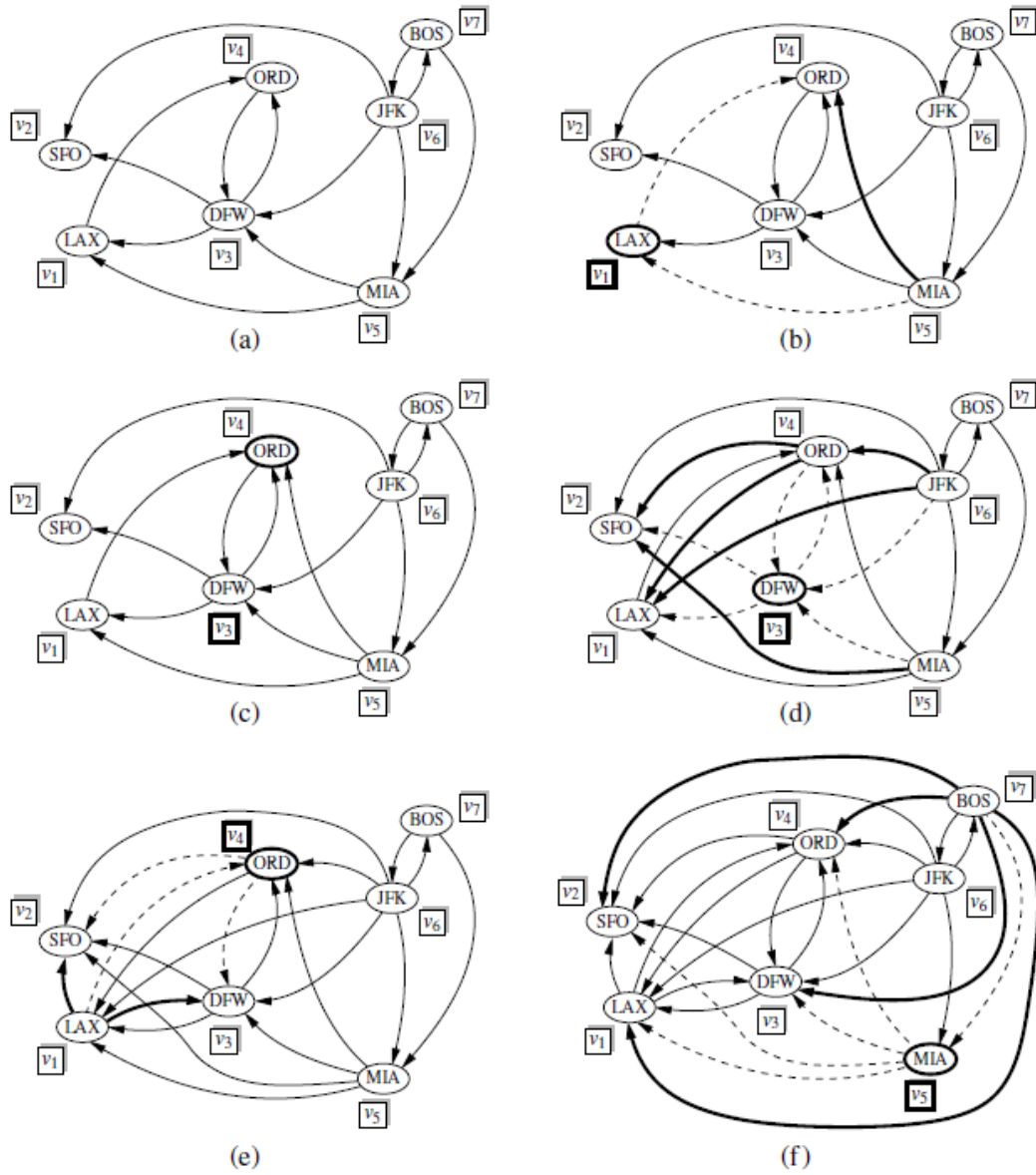


Figura 15.2: Sequenza di grafi orientati calcolati dall'algoritmo di Floyd-Warshall: (a) grafo orientato iniziale $\vec{G} = \vec{G}_0$ e numerazione dei vertici; (b) grafo orientato $\vec{G}_1$; (c) $\vec{G}_2$; (d) $\vec{G}_3$; (e) $\vec{G}_4$; (f) $\vec{G}_5$. Si noti che $\vec{G}_5 = \vec{G}_6 = \vec{G}_7$. Se il grafo orientato $\vec{G}_{k-1}$ ha gli archi (v_i, v_k) e (v_k, v_j) , ma non l'arco (v_i, v_j) , nel disegno del grafo orientato $\vec{G}_k$, mostriamo gli archi (v_i, v_k) e (v_k, v_j) con linee tratteggiate, e l'arco (v_i, v_j) con una linea spessa. Ad esempio, in (b) gli archi esistenti (MIA, LAX) e (LAX, ORD) producono il nuovo arco (MIA, ORD) .

Capitolo 16

Directed Acyclic Graph - DAG

Con **Directed Acyclic Graph**, o *DAG*, si indica un **grafo orientato** $\vec{G}$ che non contiene cicli orientati, ovvero non esiste alcun percorso che inizi e finisca nello stesso vertice seguendo la direzione degli archi. In molte applicazioni capita di avere a che fare con questo tipo di grafi; alcune di queste applicazioni includono:

- Prerequisiti tra i corsi di un programma di laurea.
- Ereditarietà tra le classi di un programma orientato agli oggetti.
- Vincoli di scheduling tra le attività di un progetto.

16.1 Topological Sorting

Sia $\vec{G}$ un grafo orientato con n vertici. Un **Topological Sorting**, o *Topological Ordering* (ordinamento topologico), di $\vec{G}$ è un ordinamento dei vertici $v_1, \dots, v_n$ di $\vec{G}$ tale che per ogni arco (v_i, v_j) di $\vec{G}$ si ha che $i < j$.

Cioè, un ordinamento topologico è un ordinamento tale che qualsiasi percorso diretto in $\vec{G}$ attraversa i vertici in ordine crescente.

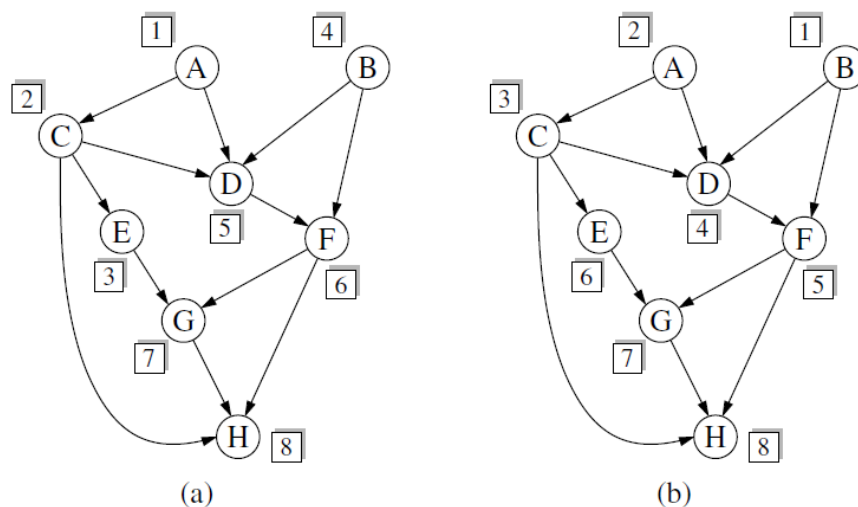


Figura 16.1: Due Topological Sorting differenti dello stesso grafo orientato aciclico.

Si noti che un grafo orientato può avere più di un ordinamento topologico (come mostrato in Figura 16.1), e che **un ordinamento topologico esiste se e solo se il grafo orientato è aciclico**. In relazione agli esempi precedenti, un ordinamento topologico di un grafo orientato può rappresentare un possibile ordine di completamento dei corsi in un piano di studi, o un possibile ordine di esecuzione delle attività di un progetto.

16.1.1 Implementazione Python

Dal momento che un grafo orientato $\vec{G}$ è aciclico, esiste almeno un vertice con *in-degree* 0 (cioè, senza archi entranti). Denotiamo tale vertice come v_1 . Rimuovendo v_1 da $\vec{G}$ insieme ai suoi archi uscenti, il grafo orientato risultante è ancora aciclico. Pertanto, il grafo orientato risultante ha anch'esso un vertice con in-degree 0, che denotiamo come v_2 . Ripetendo questo processo fino a quando il grafo orientato diventa vuoto, otteniamo un ordinamento $v_1, \dots, v_n$ dei vertici di $\vec{G}$.

L'ordinamento così ottenuto rispetta la struttura del grafo per una ragione precisa: se esiste un arco (v_i, v_j) , allora v_i agisce da 'ostacolo' per v_j . Finché v_i rimane nel grafo, il vertice v_j possiede almeno un arco entrante e, per la regola della nostra costruzione, non può essere selezionato. Pertanto, v_i deve necessariamente essere rimosso per far sì che v_j possa raggiungere un in-degree pari a zero e diventare eleggibile per la rimozione. Questo garantisce che nella sequenza finale v_i compaia sempre prima di v_j (ovvero $i < j$), soddisfacendo pienamente la definizione di ordinamento topologico. Pertanto, $v_1, \dots, v_n$ è un ordinamento topologico.

```

1 def topological_sort(g):
2     """Return a list of vertices of directed acyclic graph g in topological
3     order.
4     If graph g has a cycle, the result will be incomplete.
5     """
6     topo = [ ]          # a list of vertices placed in topological order
7     ready = [ ]         # list of vertices that have no remaining constraints
8     incount = { }       # keep track of in-degree for each vertex
9     for u in g.vertices( ):
10         incount[u] = g.degree(u, False) # parameter requests incoming degree
11         if incount[u] == 0:              # if u has no incoming edges,
12             ready.append(u)              # it is free of constraints
13     while len(ready) > 0:
14         u = ready.pop( )                 # u is free of constraints
15         topo.append(u)                   # add u to the topological order
16         for e in g.incident_edges(u):    # consider all outgoing neighbors
17             # of u
18             v = e.opposite(u)
19             incount[v] -= 1              # v has one less constraint without u
20             if incount[v] == 0:
21                 ready.append(v)
22     return topo

```

Listing 16.1: Implementazione Python dell'algoritmo di Topological Sorting.

16.1.2 Analisi della complessità

Il ciclo iniziale che registra tutte le informazioni relative agli in-degrees degli n vertici all'interno di `incount` utilizza tempo $O(n)$ basato sul metodo `degree`. Diciamo che un vertice u viene visitato dall'algoritmo di topological sorting quando u viene rimosso dalla lista `ready`. Un vertice u può essere visitato solo quando `incount(u)` è 0, il che implica che tutti i suoi predecessori (vertici con archi uscenti verso u) sono stati precedentemente visitati. Di conseguenza, qualsiasi vertice che si trova su un ciclo diretto non verrà mai visitato, e qualsiasi altro vertice verrà visitato esattamente una volta. L'algoritmo attraversa tutti gli archi uscenti di ogni vertice visitato una volta, quindi il suo tempo di esecuzione è proporzionale al numero di archi uscenti dei vertici visitati. Concludiamo che il tempo di esecuzione è $O(n + m)$. Per quanto riguarda l'uso dello spazio, osserviamo che i contenitori `topo`, `ready` e `incount` hanno al massimo una voce per vertice, e quindi utilizzano spazio $O(n)$.

In sintesi:

- Sia $\vec{G}$ un grafo orientato con n vertici e m archi, rappresentato tramite una *Adjacency List*. L'algoritmo di topological sorting viene eseguito in tempo $O(n + m)$ utilizzando spazio ausiliario $O(n)$. Il risultato finale o produce un ordinamento topologico di $\vec{G}$ oppure non include alcuni vertici, il che indica che $\vec{G}$ contiene un ciclo orientato.

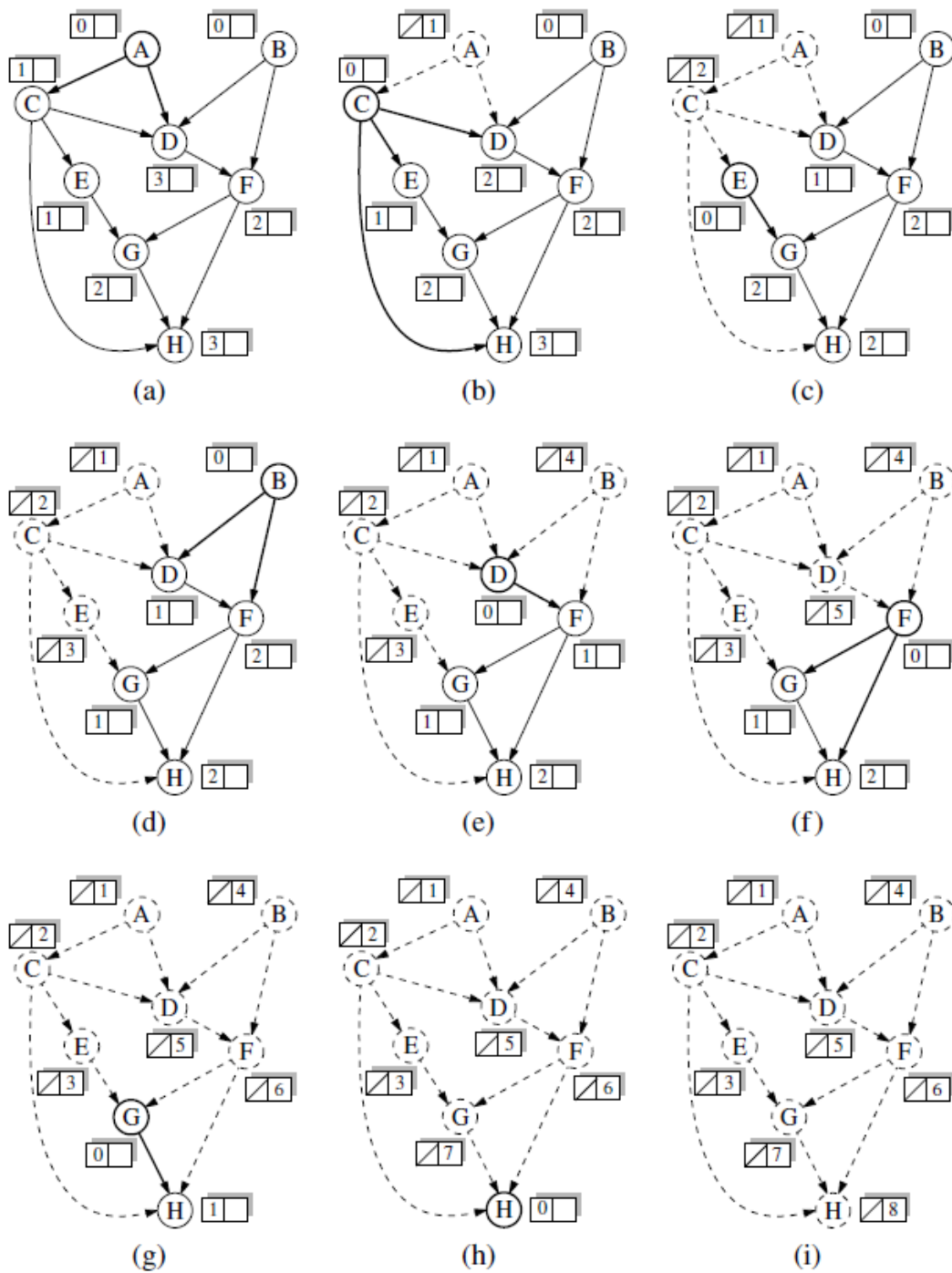


Figura 16.2: Esempio di esecuzione dell'algoritmo di topological sort. L'etichetta vicino a un vertice mostra il suo valore corrente di incount e la sua eventuale posizione nell'ordinamento topologico risultante. Il vertice evidenziato è uno con incount pari a zero che diventerà il prossimo vertice nell'ordinamento topologico. Le linee tratteggiate denotano archi che sono già stati esaminati e che non sono più conteggiati nei valori di incount.

Capitolo 17

Shortest Paths

Come abbiamo già visto, la ricerca in ampiezza (BFS) permette di trovare il cammino minimo in un grafo quando tutti gli archi sono considerati equivalenti. Tuttavia, in molte applicazioni reali questa condizione non è soddisfatta: per individuare il percorso più rapido tra due città o il tragitto più veloce per un pacchetto dati in rete, è necessario distinguere tra archi con caratteristiche diverse. Infatti, in questi contesti accade che alcune distanze tra le città saranno probabilmente molto più grandi di altre, e alcune connessioni in una rete di computer sono tipicamente molto più veloci di altre (ad esempio, alcune connessioni a bassa larghezza di banda contro connessioni ad alta velocità in fibra ottica). È quindi naturale considerare dei grafi pesati dove a ogni arco è associato un peso che ne rappresenta il costo o l'importanza.

17.1 Weighted Graphs

Un **weighted graph** (grafo pesato) è un grafo che ha un'etichetta numerica $w(e)$ associata a ciascun arco e , chiamata il peso (weight) dell'arco e . Per un arco $e = (u, v)$, indichiamo il suo peso con la notazione $w(e) = w(u, v)$.

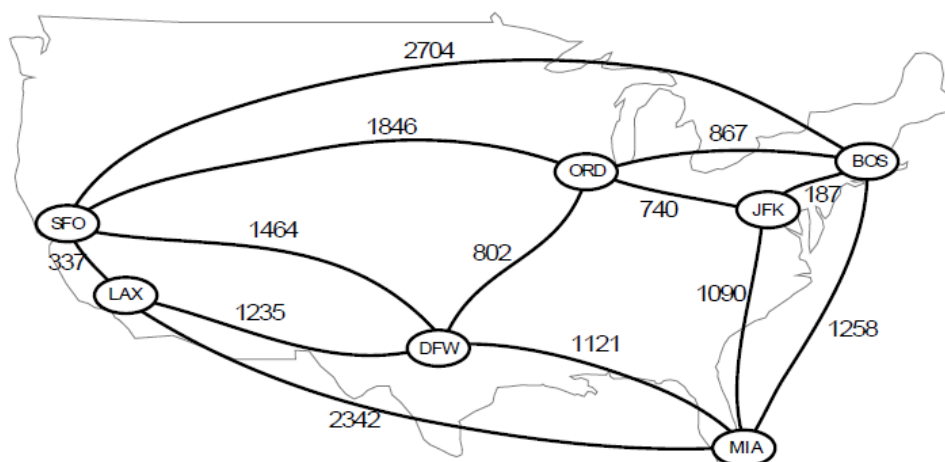


Figura 17.1: Esempio di grafo pesato i cui vertici rappresentano i principali aeroporti degli Stati Uniti e i cui pesi degli archi rappresentano le distanze in miglia.

Sia G un grafo pesato. La lunghezza (o peso) di un percorso P è la somma dei pesi degli archi di P . Cioè, se $P = ((v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k))$, allora la lunghezza di P , indicata con $w(P)$, è definita come:

$$w(P) = \sum_{i=1}^k w(v_{i-1}, v_i)$$

La **distanza** da un vertice u a un vertice v in G , indicata con $d(u, v)$, è la lunghezza di un percorso di lunghezza minima (chiamato anche **shortest path**) da u a v , se tale percorso esiste. Se non esiste alcun percorso da u a v , allora utilizziamo la convenzione $d(u, v) = \infty$.

Proprietà di uno shortest path

1. Un frammento di uno shortest path è anch'esso uno shortest path. Cioè, se P è uno shortest path da a a c , e b è un vertice su P , allora il frammento di P da a a b è uno shortest path da a a b , e il frammento di P da b a c è uno shortest path da b a c .
2. Gli shortest path da un vertice sorgente s a tutti gli altri vertici costituiscono uno **shortest path spanning tree** (o shortest path tree) radicato in s .

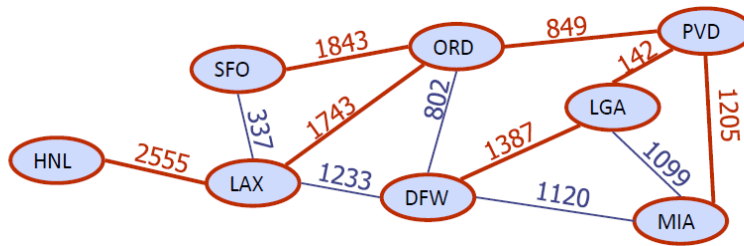


Figura 17.2: Esempio di uno shortest path tree radicato nel vertice PVD.

Problemi legati agli shortest path

- **(u, v)-shortest path**

Dati due vertici u e v , trovare lo shortest path tra u e v .

- **Single source shortest path**

Dato un vertice s , trovare lo shortest path tree radicato in s (ovvero, trova lo shortest path da s a tutti gli altri vertici).

- **All-pairs shortest paths**

Trovare gli shortest path per tutte le coppie di vertici nel grafo.

Si noti come dalla soluzione del problema single source shortest path sia possibile ottenere la soluzione del problema (u, v) -shortest path, mentre dalla soluzione del problema all-pairs shortest paths sia possibile ottenere la soluzione di entrambi gli altri problemi. Tuttavia, è anche possibile risolvere il problema all-pairs shortest paths eseguendo l'algoritmo per il single source shortest path per ogni vertice del grafo.

17.2 Algoritmo di Dijkstra

Si vuole sviluppare un **algoritmo Greedy** per la risoluzione del problema *single source shortest path*. L'idea principale nell'applicare il *metodo Greedy* al problema degli shortest path da una singola sorgente è quella di eseguire una "ricerca in ampiezza pesata" a partire dal vertice sorgente s . In particolare, possiamo utilizzare il *metodo Greedy* per sviluppare un algoritmo che cresce iterativamente una "nuvola" di vertici a partire da s , con i vertici che entrano nella nuvola in ordine di distanza da s . Quindi, in ogni iterazione, il prossimo vertice scelto è il vertice al di fuori della nuvola che è più vicino a s . L'algoritmo termina quando non ci sono più vertici al di fuori della nuvola (o quando quelli al di fuori della nuvola non sono connessi a quelli all'interno della nuvola, e quindi hanno distanza infinita), a quel punto abbiamo uno shortest path da s a ogni vertice di G raggiungibile da s . Questo approccio è un esempio semplice, ma comunque potente, del *metodo Greedy*.

L'applicazione del *metodo Greedy* al problema degli shortest path da una singola sorgente porta a un algoritmo noto come **algoritmo di Dijkstra**.

17.2.1 Edge Relaxation - Greedy Choice

Definiamo un'etichetta $D(v)$ per ogni vertice v in V , che utilizziamo per approssimare la distanza da s a v in G . Il significato di queste etichette è che $D(v)$ memorizzerà sempre la lunghezza del miglior percorso che abbiamo trovato finora da s a v . Difatti, possiamo pensare a $D(v)$ come un limite superiore della distanza minima reale $d(s, v)$. Inoltre, definiamo un insieme C , che rappresenta l'insieme di vertici nella "nuvola", ovvero quei vertici per i quali abbiamo già calcolato la distanza minima da s .

Inizialmente, definiamo $D(s) = 0$ e $D(v) = \infty$ per ogni $v \neq s$, e l'insieme C come l'insieme vuoto. Ad ogni iterazione dell'algoritmo, selezioniamo un vertice u non in C con l'etichetta $D(u)$ più piccola, e lo aggiungiamo a C (Generalmente, utilizzeremo una coda di priorità per selezionare tra i vertici al di fuori della nuvola, per via dell'efficienza del metodo `remove_min()`).

Nella prima iterazione, ovviamente, aggiungeremo s a C . Una volta che un nuovo vertice u viene aggiunto a C , aggiorniamo l'etichetta $D(v)$ di ogni vertice v adiacente a u e che è al di fuori di C , per riflettere il fatto che potrebbe esserci un nuovo e migliore modo per raggiungere v passando per u . Questa operazione di aggiornamento è nota come *rilassamento degli archi* (**edge relaxation**), poiché prende una vecchia stima e verifica se può essere migliorata per avvicinarsi al suo valore reale. L'operazione specifica di rilassamento degli archi è la seguente:

Edge Relaxation:

if $D(u) + w(u, v) < D(v)$ **then**
 $D(v) = D(u) + w(u, v)$

17.2.2 Pseudocodice ed Esempio

L'algoritmo di Dijkstra è descritto nel seguente pseudocodice:

Input: A weighted graph G with nonnegative edge weights,
and a source vertex s of G .

Output: The length of a shortest path from s to v for each
vertex v of G .

```
1. Algorithm ShortestPath( $G, s$ ):
2.   Initialize  $D[s] = 0$  and  $D[v] = +\infty$  for each vertex  $v \neq s$ 
3.   Let a priority queue  $Q$  contain all the vertices of  $G$ 
       using the  $D$  labels as keys.
4.   while  $Q$  is not empty do
5.     // pull a new vertex  $u$  into the cloud
6.      $u =$  value returned by  $Q.remove\_min()$ 
7.     for each vertex  $v$  adjacent to  $u$  such that  $v$  in  $Q$  do
8.       // perform the relaxation procedure on edge  $(u, v)$ 
9.       if  $D[u] + w(u, v) < D[v]$  then
10.         $D[v] = D[u] + w(u, v)$ 
11.        Change to  $D[v]$  the key of vertex  $v$  in  $Q$ .
12.   return the label  $D[v]$  of each vertex  $v$ 
```

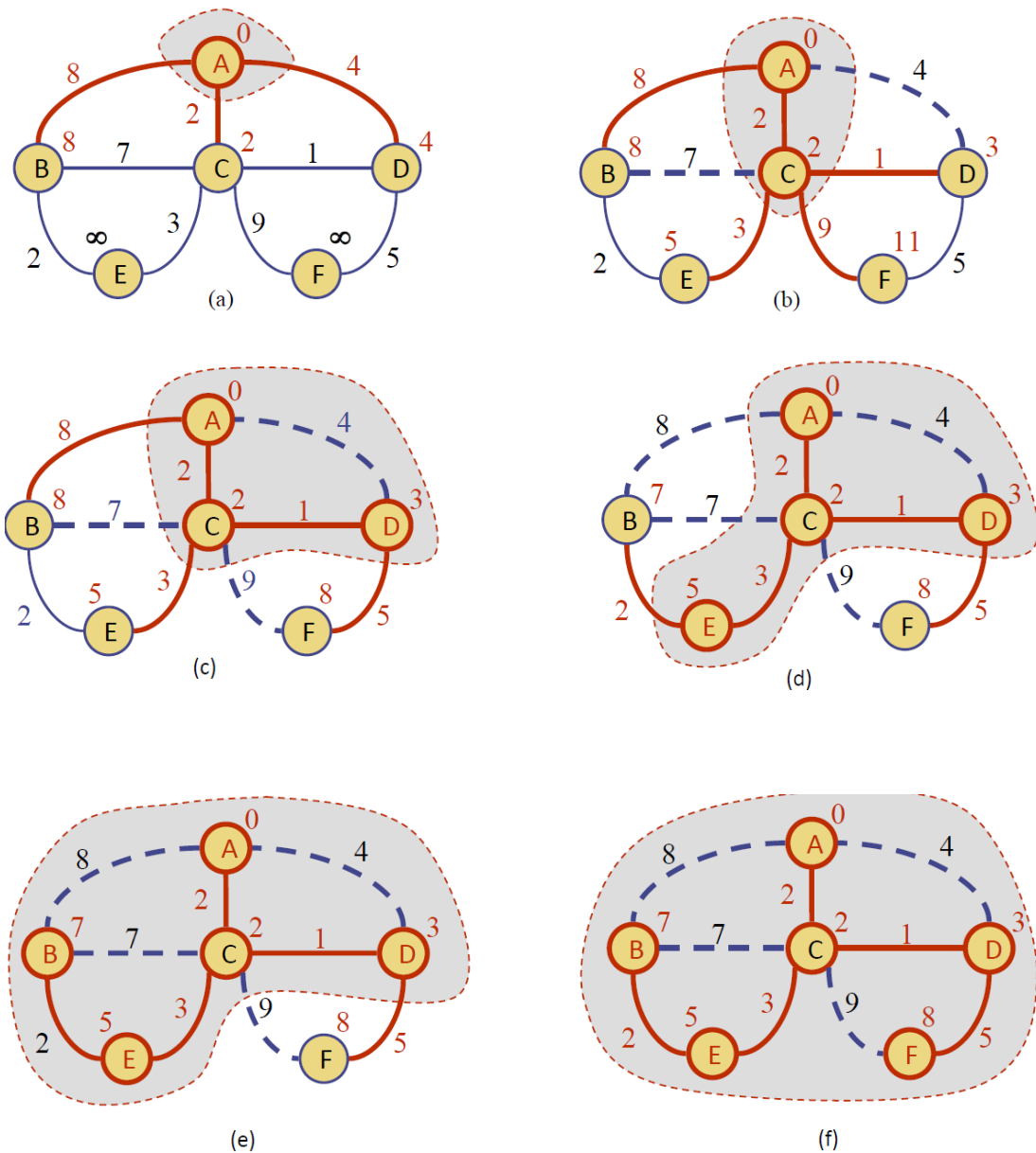


Figura 17.3: Esempio di esecuzione dell'algoritmo di Dijkstra. L'esempio inizia con il vertice sorgente A già nella nuvola (in grigio chiaro) e con le etichette dei vertici adiacenti aggiornate. Ad ogni passo, il vertice con l'etichetta minima viene aggiunto alla nuvola e le etichette dei suoi vicini vengono aggiornate di conseguenza. Al termine dell'algoritmo, le etichette rappresentano le distanze minime da A a tutti gli altri vertici. Gli archi in rosso al di fuori della nuvola rappresentano il percorso minimo attuale verso quel vertice. Gli archi in rosso nella nuvola rappresentano lo shortest path tree radicato in A. Gli archi tratteggiati sono quelli che non fanno parte dello shortest path tree.

17.2.3 Dimostrazione della correttezza dell'algoritmo

Potrebbe non essere immediatamente chiaro il motivo per cui l'algoritmo di Dijkstra trovi correttamente lo shortest path dal vertice di partenza s a ogni altro vertice u nel grafo. Perché la distanza minima da s a u è uguale al valore dell'etichetta $D(u)$ al momento in cui il vertice u viene rimosso dalla coda di priorità Q e aggiunto alla nuvola C ? La risposta a questa domanda dipende dal fatto che **non ci siano archi con peso negativo nel grafo**, poiché ciò consente al metodo Greedy di funzionare correttamente.

Teorema: Nell'algoritmo di Dijkstra, ogni volta che un vertice v viene inserito nella soluzione C , l'etichetta $D(v)$ è uguale a $d(s, v)$, la lunghezza del percorso minimo da s a v .

Dimostrazione: Per dimostrare la tesi, procediamo per assurdo. Ipotizziamo che l'algoritmo sbagli per qualche vertice. Sia z il **primo vertice** che l'algoritmo inserisce in C tale che la sua etichetta sia errata, ovvero:

$$D(z) > d(s, z)$$

Poiché z è il *primo* errore, significa che per tutti i vertici inseriti in C prima di z , l'algoritmo ha calcolato la distanza corretta.

Esiste sicuramente un percorso minimo reale da s a z (altrimenti la distanza sarebbe ∞). Chiamiamo questo percorso P . Consideriamo il momento esatto in cui l'algoritmo sta per inserire z in C . Analizziamo il percorso P partendo da s :

- Il vertice sorgente s appartiene già a C .
- Il vertice z non appartiene ancora a C .
- Di conseguenza, percorrendo il cammino P da s a z , deve necessariamente esistere un primo arco (x, y) che attraversa il confine di C , cioè tale che $x \in C$ (il vertice "dentro") e $y \notin C$ (il primo vertice "fuori").

Dunque, sia y il **primo vertice** di P che non appartiene a C , e sia x il predecessore di y lungo il percorso P .

- Poiché y è il primo vertice fuori da C , ne consegue che x deve trovarsi dentro C (al limite x potrebbe coincidere con s).

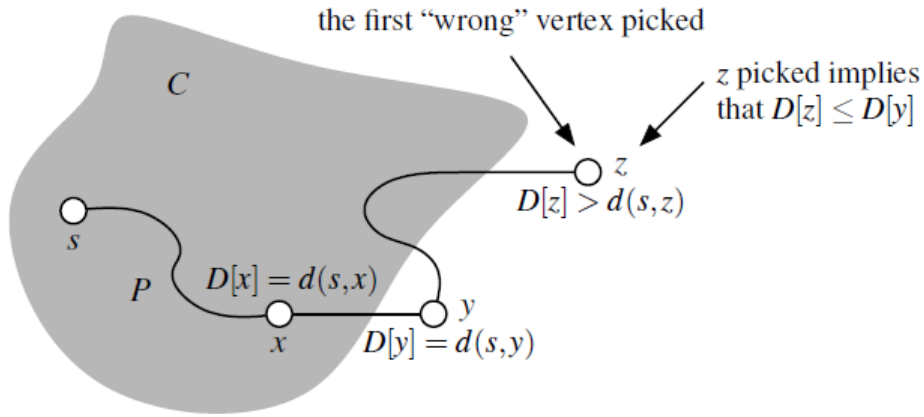


Figure 14.18: A schematic illustration for the justification of Proposition 14.23.

Figura 17.4: Rappresentazione grafica della dimostrazione della correttezza dell'algoritmo di Dijkstra.

Poiché z è stato definito come il *primo* vertice "sbagliato" aggiunto a C , e x è stato aggiunto a C prima di z , allora l'etichetta di x è sicuramente corretta:

$$D(x) = d(s, x)$$

Quando x è stato inserito in C , l'algoritmo ha eseguito la procedura di *Edge Relaxation* sui suoi vertici adiacenti, incluso y . Questo aggiornamento garantisce che:

$$D(y) \leq D(x) + w(x, y) = d(s, x) + w(x, y)$$

Poiché x e y sono vertici consecutivi sul percorso minimo P (che è un percorso minimo globale), anche il sotto-percorso da s a y deve essere un percorso minimo. Pertanto:

$$d(s, y) = d(s, x) + w(x, y)$$

Combinando le due equazioni precedenti, otteniamo che l'etichetta di y è già corretta al momento dell'estrazione di z :

$$D(y) = d(s, y)$$

Ora arriviamo al cuore della dimostrazione logica. L'algoritmo di Dijkstra è *greedy*: seleziona sempre il vertice fuori da C con l'etichetta D più piccola. In questo momento, sia z che y sono fuori da C (sono nella coda Q), ma l'algoritmo ha scelto di estrarre z . Questo implica necessariamente che:

$$D(z) \leq D(y)$$

Qui entra in gioco l'ipotesi fondamentale che non esistano archi con peso negativo. La distanza reale da s a z può essere scomposta nella distanza da s a y più la distanza rimanente da y a z :

$$d(s, z) = d(s, y) + d(y, z)$$

Poiché i pesi sono non negativi, allora si ha che $d(y, z) \geq 0$. Di conseguenza:

$$d(s, y) \leq d(s, z)$$

Mettiamo insieme la catena di disuguaglianze:

$$D(z) \leq D(y) \quad (\text{per la scelta greedy dell'algoritmo})$$

$$D(y) = d(s, y) \quad (\text{perché l'etichetta di } y \text{ è già corretta al momento dell'estrazione di } z)$$

$$d(s, y) \leq d(s, z) \quad (\text{perché i pesi sono non-negativi})$$

Unendo tutto:

$$D(z) \leq d(s, z)$$

Ma questo contraddice la nostra ipotesi iniziale per assurdo, secondo cui $D(z) > d(s, z)$.

La contraddizione dimostra che non può esistere un "primo vertice sbagliato" z . Pertanto, per ogni vertice v aggiunto a C , l'etichetta $D(v)$ rappresenta sempre la distanza minima corretta $d(s, v)$.

17.2.4 Analisi della complessità

Indichiamo con n e m rispettivamente il numero di vertici e di archi del grafo di input G . Assumiamo che i pesi degli archi possano essere sommati e confrontati in tempo costante.

Per analizzare il tempo di esecuzione dell'algoritmo di Dijkstra dobbiamo innanzitutto specificare come viene rappresentato il grafo G e come viene implementata la coda di priorità Q . Assumiamo di rappresentare il grafo G tramite *Adjacency List* o *Adjacency Map*. In questo modo possiamo attraversare i vertici adiacenti a u durante la fase di rilassamento in tempo proporzionale al loro numero. Pertanto, il tempo speso nella gestione del ciclo annidato `for`, e il numero di iterazioni di quel ciclo, è dato da:

$$\sum_{u \in V} \text{outdeg}(u)$$

che è $O(m)$. Il ciclo esterno `while` viene eseguito $O(n)$ volte, poiché in ogni iterazione viene aggiunto un nuovo vertice alla nuvola.

Tuttavia, per completare l'analisi dell'algoritmo, dobbiamo considerare anche l'implementazione della coda di priorità Q . La coda a priorità ad ogni iterazione dell'algoritmo contiene gli elementi che non sono ancora presenti nella soluzione C , per cui inizialmente vengono effettuate n operazioni di inserimento in Q (soluzione ancora vuota); queste sono le uniche operazioni di inserimento, quindi la dimensione massima della coda è n . In ciascuna delle n iterazioni del ciclo `while`, viene effettuata una chiamata a `remove_min()` per estrarre il vertice u con l'etichetta D più piccola da Q . Successivamente, per ogni vicino v di u , viene eseguito un

rilassamento dell'arco, che potrebbe comportare un aggiornamento della chiave di v nella coda ¹. Pertanto, è necessario implementare una *Adaptable Priority Queue*, in cui la chiave di un vertice v può essere modificata utilizzando il metodo `update(l, k)`, dove l è il *locator* per l'elemento della coda associato al vertice v . Nel peggiore dei casi, potrebbe esserci un aggiornamento per ogni arco del grafo. Complessivamente, il tempo di esecuzione dell'algoritmo di Dijkstra è limitato dalla somma delle seguenti operazioni:

- n operazioni di inserimento in Q .
- n chiamate al metodo `remove_min()` su Q .
- m chiamate al metodo `update()` su Q .

Se Q è una *Adaptable Priority Queue* implementata come heap, allora ciascuna delle operazioni sopra descritte viene eseguita in tempo $O(\log n)$, e quindi il tempo complessivo per l'algoritmo di Dijkstra è $O((n + m) \log n)$ (nel caso di un grafo connesso si ha $O(m \log n)$, dal momento che $m \geq n - 1$). Si noti che se si desidera esprimere il tempo di esecuzione come funzione di n soltanto, allora nel caso peggiore è $O(n^2 \log n)$.

È possibile implementare la coda a priorità Q utilizzando un'array non ordinato. In questo caso, l'operazione di estrazione del minimo richiede $O(n)$ tempo, mentre l'aggiornamento delle chiavi può essere eseguito in tempo costante. Pertanto, il tempo totale di esecuzione dell'algoritmo di Dijkstra diventa $O(n^2 + m)$, che si semplifica a $O(n^2)$ per grafi semplici.

Confronto tra le due implementazioni - Adaptable Priority Queue vs Array non ordinato

Come abbiamo visto, abbiamo due possibili scelte per implementare la coda a priorità Q nell'algoritmo di Dijkstra: una implementazione basata su heap, che porta a un tempo di esecuzione di $O((n + m) \log n)$, e una implementazione basata su array non ordinato, che porta a un tempo di esecuzione di $O(n^2)$. Dal momento che entrambe le implementazioni hanno lo stesso livello di complessità (a livello di codice), la scelta tra le due dipende principalmente dalla densità del grafo di input.

In generale, si preferisce l'implementazione basata su heap quando il grafo è sparso (ovvero, quando il numero di archi è abbastanza piccolo, e si ha $m < n^2/\log n$). Si preferisce invece l'implementazione basata su array non ordinato quando il grafo è denso (ovvero, quando il numero di archi è vicino al massimo possibile, e si ha $m > n^2/\log n$).

Implementation of the Priority Queue	Complexity	Usable for
Binary Heap	$O(m \log n)$	$m < n^2/\log n$
Unordered Array	$O(n^2)$	$m > n^2/\log n$
Fibonacci Heap	$O(m + n \log n)$	always

Tabella 17.1: Si noti che esiste un'implementazione avanzata della coda a priorità, nota come **Fibonacci heap**, che può essere utilizzata per implementare l'algoritmo di Dijkstra in tempo $O(m + n \log n)$. Si tratta di un'implementazione più complessa, ma che offre prestazioni migliori in tutti i casi, utile in applicazioni specifiche che richiedono un'elevata efficienza.

¹All'interno della coda a priorità Q , ciascun elemento ha per chiave il valore dell'etichetta $D(v)$ del vertice corrispondente v , e il valore associato è il vertice v stesso.

17.2.5 Implementazione Python dell'algoritmo di Dijkstra

```
1 def shortest_path_lengths(g, src):
2     """Compute shortest-path distances from src to reachable vertices of g.
3
4     Graph g can be undirected or directed, but must be weighted such that
5     e.element() returns a numeric weight for each edge e.
6
7     Return dictionary mapping each reachable vertex to its distance from
8     src.
9     """
10    # MAPPA: Vertice (oggetto) -> Distanza stimata (numero)
11    d = {}
12
13    # MAPPA: Vertice (oggetto) -> Distanza definitiva (numero)
14    cloud = {}
15
16    # CODA CON PRIORITA': Priorita'=Distanza (d[v]), Elemento=Vertice (v)
17    pq = AdaptableHeapPriorityQueue()
18
19    # MAPPA: Vertice (oggetto) -> Locator (oggetto handle)
20    pqlocator = {}
21
22    # for each vertex v of g, add an entry to the priority queue, with
23    # the source having distance 0 and all others having infinite distance
24    for v in g.vertices():
25        if v is src:
26            d[v] = 0
27        else:
28            d[v] = float('inf') # syntax for positive infinity
29            pqlocator[v] = pq.add(d[v], v) # save locator for future updates
30
31    while not pq.is_empty():
32        key, u = pq.remove_min()
33        cloud[u] = key # its correct d[u] value
34        del pqlocator[u] # u is no longer in pq
35        for e in g.incident_edges(u): # outgoing edges (u,v)
36            v = e.opposite(u)
37            if v not in cloud:
38                # perform relaxation step on edge (u,v)
39                wgt = e.element()
40                if d[u] + wgt < d[v]: # better path to v?
41                    d[v] = d[u] + wgt # update the distance
42                    pq.update(pqlocator[v], d[v], v) # update the pq entry
43
44    return cloud # only includes reachable vertices
```

Listing 17.1: Implementazione Python dell'algoritmo di Dijkstra per il calcolo delle distanze degli shortest path da una singola sorgente a tutti i vertici raggiungibili in un grafo pesato. Si assume che il metodo `e.element()` per un arco `e` rappresenti il peso di quell'arco.

Il codice visto finora per l'algoritmo di Dijkstra calcola correttamente le distanze minime da una sorgente s a tutti i vertici raggiungibili in un grafo pesato. Tuttavia, in molte applicazioni pratiche, è altrettanto importante poter ricostruire i percorsi effettivi che costituiscono gli shortest path, non solo le loro lunghezze.

La collezione di tutti gli shortest path che partono dalla sorgente s può essere rappresentata in modo compatto da quella che è nota come **shortest-path tree**. I percorsi formano un albero radicato perché se uno shortest path da s a v passa attraverso un vertice intermedio u , deve necessariamente iniziare con uno shortest path da s a u . In questa sezione, dimostriamo che lo shortest-path tree radicato in s può essere ricostruito in tempo $O(n + m)$, dato l'insieme dei valori $d[v]$ prodotti dall'algoritmo di Dijkstra utilizzando s come sorgente. Come abbiamo fatto quando abbiamo rappresentato gli alberi DFS e BFS, mapperemo ogni vertice $v \neq s$ a un genitore u (eventualmente può essere $u = s$), tale che u è il vertice immediatamente precedente di v su uno shortest path da s a v . Se u è il vertice immediatamente precedente di v sullo shortest path da s a v , deve essere che:

$$d[u] + w(u, v) = d[v]$$

Viceversa, se l'equazione sopra è soddisfatta, allora lo shortest path da s a u , seguito dall'arco (u, v) è uno shortest path per v . La nostra implementazione presentata di seguito ricostruisce l'albero basandosi su questa logica, testando tutti gli archi entranti per ogni vertice v , cercando un (u, v) che soddisfi l'equazione chiave. Il tempo di esecuzione è $O(n + m)$, poiché consideriamo ogni vertice e tutti gli archi entranti a quei vertici.

```

1 def shortest_path_tree(g, s, d):
2     """Reconstruct shortest-path tree rooted at vertex s, given distance
3         map d.
4
5     Return tree as a map from each reachable vertex v (other than s) to the
6     edge e=(u,v) that is used to reach v from its parent u in the tree.
7     """
8     tree = { }
9     for v in d:
10        if v is not s:
11            for e in g.incident_edges(v, False): # consider INCOMING edges
12                u = e.opposite(v)
13                wgt = e.element( )
14                if d[v] == d[u] + wgt:
15                    tree[v] = e # edge e is used to reach v
16    return tree

```

Listing 17.2: Funzione in Python che ricostruisce lo shortest-path tree radicato in un vertice sorgente s , dato il grafo g e la mappa delle distanze d calcolate dall'algoritmo di Dijkstra. La funzione restituisce un dizionario che mappa ogni vertice raggiungibile (diverso da s) all'arco utilizzato per raggiungerlo dal suo genitore nello shortest-path tree.

17.3 Algoritmo di Bellman-Ford

L'algoritmo di Dijkstra non garantisce la correttezza della soluzione in presenza di archi con peso negativo all'interno del grafo. Questa limitazione deriva dalla natura *greedy* dell'algoritmo, il quale costruisce l'insieme dei cammini minimi selezionando iterativamente i vertici in ordine crescente di distanza.

Il fondamento logico di Dijkstra risiede nell'assunzione che, estendendo un percorso, la distanza totale non possa mai diminuire. Quando un vertice u viene estratto dalla coda di priorità e aggiunto all'insieme dei nodi visitati (soluzione parziale), la sua distanza $D[u]$ è considerata **definitiva**. Se tutti i pesi sono non negativi ($w \geq 0$), questa assunzione è corretta poiché qualsiasi percorso alternativo che passi per nodi non ancora visitati avrà necessariamente una lunghezza maggiore o uguale a quella attuale.

Tuttavia, l'introduzione di archi con peso negativo invalida questa proprietà. Un arco negativo potrebbe rivelare un percorso "più lungo" in termini di numero di archi, ma con un costo totale inferiore, che passa attraverso nodi che l'algoritmo ha già scartato o finalizzato. Consideriamo il seguente scenario critico, in riferimento alla Figura 17.5:

- Un vertice C viene inserito nella soluzione definitiva con una stima di distanza $D(C) = 5$. Per l'algoritmo, questo valore è ottimale e non verrà più modificato.
- Successivamente, l'algoritmo esplora un vertice F e tenta di rilassare l'arco (F, C) . Se questo arco ha un peso fortemente negativo, potrebbe esistere un percorso che raggiunge C passando per F con un costo totale inferiore a $D(C)$ precedentemente calcolato (nel nostro caso $1 < 5$).
- Poiché l'algoritmo di Dijkstra non riconsidera i vertici già inseriti nell'insieme della soluzione (in quanto l'approccio greedy non consente backtracking), esso ignorerà questo aggiornamento, mantenendo erroneamente $D(C) = 5$ invece del valore corretto 1.

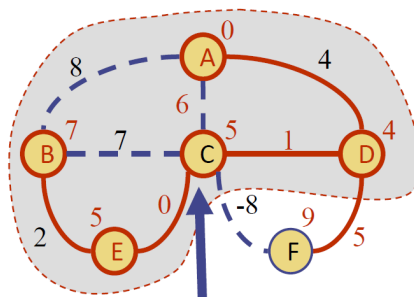


Figura 17.5: Esempio di grafo con arco a peso negativo che causa un errore nell'algoritmo di Dijkstra.

Per gestire grafi con pesi negativi (purché privi di cicli negativi²), è necessario ricorrere ad algoritmi alternativi come **l'algoritmo di Bellman-Ford**.

²Un **ciclo negativo** è un ciclo (una sequenza chiusa di nodi) il cui peso totale (la somma dei pesi degli archi che lo compongono) è negativo. In presenza di cicli negativi, il concetto di shortest path diventa indefinito, poiché si potrebbe continuare a percorrere il ciclo negativo per ridurre indefinitamente la lunghezza del percorso.

Gestione degli Archi Negativi: L'approccio di Bellman-Ford

Per risolvere il problema dei cammini minimi in presenza di archi negativi (assumendo per semplicità un grafo orientato), è necessario abbandonare la logica *greedy* di selezione dei vertici in favore di un approccio basato sul rilassamento iterativo globale. Questo metodo si fonda sulle proprietà descritte dal **Teorema di Bellman**.

Il Teorema di Bellman

Il principio fondamentale afferma che per ogni arco (u, v) nel grafo, la distanza calcolata verso v deve soddisfare la disuguaglianza:

$$d(v) \leq d(u) + w(u, v)$$

Specificamente, un arco (u, v) appartiene all'albero dei cammini minimi (Shortest Path Tree, T_s) se e solo se la condizione di uguaglianza è soddisfatta:

$$d(v) = d(u) + w(u, v)$$

Funzionamento dell'Algoritmo

A differenza di Dijkstra, non consideriamo l'ordine in cui i vertici vengono aggiunti all'albero. L'algoritmo procede invece per iterazioni fisse:

- **Strategia Globale:** Ad ogni iterazione, l'algoritmo tenta di rilassare **tutti** gli archi presenti nel grafo, cercando sistematicamente percorsi più brevi.
- **Numero di Passi:** L'algoritmo esegue esattamente $n - 1$ cicli (dove n è il numero dei vertici).
- **Significato del passo k :** Alla k -esima iterazione, l'algoritmo garantisce di aver trovato i cammini minimi che sono costituiti da esattamente (o al più) k archi.

Poiché un cammino semplice (cioè senza cicli) in un grafo di n nodi può contenere al massimo $n - 1$ archi, dopo $n - 1$ iterazioni siamo certi che tutte le distanze siano state propagate correttamente attraverso la rete, anche in presenza di pesi negativi.

Questo algoritmo è particolarmente interessante in quanto è alla base dei protocolli di routing distance-vector, essenziali per instradare i dati su Internet. Sfruttando la logica di Bellman-Ford, i router non hanno bisogno di conoscere l'intera mappa della rete: scambiandosi semplicemente le stime delle distanze con i vicini e applicando iterativamente il rilassamento, riescono a calcolare il percorso ottimale verso qualsiasi destinazione in modo distribuito e dinamico.

Pseudocodice dell'Algoritmo

```

1. Algorithm BellmanFord( $G, s$ )
2.   for all  $v$  in  $G.vertices()$  do
3.     if  $v = s$  then
4.        $d(v) = 0$ 
5.     else
6.        $d(v) = \infty$ 
7.   for  $i = 1$  to  $n - 1$  do
8.     for each  $e$  in  $G.edges()$  do
9.       // relax edge  $e$ 
10.       $u = G.origin(e)$ 
11.       $v = G.opposite(u, e)$ 
12.       $r = d(u) + weight(e)$ 
13.      if  $r < d(v)$  then
14.         $d(v) = r$ 

```

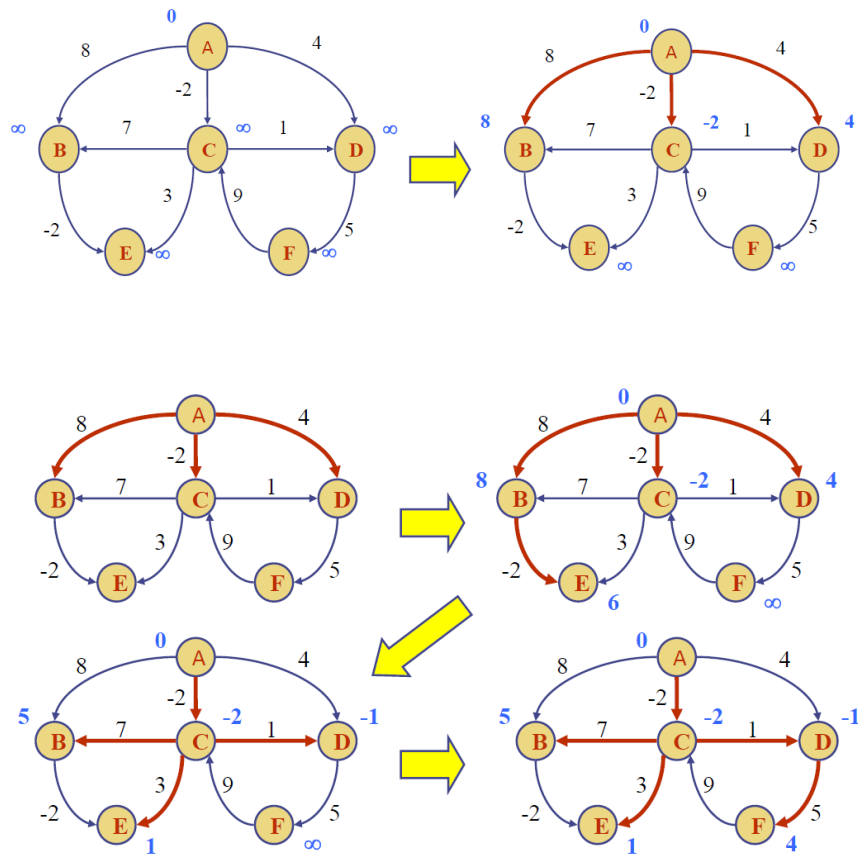


Figura 17.6: Esempio di esecuzione dell'algoritmo di Bellman-Ford.

Analisi della complessità

L'algoritmo di Bellman-Ford ha una complessità temporale di $O(n \cdot m)$, dove n è il numero di vertici e m è il numero di archi nel grafo. Questo deriva dal fatto che l'algoritmo esegue $n - 1$ iterazioni e in ciascuna iterazione rilassa tutti gli m archi del grafo. Pertanto, il tempo totale di esecuzione è proporzionale al prodotto del numero di vertici e del numero di archi, risultando in $O(n \cdot m)$. Questo rende l'algoritmo di Bellman-Ford meno efficiente rispetto all'algoritmo di Dijkstra per grafi con pesi non negativi, che può essere eseguito in tempo $O((n + m) \log n)$ utilizzando una coda di priorità basata su heap. Tuttavia, Bellman-Ford è l'unica alternativa quando si lavora con grafi che possono contenere archi con pesi negativi, purché non vi siano cicli negativi.

17.4 Shortest Paths in un DAG

Dopo aver analizzato l'algoritmo di Dijkstra (ottimo per grafi con pesi non negativi) e l'algoritmo di Bellman-Ford (necessario in presenza di pesi negativi), esaminiamo ora un caso particolare ma molto frequente: il calcolo dei cammini minimi in un **DAG** (*Directed Acyclic Graph*).

Richiamo: Cos'è un DAG

Come definito in precedenza, un DAG è un grafo diretto che non contiene cicli. Questo significa che partendo da un qualsiasi nodo u e seguendo gli archi orientati, è impossibile ritornare su u . Questa proprietà di "aciclicità" è fondamentale perché implica l'esistenza di un **ordinamento topologico** (Topological Sort).

Perché un approccio specifico?

Sebbene l'algoritmo di Bellman-Ford funzioni correttamente sui DAG (poiché non avendo cicli, non possono avere cicli negativi), esso ha una complessità di $O(n \cdot m)$. L'algoritmo di Dijkstra, d'altra parte, non supporta archi con peso negativo.

L'algoritmo specifico per i DAG supera entrambi i limiti sfruttando la struttura topologica del grafo:

- **Gestisce pesi negativi:** A differenza di Dijkstra, funziona perfettamente anche se gli archi hanno peso negativo. Essendo il grafo aciclico, non esiste il rischio di "cicli negativi" che porterebbero il costo a $-\infty$.
- **Efficienza superiore:** È molto più veloce di Bellman-Ford e persino di Dijkstra, poiché non richiede strutture dati ausiliarie complesse (come le code di priorità) e visita ogni arco esattamente una volta.

L'Algoritmo: Rilassamento in Ordine Topologico

L'idea chiave è la seguente: se visitiamo i nodi in ordine topologico, quando arriviamo a elaborare un nodo u , abbiamo la garanzia che il cammino minimo per raggiungerlo, $D(u)$, sia già stato calcolato definitivamente. Non esistono archi che tornano "indietro" da nodi successivi che potrebbero aggiornare e migliorare $D(u)$.

L'algoritmo procede in due fasi:

1. **Ordinamento Topologico:** Si ordinano linearmente i vertici del grafo in modo che per ogni arco (u, v) , u appaia prima di v nella sequenza.
2. **Rilassamento Lineare:** Si inizializzano le distanze (0 per la sorgente s , ∞ per gli altri). Si scorrono i nodi u secondo l'ordine topologico e, per ciascuno, si rilassano tutti gli archi uscenti.

```
1. Algorithm DagDistances( $G, s$ )
2.   for all  $v$  in  $G.vertices()$  do
3.     if  $v = s$  then
4.        $d(v) = 0$ 
5.     else
6.        $d(v) = \infty$ 
7.   // build a topological sort of the vertices
8.   for  $u = 1$  to  $n - 1$  do // in topological order
9.     for each  $e$  in  $G.outEdges(u)$  do
10.      // relax edge  $e$ 
11.       $v = G.opposite(u, e)$ 
12.       $r = d(u) + weight(e)$ 
13.      if  $r < d(v)$  then
14.         $d(v) = r$ 
```

Analisi della Complessità

La complessità dell'algoritmo è determinata da due passaggi:

- L'ordinamento topologico può essere eseguito in tempo $O(n + m)$.
- Il ciclo di rilassamento visita ogni vertice una volta e ogni arco esattamente una volta, impiegando anch'esso $O(n + m)$.

Pertanto, la complessità temporale totale è:

$$O(n + m)$$

Questo lo rende un algoritmo a tempo lineare, la soluzione asintoticamente ottima per questo problema.

17.5 All-Pairs Shortest Paths

Fino ad ora ci siamo concentrati sul problema *Single-Source Shortest Path*, ovvero trovare i cammini minimi da una singola sorgente verso tutti gli altri nodi. Come abbiamo già visto, esiste una generalizzazione del problema: il calcolo dei cammini minimi **tra ogni coppia di vertici** del grafo. Anche in questo caso, gli archi possono avere pesi negativi, purché non esistano cicli negativi.

Approccio Ingenuo: Iterare SSSP

Una prima soluzione intuitiva consiste nell'eseguire un algoritmo *Single-Source* per ogni vertice del grafo, considerandolo di volta in volta come sorgente.

- Se usiamo **Dijkstra**: Funziona solo se i pesi sono non negativi. La complessità sarebbe $O(n \cdot (m + n \log n)) = O(nm + n^2 \log n)$.
- Se usiamo **Bellman-Ford**: Necessario se ci sono pesi negativi. La complessità sarebbe $O(n(n \cdot m)) = O(n^2 m)$.

Se il grafo è denso (dove $m \approx n^2$), la complessità dell'approccio ingenuo con Bellman-Ford diventa $O(n^4)$, che è spesso inaccettabile.

Soluzione Efficiente: Floyd-Warshall

Per migliorare l'efficienza, Floyd e Warshall hanno proposto un algoritmo basato sulla **Programmazione Dinamica**. L'idea è simile a quella usata per calcolare la chiusura transitiva di un grafo.

Il calcolo si basa sulla seguente ricorrenza per $SP(i, j, k)$, che rappresenta il cammino minimo da i a j considerando solo i primi k vertici come intermedi:

1. **Caso Base** ($k = 0$):

$$SP(i, j, 0) = \begin{cases} w(i, j) & \text{se } (i, j) \in E \\ \infty & \text{altrimenti} \end{cases}$$

2. **Passo Ricorsivo**: La distanza minima considerando il k -esimo vertice è il minimo tra non usarlo (restare al passo $k - 1$) e usarlo come ponte:

$$SP(i, j, k) = \min\{SP(i, j, k - 1), \quad SP(i, k, k - 1) + SP(k, j, k - 1)\}$$

La complessità temporale di questo algoritmo è:

$$O(n^3)$$

17.6 Riepilogo Finale: Algoritmi di Shortest Path

Ecco una sintesi degli algoritmi trattati in questo capitolo per scegliere quello più adatto in base al contesto:

Tabella 17.2: Confronto delle complessità degli algoritmi di Shortest Path.

Algoritmo	Tipo	Vincoli	Complessità
BFS	SSSP	Grafo non pesato	$O(n + m)$
Dijkstra	SSSP	Pesi non negativi ($w \geq 0$)	$O(m + n \log n)$
Bellman-Ford	SSSP	Pesi generici (no cicli neg.)	$O(n \cdot m)$
DAG-SP	SSSP	DAG, Pesi generici (no cicli neg.)	$O(n + m)$
Floyd-Warshall	All-Pairs	Pesi generici (no cicli neg.)	$O(n^3)$

Capitolo 18

Minimum Spanning Trees (MST)

È utile cominciare richiamando alcune definizioni già viste nel capitolo sui grafi (capitolo 13).

- Una **forest** è un grafo non orientato e aciclico (ovvero privo di cicli).
- Un **tree** è una forest connessa. In altre parole, un tree è un grafo non orientato, aciclico e connesso.
- Un **sottografo** H di un grafo $G = (V, E)$ è un grafo $H = (V', E')$ tale che $V' \subseteq V$ e $E' \subseteq E$. In altre parole, un sottografo è ottenuto rimuovendo vertici e/o archi da G .
- Uno **spanning subgraph** (la traduzione corretta è "sottografo ricoprente") di un grafo $G = (V, E)$ è un sottografo $H = (V', E')$ tale che $V' = V$. In altre parole, uno spanning subgraph contiene tutti i vertici di G , ma potrebbe non contenere tutti gli archi di G .
- Uno **spanning tree** di un grafo è uno spanning subgraph che è anche un tree (tree = grafo non orientato, aciclico e connesso). Si noti come uno spanning tree di un grafo con n vertici **contenga esattamente** $n - 1$ archi.

Dato un grafo non orientato e pesato G , siamo interessati a trovare un tree T che contenga tutti i vertici di G e minimizzi la somma dei pesi degli archi in T .

$$w(T) = \sum_{(u,v) \in T} w(u,v)$$

Un tale tree T è chiamato **minimum spanning tree** (MST) di G .

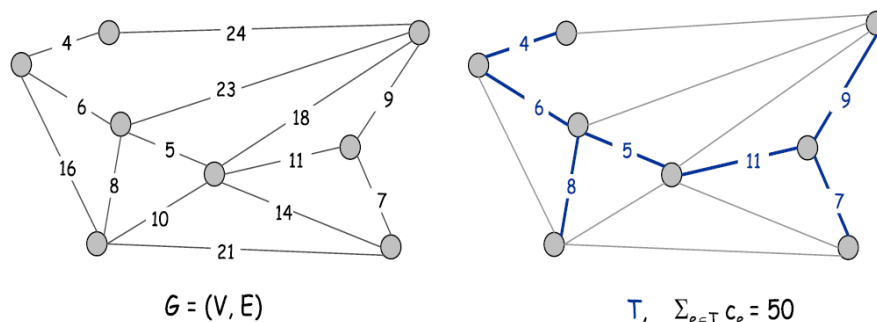


Figura 18.1: Esempio di grafo non orientato e pesato (a sinistra) e del suo Minimum Spanning Tree (a destra).

Si tratta di un problema di ottimizzazione combinatoria molto importante, con numerose applicazioni pratiche, ad esempio nella progettazione di reti (telefoniche, elettriche, idriche, ecc.).

Supponiamo, ad esempio, di dover connettere tutti i computer di un nuovo edificio per uffici utilizzando la minor quantità di cavo possibile. Possiamo modellare questo problema utilizzando un grafo non orientato e pesato G i cui vertici rappresentano i computer, e i cui archi rappresentano tutte le possibili coppie (u, v) di computer, dove il peso $w(u, v)$ dell'arco (u, v) è pari alla quantità di cavo necessaria per connettere il computer u al computer v . Piuttosto che calcolare un albero dei cammini minimi a partire da un particolare vertice v , siamo invece interessati a trovare un albero T che contenga tutti i vertici di G e che abbia il peso totale minimo tra tutti gli alberi di questo tipo.

MST vs Shortest Paths: è importante non confondere il problema del Minimum Spanning Tree con il problema degli Shortest Paths. Gli spanning tree calcolati dagli algoritmi di Shortest Paths (come Dijkstra o Bellman-Ford) dipendono dal vertice sorgente scelto, mentre il Minimum Spanning Tree è unico (a meno di casi particolari di pesi uguali) e non dipende da alcun vertice sorgente. Inoltre, gli Shortest Paths mirano a minimizzare la distanza da un singolo vertice sorgente a tutti gli altri vertici, mentre il Minimum Spanning Tree mira a minimizzare la somma totale dei pesi degli archi che connettono tutti i vertici del grafo.

Proprietà di un MST - Cycle Property

Sia T un Minimum Spanning Tree di un grafo non orientato e pesato G , e sia e un arco di G non appartenente a T . Chiamiamo C il ciclo ottenuto aggiungendo l'arco e a T . Allora, l'arco e è il più pesante tra tutti gli archi del ciclo C , ovvero $w(f) \leq w(e)$ per ogni arco $f \in C$.

Dimostrazione: Supponiamo, per assurdo, che esista un arco $f \in C$ tale che $w(f) > w(e)$. Rimuovendo l'arco f da T e aggiungendo l'arco e , otteniamo un nuovo albero $T' = T - \{f\} + \{e\}$. Poiché $w(e) < w(f)$, il peso totale di T' è minore di quello di T , ovvero $w(T') < w(T)$. Questo contraddice l'ipotesi che T sia un Minimum Spanning Tree. Pertanto, l'arco e deve essere il più pesante tra tutti gli archi del ciclo C .

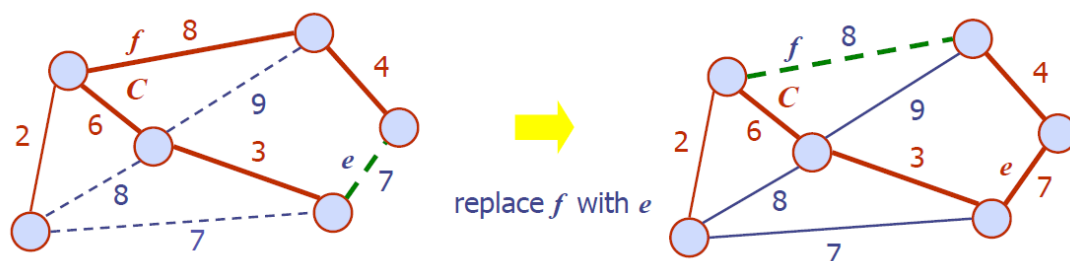


Figura 18.2: Dimostrazione della Cycle Property.

Proprietà di un MST - Partitioning Property

Sia G un grafo connesso, non orientato e pesato. Sia V_1 e V_2 una partizione dei vertici di G in due insiemi disgiunti e non vuoti. Inoltre, sia e un arco in G con peso minimo tra quelli che hanno un estremo in V_1 e l'altro in V_2 (cioè tra quelli che fanno da "ponte" tra le due partizioni). Allora, esiste un¹ Minimum Spanning Tree T che ha e come uno dei suoi archi.

Dimostrazione: Sia T un Minimum Spanning Tree di G . Se $e \in T$, la tesi è verificata. Supponiamo quindi che $e \notin T$. Poiché T è uno spanning tree, esiste un unico cammino in T che collega gli estremi dell'arco e . L'aggiunta di e a T genera dunque un ciclo C .

Poiché l'arco e ha un estremo in V_1 e l'altro in V_2 , nel ciclo C deve esistere almeno un arco $f \neq e$ appartenente a T che ha un estremo in V_1 e l'altro in V_2 . Per la definizione di e , si ha

$$w(e) \leq w(f).$$

Consideriamo ora il grafo T' ottenuto rimpiazzando l'arco f con l'arco e in T :

$$T' = (T \cup \{e\}) - \{f\}.$$

Il grafo T' è connesso, aciclico e copre tutti i vertici di G , quindi è uno spanning tree. Inoltre:

$$w(T') = w(T) + w(e) - w(f) \leq w(T).$$

Poiché T è un Minimum Spanning Tree, segue che $w(T') = w(T)$ e quindi anche T' è un Minimum Spanning Tree. Per costruzione, T' contiene l'arco e .

Pertanto, esiste un Minimum Spanning Tree di G che contiene l'arco e .

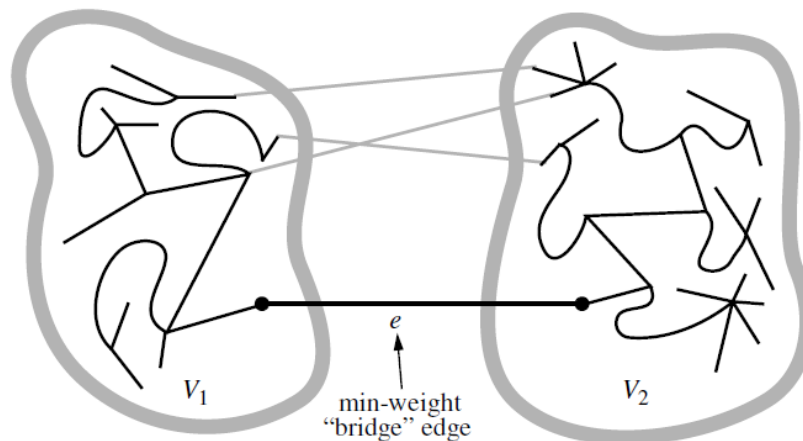


Figura 18.3: Dimostrazione della Partitioning Property.

¹ Si dice un Minimum Spanning Tree poiché, in presenza di archi con lo stesso peso, il MST non è necessariamente unico: l'arco e di peso minimo che attraversa la partizione appartiene ad almeno un MST, ma non è detto che appartenga a tutti. Se tutti gli archi hanno pesi distinti, allora il MST è unico e l'arco e appartiene necessariamente a questo unico MST.

L'approccio greedy per il calcolo di un MST

L'idea più immediata per calcolare un Minimum Spanning Tree sarebbe quella di calcolare tutti i possibili spanning tree di un grafo e scegliere quello con peso minimo. Tuttavia, questo approccio è computazionalmente inefficiente, poiché il numero di spanning tree cresce esponenzialmente con il numero di vertici del grafo, per cui per un grafo con n vertici il numero di spanning tree possibili è dell'ordine di $O(n^{n-2})$.

Ricordando che uno spanning tree con n vertici contiene esattamente $n - 1$ archi, possiamo considerare un approccio più efficiente basato sulla selezione iterativa degli archi da includere nello spanning tree. In particolare, possiamo adottare un approccio greedy, che consiste nel costruire la soluzione passo dopo passo, a partire dalla soluzione vuota, selezionando ad ogni passo l'arco (o il vertice) da aggiungere alla soluzione in base a una scelta che minimizza una certa funzione di costo.

L'idea generale è quella di scegliere, ad ogni step, l'arco di peso minimo che preservi la proprietà di **aciclicità**.

Di seguito presentiamo tre possibili algoritmi greedy per il calcolo di un Minimum Spanning Tree: l'algoritmo di **Prim-Jarník**, l'algoritmo di **Kruskal** e il **Reverse-Delete Algorithm**.

18.1 L'algoritmo di Prim-Jarník

L'algoritmo di Prim-Jarník (spesso abbreviato in Prim) è un algoritmo greedy per il calcolo di un Minimum Spanning Tree di un grafo connesso, non orientato e pesato.

L'algoritmo costruisce un minimum spanning tree operando su un singolo cluster che ha origine in un vertice 'radice' (qualsiasi) s . L'idea principale è simile a quella dell'algoritmo di Dijkstra. Iniziamo con un vertice s che definisce la "nuvola" (simile a Dijkstra) iniziale di vertici C . Poi, ad ogni iterazione, scegliamo un arco di peso minimo $e = (u, v)$ che collega un vertice u nella nuvola C a un vertice v al di fuori di C . Il vertice v viene quindi aggiunto alla nuvola C e il processo viene ripetuto fino a formare uno spanning tree. Ancora una volta, entra in gioco il fatto cruciale riguardante i minimum spanning tree: scegliendo sempre l'arco di peso minimo che collega un vertice all'interno di C a uno all'esterno di C , siamo sicuri di aggiungere sempre un arco valido al MST.

Per implementare in modo efficiente questo approccio, possiamo prendere ancora una volta spunto dall'algoritmo di Dijkstra. Manteniamo un'etichetta $D[v]$ per ogni vertice v al di fuori della nuvola C , in modo che $D[v]$ memorizzi il peso dell'arco minimo osservato per collegare v alla nuvola C . (Nell'algoritmo di Dijkstra, questa etichetta misurava la lunghezza totale del percorso dal vertice di partenza s a v , incluso un arco (u, v) .) Queste etichette servono come chiavi in una coda di priorità utilizzata per decidere quale vertice è il prossimo ad unirsi alla nuvola.

Pseudocodice

L'algoritmo Prim è descritto nel seguente pseudocodice:

Input: An undirected, weighted, connected graph G with n vertices
and m edges

Output: A minimum spanning tree T for G

```
1. Algorithm PrimJarnik( $G$ ):
2.   Pick any vertex  $s$  of  $G$ 
3.    $D[s] = 0$ 
4.   for each vertex  $v \neq s$  do
5.      $D[v] = +\infty$ 
6.   Initialize  $T =$  empty set
7.   Initialize a priority queue  $Q$  with an entry  $(D[v], (v, \text{None}))$ 
      for each vertex  $v$ , where  $D[v]$  is the key in the priority
      queue, and  $(v, \text{None})$  is the associated value.
8.   while  $Q$  is not empty do
9.      $(u, e) =$  value returned by  $Q.\text{remove\_min}()$ 
10.    if  $e$  is not None then
11.      Add edge  $e$  to  $T$ 
12.      for each edge  $e' = (u, v)$  such that  $v$  in  $Q$  do
13.        # check if edge  $(u, v)$  better connects  $v$  to  $T$ 
14.        if  $w(u, v) < D[v]$  then
15.           $D[v] = w(u, v)$ 
16.          Change the key of vertex  $v$  in  $Q$  to  $D[v]$ .
17.          Change the value of vertex  $v$  in  $Q$  to  $(v, e')$ .
17.   return the tree  $T$ 
```

Analisi della complessità

I tempi di esecuzione dell'implementazione dell'algoritmo Prim sono gli stessi di quelli dell'algoritmo di Dijkstra, poichè entrambi dipendono dalle operazioni eseguite su una Adaptable Priority Queue Q . Inizialmente vengono eseguiti n inserimenti in Q , e successivamente vengono eseguite n operazioni di $\text{remove_min}()$, e possono essere aggiornate un totale di m priorità come parte dell'algoritmo. Questi passaggi sono i principali contributi al tempo di esecuzione complessivo. Con una coda di priorità basata su heap, ogni operazione viene eseguita in tempo $O(\log n)$, e il tempo complessivo per l'algoritmo è $O((n + m) \log n)$, che è $O(m \log n)$ per un grafo connesso. In alternativa, possiamo ottenere un tempo di esecuzione di $O(n^2)$ utilizzando una lista non ordinata (unsorted list) come coda di priorità.

Implementazione Python

```
1 def MST_PrimJarnik(g):
2     """Compute a minimum spanning tree of weighted graph g.
3
4     Return a list of edges that comprise the MST (in arbitrary order).
5     """
6     d = { }                # d[v] is bound on distance to tree
7     tree = [ ]            # list of edges in spanning tree
8     pq = AdaptableHeapPriorityQueue( ) # d[v] maps to value (v, e=(u,v))
9     pqlocator = { }       # map from vertex to its pq locator
10
11     # for each vertex v, add an entry to the priority queue, with
12     # the source having distance 0 and all others having infinite distance
13     for v in g.vertices( ):
14         if len(d) == 0:    # this is the first node
15             d[v] = 0      # make it the root
16         else:
17             d[v] = float('inf') # positive infinity
18             pqlocator[v] = pq.add(d[v], (v, None))
19
20     while not pq.is_empty( ):
21         key, value = pq.remove_min()
22         u, edge = value    # unpack tuple from pq
23         del pqlocator[u]   # u is no longer in pq
24         if edge is not None: # if it's not the first node
25             tree.append(edge) # add edge to tree
26         for link in g.incident_edges(u):
27             v = link.opposite(u)
28             if v in pqlocator: # thus v not yet in tree
29                 # see if edge (u,v) better connects v to the growing tree
30                 wgt = link.element( )
31                 if wgt < d[v]: # better edge to v?
32                     d[v] = wgt # update the distance
33                     pq.update(pqlocator[v], d[v], (v, link)) # update the
34                                     pq entry
35
36     return tree
```

Listing 18.1: Implementazione Python dell'algoritmo Prim. L'MST viene restituito come una lista non ordinata di archi.

Esempio di esecuzione

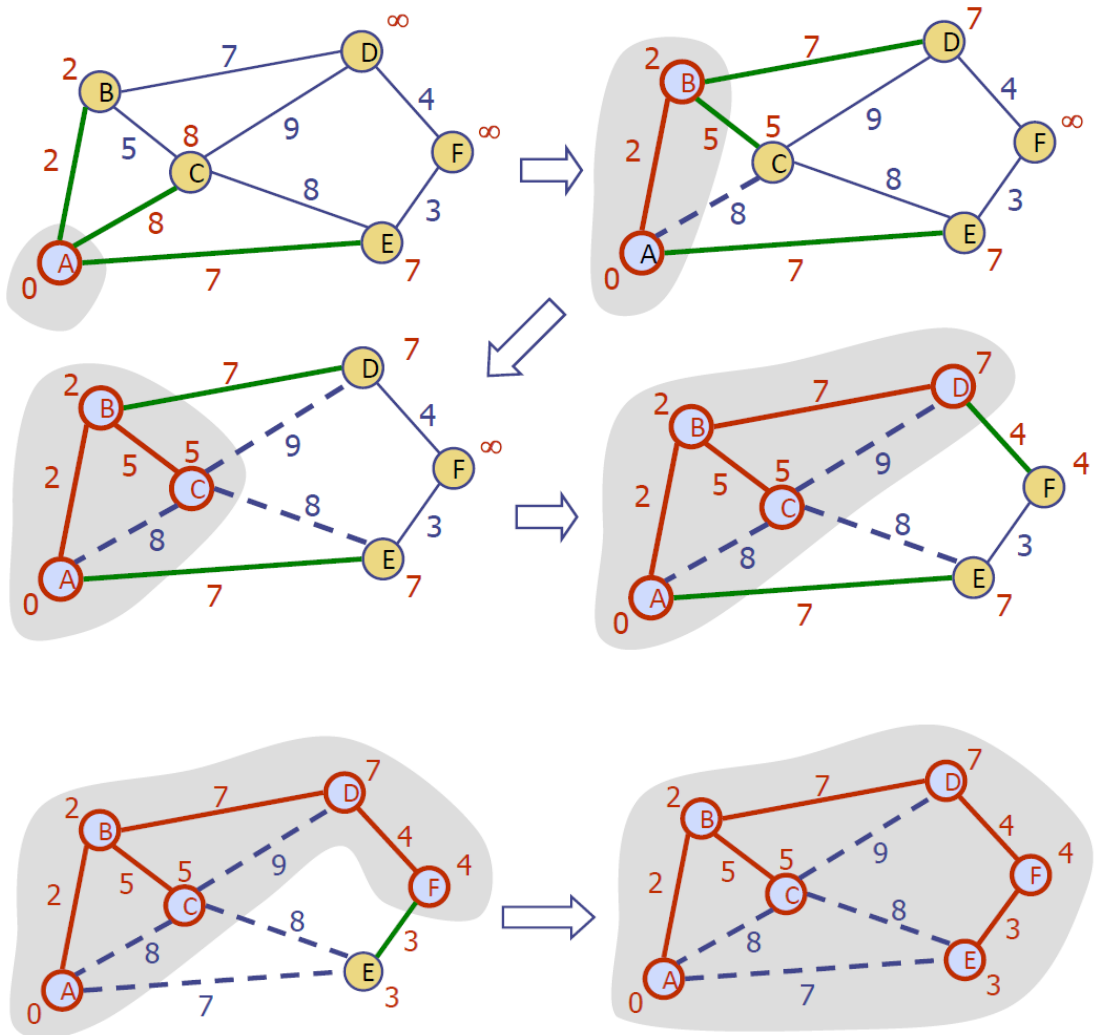


Figura 18.4: Esempio di esecuzione dell'algoritmo Prim su un grafo connesso, non orientato e pesato.

18.2 L'algoritmo di Kruskal

L'algoritmo di Kruskal è un altro algoritmo greedy per la risoluzione del problema del Minimum Spanning Tree. A differenza dell'algoritmo di Prim-Jarník, che costruisce l'albero facendo crescere un unico cluster (o "nuvola") di vertici connessi, l'algoritmo di Kruskal mantiene una **foresta di alberi disgiunti** (cluster) che vengono progressivamente fusi insieme.

Inizialmente, ogni vertice del grafo è considerato un albero separato (un cluster elementare). L'algoritmo esamina gli archi del grafo in ordine crescente di peso. Per ogni arco considerato, se questo connette due vertici che appartengono a cluster diversi, l'arco viene aggiunto alla soluzione e i due cluster vengono fusi in un unico insieme. Se invece l'arco connette due vertici che appartengono già allo stesso cluster, l'arco viene scartato, poiché la sua aggiunta creerebbe un ciclo (violando la proprietà fondamentale di aciclicità degli alberi).

L'algoritmo termina quando tutti i vertici appartengono allo stesso cluster (ovvero quando la foresta si riduce a un singolo albero che copre tutti i nodi) o quando non ci sono più archi da esaminare (se il grafo iniziale è non connesso, il risultato sarà una foresta di alberi minimi, ovvero un Minimum Spanning Forest, MSF).

Strutture Dati: Per implementare efficientemente questo approccio, sono necessarie:

1. Una **Priority Queue** per gestire gli archi ed estrarli in ordine di peso minimo.
2. Una struttura dati per la gestione di **insiemi disgiunti** (Disjoint Sets), spesso chiamata **ADT Partition**.

L'ADT Partition deve supportare le seguenti operazioni:

- **makeSet(u)**: crea un nuovo insieme contenente solo il vertice u . ($O(1)$)
- **find(u)**: restituisce l'identificativo dell'insieme a cui appartiene u . ($O(1)$)
- **union(A, B)**: unisce i due insiemi A e B in un unico insieme, aggiornando i riferimenti. $O(\min(|A|, |B|))$

Una implementazione efficiente dell'ADT Partition (basata su liste con puntatori al set di appartenenza e euristica "weighted union" che sposta sempre l'insieme più piccolo in quello più grande) permette di eseguire le operazioni di unione in modo molto rapido.

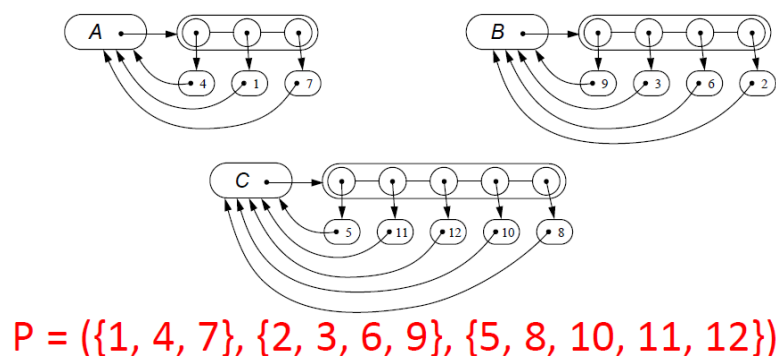


Figura 18.5: L'ADT Partition può essere rappresentato con un insieme di liste; ogni insieme è rappresentato da una lista distinta e ogni elemento nella lista ha un riferimento all'insieme a cui appartiene.

Pseudocodice

L'algoritmo Kruskal è descritto nel seguente pseudocodice:

```
Input: A simple connected weighted graph  $G$  with  $n$  vertices and  $m$  edges
Output: A minimum spanning tree  $T$  for  $G$ 

1. Algorithm Kruskal( $G$ ):
2.   for each vertex  $v$  in  $G$  do
3.     Define an elementary cluster  $C(v) = \{v\}$ .
4.   Initialize a priority queue  $Q$  to contain all edges in  $G$ , using
       the weights as keys.
5.    $T =$  empty set { $T$  will ultimately contain the edges of the MST}
6.   while  $T$  has fewer than  $n - 1$  edges do
7.      $(u, v) =$  value returned by  $Q.remove\_min()$ 
8.     Let  $C(u)$  be the cluster containing  $u$ 
9.     Let  $C(v)$  be the cluster containing  $v$ 
10.    if  $C(u) \neq C(v)$  then
11.      Add edge  $(u, v)$  to  $T$ .
12.      Merge  $C(u)$  and  $C(v)$  into one cluster.
13.  return tree  $T$ 
```

Analisi della complessità

La complessità temporale dell'algoritmo di Kruskal dipende dall'implementazione della coda di priorità e della struttura Partition.

- **Operazioni sulla Priority Queue:** La costruzione della coda richiede $O(m)$. L'estrazione degli archi richiede $O(\log m) = O(\log n)$ per operazione. Nel caso peggiore, estraiamo tutti gli archi, portando il costo globale delle operazioni sulla coda a $O(m \log m)$ o equivalentemente $O(m \log n)$.
- **Operazioni sui Cluster (Partition):** Vengono eseguite $2m$ operazioni di `find` che impiegano $O(m)$ e $n - 1$ operazioni di `union` che impiegano $O(n \log n)$. Utilizzando un'implementazione efficiente basata su liste in cui si unisce sempre l'insieme più piccolo a quello più grande, il costo ammortizzato totale è $O(m + n \log n)$.
 - La complessità temporale dell'algoritmo di Kruskal è quindi $O((n + m) \log n)$ se il grafo è rappresentato tramite liste di adiacenza/mappe, oppure $O(m \log n)$ se il grafo è connesso.

Complessivamente, il tempo di esecuzione è dominato dall'ordinamento degli archi (gestione della Priority Queue), risultando in una complessità di $O(m \log n)$ per un grafo connesso (dove $m \geq n - 1$).

Implementazione Python

Di seguito viene mostrata un'implementazione in Python che utilizza una classe `Partition` (non mostrata nel dettaglio, ma conforme all'ADT descritto) e una `HeapPriorityQueue`.

```
1 def MST_Kruskal(g):
2     """Compute a minimum spanning tree of a graph using Kruskal's algorithm.
3
4     Return a list of edges that comprise the MST.
5
6     The elements of the graph's edges are assumed to be weights.
7     """
8
9     tree = []                # list of the edges in the MST
10    pq = HeapPriorityQueue()  # keys are weights, elements are edges
11    forest = Partition()     # forest of MST
12    position = {}            # associates each vertex to its entry
13
14    # Initialize partition: each vertex is a separate group
15    for v in g.vertices():
16        position[v] = forest.make_group(v)
17
18    # Insert all edges in pq
19    for e in g.edges():
20        pq.add(e.element(), e)
21
22    size = g.vertex_count()
23    # Loop until tree has n-1 edges or pq is empty
24    while len(tree) != size - 1 and not pq.is_empty():
25        weight, edge = pq.remove_min() # extract edge with min weight
26        u, v = edge.endpoints()
27
28        # find the sets that u and v belong to
29        a = forest.find(position[u])
30        b = forest.find(position[v])
31
32        if a != b:            # if the two sets are different
33            tree.append(edge)  # add edge in tree
34            forest.union(a, b) # merge the two sets
35
36    return tree
```

Listing 18.2: Implementazione Python dell'algoritmo di Kruskal. Da notare che questa implementazione restituisce la variabile `tree` la quale conterrà esattamente $V - k$ archi, dove V è il numero di vertici e k è il numero di componenti connesse. Nel caso di un grafo connesso, $k = 1$ e quindi la lista conterrà $V - 1$ archi, come previsto per un MST.

Esempio di esecuzione

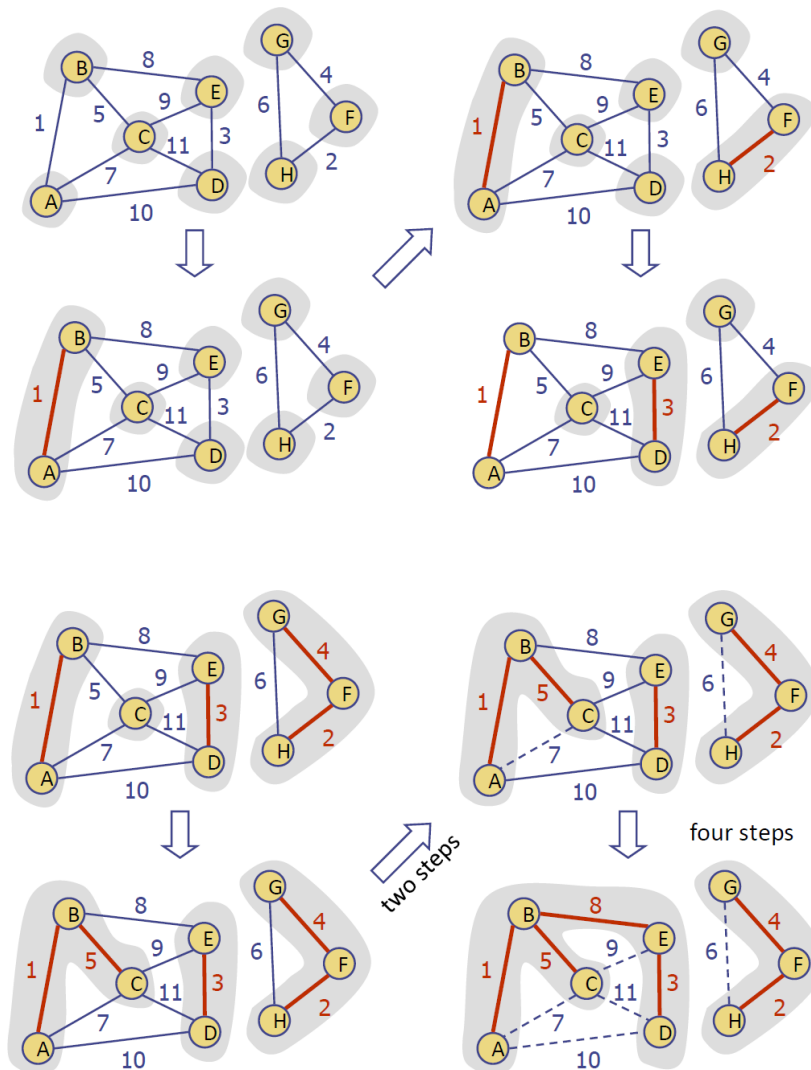


Figura 18.6: Esempio di esecuzione dell'algoritmo di Kruskal. Gli archi solidi rossi rappresentano la soluzione parziale (foresta), i verdi rappresentano gli archi che si stanno considerando in quel momento, gli archi tratteggiati sono quelli esaminati ma scartati.

Da notare, infine, che l'algoritmo di Kruskal e l'algoritmo di Prim-Jarník, pur risolvendo lo stesso problema, presentano alcune differenze chiave:

- **Approccio:** Prim costruisce un singolo albero espandendo un cluster di vertici connessi, mentre Kruskal costruisce una foresta di alberi disgiunti che vengono fusi insieme.
- **Strutture Dati:** Prim utilizza una coda di priorità per gestire i vertici, mentre Kruskal utilizza una coda di priorità per gli archi e una struttura per insiemi disgiunti (Partition) per gestire i cluster di vertici.
- **Applicabilità:** Prim richiede che il grafo sia connesso, mentre Kruskal può essere applicato anche a grafi non connessi, restituendo un Minimum Spanning Forest. D'altra parte, Prim è più semplice da implementare e occupa meno spazio in memoria.

18.3 Reverse-Delete Algorithm

Un altro algoritmo greedy per il calcolo di un Minimum Spanning Tree è il **Reverse-Delete Algorithm**. Questo algoritmo inizia con l'intero grafo e rimuove iterativamente gli archi, partendo da quelli di peso massimo, purché la rimozione non causi la disconnessione del grafo. L'idea è che, rimuovendo gli archi più pesanti che non sono necessari per mantenere la connettività, si ottiene un Minimum Spanning Tree.

Si tratta di un approccio "al contrario" rispetto agli algoritmi di Prim e Kruskal, che costruiscono il MST aggiungendo archi. Invece, il Reverse-Delete Algorithm parte da un grafo completo e rimuove archi fino a ottenere uno spanning tree. Anche in questo caso, come per l'algoritmo Prim, il grafo deve essere connesso affinché l'algoritmo funzioni correttamente.

Pseudocodice

L'algoritmo Reverse-Delete è descritto nel seguente pseudocodice:

Input: A simple connected weighted graph G with n vertices and m edges
Output: A minimum spanning tree T for G

1. Algorithm ReverseDelete(G):
2. T = set of all edges in G
3. Initialize a priority queue Q to contain all edges in G , using the weights as keys. { Q is a Max-Priority Queue}
4. **while** Q is not empty **do**
5. (u, v) = value returned by $Q.remove_max()$
6. Remove edge (u, v) from T
7. **if** there is no path between u and v in the graph (V, T) **then**
8. Add edge (u, v) back to T { (u, v) is a bridge}
9. **return** tree T

Analisi della complessità

A differenza degli algoritmi di Prim e Kruskal, l'algoritmo Reverse-Delete è generalmente meno efficiente a causa del costo elevato per la verifica della connettività.

- **Costruzione della Priority Queue:** L'inizializzazione della coda con tutti gli m archi richiede tempo $O(m)$.
- **Estrazione degli archi:** Il ciclo **while** itera m volte. In ogni iterazione, l'operazione `remove_max` impiega $O(\log m)$, portando il costo totale di questa fase a $O(m \log m)$.
- **Verifica della connettività:** Per ogni arco estratto, la verifica dell'esistenza di un cammino tramite una visita (BFS o DFS) impiega $O(n + m)$.

Nonostante l'efficienza nella gestione della coda, il costo computazionale è dominato dalla verifica della connettività ripetuta per ogni arco. Pertanto, la complessità globale è $O(m \cdot (n + m))$, ovvero $O(m^2)$ nel caso di grafi densi.

Capitolo 19

Network Flow

I problemi di **Network Flow** sono una classe fondamentale di problemi che si presentano nell'ambito del traffico in movimento in una rete di trasporto.

Concettualmente, questa tipologia di problemi nasce nel contesto della gestione del traffico all'interno di una rete di trasporto fisica. L'intuizione di base è modellare lo spostamento di entità materiali (come veicoli, merci, pacchetti di rete) attraverso un sistema interconnesso soggetto a vincoli fisici. In questa analogia con le reti di trasporto:

- **Gli archi (Edges):** rappresentano i canali di comunicazione o le strade che trasportano il traffico. Ogni arco possiede una capacità intrinseca che limita la quantità di flusso che può attraversarlo.
- **I nodi (Nodes):** fungono da giunzioni tra gli archi e operano come scambi (switches). Il loro compito è ricevere il flusso in entrata e ridistribuirlo sugli archi in uscita, rispettando solitamente un principio di conservazione (tutto ciò che entra deve uscire, salvo che per nodi speciali).
- **Il flusso (Flow):** è la quantità che deve essere spostata da specifici nodi di origine, detti *sorgenti* (sources), verso nodi di destinazione, detti *pozzi* (sinks).

Sebbene l'origine intuitiva risieda nella logistica e nei trasporti, l'astrazione matematica fornita dalla teoria dei grafi permette di applicare i problemi di Network Flow a una vasta gamma di campi totalmente distinti.

- **Reti di calcolatori:** instradamento dei pacchetti dati attraverso Internet massimizzando la banda passante.
- **Visione artificiale:** segmentazione delle immagini.
- **Scheduling e allocazione risorse:** assegnazione ottimale di compiti a macchine o persone.
- **Matching in grafi bipartiti:** accoppiamento massimo in contesti sociali o economici.

19.1 Flow Network - Rete di flusso

Una **Flow Network**¹ (*rete di flusso*) è un grafo orientato $G = (V, E)$ che rispetta dei vincoli specifici:

- Ogni arco $e \in E$ è associato con un peso non negativo che rappresenta la **capacità** c_e dell'arco. La capacità indica la quantità massima di flusso che può attraversare l'arco.
- Esiste un nodo sorgente (**source**) $s \in V$ da cui il flusso ha origine.
- Esiste un nodo destinazione (o pozzo) (**sink**) $t \in V$ dove il flusso termina.
- Tutti gli altri nodi sono detti **nodi interni**. Tutto il flusso in entrata in un nodo interno deve essere uguale al flusso in uscita da quel nodo.

Per il resto del capitolo, ci basiamo su delle assunzioni che ci aiuteranno a rendere più semplice la trattazione dei problemi di Network Flow, senza perdere di generalità.

- La capacità c_e di ogni arco e è un numero intero (non negativo).
- Il nodo sorgente s non ha archi entranti.
- Il nodo destinazione t non ha archi uscenti.
- Ciascun nodo interno ha almeno un arco incidente.

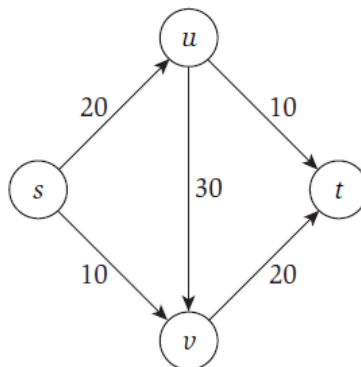


Figura 19.1: Esempio di Flow Network, con sorgente s e destinazione t , e capacità associate agli archi.

¹Con *Flow Network* (tradotto come **Rete di flusso**) si indica l'oggetto combinatorio statico, ovvero il grafo G dotato di capacità; con *Network Flow* (tradotto come **Flusso** o **Flusso su rete**) si intende invece la classe di problemi di ottimizzazione associata.

Definizione di Flusso Un **s-t Flow** in una Flow Network $G = (V, E)$ è una funzione $f : E \rightarrow R_{\geq 0}$ tale che:

- **Capacity condition:** Per ogni arco $e \in E$, il flusso $f(e)$ non può superare la capacità c_e :

$$0 \leq f(e) \leq c_e$$

- **Conservation condition:** Per ogni nodo interno v (quindi, esclusi s e t), la somma del flusso in entrata deve essere uguale alla somma del flusso in uscita:

$$\sum_{e \text{ into } v} f(e) = \sum_{e \text{ out of } v} f(e)$$

Dove $\sum_{e \text{ into } v} f(e)$ somma il valore di flusso $f(e)$ su tutti gli archi che entrano nel nodo v , mentre $\sum_{e \text{ out of } v} f(e)$ è la somma dei valori di flusso su tutti gli archi che escono dal nodo v .

La sorgente non ha archi in entrata, ma è permesso che abbia flusso in uscita; in altre parole, la sorgente può generare flusso. Simmetricamente, per la destinazione è permesso avere flusso in entrata, anche se non ha archi in uscita.

Il valore di un flusso f , denotato $v(f)$, è definito come la quantità di flusso generata dalla sorgente, ovvero la somma di tutti i flussi uscenti dalla sorgente:

$$v(f) = \sum_{e \text{ out of } s} f(e).$$

Per compattezza, definiamo per ogni nodo $v \in V$ il flusso totale in entrata e in uscita come:

$$f^{in}(v) = \sum_{e \text{ into } v} f(e) \quad \text{e} \quad f^{out}(v) = \sum_{e \text{ out of } v} f(e)$$

Con questa terminologia, la *conservation condition* per un nodo interno v si riscrive semplicemente come $f^{in}(v) = f^{out}(v)$. Il valore del flusso diventa $v(f) = f^{out}(s)$.

È utile estendere queste definizioni a un generico sottoinsieme di vertici $S \subseteq V$:

$$f^{in}(S) = \sum_{v \in S} f^{in}(v) \quad \text{e} \quad f^{out}(S) = \sum_{v \in S} f^{out}(v).$$

Una proprietà fondamentale derivante dalla conservazione puntuale è che la conservazione vale anche per gli insiemi. Se S è un insieme costituito interamente da nodi interni (cioè $s, t \notin S$), allora il flusso totale che entra nell'insieme deve essere uguale al flusso totale che ne esce:

$$f^{in}(S) = f^{out}(S)$$

Questa proprietà ci assicura che il flusso non può accumularsi né disperdersi in nessuna sottoregione della rete che non contenga la sorgente o la destinazione.

19.2 Maximum Flow Problem

Data una rete di flusso $G = (V, E)$ con sorgente s e destinazione t , un obiettivo naturale è determinare la massima capacità trasmissiva del sistema. Non ci limitiamo a cercare un instradamento qualsiasi, ma vogliamo sfruttare al massimo le capacità degli archi disponibili.

Il problema del **Maximum Flow** (Flusso Massimo) consiste nel trovare un flusso f che massimizzi il valore totale del flusso in uscita dalla sorgente $v(f) = f^{out}(s)$.

Supponiamo di voler trovare un flusso massimo in una rete. Scartata la programmazione dinamica, per la quale non esistono algoritmi noti in questo contesto, esploriamo un approccio greedy.

Partiamo da una situazione di flusso nullo: $f(e) = 0$ per ogni arco e . Sebbene questa configurazione rispetti i vincoli di capacità e conservazione, il valore del flusso è 0. Proviamo quindi ad aumentare il valore di f "spingendo" flusso lungo un cammino dalla sorgente s alla destinazione t , fino ai limiti imposti dalle capacità degli archi.

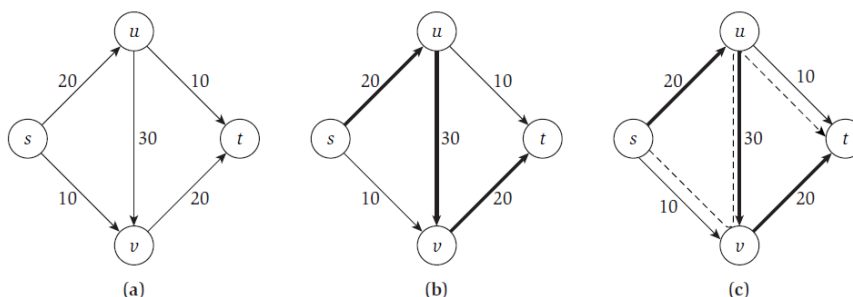


Figura 19.2: (a) La rete della Figura 19.1. (b) Un flusso valido di valore 20 che non permette ulteriori aumenti diretti. (c) L'aumento del flusso a 30 richiede di ridirigere il flusso esistente (linee tratteggiate).

Facendo riferimento alla Figura 19.2, potremmo scegliere il cammino composto dagli archi $\{(s, u), (u, v), (v, t)\}$, portando il flusso su ciascuno di essi a 20 e lasciando a 0 gli altri. Non siamo però all'ottimo (pari a 30), pur trovandoci in uno stallo: non esistono altri cammini $s - t$ con capacità disponibile per un aumento diretto.

Per sbloccare la situazione è necessario ridirigere il flusso esistente, come in Figura 19.2(c):

1. Spingiamo 10 unità di flusso lungo (s, v) ; questo crea un eccesso di flusso entrante in v .
2. Per compensare, "annulliamo" 10 unità di flusso sull'arco (u, v) . Questo ripristina la conservazione in v , ma lascia troppo poco flusso in uscita da u .
3. Infine, spingiamo 10 unità lungo (u, t) , ripristinando la conservazione in u .

Questa operazione porta il flusso totale a 30. L'intuizione fondamentale è che un algoritmo generale deve poter spingere flusso non solo in avanti (sfruttando la capacità residua), ma anche **indietro** (sugli archi già utilizzati). Tale meccanismo viene formalizzato introducendo il **Grafo Residuo**.

19.2.1 Residual Graph

Data una rete di flusso $G = (V, E)$, e un flusso f su G . Definiamo il **grafo residuo** G_f di G rispetto a f come segue.

- L'insieme dei nodi di G_f è lo stesso di quello di G .
- **Archi in avanti (forward edges):** Per ogni arco $e = (u, v) \in E$ tale che $f(e) < c_e$, includiamo in G_f l'arco $e' = (u, v)$ con capacità residua:

$$f(e) < c_e \implies c_f(e') = c_e - f(e)$$

Spiegazione: Questi archi rappresentano la disponibilità residua del canale. Indicano che è possibile spingere ulteriori $c_e - f(e)$ unità di flusso da u a v senza violare la capacità originale.

- **Archi all'indietro (backward edges):** Per ogni arco $e = (u, v) \in E$ tale che $f(e) > 0$, includiamo in G_f l'arco inverso $e' = (v, u)$ con capacità residua:

$$f(e) > 0 \implies c_f(e') = f(e)$$

Spiegazione: Questi archi rappresentano la possibilità di "annullare" fino a $f(e)$ unità di flusso precedentemente inviate, spingendo flusso all'indietro.

Da notare che, per come sono costruiti gli archi del grafo residuo, questi sono tutti strettamente positivi: $c_f(e') > 0$ per ogni arco e' in G_f .

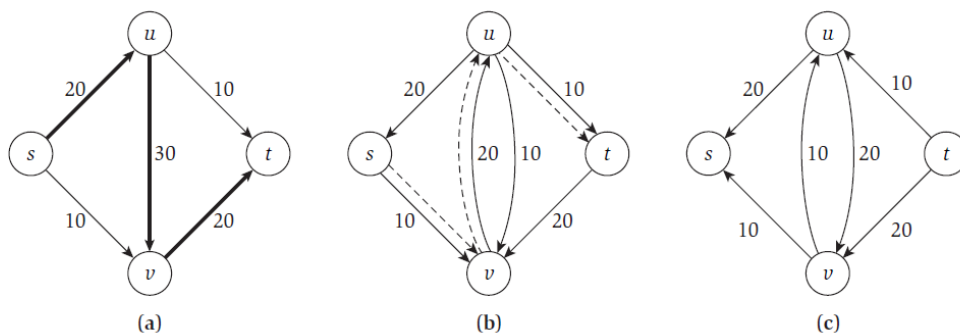


Figura 19.3: (a) Il grafo G con il cammino s, u, v, t usato per spingere le prime 20 unità di flusso. (b) Il grafo residuo del flusso risultante f , con la capacità residua accanto a ogni arco. La linea tratteggiata è il nuovo cammino aumentante (augmenting path). (c) Il grafo residuo dopo aver spinto ulteriori 10 unità di flusso lungo il nuovo cammino aumentante s, v, u, t .

Da notare che ciascun arco e in G può generare uno o due archi in G_f . Se $0 < f(e) < c_e$ risulta che sia un arco in avanti sia uno all'indietro sono inclusi in G_f . Questo comporta quindi che G_f ha al più il doppio degli archi di G .

Ci riferiamo alla capacità di un arco nel grafo residuo come **capacità residua**, per distinguere dalla capacità dell'arco corrispondente nella rete di flusso originale G .

19.2.2 Augmenting Path

Sia P un cammino semplice da s a t in G_f (cioè, P non visita alcun nodo più di una volta). Definiamo $\text{bottleneck}(P, f)$ come la **capacità residua minima** di qualsiasi arco su P , rispetto al flusso f . Di seguito definiamo l'operazione $\text{augment}(P, f)$, che produce un nuovo flusso f' in G .

Input: P è un cammino semplice s - t in G_f

f è un flusso in G

Output: f' è un flusso in G ottenuto aumentando f lungo P

```
1. Algorithm augment( $P, f$ ):
2.   let  $b = \text{bottleneck}(P, f)$ 
3.   for each edge  $e = (u, v)$  in  $P$  do #  $P$  composto da archi di  $G_f$ 
4.     if  $e$  is a forward edge then
5.        $f'(e) = f(e) + b$ 
6.     else
7.        $f'(e) = f(e) - b$ 
8.   return  $f'$ 
```

È stato proprio per poter eseguire questa operazione che abbiamo definito il grafo residuo; per riflettere l'importanza di augment , si indica spesso come **cammino aumentante** (*augmenting path*) qualsiasi cammino $s - t$ nel grafo residuo. Il risultato di $\text{augment}(P, f)$ è un nuovo flusso f' in G , ottenuto aumentando e diminuendo i valori di flusso sugli archi di P .

Proposizione Sia $G = (V, E)$ una rete di flusso, e sia f un flusso in G . Sia P un cammino semplice $s - t$ in G_f . Allora l'output f' di $\text{augment}(P, f)$ è un flusso in G .

Dimostrazione Per dimostrare che f' sia effettivamente un flusso, dobbiamo verificare che f' soddisfi le capacity conditions e conservation conditions.

Iniziamo con la capacity condition.

- **Arco in avanti:** Se (u, v) è un arco in avanti in G_f , esso corrisponde all'arco $e = (u, v) \in E$, con capacità residua $c_f(e) = c_e - f(e)$. Il nuovo flusso sarà $f'(e) = f(e) + b$. Poiché $b \leq c_e - f(e)$, otteniamo:

$$0 \leq f(e) \leq f'(e) = f(e) + b \leq f(e) + (c_e - f(e)) = c_e$$

La capacity condition è verificata.

- **Arco all'indietro:** Se (u, v) è un arco all'indietro in G_f , esso deriva dall'arco fisico $e = (v, u) \in E$, con capacità residua $c_f(e) = f(e)$. Poiché stiamo diminuendo il flusso sull'arco originale di b , abbiamo:

$$c_e \geq f(e) \geq f'(e) = f(e) - b \geq f(e) - f(e) = 0$$

La capacity condition è verificata anche in questo caso.

In entrambi i casi, il nuovo flusso rispetta i limiti imposti dalle capacità c_e e la non negatività.

Passiamo alla conservation condition. Dobbiamo verificare che per ogni nodo interno v appartenente al cammino P , la variazione della quantità di flusso in entrata in v è la stessa della variazione della quantità di flusso in uscita da v .² Siano u il nodo precedente e w il nodo successivo a v lungo il cammino P in G_f (ovvero $u \rightarrow v \rightarrow w$). Poiché f soddisfa la conservazione, è sufficiente mostrare che la variazione di flusso apportata da **augment** preserva l'equilibrio tra flusso entrante e uscente. Distinguiamo i quattro casi possibili in base alla tipologia degli archi (u, v) e (v, w) nel grafo residuo:

- **Forward $\rightarrow$ Forward:** Entrambi gli archi sono in avanti. In G esistono gli archi fisici $e_1 = (u, v)$ ed $e_2 = (v, w)$. L'operazione aumenta il flusso su entrambi di b . Di conseguenza, il flusso entrante in v aumenta di b (da e_1) e quello uscente aumenta di b (su e_2). L'equilibrio è mantenuto.
- **Backward $\rightarrow$ Backward:** Entrambi gli archi sono all'indietro. In G esistono gli archi fisici $e_1 = (v, u)$ ed $e_2 = (w, v)$. L'operazione diminuisce il flusso su entrambi di b . Diminuendo il flusso su e_1 (uscente da v), riduciamo l'uscita totale di b . Diminuendo il flusso su e_2 (entrante in v), riduciamo l'entrata totale di b . Poiché entrata e uscita diminuiscono della stessa quantità, l'equilibrio è mantenuto.
- **Forward $\rightarrow$ Backward:** L'arco entrante è in avanti, quello uscente è all'indietro. In G esistono gli archi fisici $e_1 = (u, v)$ ed $e_2 = (w, v)$. Aumentiamo il flusso su e_1 di b (aumento dell'entrata). Diminuiamo il flusso su e_2 di b (diminuzione dell'entrata). La variazione netta del flusso entrante è $+b - b = 0$, mentre il flusso uscente resta invariato. L'equilibrio è mantenuto.
- **Backward $\rightarrow$ Forward:** L'arco entrante è all'indietro, quello uscente è in avanti. In G esistono gli archi fisici $e_1 = (v, u)$ ed $e_2 = (v, w)$. Diminuiamo il flusso su e_1 di b (diminuzione dell'uscita). Aumentiamo il flusso su e_2 di b (aumento dell'uscita). La variazione netta del flusso uscente è $-b + b = 0$, mentre il flusso entrante resta invariato. L'equilibrio è mantenuto.

In tutti i casi, la proprietà di conservazione del flusso continua a valere per f' .

²Sulle slide del corso, il continuo di questa seconda parte della dimostrazione non è presente; le slide si limitano a enunciare che **augment** preserva la conservazione del flusso, senza fornire ulteriori dettagli.

19.2.3 MaxFlow Algorithm

Questa operazioni di *augmentation* ci permette di modificare un flusso esistente spingendo ulteriore flusso lungo cammini aumentanti nel grafo residuo. Ripetendo questa procedura, possiamo incrementare il valore del flusso fino a quando non esistono più cammini aumentanti nel grafo residuo. A questo punto, il flusso ottenuto è massimo.

Di seguito è riportato lo pseudocodice dell'algoritmo **MaxFlow**, o algoritmo di Ford-Fulkerson.

```
1. Algorithm Max-Flow( $G$ ):
2.   Initialize  $f(e) = 0$  for all edges  $e$  in  $G$ 
3.   Construct the residual graph  $G_f$ 
4.   while there is an  $s$ - $t$  path in  $G_f$  do
5.     Let  $P$  be a simple  $s$ - $t$  path in  $G_f$ 
6.      $f' = \text{Augment}(P, f)$ 
7.      $f = f'$ 
8.      $G_f = G_{f'}$ 
9.   return  $f$ 
```

L'algoritmo MaxFlow risulta piuttosto intuitivo. Ciò che può sembrare meno ovvio è capire se il ciclo **While** termina, e se il flusso restituito sia effettivamente massimo.

Per analizzare la correttezza e la terminazione dell'algoritmo, è necessario richiamare un'assunzione già fatta sulla natura delle capacità, che ora ripetiamo: assumiamo inizialmente che tutte le capacità degli archi c_e siano numeri interi.

Sotto questa ipotesi, dimostriamo inizialmente che l'algoritmo Max-Flow termina, e ne analizziamo la complessità; solo in un secondo momento dimostreremo che tale algoritmo produce un flusso massimo. La dimostrazione si basa su due proprietà fondamentali che valgono a ogni iterazione del ciclo **while**.

Proprietà 1: Flussi e capacità residue come numeri interi Possiamo facilmente osservare che in ogni fase intermedia dell'algoritmo i valori del flusso $f(e)$ e le capacità residue in G_f rimangono interi.

- Prima del ciclo **while**, il flusso è inizializzato a 0 ovunque, e le capacità residue coincidono con le capacità originali (che sono intere per ipotesi).
- Ad ogni step, quando troviamo un cammino aumentante P , calcoliamo il valore $b = \text{bottleneck}(P, f)$. Essendo b il minimo tra le capacità residue degli archi di P (che sono intere), anche b sarà un intero. Di conseguenza, anche le nuove capacità residue nel passo successivo rimarranno intere.

Proprietà 2: Il valore del flusso cresce strettamente Dimostriamo che ad ogni iterazione il valore del flusso aumenta di almeno 1 unità: $v(f') \geq v(f) + 1$. Consideriamo il primo arco e del cammino aumentante P . Poiché P inizia da s , e deve essere un arco uscente dalla sorgente in direzione del grafo residuo. Per definizione di grafo residuo, un arco esiste in P solo se la sua capacità residua è strettamente positiva: $c_f(e) > 0$. Poiché le capacità sono intere e P è un cammino valido in G_f , la capacità residua minima b deve essere almeno 1 ($b \geq 1$). L'operazione **Augment** aumenta il flusso su questo arco di b , incrementando di conseguenza il flusso totale uscente da s . Pertanto:

$$v(f') = f'^{out}(s) = f^{out}(s) + b \geq f^{out}(s) + 1 = v(f) + 1$$

Terminazione Per provare che l'algoritmo termina, definiamo C come la somma totale delle capacità degli archi uscenti dalla sorgente s :

$$C = \sum_{e \text{ out of } s} c_e$$

Il valore di un qualsiasi flusso ammissibile non può mai superare la capacità fisica totale degli archi che lasciano la sorgente, quindi $v(f) \leq C$. Poiché partiamo da un flusso nullo ($v(f) = 0$) e ad ogni iterazione il valore aumenta di almeno un intero ($v(f_{i+1}) \geq v(f_i) + 1$), l'algoritmo non può eseguire più di C iterazioni. Questo garantisce la terminazione.

Analisi della Complessità Valutiamo ora il costo computazionale. Il tempo totale è dato dal numero di iterazioni moltiplicato per il costo di una singola iterazione.

1. **Costo per iterazione:** In ogni passo dobbiamo trovare un cammino $s - t$ nel grafo residuo G_f e aggiornare i flussi.

- La ricerca del cammino può essere effettuata con una visita in ampiezza (BFS) o profondità (DFS).³ Questo richiede tempo $O(m + n)$.
- L'aggiornamento delle capacità e dei flussi lungo il cammino richiede tempo proporzionale alla lunghezza del cammino stesso, che è al più $O(n)$.

Pertanto, ogni iterazione costa $O(m)$ se si assume che il grafo sia connesso ($m \geq n - 1$).

2. **Numero di iterazioni:** Come dimostrato, il numero massimo di aumenti è limitato da C .

Combinando questi fattori, la complessità temporale dell'algoritmo^{4 5} Max-Flow è:

$$O(m \cdot C)$$

³In pratica si preferisce la BFS perché, trovando sempre il cammino più breve in termini di numero di archi, evita i casi patologici in cui il flusso viene incrementato di quantità minime, garantendo così una convergenza molto più rapida (polinomiale) rispetto a una scelta arbitraria o basata su DFS.

⁴È importante notare che la complessità $O(mC)$ non è polinomiale in senso stretto, ma *pseudo-polinomiale*. La complessità non dipende solo dalla dimensione dell'input (numero di nodi e archi), ma anche dal valore numerico C . Se C è grande rispetto alla dimensione del grafo, l'algoritmo può diventare inefficiente.

⁵Utilizzando tecniche più sofisticate per selezionare i cammini aumentanti, è possibile progettare algoritmi con complessità temporale $O(n^3)$, quindi con complessità polinomiale.

19.2.4 s-t Cuts

Finora abbiamo dimostrato che l'algoritmo Max-Flow termina e produce un flusso valido. Rimane da dimostrare che il flusso restituito sia effettivamente massimo.

Per fare ciò, introduciamo il concetto di **s-t Cut** (taglio s-t) in una rete di flusso.

Definizione: Un **s-t Cut** in una rete di flusso $G = (V, E)$ è una partizione dei nodi V in due insiemi disgiunti A e B tali che $s \in A$ e $t \in B$. Indichiamo gli archi che attraversano il taglio, ovvero quelli che vanno da un nodo in A a un nodo in B , come:

$$\text{cut}(A, B) = \{(u, v) \in E \mid u \in A, v \in B\}$$

La **capacità di un taglio** (A, B) , che indichiamo con $c(A, B)$, è definita come la somma delle capacità degli archi che vanno da A a B :

$$c(A, B) = \sum_{e \text{ out of } A} c_e$$

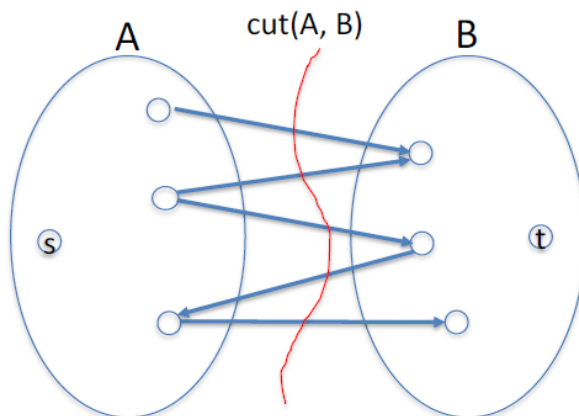


Figura 19.4: Esempio di s-t Cut in una rete di flusso.

In precedenza abbiamo visto come $C = \sum_{e \text{ out of } s} c_e$ fornisca già un limite superiore al valore di qualsiasi flusso, e si ha $v(f) \leq C$. Tuttavia, C potrebbe essere una sovrastima enorme del valore massimo di un flusso in G , per cui a volte questo limite è utile, ma altre volte si rivela molto debole.

Fortunatamente, il concetto di s-t Cuts ci permette di ottenere un limite superiore più stretto e significativo per il valore di qualsiasi flusso in G .

Proposizione Sia f un qualsiasi flusso in una rete di flusso $G = (V, E)$, e sia (A, B) un qualsiasi s-t Cut in G . Allora: $v(f) = f^{out}(A) - f^{in}(A)$.

Dimostrazione Per definizione, il valore del flusso è $v(f) = f^{out}(s)$. Dalle nostre assunzioni inoltre sappiamo che $f^{in}(s) = 0$, poiché la sorgente s non ha archi entranti e possiamo quindi scrivere: $v(f) = f^{out}(s) - f^{in}(s)$. Dal momento che ogni nodo v in A diverso da s è un nodo interno, sappiamo che $f^{out}(v) - f^{in}(v) = 0$ per tutti questi nodi. Quindi:

$$v(f) = \sum_{v \in A} (f^{out}(v) - f^{in}(v)),$$

dovuto al fatto che l'unico termine in questa somma che è diverso da zero è quello in cui v è uguale a s . Cerchiamo di riscrivere la somma a destra in un altro modo. Se un arco e ha entrambe le estremità in A , allora $f(e)$ appare una volta nella somma con un segno "+" e una volta con un segno "-", e quindi questi due termini si cancellano a vicenda. Se e ha solo la coda in A , allora $f(e)$ appare una sola volta nella somma, con un segno "+". Se e ha solo la testa in A , allora $f(e)$ appare anche una sola volta nella somma, con un segno "-". Infine, se e non ha nessuna delle due estremità in A , allora $f(e)$ non appare affatto nella somma. In vista di ciò, abbiamo:

$$\sum_{v \in A} (f^{out}(v) - f^{in}(v)) = \sum_{e \text{ out of } A} f(e) - \sum_{e \text{ into } A} f(e) = f^{out}(A) - f^{in}(A).$$

Mettendo insieme queste due equazioni, otteniamo: $v(f) = f^{out}(A) - f^{in}(A)$.

Utilizzando questa proposizione, possiamo ora dimostrare che il valore di qualsiasi flusso f è limitato superiormente dalla capacità di qualsiasi s-t Cut (A, B) in G . ■

Proposizione Sia f un qualsiasi flusso in una rete di flusso $G = (V, E)$, e sia (A, B) un qualsiasi s-t Cut in G . Allora: $v(f) \leq c(A, B)$.

Dimostrazione

$$v(f) = f^{out}(A) - f^{in}(A) \leq f^{out}(A) = \sum_{e \text{ out of } A} f(e) \leq \sum_{e \text{ out of } A} c_e = c(A, B)$$

La prima uguaglianza è data dalla proposizione precedente. Passiamo dalla prima alla seconda disuguaglianza poiché $f^{in}(A) \geq 0$, e passiamo dalla terza alla quarta applicando le condizioni di capacità a ciascun termine della somma. ■

In sostanza, questa proposizione afferma che tutto il flusso che va dalla sorgente s alla destinazione t deve attraversare gli archi che compongono il taglio $cut(A, B)$ per passare da A a B , e che al massimo $c(A, B)$ unità di flusso possono attraversare questi archi, dato che $c(A, B)$ è proprio la somma delle capacità di quegli archi.

Abbiamo quindi stabilito un limite superiore: il valore di un qualsiasi flusso non può mai superare la capacità di un qualsiasi taglio. Tuttavia, il nostro obiettivo è dimostrare che l'algoritmo Max-Flow restituisce effettivamente un flusso massimo. Per fare ciò, dobbiamo mostrare che esiste un taglio specifico la cui capacità è esattamente uguale al valore del flusso trovato dall'algoritmo.

Analisi alla terminazione dell'algoritmo Consideriamo lo stato dell'algoritmo Max-Flow nel momento in cui termina. La condizione di terminazione del ciclo `while` è che **non esistono più cammini aumentanti** (percorsi da s a t) nel grafo residuo G_f .

Basandoci su questa proprietà, possiamo costruire una specifica partizione dei nodi (A^*, B^*) definita come segue:

- $A^* = \{v \in V \mid \exists \text{ un cammino da } s \text{ a } v \text{ in } G_f\}$ (l'insieme dei nodi raggiungibili dalla sorgente nel grafo residuo).
- $B^* = V \setminus A^*$ (l'insieme di tutti gli altri nodi, non raggiungibili da s in G_f).

Verifichiamo che (A^*, B^*) sia effettivamente un **s-t Cut**:

1. La sorgente s è certamente in A^* .
2. La destinazione t deve trovarsi in B^* . Se t fosse in A^* , significherebbe che esiste un cammino da s a t in G_f , ma questo contraddice la condizione di terminazione dell'algoritmo.

Dunque, (A^*, B^*) è un taglio valido.

Caratteristiche degli archi attraverso il taglio (A^*, B^*) Esaminiamo ora il flusso f (quello calcolato dall'algoritmo al momento dell'arresto) sugli archi che attraversano questo specifico taglio. Consideriamo un arco fisico $e = (u, v)$ nel grafo originale G :

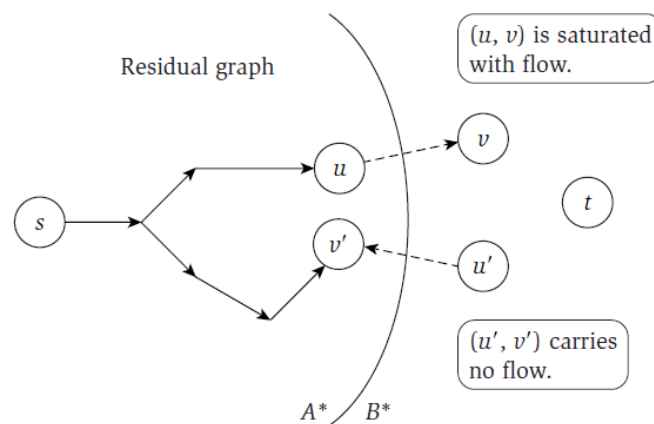


Figura 19.5: Il taglio (A^*, B^*) utilizzato nella dimostrazione.

1. **Archì in avanti (da A^* a B^*):** Sia $e = (u, v)$ un arco in G per cui $u \in A^*$ e $v \in B^*$, come mostrato in Figura 19.5. Se l'arco non fosse saturo (cioè se $f(e) < c_e$), allora nel grafo residuo G_f esisterebbe un arco in avanti (u, v) con capacità residua positiva. Ma se esiste l'arco (u, v) in G_f e u è raggiungibile da s (poiché $u \in A^*$), allora anche v sarebbe raggiungibile da s . Questo implicherebbe $v \in A^*$, contraddicendo l'ipotesi $v \in B^*$.

Conclusione: Tutti gli archi che vanno da A^* a B^* devono essere saturi.

$$\forall e \text{ out of } A^* \implies f(e) = c_e$$

2. **Archì all'indietro (da B^* a A^*):** Sia $e' = (u', v')$ un arco in G per cui $u' \in B^*$ e $v' \in A^*$, come mostrato in Figura 19.5. Vogliamo ottenere che $f(e') = 0$. Se così non fosse, l'arco e' genererebbe un arco all'indietro $e'' = (v', u')$ nel grafo residuo G_f . Poiché $v' \in A^*$, esiste un cammino da s a v' in G_f ; aggiungendo l'arco all'indietro e'' a questo cammino, otterremmo un cammino da s a u' in G_f , il che contraddice l'ipotesi che $u' \in B^*$.

Conclusione: Tutti gli archi che vanno da B^* a A^* devono avere flusso nullo.

$$\forall e \text{ into } A^* \implies f(e) = 0$$

Teorema Max-Flow Min-Cut Utilizzando le proprietà appena derivate, possiamo calcolare il valore del flusso f attraverso il taglio (A^*, B^*) usando il lemma precedente ($v(f) = f^{out}(A) - f^{in}(A)$):

$$v(f) = f^{out}(A^*) - f^{in}(A^*) = \sum_{e \text{ out of } A^*} f(e) - \sum_{e \text{ into } A^*} f(e)$$

Sostituendo i valori trovati ($f(e) = c_e$ per gli archi uscenti e $f(e) = 0$ per gli archi entranti):

$$v(f) = \sum_{e \text{ out of } A^*} c_e - \sum_{e \text{ into } A^*} 0$$

$$v(f) = \sum_{e \text{ out of } A^*} c_e = c(A^*, B^*)$$

Abbiamo quindi trovato uno specifico taglio (A^*, B^*) la cui capacità è esattamente uguale al valore del flusso f calcolato dall'algoritmo. Ricordando la disuguaglianza generale dimostrata in precedenza, secondo cui per *qualsiasi* flusso f' e *qualsiasi* taglio (A, B) vale sempre $v(f') \leq c(A, B)$, l'aver trovato un caso di uguaglianza ha due conseguenze immediate:

- **f è un flusso di valore massimo.** Poiché il valore di nessun flusso può mai superare la capacità del taglio (A^*, B^*) , e il nostro flusso f raggiunge esattamente tale capacità, è impossibile che esista un flusso con valore maggiore.
- **(A^*, B^*) è un taglio di capacità minima.** Poiché la capacità di nessun taglio può mai scendere al di sotto del valore del flusso f , e il nostro taglio (A^*, B^*) ha capacità esattamente pari a $v(f)$, è impossibile che esista un taglio con capacità minore.

Dal momento che l'algoritmo di Ford-Fulkerson termina quando non esistono più cammini aumentanti nel grafo residuo, e che abbiamo dimostrato che in questo caso il flusso calcolato è massimo, possiamo concludere l'algoritmo di Ford-Fulkerson restituisce la soluzione ottima.

Questo risultato ci porta al seguente teorema fondamentale nella teoria dei flussi di rete:

Max-Flow Min-Cut Theorem

In ogni rete di flusso, il valore del flusso massimo è uguale alla capacità del taglio minimo.

Calcolo del Min-Cut Oltre a dimostrare l'ottimalità del flusso, questa costruzione ci fornisce un algoritmo efficiente per trovare il taglio minimo in una rete. Dato un flusso massimo f (calcolato ad esempio con l'algoritmo Max-Flow), possiamo trovare il taglio di capacità minima (A^*, B^*) in tempo $O(m)$:

1. Costruiamo il grafo residuo G_f (a partire da f flusso massimo, calcolato con Max-Flow).
2. Eseguiamo una visita (BFS o DFS) partendo da s in G_f per identificare l'insieme A^* di tutti i nodi raggiungibili.
3. Poniamo $B^* = V \setminus A^*$.
4. Il taglio (A^*, B^*) è il taglio minimo cercato.

Possiamo vedere il problema del flusso massimo e quello del taglio minimo come due facce della stessa medaglia: il primo rappresenta il problema **primale**, mentre il secondo è il suo **duale**. La forza di questo legame sta nel fatto che non abbiamo bisogno di due procedure separate. Nel momento esatto in cui risolviamo il problema primale (trovando il flusso massimo), otteniamo *automaticamente* e a costo zero la soluzione per il problema duale. Questo è un grande vantaggio pratico, perché cercare direttamente il taglio minimo "da zero" sarebbe molto più complicato e meno intuitivo rispetto al calcolo del flusso.

L'algoritmo Max-Flow in presenza di capacità reali Per provare la correttezza dell'algoritmo Max-Flow, abbiamo fatto un'importante assunzione, e cioè che tutte le capacità degli archi siano numeri interi. Questa ipotesi è cruciale per garantire la terminazione dell'algoritmo, poiché ci ha permesso di dimostrare che ad ogni iterazione il valore del bottleneck fosse almeno 1, ovvero che il flusso aumenta di almeno 1 unità, limitando così il numero totale di iterazioni a C (la somma delle capacità uscenti da s).

In presenza di capacità reali, questa garanzia non vale più. La nostra analisi funziona addirittura nel caso di numeri razionali, ma se ad esempio le capacità fossero numeri irrazionali l'algoritmo potrebbe incrementare il flusso di quantità sempre più piccole, avvicinandosi asintoticamente al valore massimo senza mai raggiungerlo esattamente, e quindi non terminerebbe mai. In questo caso, è comunque possibile dimostrare che l'algoritmo Max-Flow produce un flusso di valore massimo, ma la dimostrazione richiede tecniche più avanzate e non è trattata in questo corso.

19.3 Il problema del Bipartite Matching

Una delle applicazioni più importanti del problema del Maximum-Flow è la risoluzione del problema del Bipartite Matching (accoppiamento bipartito).

Definizione: Un **grafo bipartito** $G = (V, E)$ è un grafo non orientato i cui nodi possono essere divisi in due insiemi disgiunti X e Y , e si ha $V = X \cup Y$ e $X \cap Y = \emptyset$, con la proprietà che ogni arco $e \in E$ abbia un'estremità in X e l'altra in Y .

Un **matching** M in G è un sottoinsieme degli archi $M \subseteq E$ tale che ogni nodo appare in al più un arco in M .

Il **Bipartite Matching Problem** è quello di trovare un matching in G di dimensione massima possibile.

Esempi di applicazioni del Bipartite Matching includono l'assegnazione di compiti a lavoratori, la corrispondenza tra studenti e progetti, o l'abbinamento tra offerte e richieste in un mercato.

Algoritmo di Bipartite Matching Un algoritmo molto semplice per risolvere il Bipartite Matching Problem sfrutta l'algoritmo Max-Flow. L'idea è di trasformare il grafo bipartito $G = (X \cup Y, E)$ in una rete di flusso $G' = (V', E')$ come segue:

1. Orientiamo gli archi di G da X verso Y .
2. Aggiungiamo una sorgente s e colleghiamola a tutti i nodi in X .
3. Aggiungiamo una destinazione t e colleghiamo tutti i nodi in Y a t .
4. Assegniamo capacità $c_e = 1$ a tutti gli archi in E' .
5. Calcoliamo un flusso massimo f in G' utilizzando l'algoritmo Max-Flow.

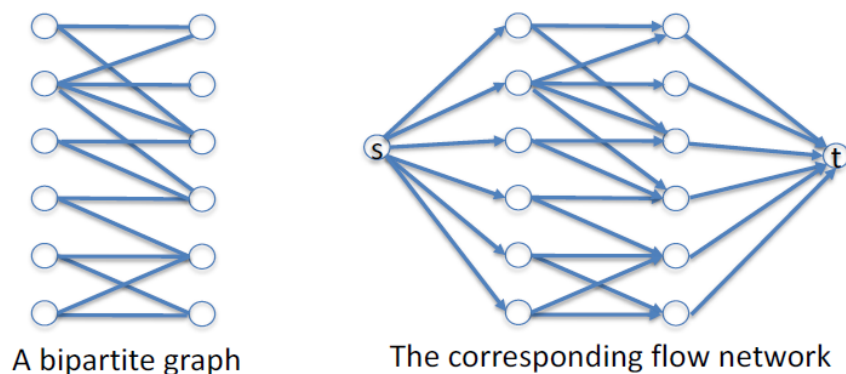


Figura 19.6: Esempio di trasformazione di un grafo bipartito in una rete di flusso per risolvere il Bipartite Matching Problem.

Per dimostrare che l'algoritmo basato sul flusso massimo risolve correttamente il problema del Bipartite Matching, dobbiamo stabilire una precisa equivalenza tra le soluzioni dei due problemi. Nello specifico, vale il seguente lemma:

Lemma La dimensione del matching massimo in G è uguale al valore del flusso massimo nella rete trasformata G' . Inoltre, gli archi che fanno parte del matching in G corrispondono esattamente agli archi che trasportano flusso positivo (flusso pari a 1) da X a Y in G' .

Dimostrazione Dimostriamo questa equivalenza analizzando le due direzioni:

1. **Da Matching a Flusso ($M \rightarrow f$):** Supponiamo che il grafo G possieda un matching $M = \{(x_1, y_1), (x_2, y_2), \dots, (x_k, y_k)\}$ di dimensione k . Possiamo costruire un flusso f in G' inviando 1 unità di flusso lungo ogni cammino della forma $s \rightarrow x_i \rightarrow y_i \rightarrow t$ per ogni coppia $(x_i, y_i) \in M$. Poiché M è un matching, ogni nodo x_i e y_i appare al più una volta, quindi i cammini sono disgiunti rispetto ai nodi interni e rispettano le capacità unitarie degli archi. La somma totale del flusso sarà quindi pari a k , dimostrando che esiste un flusso di valore almeno pari alla dimensione del matching.
2. **Da Flusso a Matching ($f \rightarrow M$):** Supponiamo di aver calcolato un flusso f in G' di valore k . Poiché tutte le capacità in G' sono intere (nello specifico, pari a 1), per la proprietà di integrità dei flussi, il flusso su ogni arco sarà intero, ovvero 0 oppure 1. Definiamo M come l'insieme degli archi (x_i, y_i) tali che il flusso tra x_i e y_i sia pari a 1. Poiché il valore totale del flusso è k , e il taglio (X, Y) separa s da t , ci devono essere esattamente k archi tra X e Y che trasportano 1 unità di flusso. Quindi $|M| = k$.

Resta da dimostrare che M è un matching valido. Dobbiamo verificare che nessun nodo sia incidente a più di un arco in M :

- **Per i nodi in X :** Ogni nodo $x \in X$ riceve flusso solo dalla sorgente s tramite l'arco (s, x) , che ha capacità 1. Pertanto, il flusso entrante in x è al massimo 1. Per la conservazione del flusso, il flusso uscente da x verso i nodi in Y non può superare 1. Se x fosse adiacente a due o più archi in M , uscirebbero 2 o più unità di flusso, il che è impossibile.
- **Per i nodi in Y :** Simmetricamente, ogni nodo $y \in Y$ invia flusso solo alla destinazione t tramite l'arco (y, t) , che ha capacità 1. Il flusso uscente da y è al massimo 1. Di conseguenza, il flusso entrante in y dai nodi di X non può superare 1. Se y fosse adiacente a due o più archi in M , riceverebbe 2 o più unità di flusso, violando il vincolo di capacità sull'arco verso la destinazione.

Concludiamo che ogni nodo partecipa a un solo arco con flusso positivo, rendendo M un matching valido.

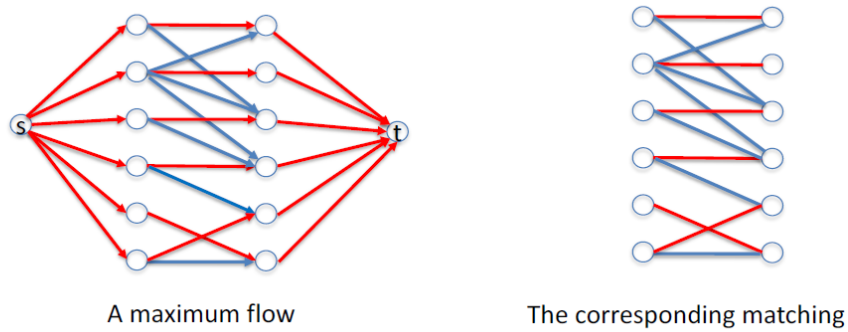


Figura 19.7: Esempio di corrispondenza tra flusso massimo e matching massimo in un grafo bipartito.

Complessità Computazionale La complessità temporale dell'algoritmo è $O(nm)$. Questo deriva dal fatto che il valore massimo del flusso (e quindi la dimensione massima del matching) non può superare $n/2$ (poiché ogni nodo può essere abbinato una sola volta). Utilizzando un algoritmo di calcolo del flusso come Ford-Fulkerson su una rete con capacità unitarie, ogni aumento di flusso richiede una visita del grafo ($O(m)$), e il numero totale di aumenti è limitato dal valore del flusso massimo ($O(n)$).